

Git & GitHub Mastery

From Zero → Internals → Production → Senior Engineer

Volume 1 of 4: Foundations and the Git data model

Baseline: Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Every Git transcript in this book is real output from the lab scripts under `labs/`, reproducible with `labs/verify-all.sh`. GitHub-side behavior is described from GitHub's documentation and is marked as such.

Contents of this volume

- Chapter 1: Fundamentals
- Chapter 2: The Mental Model
- Chapter 3: Git Internals
- Chapter 4: The Working Tree
- Chapter 5: The Index
- Chapter 6: Commits
- Chapter 7: Branches

Chapter 1: Fundamentals

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch01/`.

1.1 Why this matters

A deploy pipeline goes red on a Friday evening. The engineer on call says the fix is committed and can show the commit on screen. The pipeline fails again on the same line. Your CTO asks three questions:

1. What exactly is in the commit that the pipeline built?
2. Where does that commit exist right now: on the laptop, on the server, or on both?
3. What is the smallest change that makes the main branch correct, and how will we know that it worked?

None of these is a question about commands. Each is a question about state: which bytes Git recorded, which names point at them, and which machine holds them. An engineer who knows fifty commands and cannot answer these questions will try things until something appears to work. An engineer who can answer them needs about ten read-only commands and a method.

This chapter builds that base: a toolchain you can vouch for, a sandbox where mistakes cost nothing, a first repository watched from the inside, a diagnosis ritual, and a root-cause framework that every later chapter reuses. Section 1.12 solves the Friday problem with them.

1.2 The problem version control solves

In one sentence. A version control system records successive states of a set of files so that you can recall any state, compare any two, and see who recorded each one, when, and why.

Analogy. A bound laboratory notebook: pages are numbered, dated and signed, and a correction is a new page, never an erased one. The analogy breaks in two places: a notebook is one sequence kept by one person, while a repository holds many lines of work that people extend in parallel and later join; and each recorded state in Git is the whole project, not that day's notes.

Precisely. Four capabilities define the category.

Capability	Question it answers
Recall	What did the project look like when release 2.3 was built?
Compare	What differs between what runs in production and what I have here?
Attribute	Who changed this line, when, and with what explanation?
Integrate	Two people changed the project at the same time. What is the combined result, and where do their changes collide?

The first three are bookkeeping. The fourth is the reason the tool has a learning curve: combining states that were produced independently needs a model of history in which "independently" has a precise meaning. Chapter 2 gives you that model.

In production. The evaluation score of an LLM application drops from one week to the next. To explain the drop you need the prompt templates, the evaluation configuration and the scoring code exactly as they were when last week's number was produced. If they live in one repository and the report recorded a commit ID, one command

restores that state. If they live on a shared drive, the question has no reliable answer. Datasets and model weights need additional tools (Chapter 22: Git LFS, Chapter 28: AI/ML workflows).

1.3 Centralized versus distributed

In one sentence. In a centralized system the history lives on one server and each developer holds one checked-out version; in a distributed system every developer's copy contains the history, and "the server" is one more copy that the team agrees to treat as the meeting point.

Analogy. A centralized system is a reference library: the books stay in the building, and when the building is closed nobody reads. A distributed system gives every reader a copy of the whole library. The analogy breaks because library copies never need to agree again, while repositories must: each copy accumulates its own additions, and exchanging them is an explicit act that can find two copies in disagreement.

Precisely. The classification follows Pro Git ([About Version Control](#)).

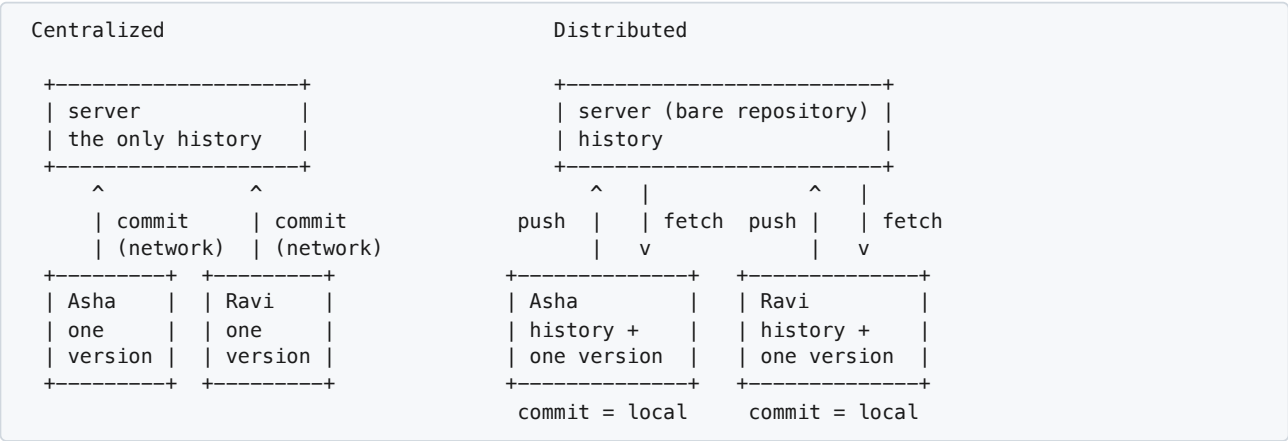
	Centralized (CVS, Subversion, Perforce)	Distributed (Git, Mercurial)
Where the history lives	On one server	In every clone
Recording a change	A network operation against the server	A local write; publishing is a separate step
Server unreachable	Nobody can record changes or collaborate	Local work continues; the exchange waits
Server disk lost, no backups	The history is gone	Every clone still holds what it had fetched

Three consequences shape daily work with Git.

- 1. Commit and publish are different operations.** `git commit` writes to the repository on your disk. `git push` copies commits to another repository. "It is committed" and "it is on the server" are independent facts, checked with different commands.
- 2. Your view of the server is a cached copy.** A name such as `origin/main` records "the last-known state of a branch in a remote repository" ([gitdata model](#)), as of your last exchange with it. Chapter 12: Remote Operations is built on this fact.
- 3. Nothing in Git makes one repository the authority.** The team does, by convention and through permissions on the hosting platform.

Inside `.git`. The whole repository, meaning the history and every name in it, is the `.git` directory at the top of your project. A clone is a copy of another repository's objects and names. The copy on a server is normally a *bare* repository: the same contents with no checked-out files beside them.

Picture.



In production. If GitHub is unreachable for an hour, you can still commit, create branches, compare versions and search history, because all of that reads your own `.git`. You cannot push, fetch, open a pull request or start a workflow. The reverse also holds: a commit that exists only on one laptop is one disk failure away from not existing. Distributed means that copies are possible, not that they are made for you.

1.4 Git's design goals

Git was written in 2005 for the Linux kernel, after the kernel project lost free use of the proprietary tool BitKeeper. Pro Git lists the goals of the new system ([A Short History of Git](#)). Each goal explains a design decision you will meet later.

Goal as listed in Pro Git	Design decision it led to	Where you study it
Speed	Almost every operation reads local files	This chapter
Simple design	Four object types named by a hash of their content, plus names that point at them	Chapter 2; Chapter 3: Git Internals
Strong support for non-linear development (thousands of parallel branches)	A branch is one small name that points at a commit, so creating one costs almost nothing	Chapter 7: Branches; Chapter 8: Merge
Fully distributed	Every clone has the history; fetch and push exchange objects and update names	Chapter 12: Remote Operations
Able to handle large projects like the Linux kernel efficiently	Compressed pack files; later, partial clone and sparse checkout	Chapter 3; Chapter 26: Performance

Git's manual describes it as "a fast, scalable, distributed revision control system with an unusually rich command set that provides both high-level operations and full access to internals" ([git\(1\)](#)). The high-level commands (`git add`, `git commit`, `git merge`) are called *porcelain*. The low-level ones (`git hash-object`, `git update-ref`) are called *plumbing*. This course uses plumbing to prove what porcelain did.

Two things are absent from the goals, and both surprise people. Git has no user accounts and no permissions: the layer that hosts a repository decides who may read or write it. And Git stores content, not intentions: it does not record that a file was renamed, and it does not track empty directories (Chapter 4: Working Tree).

1.5 Git the tool versus GitHub the platform

In one sentence. Git is a program on your machine that stores history in a `.git` directory; GitHub is a hosted service that stores Git repositories and surrounds them with its own objects: accounts, permissions, pull requests, reviews, rulesets and workflows.

Analogy. Git is a file format with its editor; GitHub is a document-management service where such files are stored, discussed and approved. The analogy breaks because GitHub also runs Git on its servers and performs Git operations for you: pressing a merge button creates commits. Platform actions change Git data, so you must know which layer did what.

Precisely. For each thing you will meet, the table gives the layer that owns it and whether it comes along with `git clone`.

Thing	Layer	Where it lives	Arrives with a clone?
Commits, directory trees, file contents, annotated tags	Git	Objects in the repository	Yes
Branches and tags	Git	Names (refs) in the repository	Yes; the server's branches arrive as remote-tracking branches such as <code>origin/main</code>

Thing	Layer	Where it lives	Arrives with a clone?
Author and committer name and email; a signature	Git	Inside each commit	Yes
The "Verified" badge next to a commit	GitHub	GitHub's database: its judgement of the signature	No
HEAD, the index, reflogs, stashes, hooks, <code>.git/config</code>	Git, local only	Your <code>.git</code>	No; each clone has its own
Pull request, review, comment	GitHub	GitHub's database, plus read-only refs that GitHub creates on the server for the pull request's commits	No
Issue, discussion, star	GitHub	GitHub's database	No
Fork	GitHub	A server-side repository linked to its parent; to Git it is one more remote	Not applicable
Ruleset, branch protection	GitHub	Repository or organization settings	No
Release	GitHub	An object that points at a Git tag and adds notes and files	The tag yes, the release no
A workflow file under <code>.github/workflows/</code>	Git	A tracked file in commits	Yes
A workflow run, its logs, artifacts and secrets	GitHub Actions	GitHub	No
Access tokens, SSH keys, collaborator roles	GitHub	Your account and the repository settings	No

The GitHub rows are described from the Phase 0 research report, which links the documentation page for each (report, sections 2 and 12). Other hosts such as GitLab, Bitbucket and Forgejo run the same Git and surround it with different platform objects.

GitHub, not Git. A pull request is not a Git feature. Git has commits, branches and merges. GitHub adds the pull request as a place to discuss a proposed merge, and performs the merge on its servers when you press the button (Chapter 17: Pull Requests).

In production. "The merge is blocked" has three root causes in three layers. Git stops a merge when both sides changed the same lines. GitHub stops one when a ruleset requires a review that has not been given. GitHub Actions stops one when a required check has not reported success. The symptoms look alike in a chat message and the fixes have nothing in common, so every explanation in this book names the layer that acted.

1.6 The two Gits on your Mac

Your Mac has two Git installations. Which one runs depends on the order of directories in the `PATH` of the shell that starts it. This transcript is specific to this machine: its lab script is marked volatile, so the verification tool checks that it runs and does not compare its output.

```
$ which -a git
/opt/homebrew/bin/git
/usr/bin/git
/opt/homebrew/bin/git
$ git --version
git version 2.55.0
$ /usr/bin/git --version
git version 2.50.1 (Apple Git-155)
```

which `-a` lists every match in search order; the first line is the program that runs when you type `git`. Here it is the Homebrew build, 2.55.0. `/usr/bin/git` is the Git that Apple ships with its developer tools, 2.50.1. The third line repeats the first because the Homebrew directory appears twice in this shell's `PATH`, which is harmless.

The difference matters. Git 2.50.1 has no `git history` and no `git last-modified`, and its manual still labels `git switch` and `git restore` as experimental, a label Git removed in 2.51 (report, section 1). Each installation also reads its own system-level configuration file. You meet the Apple Git in three situations: a script that calls `/usr/bin/git` by its full path, a scheduled job or graphical tool that starts with a shorter `PATH` than your terminal, and a colleague's Mac without Homebrew.

```
$ git version --build-options
git version 2.55.0
cpu: arm64
no commit associated with this build
sizeof-long: 8
sizeof-size_t: 8
shell-path: /bin/sh
rust: disabled
feature: fsmonitor--daemon
gettext: enabled
libcurl: 8.7.1
zlib: 1.2.12
SHA-1: SHA1_DC
SHA-256: SHA256_BLK
default-ref-format: files
default-hash: sha1
```

Three lines are worth reading now. `default-hash: sha1` means new repositories name objects with SHA-1, and `SHA-1: SHA1_DC` means the implementation detects known collision attacks. `default-ref-format: files` means branch and tag names are stored as plain files, which is why this book can show them with `cat`. Chapter 3: Git Internals covers the alternatives, SHA-256 and reftable.

Version note. Older behavior: Git 2.55.0 of 29 June 2026 is what Homebrew installed here. Current behavior: the latest release is Git 2.56.0 of 28 September 2026, which adds a few commands and no security fix that 2.55.0 lacks. Since: Git 2.56. Recommended: upgrading is optional (`brew upgrade git`); this book marks what needs it as "added in Git 2.56 (not run here)". Source: report, section 1.

GitHub, not Git. The GitHub CLI `gh` is a separate program. Installed here: 2.88.1. Latest on 1 October 2026: 2.102.0, with six security-fix releases in between, so upgrade it (`brew upgrade gh`) before the GitHub chapters. From 2.91.0 the CLI collects pseudonymous usage telemetry unless you opt out with `gh config set telemetry disabled` or `export GH_TELEMETRY=false`. Neither was run here, because the setting does not exist in 2.88.1. Sources: [GitHub changelog](#), [gh environment variables](#).

1.7 How to use this book

Conventions

You see	It means
In one sentence. Analogy. Precisely. Inside <code>.git</code> . See it. Picture. In production.	The ladder on which each important concept is taught, from plain words down to files on disk and back up to a real team
A transcript in a <code>text</code> block	Real output, produced by a script under <code>labs/</code> and inserted by a tool; never typed by hand

You see	It means
<code>\$LAB</code>	The lab root, <code>~/git-mastery-labs</code> unless you set <code>GIT_MASTERY_LABS</code>
<code>[exit status: 1] ;</code> a line starting with <code>#</code>	The exit status of the command above; a comment written by the script
A state table	What a command did to the working tree, the index, HEAD, the current branch, the rest of <code>.git</code> , the remote and GitHub
A root-cause box	A surprising behavior explained in seven fixed lines (section 1.10)
<code>> **Version note.**</code> , <code>> **GitHub, not Git.**</code> , <code>> **Outdated advice.**</code> , <code>> **Unverified.**</code>	Version-dependent behavior; a warning about which layer acts; advice from older tutorials that no longer holds; a statement that could not be confirmed

Three tools

Run them from the course folder, the directory that contains `textbook/`, `labs/` and `lab-manual/`. The lab manual's introduction has the details (`lab-manual/README.md`).

Tool	What it does
<code>labs/shell m00</code>	Opens your shell in <code>\$LAB/hands-on/m00</code> with an isolated Git configuration and the real clock. This is where you type labs by hand. <code>exit</code> leaves it
<code>labs/run ch01/first-repo</code>	Replays one demo or lab with a fixed identity and clock and prints its transcript. The sandbox stays in <code>\$LAB/ch01/first-repo</code> for inspection
<code>labs/verify-all.sh ch01</code>	Replays every script of a chapter and compares the output with the snippets printed in the book: <code>PASS</code> , <code>VOLATILE</code> or <code>FAIL</code> per script

Why the lab configuration is isolated

Git reads settings from several files, and one of them is your personal `~/.gitconfig`. If labs read it, your aliases and defaults would change the output; if labs wrote it, an experiment would change your real setup. The lab environment therefore points Git at a configuration file inside the sandbox and switches off the system-wide file.

See it. The Git environment that the lab library sets for every replay. A shell can export other variables whose names begin with `GIT_`: the course's own `GIT_MASTERY_LABS`, which only moves the lab root, or prompt settings such as `GIT_PS1_SHOWDIRTYSTATE`. The script `labs/ch01/sandbox.sh` removes those before it prints the listing, so the transcript is the same on every machine:

```
$ env | grep '^GIT_' | sort
GIT_AUTHOR_DATE=@1788755520 +0530
GIT_AUTHOR_EMAIL=you@example.com
GIT_AUTHOR_NAME=Lab User
GIT_CEILING_DIRECTORIES=$LAB:$LAB
GIT_COMMITTER_DATE=@1788755520 +0530
GIT_COMMITTER_EMAIL=you@example.com
GIT_COMMITTER_NAME=Lab User
GIT_CONFIG_GLOBAL=$LAB/ch01/sandbox/home/.gitconfig
GIT_CONFIG_NOSYSTEM=1
GIT_EDITOR=true
GIT_MERGE_AUTOEDIT=no
GIT_PAGER=cat
GIT_TERMINAL_PROMPT=0
GIT_TEST_DATE_NOW=1788755520
```

Variable	Effect	In <code>labs/shell</code> too?
<code>GIT_CONFIG_GLOBAL</code>	Git reads this file instead of <code>~/.gitconfig</code>	Yes
<code>GIT_CONFIG_NOSYSTEM=1</code>	Git skips the system-wide configuration file	Yes

Variable	Effect	In labs/shell too?
<code>GIT_CEILING_DIRECTORIES</code>	Git stops searching for a <code>.git</code> directory at the lab root, so a sandbox never acts on a repository above it	Yes
<code>GIT_AUTHOR_DATE</code> , <code>GIT_COMMITTER_DATE</code>	The fixed clock	No: real clock
<code>GIT_TEST_DATE_NOW</code>	A variable Git's own test suite uses to fix "now" for date arithmetic, so that relative dates ("2 minutes ago") and time-based selectors (<code>HEAD@{5.minutes.ago}</code>) give the same answer on every run	No: real clock
<code>GIT_AUTHOR_NAME</code> , <code>GIT_AUTHOR_EMAIL</code> and the committer pair	A fixed identity that overrides configuration	No: identity comes from the lab configuration
<code>GIT_EDITOR=true</code> , <code>GIT_PAGER=cat</code> , <code>GIT_TERMINAL_PROMPT=0</code>	No editor, pager or password prompt, because a script cannot answer them	No: your editor and pager work
<code>GIT_MERGE_AUTOEDIT=no</code>	A merge does not open an editor for its message	Yes

Inside the sandbox, a "global" setting is global only to the sandbox:

```
$ git config list --show-origin --show-scope
global file:$LAB/ch01/sandbox/home/.gitconfig user.name=Lab User
global file:$LAB/ch01/sandbox/home/.gitconfig user.email=you@example.com
global file:$LAB/ch01/sandbox/home/.gitconfig init.defaultbranch=main
global file:$LAB/ch01/sandbox/home/.gitconfig gc.reflogexpire=never
global file:$LAB/ch01/sandbox/home/.gitconfig gc.reflogexpireunreachable=never
$ git config set --global alias.st "status --short --branch"
$ git config list --show-origin --show-scope
global file:$LAB/ch01/sandbox/home/.gitconfig user.name=Lab User
global file:$LAB/ch01/sandbox/home/.gitconfig user.email=you@example.com
global file:$LAB/ch01/sandbox/home/.gitconfig init.defaultbranch=main
global file:$LAB/ch01/sandbox/home/.gitconfig gc.reflogexpire=never
global file:$LAB/ch01/sandbox/home/.gitconfig gc.reflogexpireunreachable=never
global file:$LAB/ch01/sandbox/home/.gitconfig alias.st=status --short --branch
```

`--show-scope` and `--show-origin` print the scope of every setting and the file it came from. The alias landed in the sandbox file; your real configuration was neither read nor written. The two `gc.reflogexpire...` lines are part of every sandbox configuration; the next subsection explains why they are there.

Why the clock is fixed

A commit records when it was made, and the commit ID is computed from everything in the commit, including that time. The same commands run at two different moments produce different IDs. Replays pin the identity and the clock, so the IDs printed in this book are the IDs you get.

```
$ git init -q clock && cd clock
$ git commit -q --allow-empty -m "Check the lab clock"
$ git log --format=fuller
commit c67262763db796bdaa778e529bc358323e7f96df
Author: Lab User <you@example.com>
AuthorDate: Mon Sep 7 10:07:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:07:00 2026 +0530
```

Check the lab clock

The lab clock starts at Monday 7 September 2026, 10:00 at UTC+05:30, and advances one minute before each command. `@1788755520 +0530` in the environment listing is that clock in Git's raw format: seconds since 1 January

1970 UTC, then the time-zone offset. In `labs/shell` the clock is real, so commits you type by hand get IDs that differ from the book. Chapter 6: Commits explains why in full.

A fixed clock has a side effect that the lab environment has to neutralise, and the reason is worth knowing because it is the first example in this book of a behavior that depends on *which clock* Git reads.

Observed behavior : with a fixed lab clock, some replays would print different output depending on the real date

Git state : every commit and every relog entry in a sandbox carries the fixed lab date

Mechanism : two parts of Git compare relog dates with the REAL clock, not with the commit dates:

"now"

1. git fsck (Git 2.53.0 and later) skips relog entries dated later than the real

2. git gc and git relog expire drop relog entries older than 90 days, or older than 30 days when the commit is no longer reachable from a branch

Root cause : a lab date in the future trips rule 1; a lab date in the past trips rule 2 once it is old enough

Why Git does this : expiry is a real-time retention policy (Chapter 13: Recovery)

Correct fix : keep the lab clock in the past, and switch time-based relog expiry off in the sandbox

Prevention : the sandbox configuration sets gc.relogExpire and gc.relogExpireUnreachable to "never"

In a replay your commits therefore always behave like fresh work, on whatever day you run it. Three things follow. First, the two `gc.*` lines appear in every sandbox configuration listing in this book. Second, real repositories use Git's defaults, which Chapter 13 covers; the labs demonstrate only explicit expiry (`--expire=now`), which does not depend on any clock. Third, `GIT_TEST_DATE_NOW` pins date arithmetic only: `git fsck` and relog expiry read the real clock regardless, which is why the configuration lines are needed as well.

Version note. The `git fsck` rule exists in Git 2.53.0 and later (git/git commit f6b262581a , "fsck: snapshot default refs before object walk"; Chapter 3 cites the source). Apple's Git 2.50.1 on the same Mac does not apply it.

GitHub-side material

What happens on GitHub's servers cannot be replayed on your Mac. Those parts show commands without output and describe the expected result from GitHub's documentation, with a link. For the GitHub chapters you create a free organization with public practice repositories yourself (Chapter 15: GitHub). Nothing in this book asks you to type credentials into a script.

1.8 Command risk labels

The first time a state-changing command appears in a chapter it carries one of three labels, and each chapter ends with a table of them.

Label	Meaning	Examples
 SAFE	Reads state or only adds objects. Nothing is lost	<code>git status</code> , <code>git log</code> , <code>git fetch</code> , <code>git relog</code> , <code>git commit</code>
 CAUTION	Moves refs or rewrites local history. Recoverable through the relog if you know how	<code>git reset --soft</code> , <code>git rebase</code> , <code>git commit --amend</code>
 DANGEROUS	Can destroy uncommitted work, remote history, or the safety net itself	<code>git reset --hard</code> , <code>git clean -fd</code> , <code>git push --force</code> and its conditional forms <code>--force-with-lease</code> and <code>--force-if-includes</code> , <code>git relog expire --expire=now --all</code> , <code>git gc --prune=now</code>

Before any  command this book answers five questions: what it changes, what it can destroy, how to preview it, how to recover, and when it is appropriate. Make that your own rule: if you cannot answer the five questions, you are not ready to run the command on a repository that matters.

The labels describe the worst case. `git commit` is  because it only adds, and a wrong commit is corrected by another commit. `git reset --hard` is  because it overwrites working-tree files that were never recorded anywhere, and Git cannot bring back what it never stored.

1.9 A guided first repository

You will create a repository for a small evaluation harness, `rag-eval`, and list the files under `.git` after every step. The listings leave out the sample hooks, which never change. The script is `labs/ch01/first-repo.sh`; replay it with `labs/run ch01/first-repo`.

Step 1: `git init`

```
$ git init rag-eval
Initialized empty Git repository in $LAB/ch01/first-repo/rag-eval/.git/
$ cd rag-eval
$ ls -lF .git
config
description
HEAD
hooks/
info/
objects/
refs/
$ cat .git/HEAD
ref: refs/heads/main
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

`git init` created the directory and, inside it, `.git`. Three entries matter now. `objects/` will hold every version of every file; it is empty. `refs/` will hold branch and tag names; it is empty too. `HEAD` says which branch you are on: `refs/heads/main`. No such file exists yet under `refs/`, which is why `git status` says `No commits yet`. Git calls this an *unborn* branch. The name `main` comes from the lab configuration; section 1.13 shows what unconfigured Git chooses.

Step 2: create files

```
$ echo '# rag-eval' > README.md
$ mkdir configs
$ printf 'model: small-v2\ntimeout_s: 60\n' > configs/eval.yaml
$ find .git -type f -not -path '.git/hooks/*' | sort
.git/config
.git/description
.git/HEAD
.git/info/exclude
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md
  configs/
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Two files exist in the working tree, and `.git` holds the same four files as before. Editing files is invisible to Git until you tell it. `git status` calls the files untracked and shows `configs/` as one line because nothing inside it is tracked.

Step 3: `git add`

```
$ git add README.md configs/eval.yaml
$ find .git -type f -not -path '.git/hooks/*' | sort
.git/config
.git/description
.git/HEAD
.git/index
.git/info/exclude
.git/objects/3a/79082bc80505c7543e79c4312e3e3d276a0e04
.git/objects/42/2e9c0c59f64154ed466adcfb7b7b7c0d43e89f
$ git ls-files --stage
100644 3a79082bc80505c7543e79c4312e3e3d276a0e04 0      README.md
100644 422e9c0c59f64154ed466adcfb7b7b7c0d43e89f 0      configs/eval.yaml
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
    new file:   configs/eval.yaml
```

Three files are new. `.git/index` is the index, also called the staging area: the list of what the next commit will contain. The two files under `objects/` hold the contents of `README.md` and `configs/eval.yaml`. Each object is named by a 40-digit ID, split into a two-digit directory and a 38-digit file name. `git ls-files --stage` prints the index: file mode, object ID, stage number, path. The IDs in the index are the names of the two object files. So `git add` did two things: it stored the content, and it recorded which content belongs to which path. No commit exists yet.

Step 4: `git commit`

```
$ git commit -m "Add README and evaluation config"
[main (root-commit) f7c044e] Add README and evaluation config
 2 files changed, 3 insertions(+)
 create mode 100644 README.md
 create mode 100644 configs/eval.yaml
$ find .git -type f -not -path '.git/hooks/*' | sort
.git/COMMIT_EDITMSG
.git/config
.git/description
.git/HEAD
.git/index
.git/info/exclude
.git/logs/HEAD
.git/logs/refs/heads/main
.git/objects/0d/722fc2a87c4b61afef355250e6e77f41a44cd0
.git/objects/3a/79082bc80505c7543e79c4312e3e3d276a0e04
.git/objects/42/2e9c0c59f64154ed466adcfb7b7b7c0d43e89f
.git/objects/e6/21cb090a7d826cce2a614885f534bde7015533
.git/objects/f7/c044e2033a8967176ac7d6b5c18e17042ea1cf
.git/refs/heads/main
```

`[main (root-commit) f7c044e]` reads: on branch `main`, a commit without a parent, whose ID starts with `f7c044e`. Seven files are new.

- Three objects: the commit (`f7/c044e2...`), a tree for the top directory (`e6/21cb09...`) and a tree for `configs/` (`0d/722fc2...`). A tree is Git's record of one directory. Chapter 2 opens all three.
- `refs/heads/main` : the branch now exists.
- `logs/HEAD` and `logs/refs/heads/main` : the reflogs, journals of where `HEAD` and `main` have pointed.
- `COMMIT_EDITMSG` : the text of the last commit message.

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
f7c044e2033a8967176ac7d6b5c18e17042ea1cf
$ git log
commit f7c044e2033a8967176ac7d6b5c18e17042ea1cf
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:16:00 2026 +0530

    Add README and evaluation config
$ git status
On branch main
nothing to commit, working tree clean
```

`HEAD` still contains the branch name. The branch file contains the commit ID. That is the whole mechanism of being "on a branch": `HEAD` names a branch, and the branch names a commit. `git status` compares that commit, the index and the working tree, finds them identical, and reports a clean working tree.

Step 5: a second commit

```
$ echo 'top_k: 5' >> configs/eval.yaml
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   configs/eval.yaml

no changes added to commit (use "git add" and/or "git commit -a")
$ git diff
diff --git a/configs/eval.yaml b/configs/eval.yaml
index 422e9c0..327cd0b 100644
--- a/configs/eval.yaml
+++ b/configs/eval.yaml
@@ -1,2 +1,3 @@
 model: small-v2
 timeout_s: 60
+top_k: 5
$ git add configs/eval.yaml
$ git commit -m "Add top_k to evaluation config"
[main 2da8d74] Add top_k to evaluation config
1 file changed, 1 insertion(+)
$ git log --oneline
2da8d74 Add top_k to evaluation config
f7c044e Add README and evaluation config
```

After the edit, `git status` lists the file under "Changes not staged for commit": the working tree differs from the index, and `git diff` shows that difference. `git add` copies the new content into the index, and `git commit` records the index.

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
2da8d743d511369d13dcc318644c8c1ba33eea28
$ cat .git/logs/HEAD
0000000000000000000000000000000000000000 f7c044e2033a8967176ac7d6b5c18e17042ea1cf Lab User
<you@example.com> 1788756360 +0530    commit (initial): Add README and evaluation config
f7c044e2033a8967176ac7d6b5c18e17042ea1cf 2da8d743d511369d13dcc318644c8c1ba33eea28 Lab User
<you@example.com> 1788756960 +0530    commit: Add top_k to evaluation config
```

```
$ find .git/objects -type f | wc -l
9
```

HEAD did not change. The branch file did: it now holds 2da8d74... A commit moves the current branch, and HEAD follows only because it names that branch. logs/HEAD has one line per movement: old ID, new ID, who, when (the lab clock) and why. The first old ID is all zeros because nothing came before. Nine objects exist: five from the first commit and four from the second, namely one new file content, two new trees because configs/ and the top directory both changed, and the commit. README.md was not stored a second time. Chapter 2 explains why.

Picture. The state after the second commit, with the abbreviated IDs from the transcript.



Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git init rag-eval	Empty directory created	Does not exist yet	Created: ref: refs/heads/main	Does not exist (unborn)	config, description, hooks/, info/, empty objects/ and refs/	unchanged	unchanged
Create or edit a file	Changed	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged
git add <path>	unchanged	The entry for the path points at the new content	unchanged	unchanged	One new object per new file content	unchanged	unchanged
git commit	unchanged	unchanged	unchanged: still names the branch	Created, or moved to the new commit	New tree and commit objects, a line in each reflog, COMMIT_EDITMSG	unchanged	unchanged

In production. A build server checks out a commit. It never sees your working tree or your index. A change that was edited but not added, or added but not committed, does not exist for anyone else, however real it looks in your editor.

1.10 The root-cause framework

Every problem in this book is handled in the same eleven steps.

```
SYMPTOM -> OBSERVE -> COLLECT EVIDENCE -> UNDERSTAND STATE -> FORM HYPOTHESES -> TEST HYPOTHESES
-> IDENTIFY ROOT CAUSE -> SELECT LOWEST-RISK FIX -> EXECUTE -> VERIFY -> PREVENT
```

Step	What you do
SYMPTOM	Write down what was observed, without interpretation. "CI fails with a timeout" is a symptom; "the commit is broken" is already a guess
OBSERVE	Look before you touch. Start with git status
COLLECT EVIDENCE	Run the diagnosis ritual of section 1.11 and keep the output
UNDERSTAND STATE	State what the working tree, the index, HEAD, the branches and the remote contain

Step	What you do
FORM HYPOTHESES	List every mechanism that could produce the symptom from that state. One hypothesis is a belief, not an analysis
TEST HYPOTHESES	For each, name the command whose output would differ if it were true, and run it
IDENTIFY ROOT CAUSE	Name the mechanism that survived and the layer it belongs to: Git, GitHub or GitHub Actions
SELECT LOWEST-RISK FIX	List the possible fixes with their risk labels. Choose the one that destroys least and is simplest to undo
EXECUTE	Preview, then run, one change at a time
VERIFY	Repeat the commands that showed the problem. The evidence must have changed for the reason you predicted
PREVENT	Change a habit, a setting or a rule so the problem cannot recur silently

Two rules make the framework safe. First, everything up to and including IDENTIFY ROOT CAUSE is read-only. Most damage in Git incidents is done by a command typed before the state was understood. Second, every explanation ends in the same seven-line box, so that a reader can check whether it is complete:

```

Observed behavior : what was seen
Git state         : what the working tree, index, HEAD, refs and remote held
Mechanism         : what Git does with that state
Root cause        : the one fact that, had it been different, would have prevented the symptom
Why Git does this : the design reason
Correct fix        : the lowest-risk change that repairs the state
Prevention        : what stops a silent recurrence

```

1.11 The ten-command diagnosis

COLLECT EVIDENCE always starts with the same ten commands. `git diff` is run twice, once for each of its two comparisons, so you type eleven lines. Run them every time, in this order, including when you think you already know the answer. The ritual exists because people under pressure skip the one command that would have shown the cause.

The repository below is `rag-eval` some days later, with a real problem. **Symptom:** the evaluation job in CI fails with a timeout after 60 seconds, although the commit "Raise eval timeout to 120s" is on `main`; on the author's laptop the same test passes. A bare repository on disk plays the server. The script is `labs/ch01/diagnosis.sh`.

1. `git status`: where am I, and what is uncommitted?

```

$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   eval/runner.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        run.log

no changes added to commit (use "git add" and/or "git commit -a")

```

You are on `main`. It matches `origin/main` as of the last exchange with the server. One tracked file is modified and not staged, and one file is untracked.

2. `git branch -vv`: which branches exist, and what do they follow?

```
$ git branch -vv
feature/cache 1c96817 Cache embeddings between runs
* main        f29df3b [origin/main] Raise eval timeout to 120s
```

The star marks the current branch. `[origin/main]` is its upstream; no "ahead" or "behind" appears, so the last commit has been pushed. `feature/cache` has no upstream: it exists only here.

3. `git remote -v`: which other repositories does this one talk to?

```
$ git remote -v
origin $LAB/ch01/diagnosis/origin.git (fetch)
origin $LAB/ch01/diagnosis/origin.git (push)
```

One remote named `origin`, with the same address for fetching and pushing. In your own projects this is a GitHub URL.

4. `git log --graph --decorate --oneline --all`: what does the history look like?

```
$ git log --graph --decorate --oneline --all
* f29df3b (HEAD -> main, origin/main) Raise eval timeout to 120s
| * 1c96817 (feature/cache) Cache embeddings between runs
|/
* 25fbbb0 Add evaluation runner and config
```

`--all` includes every branch, `--graph` draws the lines between commits, and `--decorate` prints the names that point at each commit. `HEAD`, `main` and `origin/main` all sit on `f29df3b`.

5. `git reflog`: what was done in this repository, and in which order?

```
$ git reflog
f29df3b HEAD@{0}: commit: Raise eval timeout to 120s
25fbbb0 HEAD@{1}: checkout: moving from feature/cache to main
1c96817 HEAD@{2}: commit: Cache embeddings between runs
25fbbb0 HEAD@{3}: checkout: moving from main to feature/cache
25fbbb0 HEAD@{4}: commit (initial): Add evaluation runner and config
```

The reflog lists every position `HEAD` has had, newest first. It is a local journal that no other clone has. The latest action here was the commit under suspicion.

6. `git rev-parse`: which exact object does a name stand for?

```
$ git rev-parse HEAD origin/main
f29df3b2e662b85dc3a9228a7b1b36248235bb3d
f29df3b2e662b85dc3a9228a7b1b36248235bb3d
$ git rev-parse --abbrev-ref HEAD
main
$ git rev-parse --show-toplevel
$LAB/ch01/diagnosis/rag-eval
```

Names can be ambiguous; object IDs cannot. `HEAD` and `origin/main` resolve to the same ID. `--abbrev-ref HEAD` prints the current branch, and `--show-toplevel` prints the root of the working tree, which settles whether you are in the repository you think you are in.

7. `git show`: what is in this commit?

```
$ git show --stat HEAD
commit f29df3b2e662b85dc3a9228a7b1b36248235bb3d
Author: Lab User <you@example.com>
```

Date: Mon Sep 7 10:11:00 2026 +0530

Raise eval timeout to 120s

```
configs/eval.yaml | 2 +-  
1 file changed, 1 insertion(+), 1 deletion(-)
```

The commit changes one file, `configs/eval.yaml`. With `--stat` you get the list of changed files; without it, the full difference.

8 and 9. `git diff` and `git diff --cached`: what is not committed?

```
$ git diff  
diff --git a/eval/runner.py b/eval/runner.py  
index 817bce0..2d928f5 100644  
--- a/eval/runner.py  
+++ b/eval/runner.py  
@@ -1,6 +1,6 @@  
     """Run one evaluation batch against the retrieval service."""  
  
-MAX_TIMEOUT_S = 60  
+MAX_TIMEOUT_S = 120  
  
def effective_timeout(cfg):  
$ git diff --cached
```

`git diff` compares the working tree with the index: the change to `MAX_TIMEOUT_S` exists on disk and is not staged. `git diff --cached` compares the index with the last commit and prints nothing: nothing is staged. (`--staged` is a synonym of `--cached`.)

10. `git config list --show-origin --show-scope`: which settings are in effect?

```
$ git config list --show-origin --show-scope  
global file:$LAB/ch01/diagnosis/home/.gitconfig user.name=Lab User  
global file:$LAB/ch01/diagnosis/home/.gitconfig user.email=you@example.com  
global file:$LAB/ch01/diagnosis/home/.gitconfig init.defaultbranch=main  
global file:$LAB/ch01/diagnosis/home/.gitconfig gc.reflogexpire=never  
global file:$LAB/ch01/diagnosis/home/.gitconfig gc.reflogexpireunreachable=never  
local file:./git/config core.repositoryformatversion=0  
local file:./git/config core.filemode=true  
local file:./git/config core.bare=false  
local file:./git/config core.logallrefupdates=true  
local file:./git/config core.ignorecase=true  
local file:./git/config core.precomposeunicode=true  
local file:./git/config remote.origin.url=$LAB/ch01/diagnosis/origin.git  
local file:./git/config remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*  
local file:./git/config branch.main.remote=origin  
local file:./git/config branch.main.merge=refs/heads/main
```

Identity, the remote's address, and the two `branch.main.*` lines that make `origin/main` the upstream of `main`. When a command behaves differently on two machines, this listing is where the difference shows.

11. `git ls-files`: what does Git track?

```
$ git ls-files  
README.md  
configs/eval.yaml  
eval/runner.py
```

`eval/runner.py` is tracked. `run.log` is absent, which matches "untracked" in the status output.

Command	What it reveals
<code>git status</code>	Current branch, relation to its upstream as of the last exchange, staged, unstaged and untracked paths
<code>git branch -vv</code>	Each local branch, the commit it points at, its upstream, and how far ahead or behind it is
<code>git remote -v</code>	The other repositories this one exchanges data with
<code>git log --graph --decorate --oneline --all</code>	The shape of the history and where every name points
<code>git reflog</code>	What happened to HEAD in this repository, newest first
<code>git rev-parse</code>	The exact object ID behind a name; the current branch; the repository root
<code>git show</code>	One commit: author, date, message and change
<code>git diff</code>	Working tree against index: what is not staged
<code>git diff --cached</code>	Index against the last commit: what the next commit would record
<code>git config list --show-origin --show-scope</code>	Every setting in effect and the file it comes from
<code>git ls-files</code>	The paths in the index, which are the tracked files

1.12 Worked example: the fix that was committed and did not ship

The evidence is collected. The remaining steps of the framework follow.

UNDERSTAND STATE. Outputs 6, 7 and 8 give the content of the two files in each place.

	Last commit f29df3b (also on the server)	Index	Working tree
configs/eval.yaml	<code>timeout_s: 120</code>	same	same
eval/runner.py	<code>MAX_TIMEOUT_S = 60</code>	<code>MAX_TIMEOUT_S = 60</code>	<code>MAX_TIMEOUT_S = 120</code>

Your clone	The server (origin.git, bare)
<pre> +-----+ 25fbbb0---f29df3b main, origin/main \ (HEAD -> main) 1c96817 feature/cache index: runner.py has 60 working tree: runner.py has 120 (the fix) +-----+ </pre>	<pre> +-----+ 25fbbb0---f29df3b main CI builds f29df3b: MAX_TIMEOUT_S = 60 +-----+ </pre>

FORM HYPOTHESES and TEST HYPOTHESES.

Hypothesis	Evidence that would confirm it	Result
H1: the change to <code>runner.py</code> was never saved	The working tree shows 60	Rejected: it shows 120
H2: it was saved and never staged, so no commit contains it	The commit shows 60; <code>git diff</code> shows the change	Confirmed
H3: it was committed on another branch	Another commit touches <code>runner.py</code>	Rejected: only 25fbbb0 does
H4: it was committed and not pushed, so CI built an older commit	<code>HEAD</code> and <code>origin/main</code> differ	Rejected: same ID

```

$ git log --oneline --all -- eval/runner.py
25fbbb0 Add evaluation runner and config
$ git show HEAD:eval/runner.py | grep '^MAX_TIMEOUT_S'

```

```
MAX_TIMEOUT_S = 60
$ grep '^MAX_TIMEOUT_S' eval/runner.py
MAX_TIMEOUT_S = 120
```

`git log --all -- <path>` lists the commits on any branch that changed the path. `git show HEAD:eval/runner.py` prints the file as the last commit recorded it.

IDENTIFY ROOT CAUSE.

```
Observed behavior : CI fails with the old 60-second limit although the fix commit is on main.
                    The test passes on the author's laptop.
Git state          : HEAD = origin/main = f29df3b, which changes configs/eval.yaml only.
                    eval/runner.py is modified in the working tree and not staged.
Mechanism          : git commit records the index. Two files were edited; one was staged.
                    The local test reads the working tree. CI checks out the commit.
Root cause         : The second file was never added to the index, so it is in no commit.
Why Git does this  : The index exists so that you choose what a commit contains. A commit is
                    what was staged, not what is on disk.
Correct fix        : Stage eval/runner.py, commit, push. One new commit; nothing is rewritten.
Prevention         : Read git status and git diff --cached before each commit.
                    Trust the test that runs on the commit, which is what CI does.
```

The layer is Git. GitHub and Actions did their job: the pipeline tested the commit it was given.

SELECT LOWEST-RISK FIX.

Option	Risk	Verdict
A new commit with the missing file	🟢 adds a commit; anyone who already fetched f29df3b is unaffected	Chosen
Amend f29df3b and force-push	🟡 then 🛑 : replaces a commit that the server and possibly colleagues already have	Rejected: a tidier log is not worth rewriting shared history

EXECUTE. `git push` 🟡 changes shared state, so preview it with `--dry-run`.

```
$ git add eval/runner.py
$ git diff --cached --stat
eval/runner.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git commit -m "Raise runner timeout cap to 120s"
[main 230ef10] Raise runner timeout cap to 120s
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push --dry-run
To $LAB/ch01/diagnosis/origin.git
f29df3b..230ef10  main -> main
$ git push
To $LAB/ch01/diagnosis/origin.git
f29df3b..230ef10  main -> main
```

`git diff --cached` now shows exactly what the commit will record. `f29df3b..230ef10 main -> main` means the server's `main` moved from the old commit to the new one.

VERIFY.

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    run.log

nothing added to commit but untracked files present (use "git add" to track)
$ git log --oneline -3
```

```

230ef10 Raise runner timeout cap to 120s
f29df3b Raise eval timeout to 120s
25fbabb0 Add evaluation runner and config
$ git rev-parse HEAD origin/main
230ef105d2234274936997629350568cbac83839
230ef105d2234274936997629350568cbac83839
$ git show HEAD:eval/runner.py | grep '^MAX_TIMEOUT_S'
MAX_TIMEOUT_S = 120

```

Nothing tracked is modified, the new commit is on top, `HEAD` and `origin/main` agree, and the committed file has the new value. On GitHub, the pipeline run for `230ef10` must pass; that part cannot be shown from a local sandbox.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git push</code> 🟡	unchanged	unchanged	unchanged	unchanged	<code>refs/remotes/origin/main</code> moves to the pushed commit	The server's <code>main</code> moves; missing objects are copied to it	If the remote is on GitHub, workflows that listen for pushes can start (Chapter 20A)

PREVENT. Before every commit, read both lists in `git status`, and read `git diff --cached` as the text of what you are about to record. `git commit -a` would have included the second file, because it stages every modified tracked file first; for the same reason it also includes changes you did not mean to commit, so it is not a substitute for looking.

1.13 What can go wrong

Four failures that people meet on the first day, from `labs/ch01/pitfalls.sh`.

The first branch is not called `main`.

```

# With no configuration at all, Git 2.55 still names the first branch master.
$ GIT_CONFIG_GLOBAL=/dev/null git init plain
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in $LAB/ch01/pitfalls/plain/.git/
$ cat plain/.git/HEAD
ref: refs/heads/master
$ git -C plain branch -m main
$ cat plain/.git/HEAD
ref: refs/heads/main

```

With no configuration at all, Git 2.55 names the first branch `master` and prints a hint. The lab configuration sets `init.defaultBranch=main`, which is why every other transcript says `main`. `git branch -m main` 🟡 renames the unborn branch; the last line shows that only the text in `HEAD` changed.

Git says there is no repository.

```
$ mkdir notes && cd notes
$ git status
fatal: not a git repository (or any of the parent directories): .git
[exit status: 128]
$ cd ..
```

Git searched the current directory and its parents for a `.git` directory and found none. Lab 0.2 produces the same message a second way, by damaging `.git` itself.

`git commit` records nothing.

```
$ echo 'Evaluation harness.' >> README.md
$ git commit -m "Describe the project"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
[exit status: 1]
```

Exit status 1 and no commit: the change is in the working tree and the index was never updated. The partial form of this failure, some files staged and others not, is the worked example of section 1.12.

A repository inside a repository.

```
$ git status --short
?? vendor/
$ git add .
warning: adding embedded git repository: vendor/tokenizer
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> vendor/tokenizer
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached vendor/tokenizer
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
$ git ls-files --stage
100644 3a79082bc80505c7543e79c4312e3e3d276a0e04 0      README.md
160000 26803e94675c7c4dba579a4a20be4109737ce628 0      vendor/tokenizer
```

`vendor/tokenizer` has its own `.git` directory. Git does not add its files. It records one entry with mode `160000`, a *gitlink*: the ID of a commit in the inner repository. A clone of the outer repository would contain an empty directory there. This happens when you run `git init` or `git clone` inside an existing working tree. The hint tells you how to undo it:

```
# The command from the hint is refused. Unstaging with git restore works.
$ git rm --cached vendor/tokenizer
error: the following file has staged content different from both the
file and the HEAD:
    vendor/tokenizer
(use -f to force removal)
[exit status: 1]
$ git restore --staged vendor/tokenizer
```

```
$ git status --short
?? vendor/
```

Observed behavior : `git rm --cached vendor/tokenizer`, the command printed in the hint, is refused.

Git state : The index holds a new gitlink for `vendor/tokenizer`. HEAD has no entry for it. `vendor/tokenizer` still contains its own `.git` directory.

Mechanism : `git rm` refuses to drop an index entry that matches neither HEAD nor the working tree. For a gitlink, the working-tree half of that test treats a nested repository that keeps its own `.git` directory as something removal could lose.

Root cause : A freshly added embedded repository fails both halves of the test.

Why Git does this : The test protects staged content that exists nowhere else.

Correct fix : `git restore --staged vendor/tokenizer` resets only the index entry. Before the first commit there is no HEAD to restore from; there, `git rm --cached -f` works, and `--cached` guarantees that the working tree is not touched.

Prevention : Run `git rev-parse --show-toplevel` before `git init` or `git clone`.

The mechanism was read from the Git 2.55.0 source ([builtin/rm.c](#), [submodule.c](#)). After unstaging, decide what the inner repository is: a separate project to ignore, or a dependency to add properly as a submodule (Chapter 23: Submodules).

Symptom	Diagnosis	Fix	Prevention
A command from this book is "not a git command"	<code>which -a git</code> , <code>git --version</code>	Put the Homebrew directory first in <code>PATH</code> ; do not call <code>/usr/bin/git</code> by full path	Print <code>git --version</code> at the top of scripts and CI logs
The first branch is <code>master</code>	<code>git config list --show-origin</code> shows no <code>init.defaultBranch</code>	<code>git branch -m main</code> in the new repository	Set <code>init.defaultBranch</code> once (Chapter 14B)
"not a git repository"	<code>pwd</code> , <code>git rev-parse --show-toplevel</code> , then look at <code>.git/HEAD</code>	Change into the project, or restore HEAD (Lab 0.2)	Know where the repository root is before you act
<code>git commit</code> makes no commit, or one that lacks a file	<code>git status</code> , <code>git show --stat HEAD</code> , <code>git diff</code>	<code>git add</code> the paths and commit again	Read <code>git diff --cached</code> before committing
"adding embedded git repository"	<code>git ls-files --stage</code> shows mode <code>160000</code>	<code>git restore --staged <path></code> , then ignore it or make it a submodule	<code>git rev-parse --show-toplevel</code> before <code>git init</code> or <code>git clone</code>
Every folder in your home directory shows as untracked	<code>git rev-parse --show-toplevel</code> prints your home directory	Remove the stray <code>.git</code> after checking that it holds no commits you need	Give <code>git init</code> a directory argument

1.14 When not to use it, and dangerous edge cases

When Git alone is the wrong tool.

- Large binary data such as datasets, model weights and media. Every version of every file stays in the history and travels to every clone. Keep a pointer in Git and the data elsewhere (Chapter 22: Git LFS, Chapter 28: AI/ML workflows).
- Secrets. A committed and pushed secret is in every clone. Deleting the file in a later commit does not remove it from history, and the remedy starts with rotating the secret (Chapter 21B).
- Generated files: build output, virtual environments, caches. Ignore them (Chapter 4: Working Tree).

When the ritual is not enough. The ten commands describe one clone. They say nothing certain about the server until you fetch, and nothing at all about GitHub objects such as pull requests, rulesets and check results.

Dangerous edge cases.

- `git status` can write. By default it refreshes the cached file information in `.git/index` and writes the file back. That never changes history, but it matters in two situations: when you investigate a damaged repository, work on a copy; and in scripts that run in the background, use `git --no-optional-locks status`, as the `git-status` manual advises.
- Editing `.git` by hand. Lab 0.2 does it in a sandbox to show which files Git cannot live without. In a real repository, use the plumbing command that performs the same edit with locking and a relog entry, for example `git symbolic-ref` or `git update-ref`.
- `git init` in the wrong place. In your home directory it turns everything you own into one working tree. Inside another repository it creates the embedded repository of section 1.13.
- `rm -rf .git` . **What it changes:** it deletes the repository and leaves the working-tree files. **What it can destroy:** every commit that was never pushed, every local branch, stash and relog; that is, the safety net itself. **Preview:** `git log --oneline --all` and `git branch -vv` show what exists only here. **Recovery:** none through Git; another clone or a disk backup is the only source. **When appropriate:** a repository created by mistake that holds no commit you need.

1.15 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git init</code>		Creates <code>.git</code> ; in an existing repository it only adds missing template files	<code>git rev-parse --show-toplevel</code> tells you whether you are already inside a repository	Remove the new <code>.git</code> if it was created in the wrong place and holds nothing
<code>git add <path></code>		Stores content as objects and points index entries at it	<code>git status</code> , <code>git diff</code>	<code>git restore --staged <path></code>
<code>git commit</code>		Adds tree and commit objects, moves the current branch, appends to the reflogs	<code>git diff --cached</code>	A further commit, or Chapter 11: Reset, Revert, Restore
<code>git restore --staged <path></code> , <code>git rm --cached <path></code>		Change or remove an index entry; files on disk stay	<code>git diff --cached</code> , <code>git ls-files --stage</code>	<code>git add <path></code>
<code>git branch -m <name></code>		Renames the current branch	<code>git branch -vv</code>	Rename it back
<code>git config set --global <key> <value></code>		Writes your personal configuration file (the sandbox file in the lab)	<code>git config list --show-origin --show-scope</code>	<code>git config unset --global <key></code>
<code>git push</code>		Moves a branch on the remote and copies objects to it	<code>git push --dry-run</code>	Cannot be taken back quietly once others have fetched; publish a correcting commit
<code>rm -rf .git</code>		Deletes the repository	<code>git log --oneline --all</code> , <code>git branch -vv</code>	None through Git

1.16 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
<code>git config</code>	<code>--list</code> , <code>--get</code> , <code>--unset</code> options	<code>list</code> , <code>get</code> , <code>set</code> , <code>unset</code> subcommands; the options still work	Git 2.46	Use the subcommands, and remember that they fail on older

Topic	Older behavior	Current behavior	Since	Recommended
				Gits such as the 2.43.0 that Ubuntu 24.04 ships
Name of the first branch	Always <code>master</code>	Still <code>master</code> when unconfigured, with a hint; <code>init.defaultBranch</code> and <code>git init -b</code> choose it	Setting since Git 2.28; <code>main</code> is planned as the default in Git 3.0, which has no release date	Set <code>init.defaultBranch</code> ; never hard-code a branch name in a script
<code>git switch</code> , <code>git restore</code>	Labelled experimental	Stable	Introduced in Git 2.23; label removed in 2.51	Use them; read <code>git checkout</code> in older scripts
Object IDs and ref storage	Not applicable	40-digit SHA-1 IDs and refs stored as files are the defaults	Git 3.0 plans SHA-256 and reftable as defaults for new repositories	Never assume the length of an ID, or that a ref is a file (Chapter 3)
<code>git datamodel</code> manual page	Did not exist	<code>git help datamodel</code>	Git 2.53	Read it after Chapter 2
Installed versus latest	Git 2.55.0; <code>gh</code> 2.88.1	Git 2.56.0 (28 September 2026); <code>gh</code> 2.102.0 (30 September 2026)	Not applicable	Git: optional. <code>gh</code> : upgrade before the GitHub chapters

All rows come from sections 1 and 4 of the research report, which links the release notes.

Outdated advice. Tutorials written before 2020 switch branches and discard changes with `git checkout` and assume a branch called `master`. Neither is wrong: `git checkout` remains supported, and unconfigured Git still creates `master`. Do not "correct" a script because it uses them. Do write new commands with `git switch` and `git restore`.

1.17 Practice

1. Lab 0.1: verify the toolchain and build the sandbox.
2. Lab 0.2: read every file in an empty `.git`, then find out which of them Git cannot live without.
3. Replay `labs/run ch01/first-repo`, then repeat section 1.9 by hand in `labs/shell m00`. Predict before you look: which of your object IDs will equal the book's, and which will not?
4. Replay `labs/run ch01/diagnosis` and inspect the sandbox it leaves in `$LAB/ch01/diagnosis/rag-eval`. Run the ritual yourself before reading section 1.12 again.
5. Then continue with Chapter 2, which explains the objects you saw appear under `.git/objects`.

1.18 Interview questions

1. A colleague says "the fix is committed". List the distinct places where that statement can be true or false, and the command that checks each.
2. Name three things that Git stores and GitHub only displays, and three things that exist only on GitHub. Why does the distinction matter during an incident?
3. `git status` reports that your branch is up to date with `origin/main`. What exactly did Git compare, and how old can that information be?
4. Why can `git log` work without a network connection while `git push` cannot? What does your answer imply about backups?
5. Between `git add` and `git commit`, which files under `.git` have changed, and what does each contain?

6. What makes a directory a Git repository? Which files in a fresh `.git` can you delete without breaking it?
7. Two engineers type the same Git command on two Macs and see different behavior. Name three causes outside the repository and the command that detects each.
8. A fix must reach a shared branch that already contains the faulty commit. Compare a new commit with amending and force-pushing. Which do you choose, and what would change your answer?
9. The same commands typed at two different times produce different commit IDs. Why? What did the lab environment pin to prevent it, and what did it not need to pin?
10. Which of the ten diagnosis commands can write to disk, and why does that matter when you investigate a damaged repository?
11. Git prints a hint that recommends `git rm --cached <path>` and then refuses that command. Explain how you would find out why.
12. Walk through the eleven steps of the root-cause framework for the symptom "my commit is not on GitHub".

1.19 Sources

Primary sources

- [git\(1\)](#): description, environment variables, `--no-optional-locks`.
- [gitrepository-layout](#): every file in `.git`.
- [gitglossary](#): bare repository, unborn branch, plumbing, porcelain.
- [gitdatamodel](#): the official data model page, on your machine as `git help datamodel`.
- [git-init](#), [git-status](#), [git-config](#), [git-rev-parse](#), [git-ls-files](#), [git-rm](#).
- Git source at the v2.55.0 tag: [builtin/rm.c](#), [submodule.c](#) and [builtin/fsck.c](#), read for the two root-cause notes in sections 1.7 and 1.13.
- Pro Git, second edition: [About Version Control](#), [A Short History of Git](#), [What is Git?](#). The book is hosted on the official site and has been frozen since May 2024, so its command style is older than this chapter's.
- GitHub: [changelog entry on CLI telemetry](#), [gh environment variables](#).

Secondary sources

- Phase 0 research report, sections 1, 2, 4 and 12: versions, dates, the two Gits, and the Git and GitHub layers, each with a link to its primary source.

Videos

The report assessed these from captions and chapter lists, not by full viewing.

- ["Learn Git & GitHub for Beginners \(2026\) Tutorial"](#), Coder Coder, 56 minutes, 28 June 2026. Current commands, including `git switch` and `git restore`. Beginner scope only.
- ["Git Tutorial for Beginners: Learn Git in One Video"](#), CodeWithHarry, Hindi, 2 hours 32 minutes, 12 July 2026. Current commands. It teaches no `reset`, `revert` or `reflog`, uses a local `master`, and first calls a branch a copy before correcting that to a pointer.
- ["Tech Talk: Linus Torvalds on git"](#), 2007, and ["Two decades of Git"](#), 2025. Design intent and history. Polemical; his 2025 remark that the SHA-256 work was needless churn is his opinion, not the project's position.

Further reading

- [gittutorial](#) and [giteveryday](#), also available as `git help tutorial` and `git help everyday`.
- [The Git User's Manual](#).

Chapter 2: The Mental Model

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch02/`; one continues a script in `labs/ch01/`.

2.1 Why this matters

An evaluation report from three weeks ago says that its numbers were produced "at commit `2e76f67`". Today the numbers are disputed, and your CTO asks:

1. What does that ID pin down: one file, one change, or the whole project?
2. Could anything behind that ID have been altered since then, by accident or on purpose?
3. The same fix is on `main` as `cf6a5b3` and on the release branch as `e460212`. Is that one fix or two different things?

No command answers these. A model does: what Git stores, how it names what it stores, and how a name such as `main` relates to it. With the model, each answer is one sentence. The ID pins down every byte of every tracked file, the complete history before it, and who recorded it when. Nothing behind the ID can change without the ID changing. The two commits are the same change recorded as two different snapshots, which is what a cherry-pick produces.

Sections 2.2 to 2.9 build the model. Section 2.10 reconciles it with the intuition that a commit is a diff, section 2.11 lists the wrong models behind most Git confusion, and section 2.12 puts GitHub on the map.

2.2 Start from what you have: the first commit of Chapter 1

In Chapter 1, section 1.9, five files appeared under `.git/objects` when you made the first commit of `rag-eval`. Each file is one *object*. `git cat-file -t` prints the type of an object and `git cat-file -p` prints its content in readable form. This transcript continues the Chapter 1 script (`labs/ch01/first-repo.sh`):

```
$ git cat-file -t f7c044e
commit
$ git cat-file -p f7c044e
tree e621cb090a7d826cce2a614885f534bde7015533
author Lab User <you@example.com> 1788756360 +0530
committer Lab User <you@example.com> 1788756360 +0530

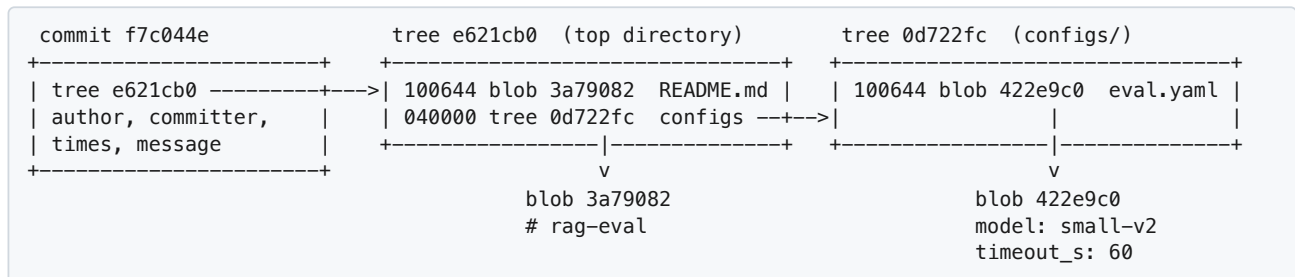
Add README and evaluation config
$ git cat-file -p 'f7c044e^{tree}'
100644 blob 3a79082bc80505c7543e79c4312e3e3d276a0e04    README.md
040000 tree 0d722fc2a87c4b61afef355250e6e77f41a44cd0    configs
$ git cat-file -p f7c044e:configs
100644 blob 422e9c0c59f64154ed466adcfb7b7b7c0d43e89f    eval.yaml
$ git cat-file -p f7c044e:configs/eval.yaml
model: small-v2
timeout_s: 60
```

- The commit `f7c044e` contains no file name and no file content: one `tree` line, author and committer with their times, the message.
- The tree `e621cb0` is the top directory at that moment. Each entry has a mode, the type and ID of an object, and a name. `README.md` is a *blob*, an object that holds file content; `configs` is another tree.
- The tree `0d722fc` is the directory `configs/`, with one blob for `eval.yaml`.

- The blob `422e9c0` is the content of that file and nothing else: no name, no date.

`<commit>^{tree}` means "the tree of this commit", and `<commit>:<path>` means "the object at this path in this commit" ([gitrevisions](#)).

Picture. Five objects, connected only by the IDs they contain.



Git's manual counts four kinds of data in a repository ([gitdatamodel](#)). The working tree is a fifth thing, and it is not repository data.

Kind of data	What it is	Default location	Read more
Objects	Immutable commits, trees, blobs and tag objects, named by their IDs	<code>.git/objects/</code>	2.3 to 2.7; Chapter 3: Git Internals
References (refs)	Names for object IDs: branches, tags, remote-tracking branches, HEAD	<code>.git/refs/</code> , <code>.git/HEAD</code>	2.8; Chapter 7: Branches
The index	The next commit, as a flat list of paths	<code>.git/index</code>	2.9; Chapter 5: Index
Reflogs	A local journal of the values each ref has had	<code>.git/logs/</code>	2.7; Chapter 13: Recovery
Working tree	The files you edit	The project directory	2.9; Chapter 4

2.3 Snapshots, not diffs

In one sentence. A commit records the complete state of every tracked file at one moment, as the ID of one tree; it does not record what changed.

Analogy. A photograph of the whole whiteboard after every meeting, instead of a list of what was wiped and written. Each photograph can be read on its own, and any two can be compared. The analogy breaks in one place: an album stores every photograph in full, while Git stores each distinct file content once and lets every snapshot that contains it refer to that object.

Precisely. The official data model lists what a commit must contain: the directory structure and file contents of that version, stored as the ID of the top-level tree; the IDs of its parent commits; an author and a committer, each with a time; and a message. It states that Git stores no diff for a commit and that `git show` calculates one from the parent when you ask ([gitdatamodel](#)). Full snapshots stay cheap because a file whose content did not change is not stored again: the new tree lists the ID of the blob that already exists.

See it. A scorer with three files is committed; then one file changes and is committed again (`labs/ch02/snapshots.sh`). `git ls-tree` prints the tree of a commit.

```

$ ls
config.yaml
judge_prompt.txt
metrics.py
$ git add .
$ git commit -m "Add scorer: metrics, judge prompt, config"
[main (root-commit) 916dec3] Add scorer: metrics, judge prompt, config
3 files changed, 5 insertions(+)

```

```

create mode 100644 config.yaml
create mode 100644 judge_prompt.txt
create mode 100644 metrics.py
$ git ls-tree HEAD
100644 blob e09e51ff381d1fdcf4ca91fce8852f43d3593e26    config.yaml
100644 blob f5970a3d9f36718a1c1e51b244e7d891dbf365d5    judge_prompt.txt
100644 blob 652e0e2f7afd2080aef8c6b86ce436380d11ecc4    metrics.py

```

```

$ printf 'model: small-v2\ntemperature: 0.2\n' > config.yaml
$ git commit -am "Raise judge temperature to 0.2"
[main 2e76f67] Raise judge temperature to 0.2
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git ls-tree HEAD~1
100644 blob e09e51ff381d1fdcf4ca91fce8852f43d3593e26    config.yaml
100644 blob f5970a3d9f36718a1c1e51b244e7d891dbf365d5    judge_prompt.txt
100644 blob 652e0e2f7afd2080aef8c6b86ce436380d11ecc4    metrics.py
$ git ls-tree HEAD
100644 blob f72ae7508cafa27bf9eb379d094c09b65825fe07    config.yaml
100644 blob f5970a3d9f36718a1c1e51b244e7d891dbf365d5    judge_prompt.txt
100644 blob 652e0e2f7afd2080aef8c6b86ce436380d11ecc4    metrics.py

```

Compare the two listings. `judge_prompt.txt` and `metrics.py` have the same blob IDs in both commits, `f5970a3` and `652e0e2`. Only `config.yaml` has a new one, `f72ae75` instead of `e09e51f`. The second snapshot is complete, and it needed one new blob. (`git commit -a` first stages every tracked file that was modified or deleted.)

```

$ git cat-file --batch-all-objects --batch-check
2e76f6786049b00c41eedcc3833d89ab882de618 commit 231
4798110f4ba90e7736ec51e1ad0ef9a5fc55912b tree 121
652e0e2f7afd2080aef8c6b86ce436380d11ecc4 blob 69
916dec36a1932b80b97f7a15d37d621c5830e6a7 commit 194
e09e51ff381d1fdcf4ca91fce8852f43d3593e26 blob 33
f3b0ea8770296178f91e353cda6f362acf83c1e7 tree 121
f5970a3d9f36718a1c1e51b244e7d891dbf365d5 blob 44
f72ae7508cafa27bf9eb379d094c09b65825fe07 blob 33

```

`git cat-file --batch-all-objects --batch-check` lists every object with its type and size in bytes: two commits, two trees and four blobs, where six blobs would be needed if nothing were shared.

```

$ git cat-file -p HEAD
tree 4798110f4ba90e7736ec51e1ad0ef9a5fc55912b
parent 916dec36a1932b80b97f7a15d37d621c5830e6a7
author Lab User <you@example.com> 1788755880 +0530
committer Lab User <you@example.com> 1788755880 +0530

Raise judge temperature to 0.2

```

The commit object holds a tree, a parent, two identities with times, and a message. It holds no diff. The diff that `git show` prints is computed when you ask for it:

```

$ git show HEAD
commit 2e76f6786049b00c41eedcc3833d89ab882de618
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:08:00 2026 +0530

    Raise judge temperature to 0.2

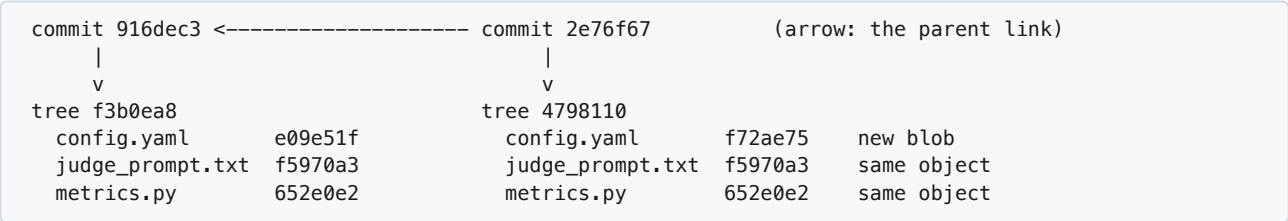
diff --git a/config.yaml b/config.yaml
index e09e51f..f72ae75 100644
--- a/config.yaml
+++ b/config.yaml
@@ -1,2 +1,2 @@
 model: small-v2
-temperature: 0.0
+temperature: 0.2

```

The line `index e09e51f..f72ae75` names the two blobs that Git compared.

Inside `.git`. The second commit added three object files: the new blob, the new top-level tree and the commit. In Chapter 1 the second commit added four, because the changed file was in a subdirectory, so the tree of `configs/` and the top-level tree both changed. `README.md` was not stored again. That was the question section 1.9 left open.

Picture.



In production. Any commit can be checked out, built or compared on its own; no chain of patches is replayed. A commit costs what changed, not what exists: in a repository of 40,000 files, an edit to two files in `services/ranker/` writes two blobs, three trees (`ranker/`, `services/`, the top directory) and one commit. The same rule is the weak point for large binary files: every version of a 2 GB model file is a new 2 GB blob (Chapter 22: Git LFS).

The model says "full snapshots". Git may later pack objects into packfiles and store some as differences against similar objects (Chapter 3). That is compression below the model: `git cat-file -p` returns the full content either way.

2.4 Content addressing

In one sentence. An object's ID is a hash of the object's type and content, so identical content has the identical ID in every repository on every machine, and different content has, for all practical purposes, a different ID.

Analogy. A warehouse that shelves each parcel at an address computed from its content. Two warehouses in different cities put identical parcels at identical addresses without talking to each other, and an altered parcel no longer belongs at its address. The analogy breaks because nobody finds a parcel by guessing its content: you need a catalogue from names to addresses, and in Git that catalogue is the trees and the refs.

Precisely. The manual defines the ID, also called the object name, as a cryptographic hash of the object's type and contents ([gitdatamodel](#)). With the default hash, SHA-1, an ID has 40 hexadecimal digits; commands accept and print unique prefixes such as `b5fee68`. Chapter 3 reproduces an ID with `shasum`. What is hashed decides what an ID identifies:

Object	What is hashed	What the ID identifies
Blob	The bytes of the file; not its name, its mode or any time	One exact file content, wherever it occurs
Tree	For each entry: mode, name, object ID	One exact directory state, including everything below it
Commit	Tree ID, parent IDs, author, committer, both times, message	One project state, the whole history behind it, and who recorded it when

Four properties follow. **Deduplication:** equal content is stored once, whatever its path, commit or branch.

Immutability: an object cannot be edited, because other content is another object with another ID, so "changing a commit" always means creating a new one (Chapter 9, Chapter 11). **Integrity:** Git can recompute the hash of whatever it reads, so corruption and tampering are detectable (`git fsck`, Chapter 3). **Cheap comparison:** two files, directories or project states are equal when their IDs are equal, and merge and diff skip everything whose IDs match (Chapter 8).

See it. `git hash-object` computes the ID that a file's content has as a blob. First outside any repository (`labs/ch02/content-ids.sh`):

```
# No repository here: hash-object only computes.
$ printf 'temperature: 0.2\n' | git hash-object --stdin
b5fee68e16090ca243c174fd28853ad0abe8b77a
$ printf 'temperature: 0.2\n' | git hash-object --stdin
b5fee68e16090ca243c174fd28853ad0abe8b77a
$ printf 'temperature: 0.3\n' | git hash-object --stdin
885f7cec8d3b132ef5144951a0fc37ee1cb80fba
```

The same bytes give the same ID twice; one changed character gives an unrelated ID. No repository exists here, so the ID cannot depend on one. Next, two repositories and two file names with the same bytes; with `-w`  the command also stores the object:

```
$ git init -q laptop
$ git init -q server
$ printf 'temperature: 0.2\n' > laptop/eval.yaml
$ printf 'temperature: 0.2\n' > server/settings.yaml
$ git -C laptop hash-object -w eval.yaml
b5fee68e16090ca243c174fd28853ad0abe8b77a
$ git -C server hash-object -w settings.yaml
b5fee68e16090ca243c174fd28853ad0abe8b77a
$ find laptop/.git/objects server/.git/objects -type f
laptop/.git/objects/b5/fee68e16090ca243c174fd28853ad0abe8b77a
server/.git/objects/b5/fee68e16090ca243c174fd28853ad0abe8b77a
```

Inside `.git`. Both repositories hold an object file at the same path, `objects/b5/fee68e...`: the first two digits of the ID name a directory, the other 38 the file. Neither file name is recorded in it.

Names are stored in trees, so they show up in tree IDs. `git write-tree`  writes the index as a tree and prints its ID (section 2.6):

```
$ git -C laptop add eval.yaml
$ git -C server add settings.yaml
$ git -C laptop write-tree
097da7ea458de812ac723fb7085d40be43fdae05
$ git -C server write-tree
8bc25e61fADF002956ecddf2f643b04dd386a8dc
$ mv server/settings.yaml server/eval.yaml
$ git -C server add -A
$ git -C server write-tree
097da7ea458de812ac723fb7085d40be43fdae05
```

With different file names the two trees differ. After the rename (`git add -A` stages the removal of the old name with the new file), both repositories contain the same directory state and the tree IDs are equal: `097da7e`.

Picture. Each ID is computed from the IDs below it, so one changed byte changes every ID above it.

```
blob ID   = hash(file content)
tree ID   = hash(names, modes, IDs of the blobs and trees in the directory)
commit ID = hash(tree ID, parent commit IDs, author, committer, times, message)

one byte changes in one file
-> new blob ID -> new ID for every tree above it -> new commit ID
-> new ID for every commit that is later built on that commit
```

In production. After a rollback, `git rev-parse 'HEAD^{tree}'` printing the tree ID of the last good commit proves that every tracked file is back in that state, with no file-by-file comparison (Lab 1.2). An ID in a report or an incident ticket means one exact state to everyone who has the repository; write all 40 digits, because an abbreviation is unique only in one repository at one time (`core.abbrev`). Files that look identical and have different IDs differ in bytes you cannot see, such as a carriage return (Lab 1.3). The same files committed one second later give the same

tree ID and a different commit ID, which is why commits you type by hand never have the book's IDs (Chapter 1, section 1.7).

2.5 The four object types

Type	Holds	Refers to	Written by
Blob	The bytes of one file	Nothing	<code>git add</code>
Tree	One directory: for each entry a mode, a name and an object ID	Blobs and other trees	<code>git commit</code> , from the index
Commit	One tree ID, the parent commit IDs, author, committer, message	Its tree and its parents	<code>git commit</code> , <code>git merge</code> , <code>git cherry-pick</code> and others
Tag object	The ID and type of another object, a tag name, the tagger, a message	Usually a commit	<code>git tag -a</code>

See it. A repository `ranker` with one commit and one annotated tag (`labs/ch02/object-types.sh`):

```
$ git cat-file --batch-all-objects --batch-check
08dd0d94a4c377361126a24ed13ad97267786d6f tree 35
0f7f193a9e915f3327b1349744e32c6b87e4f3a2 tag 146
572a34a9da9d2cc636c2b011e3956e1894059ee0 blob 51
895d61e52d8c1f55dfe0af9f20403cc940a44742 blob 9
95722c300fc5cfe6cc70305e21038a12ea6dfd61 tree 37
aa1a031ca30744c677a82eac91b9c4672542593f blob 30
b5688c9112aa76d0d5d5ff35e0bec6723ff89f02 tree 101
e375b7b7f8ad9877dd3bd0ff5272ef6323a9b55f commit 182
```

Eight objects: three blobs, three trees, one commit, one tag object.

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree b5688c9112aa76d0d5d5ff35e0bec6723ff89f02
author Lab User <you@example.com> 1788755580 +0530
committer Lab User <you@example.com> 1788755580 +0530

Add reranker with test script
```

This commit has no `parent` line, so it is a *root commit*. `1788755580 +0530` is a time in Git's raw format: seconds since 1 January 1970 UTC, then the offset from UTC.

```
$ git cat-file -p 'HEAD^{tree}'
100644 blob 895d61e52d8c1f55dfe0af9f20403cc940a44742 README.md
040000 tree 08dd0d94a4c377361126a24ed13ad97267786d6f scripts
040000 tree 95722c300fc5cfe6cc70305e21038a12ea6dfd61 src
$ git cat-file -p HEAD:scripts
100755 blob aa1a031ca30744c677a82eac91b9c4672542593f test.sh
$ git ls-tree -r HEAD
100644 blob 895d61e52d8c1f55dfe0af9f20403cc940a44742 README.md
100755 blob aa1a031ca30744c677a82eac91b9c4672542593f scripts/test.sh
100644 blob 572a34a9da9d2cc636c2b011e3956e1894059ee0 src/rerank.py
```

`git cat-file -p` prints one level of a tree. `git ls-tree -r` walks down and prints every file with its full path. The first column is the mode ([gitdatamodel](#)):

Mode	Kind of entry
100644	Regular file
100755	Executable file

Mode	Kind of entry
120000	Symbolic link
040000	Directory: the entry points at a tree
160000	Gitlink: a commit of another repository (Chapter 1, section 1.13; Chapter 23: Submodules)

`scripts/test.sh` has mode `100755`. Executable or not is the only permission that Git records; it stores no owner and no other permission bits.

```
$ git cat-file -t HEAD:src/rerank.py
blob
$ git cat-file -p HEAD:src/rerank.py
def rerank(docs):
    return sorted(docs, key=len)
```

A blob is content without a name. The name `rerank.py` is stored in the tree `95722c3`.

```
$ git cat-file -t v0.1.0
tag
$ git cat-file -p v0.1.0
object e375b7b7f8ad9877dd3bd0ff5272ef6323a9b55f
type commit
tag v0.1.0
tagger Lab User <you@example.com> 1788755640 +0530

First internal release
```

The tag object `0f7f193` names the commit `e375b7b` and its type, repeats the tag name, and adds a tagger and a message. The ref `v0.1.0` points at the tag object. A *lightweight* tag has no object: it is a ref that points at the commit directly (Chapter 14B).

Picture. All eight objects and the two names that lead to them.

```
refs/tags/v0.1.0 --> tag 0f7f193 --+
                               v
refs/heads/main -----> commit e375b7b --> tree b5688c9 --+--> blob 895d61e README.md
                                           +--> tree 08dd0d9 scripts --> blob
aa1a031 test.sh
                                           +--> tree 95722c3 src -----> blob
572a34a rerank.py
```

In production. Author and committer are text supplied by whoever creates the commit; Git does not check them. A signature makes authorship verifiable (Chapter 14B, Chapter 21B).

2.6 Building a commit by hand

`git add` and `git commit` are porcelain. Five plumbing commands do the same work in separate, visible steps.

Step	Plumbing command	Porcelain that does it for you
1	<code>git hash-object -w</code>  stores content as a blob	<code>git add</code>
2	<code>git update-index --add --cacheinfo <mode>,<id>,<path></code>  records in the index that this path has this content	<code>git add</code>
3	<code>git write-tree</code>  writes the index as tree objects	<code>git commit</code>
4	<code>git commit-tree <tree> [-p <parent>] -m <message></code>  writes a commit object	<code>git commit</code>
5	<code>git update-ref refs/heads/<branch> <commit></code>  points the branch at the commit	<code>git commit</code>

See it. A repository with no working-tree file at all (labs/ch02/commit-by-hand.sh); the content arrives through a pipe:

```
$ git init handmade
Initialized empty Git repository in $LAB/ch02/commit-by-hand/handmade/.git/
$ cd handmade
$ printf 'Answer only from the provided context.\n' | git hash-object -w --stdin
a7f6bbccac4db25937e64faa9292a454e408d871
$ find .git/objects -type f
.git/objects/a7/f6bbccac4db25937e64faa9292a454e408d871
$ git cat-file -t a7f6bbccac4db25937e64faa9292a454e408d871
blob
$ git cat-file -p a7f6bbccac4db25937e64faa9292a454e408d871
Answer only from the provided context.
```

One object exists. There is no index yet.

```
$ git update-index --add --cacheinfo
100644,a7f6bbccac4db25937e64faa9292a454e408d871,prompts/system.txt
$ git ls-files --stage
100644 a7f6bbccac4db25937e64faa9292a454e408d871 0      prompts/system.txt
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   prompts/system.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    prompts/system.txt
```

`--cacheinfo` takes a mode, a full object ID and a path; `--add` permits a path that the index does not have yet. `git status` reports the path twice, and both lines are correct: "new file" compares the index with HEAD, which has no commit, and "deleted" compares the working tree with the index, which lists a file that is not on disk (section 2.9).

```
$ git write-tree
5445e77320236121d609466c32a9b87e30cc3b93
$ git cat-file -p 5445e77320236121d609466c32a9b87e30cc3b93
040000 tree 4015b7b95e10b15e3f75a77197c1fa3e0999d344  prompts
$ git cat-file -p 4015b7b95e10b15e3f75a77197c1fa3e0999d344
100644 blob a7f6bbccac4db25937e64faa9292a454e408d871  system.txt
$ git ls-tree -r 5445e77320236121d609466c32a9b87e30cc3b93
100644 blob a7f6bbccac4db25937e64faa9292a454e408d871  prompts/system.txt
```

The index is a flat list of full paths; trees are nested, one object per directory. So `git write-tree` wrote two trees: `5445e77` for the top directory and `4015b7b` for `prompts/`.

```
$ git commit-tree 5445e77320236121d609466c32a9b87e30cc3b93 -m 'Add system prompt'
cc0ef1583aec692cf787228d912e5e33ff228a9a
$ git cat-file -p cc0ef1583aec692cf787228d912e5e33ff228a9a
tree 5445e77320236121d609466c32a9b87e30cc3b93
author Lab User <you@example.com> 1788756300 +0530
committer Lab User <you@example.com> 1788756300 +0530

Add system prompt
$ git log --oneline
fatal: your current branch 'main' does not have any commits yet
[exit status: 128]
$ git fsck
notice: No default references
dangling commit cc0ef1583aec692cf787228d912e5e33ff228a9a
```

`git commit-tree` wrote the commit `cc0ef15`, with the author and the committer taken from the environment, as `git commit` does. `git log` fails, because `HEAD` names the branch `main` and no ref `refs/heads/main` exists. `git fsck`, which checks the object database, calls the commit *dangling*: it exists, and nothing refers to it.

```
$ git update-ref refs/heads/main cc0ef1583aec692cf787228d912e5e33ff228a9a
$ git log --oneline
cc0ef15 Add system prompt
$ git fsck
$ git status --short
D prompts/system.txt
```

One ref makes the difference: `git log` works and `git fsck` is silent. `git status --short` prints `D prompts/system.txt`: the commit and the index contain the file, the working tree does not.

```
$ git restore prompts/system.txt
$ cat prompts/system.txt
Answer only from the provided context.
$ git status
On branch main
nothing to commit, working tree clean
```

`git restore <path>`  copies the file from the index into the working tree. Nothing can be lost here, because no file existed; on a file with uncommitted edits the same command overwrites them (Chapter 4, section 4.7).

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git hash-object -w</code>	unchanged	unchanged	unchanged	unchanged	One new blob	unchanged	unchanged
<code>git update-index --add --cacheinfo</code>	unchanged	One entry added or replaced	unchanged	unchanged	<code>.git/index</code> is written	unchanged	unchanged
<code>git write-tree</code>	unchanged	unchanged	unchanged	unchanged	One tree per directory, unless it exists already	unchanged	unchanged
<code>git commit-tree</code>	unchanged	unchanged	unchanged	unchanged	One commit object; no ref, no reflog line	unchanged	unchanged
<code>git update-ref refs/heads/main <id></code>	unchanged	unchanged	unchanged	Created, or moved to <code><id></code>	Reflog lines for the branch and for <code>HEAD</code> , without a message unless you pass <code>-m</code>	unchanged	unchanged

`git commit` does more than steps 3 to 5: it refuses to record nothing (Chapter 1, section 1.13), runs hooks, and writes a reason into the reflogs. The plumbing is still worth knowing. Every porcelain command is a combination of these moves, so when a command surprises you, ask which objects it wrote and which refs it moved. Recovery uses the same tools: a commit found by `git fsck` or in a reflog is reattached with one ref (Lab 1.1, Chapter 13).

2.7 The commit graph

In one sentence. Every commit names its parent commits, so the commits form a graph that is walked from child to parent, and "the history of X" means everything you can reach from X by following those links.

Analogy. A family tree in which each record lists the parents and none lists the children: from any person you can find all ancestors, but to find descendants you must start from someone younger. The analogy breaks on the counts: a commit has no parent (a root commit), one (an ordinary commit) or several (a merge).

Precisely. The commits form a directed acyclic graph, a DAG ([gitglossary](#)). It is directed because the link is stored in the child and points at the parent. It is acyclic because a commit's ID is computed from its parents' IDs: a parent must exist before its child, so no chain of links can return to its start. An object is *reachable* from another if a chain leads to it that follows tags to what they tag, commits to their parents and trees, and trees to their entries. Three things follow.

- A commit does not know its children, so "what came after this commit?" has an answer only relative to a starting point. History commands start from a ref.
- "The commits on a branch" is not a stored list but the set reachable from the commit that the branch names.
- "A is an ancestor of B" means that A is reachable from B. `git merge-base --is-ancestor A B` answers with its exit status.

See it. Five commits, one of them a merge (`labs/ch02/graph.sh`):

```
$ git log --graph --decorate --oneline --all
* 2511274 (HEAD -> main) Merge branch 'feature/dedupe'
|\
| * ea0d3fa (feature/dedupe) Add dedupe rules
* | 6d7e946 Add split step
|/
* 0fd50fb Add clean step
* dec99b5 Add load step
```

```
$ git log --format='%h parents: %p' main
2511274 parents: 6d7e946 ea0d3fa
6d7e946 parents: 0fd50fb
ea0d3fa parents: 0fd50fb
0fd50fb parents: dec99b5
dec99b5 parents:
$ git cat-file -p main
tree a9772be4c20443bc529f1b4fd1072be849341e67
parent 6d7e94608a939094e01cc57ba109f162c33cf7cd
parent ea0d3faa4a9e73d5df7cf0b7df41e67546ae932f
author Lab User <you@example.com> 1788755940 +0530
committer Lab User <you@example.com> 1788755940 +0530

Merge branch 'feature/dedupe'
```

`%p` prints the parents. The merge commit `2511274` has two `parent` lines: first the commit that `main` pointed at when the merge was made, `6d7e946`, then the tip of the merged branch, `ea0d3fa`. A merge commit is an ordinary commit with more than one parent, and its tree is the merged snapshot (Chapter 8).

```
$ git rev-list --count main
5
$ git rev-list --count feature/dedupe
3
$ git log --oneline feature/dedupe
ea0d3fa Add dedupe rules
0fd50fb Add clean step
dec99b5 Add load step
$ git merge-base --is-ancestor feature/dedupe main
[exit status: 0]
$ git merge-base --is-ancestor main feature/dedupe
[exit status: 1]
```

Five commits are reachable from `main`, and three from `feature/dedupe`: its own commit and the two it shares with `main`. The branch is an ancestor of `main` (status 0); `main` is not an ancestor of the branch (status 1).

```
$ git commit-tree 'main^{tree}' -p main -m 'Experiment that no ref points at'
2c65cfc7d0ba9f3dcd757786eba7734820034807
$ git cat-file -t 2c65cfc7d0ba9f3dcd757786eba7734820034807
commit
$ git rev-list --count --all
```

```
5
$ git fsck
dangling commit 2c65cfc7d0ba9f3dcd757786eba7734820034807
```

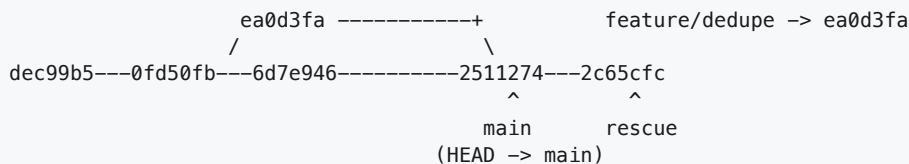
`git commit-tree` created a sixth commit whose parent is the tip of `main`. The object exists, but no ref leads to it: `git rev-list --count --all` still counts five, and `git fsck` reports a dangling commit.

```
$ git branch rescue 2c65cfc7d0ba9f3dcd757786eba7734820034807
$ git rev-list --count --all
6
$ git fsck
$ git log --graph --decorate --oneline --all
* 2c65cfc (rescue) Experiment that no ref points at
* 2511274 (HEAD -> main) Merge branch 'feature/dedupe'
| \
| * ea0d3fa (feature/dedupe) Add dedupe rules
* | 6d7e946 Add split step
| /
* 0fd50fb Add clean step
* dec99b5 Add load step
```

`git branch rescue <id>`  created a ref for it. Six commits are reachable, `git fsck` is silent, and the graph shows the commit. No object changed; a name was added.

Inside .git. The graph is not stored anywhere as a whole. Parent IDs are inside commit objects, and the starting points are refs.

Picture.



In production. "Is the fix in what we shipped?" is an ancestry question: `git merge-base --is-ancestor <fix> <released commit>`. Deleting a branch deletes a name, never a commit, and unreachable is not gone: `git gc` removes unreachable objects only after a grace period, two weeks by default (`gc.pruneExpire`). Reflog entries also count as starting points and are kept for 90 days by default, or 30 days once the commit is no longer reachable from the tip of its ref (`gc.reflogExpire`, `gc.reflogExpireUnreachable`; the lab configuration sets both to `never`). Chapter 13: Recovery is built on these numbers.

2.8 Refs and HEAD

In one sentence. A ref is a name for an object ID, a branch is a ref that moves forward when you commit, and HEAD records which branch, or which commit, you are on.

Analogy. Sticky labels on a wall chart of the commit graph. Moving a label changes nothing on the chart, and removing one removes no commit. HEAD is the "you are here" marker, normally stuck onto a label rather than onto the chart. The analogy breaks because labels are local: every clone has its own set and learns about another repository's labels only when it fetches (section 2.12).

Precisely. A ref refers either to an object ID or to another ref, in which case it is a *symbolic ref* ([gitdatamodel](#)). Its place in the hierarchy decides how Git treats it.

Ref	Refers to	Moves when
refs/heads/<name> , a branch	A commit	You commit, merge, reset or rebase while it is the current branch
refs/tags/<name> , a tag	Usually a commit or a tag object	Normally never
refs/remotes/<remote>/<name> , a remote-tracking branch	A commit	You fetch from that remote or push to it
HEAD	The current branch, as a symbolic ref; or one commit ID directly (<i>detached HEAD</i>)	You switch

A branch contains no commits: it names one, and its history is what is reachable from there. Creating a branch writes one ref and copies nothing. Git records no "parent branch", so merge and rebase must be told the other side (Chapter 9).

See it. One commit on `main` (`labs/ch02/refs-head.sh`):

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git rev-parse HEAD main
3046dc6147cc4993920eb20efeb20d44f2b73d77
3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git symbolic-ref HEAD
refs/heads/main
```

HEAD contains the name of a branch, and the branch file contains a commit ID. `git rev-parse` resolves both names to that ID; `git symbolic-ref HEAD` prints the branch that HEAD names.

```
$ git cat-file --batch-all-objects --batch-check
3046dc6147cc4993920eb20efeb20d44f2b73d77 commit 171
3a19f43c4181bfd7a3b52b74fbb1b1079e9c2bac tree 37
bfb2990471be330c8cdd6365bd10b4d3fac0ddbc blob 10
$ git branch feature/unicode
$ cat .git/refs/heads/feature/unicode
3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git cat-file --batch-all-objects --batch-check
3046dc6147cc4993920eb20efeb20d44f2b73d77 commit 171
3a19f43c4181bfd7a3b52b74fbb1b1079e9c2bac tree 37
bfb2990471be330c8cdd6365bd10b4d3fac0ddbc blob 10
$ git for-each-ref
3046dc6147cc4993920eb20efeb20d44f2b73d77 commit refs/heads/feature/unicode
3046dc6147cc4993920eb20efeb20d44f2b73d77 commit refs/heads/main
```

The object list is identical before and after `git branch feature/unicode` ; the command wrote one small file holding one ID. `git for-each-ref` lists every ref with the object it names.

```
$ git switch feature/unicode
Switched to branch 'feature/unicode'
$ cat .git/HEAD
ref: refs/heads/feature/unicode
$ printf 'lowercase\nnormalize NFC\n' > rules.txt
$ git commit -am "Normalize to NFC"
[feature/unicode 4a3fb80] Normalize to NFC
1 file changed, 1 insertion(+)
$ cat .git/HEAD
ref: refs/heads/feature/unicode
$ git for-each-ref
4a3fb8058851f5b863aea726837405da25a9a7e9 commit refs/heads/feature/unicode
3046dc6147cc4993920eb20efeb20d44f2b73d77 commit refs/heads/main
$ git log --graph --decorate --oneline --all
```

```
* 4a3fb80 (HEAD -> feature/unicode) Normalize to NFC
* 3046dc6 (main) Add lowercase rule
```

`git switch`  rewrote HEAD to name the other branch. The commit then moved `feature/unicode` to `4a3fb80`; `main` stayed at `3046dc6`, and HEAD reads the same before and after. A commit moves the current branch, and HEAD follows because it names that branch.

```
# Older scripts switch branches with git checkout. Same effect.
$ git checkout main
Switched to branch 'main'
$ cat .git/HEAD
ref: refs/heads/main
```

```
# The second form of HEAD: a commit ID instead of a branch name.
$ git switch --detach feature/unicode
HEAD is now at 4a3fb80 Normalize to NFC
$ cat .git/HEAD
4a3fb8058851f5b863aea726837405da25a9a7e9
$ git symbolic-ref HEAD
fatal: ref HEAD is not a symbolic ref
[exit status: 128]
$ git switch main
Previous HEAD position was 4a3fb80 Normalize to NFC
Switched to branch 'main'
$ cat .git/HEAD
ref: refs/heads/main
```

`git switch --detach` wrote a commit ID into HEAD. No branch is current, so a new commit would move no branch (Chapter 7: Branches). Switching back to `main` restores the symbolic form.

Inside `.git`. Here each ref is a file that `cat` can read. That is the default "files" format, not a guarantee: refs can be packed into one file, and the refable format stores them in binary (Chapter 3). `git rev-parse`, `git symbolic-ref` and `git for-each-ref` work with every format.

Picture.

before the commit	after the commit
<pre>3046dc6 ^ +-- main +-- feature/unicode <-- HEAD</pre>	<pre>3046dc6 <----- 4a3fb80 ^ ^ main feature/unicode <-- HEAD</pre>

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git branch <name></code>	unchanged	unchanged	unchanged	unchanged	New ref <code>refs/heads/<name></code> at the current commit, with its own reflog	unchanged	unchanged
<code>git switch <branch></code> , <code>git switch --detach <commit></code>	Files that differ between the two commits are updated; refused if that would overwrite uncommitted changes	Updated in the same way	Rewritten: the branch name, or the commit ID	unchanged; another branch, or none, is now current	A reflog line for HEAD	unchanged	unchanged

In production. "Which commit is deployed?" must be answered with a commit ID. `main` is a moving name: it meant another commit yesterday and may mean a different one in a colleague's clone today. A tag is a name that is

not supposed to move; whether it can is decided by the rules on the server (Chapter 18).

2.9 The three trees: a preview

In one sentence. A file can exist in three states at the same time, in the commit that HEAD names, in the index and in the working tree, and `git status` reports the differences between them.

Precisely. "Tree" here means a complete set of files, not a tree object. The commit that HEAD names is the last snapshot. The index is the proposed next snapshot: every tracked path with the ID of a blob. The working tree is the files on disk. `git add` copies content from the working tree into the index, and `git commit` turns the index into tree objects and a commit.

See it. One file is committed, then changed and staged, then changed again (`labs/ch02/three-trees.sh`):

```
$ printf 'lowercase\nstrip accents\n' > rules.txt
$ git add rules.txt
$ printf 'lowercase\nstrip accents\ncollapse whitespace\n' > rules.txt
$ git show HEAD:rules.txt
lowercase
$ git show :rules.txt
lowercase
strip accents
$ cat rules.txt
lowercase
strip accents
collapse whitespace
```

`git show HEAD:rules.txt` prints the committed version, and `git show :rules.txt` prints the version in the index. Three versions exist at once.

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   rules.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   rules.txt
```

The file is listed twice, once for each comparison.

```
$ git diff --cached
diff --git a/rules.txt b/rules.txt
index bfb2990..298e6b4 100644
--- a/rules.txt
+++ b/rules.txt
@@ -1 +1,2 @@
 lowercase
+strip accents
$ git diff
diff --git a/rules.txt b/rules.txt
index 298e6b4..969b4fa 100644
--- a/rules.txt
+++ b/rules.txt
@@ -1,2 +1,3 @@
 lowercase
 strip accents
+collapse whitespace
```

`git diff --cached` compares HEAD with the index, and `git diff` compares the index with the working tree. Neither shows the whole distance from HEAD to the working tree; `git diff HEAD` does.

Picture.



A commit made now would record the middle version: the index, not the file on disk. That is the mechanism behind the worked example in Chapter 1, section 1.12. Chapter 4 and Chapter 5: Index take the three trees apart.

2.10 A commit is a snapshot, and it can be read as a change

"A commit is a diff" is wrong as a statement about storage and right as a way to read history. You need both views, and you need to know which one a command uses.

View	Definition	Used by
Snapshot	The tree that the commit records	Checkout, builds, <code>git ls-tree</code> , <code>git diff A B</code> , merge (three snapshots)
Change	The difference between the commit's tree and its parent's, computed on demand	<code>git show</code> , <code>git log -p</code> , cherry-pick, rebase, revert

See it. `main` is two commits ahead of `release/1.0` (`labs/ch02/two-views.sh`):

```
$ git log --graph --decorate --oneline --all
* cf6a5b3 (HEAD -> main) Retry failed calls three times
* 92b1ad4 Run four workers
* 68dcb3b (release/1.0) Add client and server settings
$ git show --stat --format="%h %s" main
cf6a5b3 Retry failed calls three times

client.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

`git cherry-pick` 🟡 copies the last commit onto the release branch:

```
$ git switch release/1.0
Switched to branch 'release/1.0'
$ git cherry-pick main
[release/1.0 e460212] Retry failed calls three times
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --graph --decorate --oneline --all
* e460212 (HEAD -> release/1.0) Retry failed calls three times
| * cf6a5b3 (main) Retry failed calls three times
| * 92b1ad4 Run four workers
|/
* 68dcb3b Add client and server settings
```

The result is a new commit, `e460212`, with the same message. The `Date:` line is the author date, which a cherry-pick takes over from the original (Chapter 10, section 10.3).

```
$ git ls-tree main
100644 blob efc3dd6049f31961d7de37bf366cbe5552aee146    client.yaml
100644 blob d333cab4da5c3a4c1f455a8626c14aa1a1647aca    server.yaml
$ git ls-tree release/1.0
100644 blob efc3dd6049f31961d7de37bf366cbe5552aee146    client.yaml
100644 blob c44fe52484fbfa487bfb4b29c7b91e44fd76022e    server.yaml
```

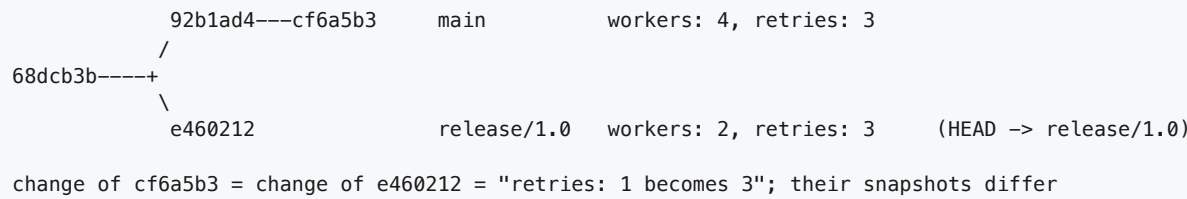
The snapshots differ. Both contain the new `client.yaml`, blob `efc3dd6`. But `server.yaml` is `d333cab` on `main` and `c44fe52` on the `release` branch, which never received the commit "Run four workers". Had cherry-pick copied the snapshot, the `release` would now run four workers.

```
$ git show --stat --format="%h %s" release/1.0
e460212 Retry failed calls three times

client.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git show main | git patch-id --stable
22602bf345cdd3c410ce2340ba3477e68e08f9a2 cf6a5b3ed8609aa40fd89ecc364a90e59d07d6fb
$ git show release/1.0 | git patch-id --stable
22602bf345cdd3c410ce2340ba3477e68e08f9a2 e4602121961240cdf7e21b59834c799639f5c253
```

The change is the same. `git patch-id` reduces a diff to an ID that ignores line numbers and whitespace, and both commits give `22602bf...`. Same change, different snapshot, different parent, different commit ID: that answers the third question of section 2.1.

Picture.



Precisely. The manual describes cherry-pick in the change view: it applies the change that a commit introduces and records a new commit ([git-cherry-pick](#)). The implementation is a three-way merge whose base is the parent of the picked commit, so it works on three snapshots ([how cherry-pick and revert work](#)). `Rebase` repeats this for a series of commits (Chapter 9); `revert` applies the inverse change (Chapter 11). The result is always a new commit with a new ID.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git cherry-pick <commit></code>	Updated to the new commit	Updated to the new commit	unchanged	Moves to the new commit	New commit, and new trees and blobs where content is new; reflog lines	unchanged	unchanged

In production. In a poll of 2,466 mostly professional developers, 50% think of a commit as a diff and 42% as a snapshot ([poll results](#); a self-selected sample). Each group holds half of the model. The snapshot view says what a build contains and why two IDs differ; the change view says what a cherry-pick or a rebase will carry over and what it will leave behind.

2.11 Common wrong mental models

The rows come from section 12 of the research report, which has the complete table. "Proof" says where this book demonstrates the reality.

Wrong model	Verified reality	Layer	Proof	Source
A commit stores a diff	A commit records a full snapshot, the ID of the top-level tree, with its parents, author, committer and message. Diffs are computed on demand	Git	2.3	gitdatamodel
Cherry-pick and revert apply patches	Cherry-pick is a three-way merge whose base is the picked commit's parent; revert swaps the base and "theirs"	Git	2.10, Chapter 10	jvns.ca
A branch is a copy of the code, with a parent branch	A branch is a ref that names one commit. Git has no notion of a parent branch	Git	2.8	gitdatamodel , jvns.ca
HEAD is the latest commit	HEAD is a symbolic ref to the current branch, or a direct reference to a commit, in which case no branch is current	Git	2.8	gitdatamodel
<code>origin/main</code> is the branch on the server	A remote-tracking branch records the last-known state of the remote branch, as of the last fetch	Git	2.12, Chapter 12	gitdatamodel , jvns.ca
The staging area is a list of files to commit	The index is a flat list of entries, each with a file type, a blob ID, a stage number and a path; it describes the whole next snapshot	Git	2.6, 2.9	gitdatamodel , gitformat-index
Amend and rebase edit commits	Objects never change. Amend and rebase create new commits with new IDs; the old ones remain until their reflog entries expire	Git	2.4, Chapters 9 and 11	gitdatamodel , gc configuration
The reflog will always save me, even for a deleted branch or on the server	Reflogs are local and are not shared with remotes. Deleting a branch deletes its reflog. Bare repositories keep none by default	Git	2.12, Chapter 13	gitdatamodel , git-branch
Git tracks renames and directories	Git does not record renames, and it tracks file content, not directories	Git	2.5, Chapter 4	Git User's Manual
A pull request is a Git feature	It is a GitHub object, backed by hidden refs, a test merge commit and a three-dot diff	GitHub	2.12, Chapter 17	pull requests reference
Deleting the file or force-pushing removes a leaked secret	The commits stay accessible in clones and forks, by ID in cached views, and through pull-request refs; rotation is the remedy	GitHub	2.7, Chapter 21B	removing sensitive data

These are not beginner errors: in the same polls, only 10% of respondents were fully confident that they understood HEAD.

2.12 The mental map of Git versus GitHub

Chapter 1, section 1.5 sorted things into layers. This chapter's model adds the rule for what can cross between repositories at all: **a push or a fetch transfers objects and updates refs; nothing else in `.git` leaves your machine.**

See it. A bare repository on disk plays the Git layer of the server (`labs/ch02/what-travels.sh`). Your clone has one commit, a second branch `wip/unicode`, a staged change, and an alias `st` in its `.git/config`:

```
# Your clone: one commit, a second branch, one staged change, one alias in .git/config.
$ git st
## main
M rules.txt
$ git push -u origin main
To ../hub.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
$ git for-each-ref
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/heads/main
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/heads/wip/unicode
```

```
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/remotes/origin/main
$ git cat-file --batch-all-objects --batch-check
298e6b4d3fbad90aacecaf555620943c308cd90a blob 24
3a19f43c4181bfd7a3b52b74fbb1b1079e9c2bac tree 37
bfb2990471be330c8cdd6365bd10b4d3fac0ddbc blob 10
c0c5420b9f771c668129cf6f5a885ab951d15263 commit 171
```

Four objects exist locally: the commit, its tree, the committed blob and the blob of the staged change. `git push` published `main` (its state table is in Chapter 1, section 1.12).

```
$ cd ../hub.git
$ find . -type f -not -path './hooks/*' | sort
./config
./description
./HEAD
./info/exclude
./objects/3a/19f43c4181bfd7a3b52b74fbb1b1079e9c2bac
./objects/bf/b2990471be330c8cdd6365bd10b4d3fac0ddbc
./objects/c0/c5420b9f771c668129cf6f5a885ab951d15263
./refs/heads/main
$ git for-each-ref
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/heads/main
$ git cat-file --batch-all-objects --batch-check
3a19f43c4181bfd7a3b52b74fbb1b1079e9c2bac tree 37
bfb2990471be330c8cdd6365bd10b4d3fac0ddbc blob 10
c0c5420b9f771c668129cf6f5a885ab951d15263 commit 171
$ git status
fatal: this operation must be run in a work tree
[exit status: 128]
```

The server received three objects and one ref. The staged blob `298e6b4` is absent, because no pushed commit refers to it. The branch `wip/unicode` is absent, because it was not pushed. There is no index, no `logs/` directory and no working tree, so `git status` cannot run.

```
$ cd ..
$ git clone -q hub.git colleague
$ cd colleague
$ git for-each-ref
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/heads/main
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/remotes/origin/HEAD
c0c5420b9f771c668129cf6f5a885ab951d15263 commit refs/remotes/origin/main
$ git reflog
c0c5420 HEAD@{0}: clone: from $LAB/ch02/what-travels/hub.git
$ git st
git: 'st' is not a git command. See 'git --help'.
```

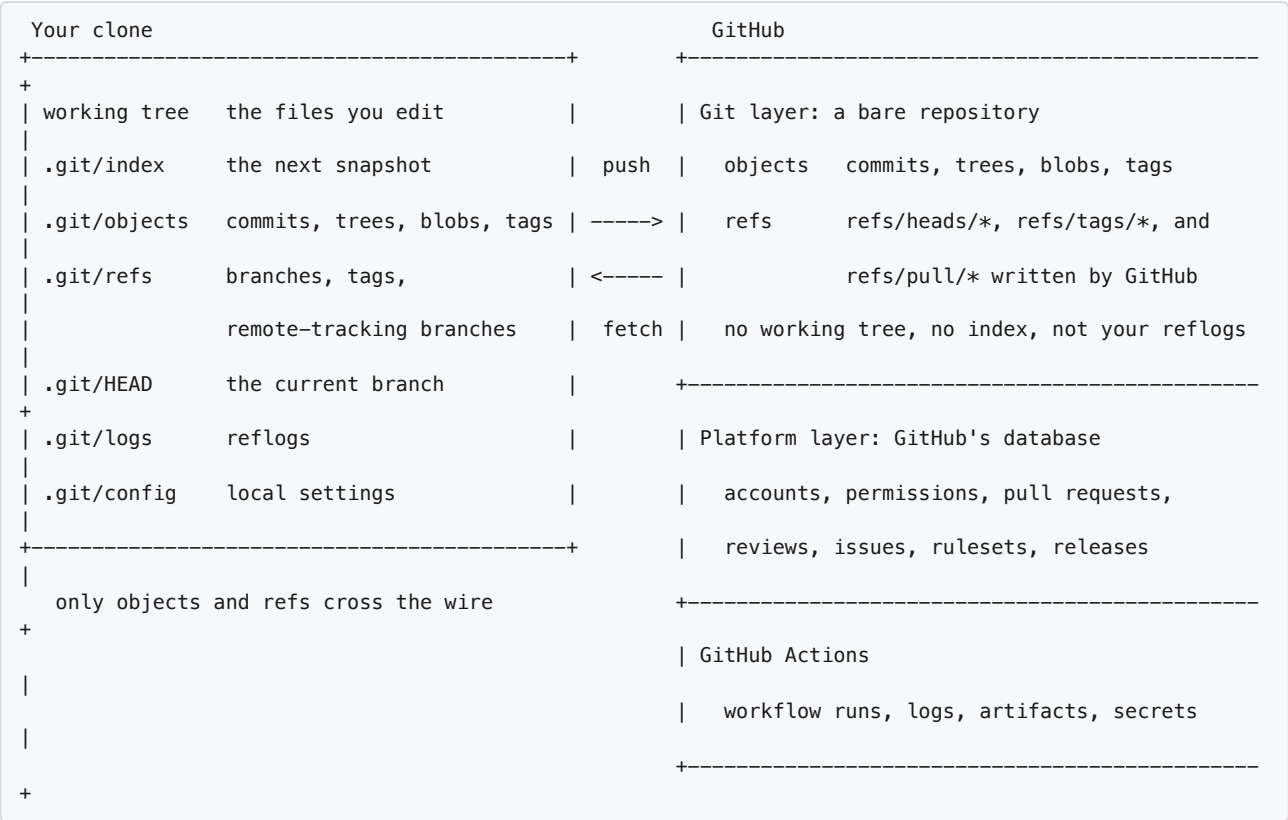
The most similar commands are

```
status
reset
stage
stash
[exit status: 1]
```

A colleague's clone has the server's branch as the remote-tracking branch `refs/remotes/origin/main`, and Git created a local `main` from it. The reflog begins with the clone, and the alias is unknown. Your reflogs and your `.git/config` stayed with you.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git clone <url></code> 	Created from the default branch	Created	Created; names the default branch	Created from the remote-tracking branch	All reachable objects, <code>refs/remotes/origin/*</code> , a new reflog, a <code>.git/config</code> that names the remote	unchanged	unchanged

Picture.



GitHub, not Git. The two lower boxes exist only in GitHub's systems. A pull request, for example, is a database record plus read-only refs under `refs/pull/` that GitHub maintains on the server, as described in GitHub's documentation ([pull requests reference](#), [removing sensitive data](#)). A clone copies none of it, and deleting your clone loses none of it.

In production. "It is on GitHub" translates to: which objects were pushed, and which ref on the server points at them? "GitHub shows X" raises the question whether X is Git data that any clone would show, or a platform record. Chapter 12 develops the arrows of the picture; Chapters 15 to 21 the right-hand boxes.

2.13 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A commit made with <code>git commit-tree</code> is not in <code>git log</code>	<code>git fsck</code> reports a dangling commit	<code>git update-ref refs/heads/<branch> <id></code> , or <code>git branch <name> <id></code>	In scripts, create the commit and move the ref together (Lab 1.1)
After <code>git update-ref</code> moved the current branch, <code>git status</code> shows changes that nobody made	The ref moved; the index and the working tree did not	Decide which state you want; <code>git restore --staged --worktree <paths></code>  takes them from the new HEAD	Move the current branch with porcelain, which updates all three (Chapter 11)
Git warns that a name such as <code>main</code> is ambiguous	A stray ref outside <code>refs/</code> (below)	Delete the stray file	Give <code>git update-ref</code> full names that begin with <code>refs/</code>
Two files look identical and have different IDs	<code>cmp</code> , <code>od -c</code> and <code>git hash-object</code> on both	Remove the invisible bytes (Lab 1.3)	Normalize line endings by rule (Chapter 4, section 4.13)

Symptom	Diagnosis	Fix	Prevention
fatal: bad object, or git fsck reports a missing object	git fsck names the type and the ID	Write the same content back, or copy the object from another clone (Lab 1.2, Chapter 13)	Never edit .git/objects; keep more than one clone

git update-ref **takes the name literally**. The intent below is to move the branch `main`. The name given is `main` instead of `refs/heads/main` (`labs/ch02/update-ref-trap.sh`):

```
# Intent: point the branch main at the first commit. Mistake: the short name.
$ git log --oneline
cfa6dbc Normalize to NFC
3046dc6 Add lowercase rule
$ git update-ref main 3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git branch -vv
* main cfa6dbc Normalize to NFC
$ git rev-parse main
warning: refname 'main' is ambiguous.
3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git rev-parse refs/heads/main
cfa6dbcdb38cee2b5a32420a51a8dea71fab1dc1
```

The command printed nothing, and the branch did not move: `git branch -vv` still shows `cfa6dbc`. But the name `main` now resolves to the old commit, with a warning.

```
$ git for-each-ref
cfa6dbcdb38cee2b5a32420a51a8dea71fab1dc1 commit refs/heads/main
$ find .git -maxdepth 1 -type f | sort
.git/COMMIT_EDITMSG
.git/config
.git/description
.git/HEAD
.git/index
.git/main
$ cat .git/main
3046dc6147cc4993920eb20efeb20d44f2b73d77
$ git update-ref -d main
error: refusing to update ref with bad name 'main'
[exit status: 1]
$ rm .git/main
$ git rev-parse main
cfa6dbcdb38cee2b5a32420a51a8dea71fab1dc1
```

```
Observed behavior : git update-ref main <id> prints nothing. Afterwards "main" is ambiguous and
                    resolves to <id>, while the branch main still points at its own commit.
Git state          : a file .git/main holds <id>. refs/heads/main is unchanged. git for-each-ref,
                    which lists refs/, does not show the stray ref.
Mechanism          : update-ref writes exactly the ref it is given. For an update it checks only
                    that the name is well-formed. Name lookup tries <name> at the top of .git
                    before refs/heads/<name>. For a deletion Git accepts only names under refs/
                    and names in capitals such as ORIG_HEAD, so "git update-ref -d main" is refused.
Root cause         : a plumbing command received a short name where it needs the full ref name.
Why Git does this  : plumbing serves scripts. It does what it is told and guesses nothing.
Correct fix        : confirm that .git/main is the stray ref, then delete that file.
Prevention         : write refs/heads/<name> for update-ref; move branches with porcelain.
```

The lookup order is in [gitrevisions](#); the two name checks were read from the Git 2.55.0 source ([refs.c](#)). The fix applies to the files format, where a ref is a file.

2.14 When not to use it, and dangerous edge cases

When not to use plumbing. For daily work: `git add`, `git commit`, `git switch` and `git branch` make the same objects and refs and add checks. The read-only plumbing (`git cat-file`, `git ls-tree`, `git rev-parse`, `git`

`for-each-ref`, `git merge-base`) is always appropriate and settles arguments about state fastest.

Dangerous edge cases.

- `git update-ref` verifies little. It refuses a blob as the value of a branch and a stale expected value (Lab 1.1), but it accepts any existing commit and never looks at the index or the working tree. Porcelain refuses more: `git branch -f` will not move the checked-out branch.
- `git update-ref -d <ref>` . **What it changes:** it deletes the ref and its reflog, even for the current branch. **What it can destroy:** the only name that leads to those commits. **Preview:** `git rev-parse <ref>`; write the ID down. **Recovery:** `git update-ref <ref> <id>` with that ID; without it, `git fsck --no-reflogs` lists the commits as dangling while they exist. **When appropriate:** scripts that remove refs they created themselves.
- Deleting or editing files under `.git/objects` . One object can be part of every snapshot in the history (Lab 1.2).
- `git add` and `git hash-object -w` store content at once. A secret that was staged for a second is an object in `.git/objects` until garbage collection removes it, even if it was never committed (Chapter 21B).
- An abbreviated ID in a script or a ticket can become ambiguous as the repository grows; store full IDs.

2.15 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git cat-file</code> , <code>git ls-tree</code> , <code>git rev-parse</code> , <code>git for-each-ref</code> , <code>git rev-list</code> , <code>git merge-base</code> , <code>git fsck</code> , <code>git patch-id</code> , <code>git hash-object</code> without <code>-w</code>		Nothing	not needed	not needed
<code>git hash-object -w</code> , <code>git write-tree</code> , <code>git commit-tree</code>		Add objects; no ref and no index entry changes	not needed	Garbage collection removes objects that nothing refers to
<code>git update-index --add --cacheinfo</code>		One index entry	<code>git ls-files --stage</code>	<code>git restore --staged <path></code>
<code>git update-ref <ref> <new> [<old>]</code>		One ref, and nothing else	<code>git rev-parse <ref></code> ; pass <code><old></code> so that a surprise stops the command	<code>git update-ref <ref> <previous ID></code> , taken from the reflog
<code>git update-ref -d <ref></code>		Deletes the ref and its reflog	<code>git rev-parse <ref></code>	Recreate the ref from the ID; <code>git fsck --no-reflogs</code> if you did not keep it
<code>git branch <name> [<commit>]</code>		Adds one ref	<code>git branch -vv</code>	<code>git branch -d <name></code>
<code>git switch <branch></code> , <code>git checkout <branch></code> , <code>git switch -detach <commit></code>		HEAD, index and working tree; refuses to overwrite uncommitted changes (exception: Chapter 4, section 4.17)	<code>git status</code>	<code>git switch -</code>
<code>git cherry-pick <commit></code>		Adds one commit to the current branch, which moves it, and updates index and working tree	<code>git show <commit></code>	Chapter 10
<code>git restore <path></code>		Overwrites the file from the index	<code>git diff --<path></code>	None for content that was never staged

Command	Label	What it changes	Preview	Recovery
<code>git push</code> , <code>git clone</code>	● , ●	Chapter 1, section 1.15 and Chapter 12	<code>git push --dry-run</code>	Chapter 12

2.16 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
Object IDs	SHA-1, 40 digits	Still the default. SHA-256 repositories, with 64 digits, are supported and cannot exchange data with SHA-1 repositories	Supported since Git 2.42; planned default for new repositories in Git 3.0	Never assume 40 digits in a script or a regular expression
Ref storage	One file per ref, plus <code>packed-refs</code>	Still the default; reftable is optional	Git 2.45; planned default for new repositories in Git 3.0	Read refs with <code>git rev-parse</code> , <code>git for-each-ref</code> and <code>git symbolic-ref</code>
<code>git datamodel</code>	Not available	<code>git help datamodel</code>	Git 2.53	Read it next
<code>git switch</code>	<code>git checkout</code> for switching	A stable command; <code>git checkout</code> remains supported	Git 2.23; no longer experimental since 2.51	Use <code>git switch</code> ; read <code>git checkout</code> in older scripts

All rows come from sections 1 and 4 of the research report, which links the release notes.

Outdated advice. Tutorials that read `.git/refs/heads/master` to find "the latest commit" assume the files format and an old default branch name. `git rev-parse <branch>` works in every repository.

2.17 Practice

1. Lab 1.1: build two commits by hand, lose a third on purpose and get it back.
2. Lab 1.2: prove that snapshots share unchanged blobs and trees, then delete a shared object and rebuild it.
3. Lab 1.3: the same content in two repositories, down to an identical commit ID, and one invisible byte that breaks it.
4. Replay `labs/run ch02/graph` and `labs/run ch02/what-travels`, then inspect the sandboxes under `$LAB/ch02/` with `git cat-file -p` and `git for-each-ref`.
5. Read `git help datamodel` from top to bottom. After this chapter every sentence in it should be familiar.

2.18 Interview questions

1. A report quotes a commit ID. What does that ID guarantee about file contents, history and authorship, and what does it not guarantee?
2. A commit changes one file out of 5,000, three directories deep. How many objects does it create, and of which types?
3. Why can a commit object not be edited? What do `git commit --amend` and `git rebase` do instead?
4. Two engineers commit identical content with identical messages on two laptops. Which IDs are equal, which differ, and why?
5. Define "reachable". Why is a commit that no ref can reach still in the repository, and for how long by default?
6. What is a branch, what does creating one cost, and what does Git know about the branch it was created from?
7. Describe the two forms of HEAD and how you would tell them apart on a machine you have never seen.

8. `git status` lists one file under "Changes to be committed" and under "Changes not staged for commit". Explain the state of the three trees.
9. A cherry-picked commit has a different ID from the original. Is it the same commit, the same change, or the same snapshot, and how would you prove it?
10. Which parts of `.git` reach the server when you push, and which never do? What does GitHub hold that no clone contains?

2.19 Sources

Primary sources

- [gitdatamodel](#): objects, refs, the index and reflogs; on your machine as `git help datamodel`.
- [gitglossary](#): DAG, reachable, dangling object, HEAD, ref, symref.
- [gitrevisions](#): `<rev>^{tree}`, `<rev>:<path>`, `:<path>`, and the lookup order of a short ref name.
- Manual pages: [git-hash-object](#), [git-update-index](#), [git-write-tree](#), [git-commit-tree](#), [git-update-ref](#), [git-cat-file](#), [git-ls-tree](#), [git-fsck](#), [git-patch-id](#), [git-cherry-pick](#).
- Git source at the v2.55.0 tag: [refs.c](#), read for the root-cause box in section 2.13.
- Pro Git, second edition, chapter 10: [Git Objects](#), [Git References](#). Frozen since May 2024; the object model is current, the commands use `master` and `git checkout`.
- GitHub Docs: [pull requests reference](#), [removing sensitive data from a repository](#).

Secondary sources

- Phase 0 research report: section 12 for the misconceptions, sections 1 and 4 for versions.
- Julia Evans: [how cherry-pick and revert work](#), [branches: intuition and reality](#), [confusing Git terminology](#), [poll results](#). The polls are self-selected samples: direction, not population figures.

Videos

The report assessed these from caption searches, chapter lists and descriptions, not by full viewing.

- ["Lecture 5: Version Control and Git"](#), MIT Missing Semester 2026, 1 hour 10 minutes, 19 February 2026, with [notes and exercises](#). The data model before the commands. SHA-1 only; the demo starts on `master`; nothing on GitHub.
- ["Git Internals by John Britton of GitHub - CS50 Tech Talk"](#), 58 minutes, 11 April 2018. Blobs, trees, commits, hashing, refs as files. Correct and clear; `master`, `git checkout`, SHA-1 only.
- ["Complete git and Github course in Hindi"](#), Chai aur Code, Hindi, 2 hours 55 minutes, 8 June 2024, from 55:16. Commit, tree and blob by name; HEAD as a pointer to the current branch. Local `master`; no revert, restore or cherry-pick.

Further reading

- [gitcore-tutorial](#), the Git project's own walk through the plumbing, and [The Git User's Manual](#).

Chapter 3: Git Internals

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch03/`.

3.1 Why this matters

Four questions a CTO can ask on an ordinary day:

1. "A laptop lost power during a commit, and Git now says `fatal: index file corrupt`. What did we lose: the history, the work in progress, or nothing?"
2. "The release script stamps builds by reading `.git/refs/heads/main`. On the new build image that file does not exist. Is the repository damaged?"
3. "`git fsck` reports a dangling commit on the build server. Is data at risk? Should somebody run a cleanup?"
4. "We keep four hundred versions of a 16 MB evaluation set in one repository. Are we storing 6 GB?"

None of these can be answered with the commands of the other chapters alone. They are questions about how Git stores things. Chapter 2 gave you the model: four object types, names that point at them, a staging area. This chapter opens each box: you will recompute an object ID with `shasum`, read a tree, a commit and a tag in raw form, watch loose objects become a pack with a delta chain, list what is reachable and what is not, read one ref in three storage forms, and decode the header of the index with `xxd`.

The answers, in order. The index is rebuilt with one command, and only content that was staged and never committed needs a second look (section 3.12). The ref is in `packed-refs` or in a refable, and the script should have asked Git (sections 3.9 and 3.13). "Dangling" describes where `git fsck` started looking, not what will be deleted, and the cleanup is the risky part (section 3.8). And probably not: a pack stores versions that differ in a few lines as deltas while every snapshot stays complete (section 3.7); files that change in every byte are another matter (Chapter 22: Git LFS).

Two habits come out of this chapter. Read repository state through plumbing (`git rev-parse`, `git for-each-ref`, `git cat-file`, `git ls-files`), because the files underneath exist in more than one format. And before you delete or repair anything under `.git`, decide whether it is primary or derived data: an index or a pack index can be rebuilt, an object or a reflog cannot.

3.2 The `.git` directory, file by file

In one sentence. The `.git` directory is the repository: an object database, names for objects, a log of how those names moved, one staging file, and settings; the files next to it are a working tree that Git can rebuild from it.

Analogy. A records office. The vault holds sealed documents filed under a number computed from their content (objects). A card index says which document is current for each project (refs). A visitors' book records every change to a card (reflogs). One tray on the clerk's desk holds the draft of the next filing (the index). The analogy breaks in one place that matters: a sealed document can never be amended, not even by the office. Every "change" files a new document and moves a card.

Precisely. A repository is either a `.git` directory at the top of a working tree or a bare directory such as `server.git` without one; a `.git` that is a plain file containing `gitdir: <path>` points at a repository stored elsewhere, which is how linked worktrees and submodules work ([gitrepository-layout](#)). The official data model names four kinds of data inside: objects, references, the index and reflogs ([gitdatamodel](#)); everything else is

configuration or the state of an operation in progress. Ask Git where the directory is with `git rev-parse --git-dir`; never assume `./.git`.

See it. What `git init` creates, before anything is recorded:

```
$ git init inference-service
Initialized empty Git repository in $LAB/ch03/gitdir-tour/inference-service/.git/
$ cd inference-service
# Everything Git knows about this repository, minus the 14 sample hooks:
$ find .git -not -name '*.sample' | sort
.git
.git/config
.git/description
.git/HEAD
.git/hooks
.git/info
.git/info/exclude
.git/objects
.git/objects/info
.git/objects/pack
.git/refs
.git/refs/heads
.git/refs/tags
```

No object, no ref, no index. `refs/heads` is an empty directory, and yet the repository already has a current branch:

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
$ cat .git/description
Unnamed repository; edit this file 'description' to name the repository.
$ cat .git/info/exclude
# git ls-files --others --exclude-from=.git/info/exclude
# Lines that start with '#' are comments.
# For a project mostly in C, the following would be a good set of
# exclude patterns (uncomment them if you want to use them):
# *.lo
# *~
$ ls .git/hooks | wc -l
14
```

`HEAD` names `refs/heads/main`, a ref that does not exist yet: an "unborn" branch, which is legal, and the first commit creates the ref. `repositoryformatversion = 0` says that no extension is in use (sections 3.13 and 3.14 show version 1). `ignorecase` was probed from this Mac's filesystem, and `precomposeunicode` is set on macOS. `description` is read by the gitweb interface and by one sample hook; no Git command needs it. The fourteen hooks are samples that do nothing until renamed.

The first commit fills in the rest:

```
$ git add config.toml src/server.py
$ git commit -m "Add inference service skeleton"
[main (root-commit) 08ffd04] Add inference service skeleton
2 files changed, 4 insertions(+)
create mode 100644 config.toml
create mode 100644 src/server.py
# What the first commit added to .git:
$ find .git -type f -not -name '*.sample' | sort
.git/COMMIT_EDITMSG
.git/config
```

```
.git/description
.git/HEAD
.git/index
.git/info/exclude
.git/logs/HEAD
.git/logs/refs/heads/main
.git/objects/08/ffd041265ea03746a748c35f7d35df9a8e6900
.git/objects/36/35c8470ebd3d3e70fb74c4705d011de687e7f9
.git/objects/81/e5c25d60a924e6afc7e8f121dd6e78fded11fc
.git/objects/bc/cfc53fe1c451c013ac9e612812cfd3543c029a
.git/objects/e2/238784c412f0a5151764c07c7a44e7b3a9c073
.git/refs/heads/main
```

Two files in the working tree became five objects: two blobs, the tree for `src`, the top-level tree, and the commit. Three things appeared besides the objects: the index, the ref `refs/heads/main`, and two reflog files.

```
$ cat .git/refs/heads/main
08ffd041265ea03746a748c35f7d35df9a8e6900
$ cat .git/COMMIT_EDITMSG
Add inference service skeleton
$ cat .git/logs/HEAD
0000000000000000000000000000000000000000 08ffd041265ea03746a748c35f7d35df9a8e6900 Lab User
<you@example.com> 1788756240 +0530      commit (initial): Add inference service skeleton
$ git count-objects
5 objects, 20 kilobytes
```

The whole branch is one line of 41 bytes: forty hexadecimal digits and a newline. The reflog line reads: old value (all zeros, because the ref did not exist), new value, who, when, and why.

Picture. The four kinds of data and where each lives in a new repository:

<code>.git/</code>			
<code>objects/</code>	1. OBJECTS	blobs, trees, commits, tags	
<code>08/ffd041...</code>		loose: one compressed file per object	section 3.3
<code>pack/</code>		packed: many per file, plus an index	section 3.7
<code>refs/</code>	2. REFS	names for object IDs	section 3.9
<code>packed-refs</code>		the current branch; root refs	section 3.10
<code>HEAD, ORIG_HEAD, ...</code>		how each name moved, when, why	section 3.11
<code>logs/</code>	3. REFLOGS	the proposed next commit	section 3.12
<code>index</code>	4. INDEX	settings and local policy	
<code>config</code>			
<code>info/</code>			
<code>hooks/</code>			
<code>description</code>			

The complete inventory, for a repository in the default format. "Primary" means that Git cannot recreate the file from anything else.

Path	What it holds	Written by	Primary or derived
HEAD	ref: refs/heads/<branch>, or a commit ID when detached	git init, git switch, git checkout	primary
config	repository-level settings, remotes, upstream branches	git init, git config set, git remote	primary
description	one line for gitweb	the template	unused by Git
hooks/	scripts that Git runs at fixed points; samples end in <code>.sample</code>	you (Chapter 14C)	primary, never cloned
info/exclude	ignore patterns for this clone only	you (Chapter 4)	primary, never cloned
objects/<2 hex>/<38 hex>	loose objects	any command that records content	primary

Path	What it holds	Written by	Primary or derived
objects/pack/pack-*.pack	packed objects	git gc, git repack, git fetch, git clone	primary
objects/pack/pack-*.idx, *.rev	lookup tables for one pack	the same commands	derived from the pack
objects/info/	packs (a list for dumb transports), commit-graph, alternates	git gc; alternates by git clone --shared	derived, except alternates
refs/heads/, refs/tags/, refs/remotes/, refs/stash	loose refs, one file each	every command that moves a ref	primary
packed-refs	many refs in one text file	git pack-refs, git gc, git clone	primary
logs/HEAD, logs/refs/...	reflogs	every ref update	primary, never cloned or pushed
index	the staging area	git add, git commit, git switch, git merge, git status	derived, except for what is staged
COMMIT_EDITMSG	the message of the commit in progress or of the last one	git commit	scratch
ORIG_HEAD, FETCH_HEAD, MERGE_HEAD, CHERRY_PICK_HEAD, REVERT_HEAD, REBASE_HEAD, AUTO_MERGE, MERGE_MSG, MERGE_MODE, rebase-merge/, BISECT_*	root refs and the state of an operation in progress	the operation	state (section 3.10)
reftable/	refs and reflogs in the reftable format	every ref update, in a reftable repository	primary (section 3.13)
modules/, worktrees/, shallow	repositories of submodules; data of linked worktrees; the boundary of a shallow clone	Chapters 23, 25 and 26	primary

In production. The directory is the whole history: a container image, a CI cache or a backup that includes `.git` includes every version of every file, also the credentials file that was "deleted" two years ago. A copy of `.git` taken while no Git process is writing is a complete backup, and the only kind that includes reflogs, hooks, `info/exclude` and local configuration. And the right column of the table is your triage sheet in an incident: derived data is rebuilt, primary data is restored from another copy.

3.3 The loose object format

In one sentence. A loose object is one compressed file whose content is a short header, a NUL byte and the data, and whose file name is the hash of exactly those bytes.

Analogy. A sealed envelope filed under a number that is computed from what is inside it. Two envelopes with the same content get the same number, so the office keeps one; an envelope whose content differs by one character gets an unrelated number. The analogy breaks at correction: a real archive can fix a filing mistake by relabelling, and here the label cannot be changed without changing the content.

Precisely. The manual defines the format in three sentences: the object is the prefix `<type> <size>\0` followed by the data, where the type is `blob`, `tree`, `commit` or `tag` and the size is the length of the data in decimal; prefix and data together are compressed with zlib and stored; and "the object ID of the object is the SHA-1 or SHA-256 (as

appropriate) hash of the uncompressed data" ([gitformat-loose](#)). The file sits under `objects/`, in a directory named after the first two hexadecimal digits of the ID. `git hash-object <file>`  computes the ID that the file's content has as a blob, and writes the object with `-w`. It hashes what `git add` would store: if a clean filter or a line-ending conversion applies to the path, that is the converted content (`--no-filters` hashes the file as it is; Chapter 14C).

Inside `.git`. One new file under `objects/`, created read-only. Nothing else: no index entry, no ref, no reflog line.

See it.

```
$ git init inference-service
Initialized empty Git repository in $LAB/ch03/loose-object/inference-service/.git/
$ cd inference-service
$ printf 'retry_limit = 3\n' > config.toml
# Compute the ID only. Nothing is stored yet:
$ git hash-object config.toml
f784b58423ef67be5af8d1cfd9bea5eaa26bae
$ find .git/objects -type f
# Now store it (-w). One file appears, named after the ID:
$ git hash-object -w config.toml
f784b58423ef67be5af8d1cfd9bea5eaa26bae
$ find .git/objects -type f
.git/objects/f7/84b58423ef67be5af8d1cfd9bea5eaa26bae
```

The object file is not the working tree file. Decompress it and hash it yourself:

```
# The file is a zlib stream. Inflate it:
$ python3 -c "import sys, zlib; print(zlib.decompress(open(sys.argv[1], 'rb').read()))"
.git/objects/f7/84b58423ef67be5af8d1cfd9bea5eaa26bae
b'blob 16\x00retry_limit = 3\n'
$ wc -c < config.toml
16
# Hash the same bytes yourself: header, NUL, content.
$ printf 'blob 16\x00retry_limit = 3\n' | shasum
f784b58423ef67be5af8d1cfd9bea5eaa26bae -
```

`shasum` knows nothing about Git and prints the same forty digits. That is all an object ID is: a hash of `blob 16`, a NUL byte, and sixteen bytes of content. The compression is a storage detail and is not part of the identity.

```
$ git cat-file -t f784b584
blob
$ git cat-file -s f784b584
16
$ git cat-file -p f784b584
retry_limit = 3
# The ID depends on content only: another name, another directory, same ID.
$ mkdir -p deploy && cp config.toml deploy/production.toml
$ git hash-object deploy/production.toml
f784b58423ef67be5af8d1cfd9bea5eaa26bae
$ printf 'retry_limit = 3\n' | git hash-object --stdin
f784b58423ef67be5af8d1cfd9bea5eaa26bae
# One changed byte gives an unrelated ID:
$ printf 'retry_limit = 4\n' | git hash-object --stdin
eb1ea65a46ac10abf5e01a4af8d6a219f5182835
```

`-t` prints the type, `-s` the size of the data without the header, `-p` the content. The ID depends on nothing except the content: not the file name, not the directory, not the time, not the author, not the repository. One changed byte gives an ID that has nothing in common with the old one.

Picture.

config.toml, 16 bytes		the object, 24 bytes before compression
+-----+		+-----+
retry_limit = 3\n	----->	blob 16 \0 retry_limit = 3\n
+-----+		+-----+

	SHA-1 of these bytes	zlib
	v	v
f784b58423ef67be5af8...		.git/objects/f7/84b58423ef67be5af8...
the object ID		the file, named after the ID

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git hash-object -w <file></code>	unchanged	unchanged	unchanged	unchanged	one new file under <code>objects/</code> , unless the object already exists; without <code>-w</code> , nothing	unchanged	unchanged

An object in the database is not a tracked file. Tracking is a matter of the index:

```
# An object in the database is not a tracked file. The index is still empty:
$ git status --short
?? config.toml
?? deploy/
$ git ls-files --stage
$ git add config.toml
$ git ls-files --stage
100644 f784b58423ef67be5af8d1cfd9bea5eaa26bae 0      config.toml
$ find .git/objects -type f
.git/objects/f7/84b58423ef67be5af8d1cfd9bea5eaa26bae
```

`git add` created the index entry and wrote no second object: the blob it needed was already there, under the name it would have computed.

In production. Content addressing gives you three properties for free. Deduplication: the same lock file on forty branches is one object. Integrity: an ID is a checksum, so a copy of an object from any clone, backup or colleague is known to be the right bytes if its ID matches. Idempotence: recording the same content twice changes nothing. The second property is the foundation of every recovery in this book; Lab 16.2 uses it to repair a damaged pack from a file in the working tree.

3.4 The four object types in full

In one sentence. A blob is the bytes of one file, a tree is one directory listing, a commit is a snapshot with its ancestry and its metadata, and a tag object is a named, dated pointer to another object.

Analogy. A filesystem taken apart. Blobs are file contents with the name torn off, trees are the directory pages that put names back, and a commit is a dated cover sheet stapled to the top directory page, noting which cover sheets came before. The analogy breaks at mutation: a directory entry points at a place whose content can change, a tree entry points at content. Change one file, and its blob, every tree above it and the commit are new objects with new IDs.

Precisely. The four types and what each does not contain:

Type	Content	Deliberately absent
blob	the bytes of a file, or the target path of a symbolic link	name, mode, timestamps
tree	entries of mode, name and object ID, sorted by name	its own path; any history
commit	headers <code>tree</code> , <code>parent</code> , <code>author</code> , <code>committer</code> , optional headers, a blank line, the message	a diff; a branch name

Type	Content	Deliberately absent
tag	headers <code>object</code> , <code>type</code> , <code>tag</code> , <code>tagger</code> , a blank line, the message, optionally a signature	any guarantee that the ref of the same name still points at it

Inside `.git`. Each is one entry in the object database, loose or packed, stored and named exactly as section 3.3 describes; the type is the first word of the header.

Trees and the five entry modes

The demo repository is a small inference service whose top-level directory contains every kind of entry Git can record:

```
$ ls -F
config.toml
current-model@
models/
run.sh*
src/
tokenizer/
$ git ls-tree HEAD
100644 blob 60a72d9888260645df622ecd424e474d09d40560 config.toml
120000 blob ac850302da978fa7e1ffb2915871a2a50175cfea current-model
040000 tree a67ce55a4207beac5330d0947d65bffa4354fc52 models
100755 blob ea66be4ca05594e98649f4f08ac9fa9ff25578dc run.sh
040000 tree c009bc4782e140b36dc7a2136770393623a345bf src
160000 commit 9eb542d53a49342a2e19fbca1482a3d10d638739 tokenizer
# A symbolic link is a blob whose content is the target path:
$ git cat-file -p HEAD:current-model; echo
models/v2.bin
```

Mode	The entry is	The ID names
100644	a regular file	a blob
100755	an executable file	a blob
120000	a symbolic link	a blob whose content is the link target
040000	a directory	another tree
160000	a gitlink, the record of a submodule	a commit in another repository

These five are the complete list ([gitdatamodel](#)). The modes look like Unix permissions and are not: the executable bit is the only permission Git records; owner, group, the other bits and all timestamps are not in the repository. A directory exists only as a tree with entries, which is why an empty directory cannot be committed. The commit that a gitlink names is not stored here at all; `git cat-file` reports it as missing in section 3.5, correctly (Chapter 23).

`git ls-tree` and `git cat-file -p` print a tree in a readable form. The stored form is binary:

```
$ git cat-file -p HEAD:src
040000 tree 1722c9815eb7036ae06efcaf8d93c9c3140fb744 handlers
100644 blob e2238784c412f0a5151764c07c7a44e7b3a9c073 server.py
# The same tree as stored: mode, space, name, NUL, then the ID as 20 raw bytes.
$ git cat-file tree HEAD:src | xxd
00000000: 3430 3030 3020 6861 6e64 6c65 7273 0017 40000 handlers..
00000010: 22c9 815e b703 6ae0 6efc af8d 93c9 c314 "..^..j.n.....
00000020: 0fb7 4431 3030 3634 3420 7365 7276 6572 ..D100644 server
00000030: 2e70 7900 e223 8784 c412 f0a5 1517 64c0 .py..#.....d.
00000040: 7c7a 44e7 b3a9 c073 |zD....s
```

Read the hex dump against the listing: each entry is the mode in ASCII (40000, without the leading zero the listing prints), a space, the name, a NUL byte, and the object ID as twenty raw bytes (17 22 c9 81 ... is 1722c981...). The type column of the listing is not stored; Git derives it from the mode.

Commits, header by header

```
$ git log --graph --oneline
* 0c2cf43 Merge feature/batching
|\
| * 62001eb Add batch size setting
* | 4f2cc0c Add readiness handler
|/
* b602c1f Add inference service skeleton
# A merge commit: one tree, two parent headers.
$ git cat-file -p HEAD
tree 31fc0d39799d6931bb12f7befeb250a539f84a96
parent 4f2cc0c5f842120f109977a97bc72acef5aa5ccd
parent 62001eb879c6506f6d3c8e625dfadfa5029367b7
author Lab User <you@example.com> 1788756720 +0530
committer Lab User <you@example.com> 1788756720 +0530

Merge feature/batching
# A root commit has no parent header at all:
$ git cat-file -p HEAD~2
tree b6f6972aea97f7f084242c2b859bf307959a0920
author Lab User <you@example.com> 1788756360 +0530
committer Lab User <you@example.com> 1788756360 +0530

Add inference service skeleton
```

Header	How many	Meaning
tree	exactly one	the ID of the top-level tree: the complete snapshot
parent	none, one or several	the commits this one follows, in order: the first is the commit you were on, the second is the one you merged
author	one	who wrote the change: name, email, seconds since 1 January 1970 UTC, time-zone offset
committer	one	who created this commit object, in the same format; it differs from the author after an amend, a rebase, a cherry-pick or an applied patch (Chapter 6)
encoding	optional	the encoding of the message when it is not UTF-8
gpgsig	optional	a signature over the rest of the commit; the header has this name for OpenPGP, SSH and X.509 signatures alike (Chapter 14B)
mergetag	optional	the complete signed tag object that this commit merged

After the headers come one blank line and the message. A header value that spans several lines continues on lines that begin with a space. `gpgsig` and `mergetag` are documented in `git help gitformat-signature`. Both were checked on Git 2.55.0 with a throwaway SSH key, which also showed that the header is named `gpgsig-sha256` in a SHA-256 repository; the output is not printed because a signature differs on every run. The `encoding` header is deterministic:

```
# An optional header: the encoding of the message when it is not UTF-8.
$ printf 'max_tokens = 256\n' >> config.toml
$ git -c i18n.commitEncoding=ISO-8859-1 commit -q -am "Add token limit"
$ git cat-file -p HEAD
tree 4274d7332863c24a4bebec4082f0ea6c6fdbd467
parent 0c2cf4371cac2e5d412153dc58293c0cb484c4ba
author Lab User <you@example.com> 1788757800 +0530
committer Lab User <you@example.com> 1788757800 +0530
encoding ISO-8859-1

Add token limit
```

A commit ID is computed like a blob ID, over the header `commit <size>`, a NUL byte, and the text you have been reading:

```
# A commit ID is the hash of "commit <size>", a NUL byte, and exactly the text above.
$ git cat-file -s HEAD
271
$ (printf 'commit %s\0' "$(git cat-file -s HEAD)"; git cat-file commit HEAD) | shasum
0c2cf4371cac2e5d412153dc58293c0cb484c4ba -
$ git rev-parse HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
```

Follow that through. The commit contains the ID of its tree, which contains the IDs of every blob and subtree; it contains the IDs of its parents, which contain the IDs of theirs. One commit ID therefore fixes every file in the snapshot and every commit behind it, and nothing in a commit can be edited: a different message, timestamp or parent is a different text with a different hash. "Amending" and "rebasing" always create new commits (Chapters 6 and 9).

Annotated tags

```
$ git cat-file -t v1.0.0
tag
$ git cat-file -t v1.0.0-rc1
commit
$ git cat-file -p v1.0.0
object 0c2cf4371cac2e5d412153dc58293c0cb484c4ba
type commit
tag v1.0.0
tagger Lab User <you@example.com> 1788756780 +0530

Release 1.0.0
# The ref holds the ID of the tag object; ^{} peels it to the commit it tags.
$ git rev-parse v1.0.0 "v1.0.0^{}" HEAD
0b624edf4a556c702b6ed110ef2702570ced1558
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
```

`v1.0.0-rc1` is a lightweight tag: a ref that holds a commit ID, and nothing else. `v1.0.0` is an annotated tag: the ref holds the ID of a tag object, and the tag object names the commit, its type, the tag's own name, who tagged and when. `v1.0.0^{}` peels the tag, that is, follows it until it reaches something that is not a tag.

Picture. The objects of the demo repository and how they point at each other:

```

refs/tags/v1.0.0 --> tag 0b624ed --> commit 0c2cf43 --tree--> tree 31fc0d3
      |               |
1st parent |         | 2nd parent 120000 blob 60a72d9 config.toml
      v               v
commit 4f2cc0c      commit 62001eb 040000 tree a67ce55 models
      |               |
      v               v
      commit b602c1f (root) 100755 blob ea66be4 run.sh
                             040000 tree c009bc4 src
                             160000 commit 9eb542d tokenizer
                                 (in another repository)

```

Every arrow is an object ID stored inside the object at its tail, and there are none in the other direction: a blob does not know which trees contain it, and a commit does not know its children or its branches.

In production. A deploy step fails with "permission denied" on `run.sh` on the server and works on every laptop: `git ls-tree HEAD run.sh` shows `100644`, so the executable bit was never recorded, and the fix is a commit that changes the mode (`git update-index --chmod=+x run.sh`, Chapter 4). A release is "the same code" as the last one according to its notes: compare `git rev-parse v1.0.0^{tree}` with the tree of the other tag. Equal tree IDs mean identical content in every file.

3.5 Reading objects: `git cat-file`, `git ls-tree`, `git show`

In one sentence. `git cat-file` answers questions about objects (type, size, content, existence) for one object or for millions, `git ls-tree` lists a tree, and `git show` formats any object for a person.

Precisely. All three only read. The forms of `git cat-file`:

Form	Answers
<code>git cat-file -t <object></code>	the type
<code>git cat-file -s <object></code>	the size of the content in bytes, without the header
<code>git cat-file -p <object></code>	the content, formatted according to its type
<code>git cat-file -e <object></code>	whether the object exists, by exit status only
<code>git cat-file <type> <object></code>	the raw content; <code>git cat-file tree HEAD</code> also steps from the commit to its tree
<code>git cat-file --batch-check[=<format>]</code>	one line per name read from standard input
<code>git cat-file --batch</code>	the same line followed by the content
<code>... --batch-all-objects</code>	every object in the database, reachable or not, without reading standard input

`<object>` is anything that `git rev-parse` can resolve (section 3.6): an ID, a ref, `HEAD^{tree}`, `HEAD:path`.

See it.

```
# Type, size and content of one object, named three different ways.
$ git cat-file -t HEAD
commit
$ git cat-file -t "HEAD^{tree}"
tree
$ git cat-file -t HEAD:config.toml
blob
$ git cat-file -s HEAD:config.toml
46
$ git cat-file -p HEAD:config.toml
retry_limit = 3
timeout_s = 30
batch_size = 8
# -e answers "does this object exist?" with the exit status only.
$ git cat-file -e HEAD:config.toml
[exit status: 0]
$ git cat-file -e 1234567890123456789012345678901234567890
[exit status: 1]
```

Starting one process per object is slow in a script. The batch modes read names from standard input and answer in one process:

```
# Many objects in one process: names on standard input, one line of output each.
$ printf 'HEAD\nHEAD^{tree}\nHEAD:run.sh\nv1.0.0\nno-such-branch\n' | git cat-file --batch-check
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit 271
31fc0d39799d6931bb12f7befeb250a539f84a96 tree 214
ea66be4ca05594e98649f4f08ac9fa9ff25578dc blob 42
0b624edf4a556c702b6ed110ef2702570ced1558 tag 137
no-such-branch missing
# A custom format. %(rest) echoes whatever followed the name on the input line.
$ git ls-tree -r HEAD | awk '{print $3, $4}' | git cat-file --batch-check='%(objectsize) %(objecttype)
%(rest)
46 blob config.toml
13 blob current-model
11 blob models/v2.bin
42 blob run.sh
30 blob src/handlers/health.py
29 blob src/handlers/ready.py
```

```
40 blob src/server.py
9eb542d53a49342a2e19fbca1482a3d10d638739 missing
```

The default line is ID, type, size. A name that does not resolve gives `<name> missing` and the command carries on. With a custom format, `%(rest)` repeats whatever followed the name on the input line, which is how the second command carries each path through to the output. Its last line is the gitlink: the tree names commit `9eb542d`, and this object database does not contain it.

```
# Every object in the database, reachable or not, without walking any history:
$ git cat-file --batch-all-objects --batch-check='%(objecttype)' | sort | uniq -c
  8 blob
  4 commit
  1 tag
  9 tree
# The three largest blobs in the whole history, with the path that leads to them:
$ git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %(objectsize) %(rest)' |
grep '^blob' | sort -k2 -n | tail -3
blob 40 src/server.py
blob 42 run.sh
blob 46 config.toml
```

Twenty-two objects for four commits. `--batch-all-objects` visits loose and packed objects without walking any history, so it also finds objects that nothing refers to. The second command is the first one to run when a repository is larger than expected: `git rev-list --objects --all` prints every reachable object with the path that leads to it, and `cat-file` adds the sizes.

`git ls-tree` reads one tree, or with `-r` every tree below it:

```
$ git ls-tree HEAD src/
040000 tree 1722c9815eb7036ae06efcaf8d93c9c3140fb744    src/handlers
100644 blob e2238784c412f0a5151764c07c7a44e7b3a9c073    src/server.py
# -r recurses into subtrees and lists only the leaves; -t also shows the trees on the way.
$ git ls-tree -r HEAD
100644 blob 60a72d9888260645df622ecd424e474d09d40560    config.toml
120000 blob ac850302da978fa7e1ffb2915871a2a50175cfea    current-model
100644 blob ba6f678b335f69ce9ddd1551c01e264366b80a6b    models/v2.bin
100755 blob ea66be4ca05594e98649f4f08ac9fa9ff25578dc    run.sh
100644 blob 7dd511683bc85b8cb7d31982e55a6d1a06f814f9    src/handlers/health.py
100644 blob 1ea8895d490a68d48ff799a4b567792e5fb4c2fb    src/handlers/ready.py
100644 blob e2238784c412f0a5151764c07c7a44e7b3a9c073    src/server.py
160000 commit 9eb542d53a49342a2e19fbca1482a3d10d638739    tokenizer
$ git ls-tree -r -t HEAD src
040000 tree c009bc4782e140b36dc7a2136770393623a345bf    src
040000 tree 1722c9815eb7036ae06efcaf8d93c9c3140fb744    src/handlers
100644 blob 7dd511683bc85b8cb7d31982e55a6d1a06f814f9    src/handlers/health.py
100644 blob 1ea8895d490a68d48ff799a4b567792e5fb4c2fb    src/handlers/ready.py
100644 blob e2238784c412f0a5151764c07c7a44e7b3a9c073    src/server.py
# -l adds the blob size; -d lists only trees.
$ git ls-tree -l HEAD
100644 blob 60a72d9888260645df622ecd424e474d09d40560    46    config.toml
120000 blob ac850302da978fa7e1ffb2915871a2a50175cfea    13    current-model
040000 tree a67ce55a4207beac5330d0947d65bffa4354fc52    -    models
100755 blob ea66be4ca05594e98649f4f08ac9fa9ff25578dc    42    run.sh
040000 tree c009bc4782e140b36dc7a2136770393623a345bf    -    src
160000 commit 9eb542d53a49342a2e19fbca1482a3d10d638739    -    tokenizer
$ git ls-tree -d --name-only HEAD
models
src
tokenizer
```

Without `-r` you see one directory level, with trees as entries. With `-r` you see only the leaves, which is the flat list of paths that a checkout would create; `-t` adds the trees that were passed on the way. `-l` prints sizes, and prints `-` where an entry is not a blob. `-d` keeps the entries that are directories, the gitlink among them.

`git show` is the porcelain on top and picks a presentation by object type:

```
# git show adapts to the object type. A blob: its content.
$ git show HEAD:config.toml
retry_limit = 3
timeout_s = 30
batch_size = 8
# A tree: the names in it.
$ git show "HEAD^{tree}"
tree HEAD^{tree}

config.toml
current-model
models/
run.sh
src/
tokenizer
# An annotated tag: the tag object, then the commit it points at.
$ git show --no-patch v1.0.0
tag v1.0.0
Tagger: Lab User <you@example.com>
Date:   Mon Sep 7 10:23:00 2026 +0530

Release 1.0.0

commit 0c2cf4371cac2e5d412153dc58293c0cb484c4ba
Merge: 4f2cc0c 62001eb
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:22:00 2026 +0530

    Merge feature/batching
# A commit in raw form: the object headers, then the message.
$ git show --no-patch --format=raw HEAD^2
commit 62001eb879c6506f6d3c8e625dfadfa5029367b7
tree 537b759e6f31d04b14514d85b1cca6e8db79933c
parent b602c1fa61adb577fc9ca3113292b7cf734e856a
author Asha Rao <asha@example.com> 1788756480 +0530
committer Asha Rao <asha@example.com> 1788756480 +0530

    Add batch size setting
```

In production. Reading a file as it was in another commit without touching the working tree: `git show <commit>:<path>`. Finding what makes a repository large: the `rev-list` and `--batch-check` pipeline above, sorted by size. Checking in CI that a commit exists before deploying it: `git cat-file -e "$id^{commit}"`, which fails for a missing object and for an ID that names anything other than a commit.

3.6 Turning names into IDs: `git rev-parse`

In one sentence. `git rev-parse` resolves anything that can name an object to its object ID and answers questions about the repository itself, by the same rules that every other Git command applies to its arguments.

Precisely. The revision syntax, as defined in `git help revisions`:

You write	It means	In the demo repository
<code>main</code> , <code>v1.0.0</code> , <code>HEAD</code>	the object that the ref names; for an annotated tag, the tag object	<code>0c2cf43</code> ; the tag <code>0b624ed</code>
<code>HEAD~2</code>	two steps back, following first parents only	<code>b602c1f</code>
<code>HEAD^2</code>	the second parent of a merge; <code>HEAD^1</code> or <code>HEAD^</code> is the first	<code>62001eb</code>
<code>HEAD^{tree}</code>	the object of that type reached by following the name: here the commit's tree	<code>31fc0d3</code>
<code>v1.0.0^{commit}</code> , <code>v1.0.0^{} </code>	the tag peeled to a commit; peeled until it is no longer a tag	<code>0c2cf43</code>
<code>HEAD:src/server.py</code>	the blob or tree at that path in that commit	<code>e223878</code>

You write	It means	In the demo repository
:config.toml, :2:config.toml	the entry for the path in the index; the entry at stage 2	section 3.12
main@{1}	the previous value of the ref, read from its reflog	4f2cc0c

A bare name is looked up in a fixed order: `$GIT_DIR/<name>`, then `refs/<name>`, `refs/tags/<name>`, `refs/heads/<name>`, `refs/remotes/<name>` and `refs/remotes/<name>/HEAD`. A tag therefore wins over a branch of the same name; Git 2.55 prints `warning: refname 'main' is ambiguous.` and uses the tag. In scripts, write the full name, `refs/heads/main`.

See it.

```
$ git log --graph --oneline
* 0c2cf43 Merge feature/batching
|\
| * 62001eb Add batch size setting
* | 4f2cc0c Add readiness handler
|/
* b602c1f Add inference service skeleton
$ git rev-parse HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
$ git rev-parse --short HEAD
0c2cf43
# ~2 follows first parents twice. ^2 is the second parent of a merge. ^1 is the first.
$ git rev-parse HEAD~2 HEAD^2 HEAD^1
b602c1fa61adb577fc9ca3113292b7cf734e856a
62001eb879c6506f6d3c8e625dfadfa5029367b7
4f2cc0c5f842120f109977a97bc72acef5aa5ccd
# From a commit to its tree, to a subtree, to a blob:
$ git rev-parse "HEAD^{tree}" HEAD:src HEAD:src/server.py
31fc0d39799d6931bb12f7bfebf250a539f84a96
c009bc4782e140b36dc7a2136770393623a345bf
e2238784c412f0a5151764c07c7a44e7b3a9c073
# A tag object, and the commit it peels to:
$ git rev-parse v1.0.0 "v1.0.0^{commit}"
0b624edf4a556c702b6ed110ef2702570ced1558
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
```

Picture. The same answers on the graph:

```

      62001eb   feature/batching           HEAD~2
      /         \
b602c1f ---+      +--- 0c2cf43   main, tag v1.0.0   (HEAD -> main)
      \         /
      4f2cc0c   tag v1.0.0-rc1             HEAD^1 = HEAD~1

HEAD~2 = HEAD^1^1 = b602c1f           ~ counts generations, ^ chooses a parent
```

The other direction, from an ID or a short name to a full name, and the reflog and index forms:

```
$ git rev-parse --abbrev-ref HEAD
main
$ git rev-parse --symbolic-full-name HEAD
refs/heads/main
$ git rev-parse --symbolic-full-name v1.0.0 feature/batching
refs/tags/v1.0.0
refs/heads/feature/batching
# The previous value of a branch comes from its reflog:
$ git rev-parse "main@{1}"
4f2cc0c5f842120f109977a97bc72acef5aa5ccd
# A leading colon reads the index, not a commit:
$ git rev-parse :config.toml
60a72d9888260645df622ecd424e474d09d40560
```

The second family of options describes the repository. These are what a script should use in place of assumptions about paths:

```
$ git rev-parse --git-dir --show-toplevel
.git
$LAB/ch03/rev-parse/inference-service
$ cd src/handlers
$ git rev-parse --git-dir
$LAB/ch03/rev-parse/inference-service/.git
$ git rev-parse --show-prefix --show-cdup
src/handlers/
../../
$ git rev-parse --is-inside-work-tree --is-bare-repository
true
false
$ git rev-parse --show-object-format --show-ref-format
sha1
files
$ git rev-parse --git-path hooks/pre-commit
../../.git/hooks/pre-commit
$ cd ../../
```

`--git-dir` answers relative to where you are. `--show-prefix` and `--show-cdup` are the path from the top level down to the current directory and back up. `--show-object-format` and `--show-ref-format` name the two storage formats of sections 3.13 and 3.14. `--git-path` gives the location of a file inside the repository, and stays right when the repository is a linked worktree or an environment variable relocates part of it.

Error handling is where scripts go wrong:

```
# Without --verify, rev-parse prints what it cannot resolve on standard output:
$ id=$(git rev-parse no-such-branch 2>/dev/null); echo "status=$? captured=[$id]"
status=128 captured=[no-such-branch]
# With --verify it prints one object ID or nothing, and --quiet drops the message:
$ id=$(git rev-parse --verify --quiet no-such-branch); echo "status=$? captured=[$id]"
status=1 captured=[]
$ git rev-parse --verify no-such-branch
fatal: Needed a single revision
[exit status: 128]
$ git rev-parse --verify --quiet "v1.0.0^{commit}"
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
[exit status: 0]
# --short implies --verify, so it accepts exactly one revision:
$ git rev-parse --short HEAD~2 HEAD^2
fatal: Needed a single revision
[exit status: 128]
# A merge with two parents has no third parent:
$ git rev-parse --verify "HEAD^3"
fatal: Needed a single revision
[exit status: 128]
```

Observed behavior	: a deploy script stores the text "no-such-branch" in a variable that should hold a commit ID
Git state	: the name resolves to nothing
Mechanism	: without <code>--verify</code> , git <code>rev-parse</code> prints an argument it cannot resolve on standard output, writes the error to standard error, and exits with status 128
Root cause	: the script captured standard output and never looked at the exit status
Why Git does this	: the first purpose of <code>rev-parse</code> is to sort the arguments of a calling script into revisions and everything else, so what is not a revision is passed through
Correct fix	: <code>id=\$(git rev-parse --verify --quiet "\$name^{commit}") exit 1</code>
Prevention	: use <code>--verify</code> in every script; add <code>^{commit}</code> when the object must exist and be a commit

`--verify` accepts exactly one argument and prints one ID or nothing. It checks that the argument can be turned into an ID, not that the object is in the database: a full forty-digit ID passes even if no such object exists. The

peeling suffix closes that gap, because `^{commit}` has to read the object.

In production. `git rev-parse --show-toplevel` at the start of a script makes it independent of the calling directory; `git rev-parse --verify --quiet "refs/heads/$branch^{commit}"` tests whether a branch exists with nothing to parse; and when two people disagree about what a build contained, `git rev-parse HEAD^{tree}` in both checkouts settles it.

3.7 Packfiles, pack indexes and delta compression

In one sentence. A packfile stores many objects in one file and may store an object as a difference from a similar object, an index file beside it finds any object by ID, and none of this changes what an object is.

Analogy. A warehouse keeps one complete copy of a long contract and, for each earlier draft, a sheet of instructions: "copy bytes 0 to 2,400 of the complete copy, insert these twelve bytes, copy the rest". You ask for a draft by its number and always receive the complete draft. The analogy breaks at who decides: the warehouse chooses which version is kept whole, may choose differently at every reorganisation, and nothing you see through Git depends on it.

Precisely. Every command that records content writes loose objects. Maintenance (`git gc`  here; the automatic strategies are the subject of Chapter 26) later consolidates them into a pack:

- `pack-<checksum>.pack`: a 12-byte header (the signature `PACK`, a version, the number of objects), one entry per object, and a checksum of everything before it. An entry is a type, a size and zlib-compressed data; in the deltified form the data is a list of copy and insert instructions against a base object, named by its offset in the same pack or by its ID ([gitformat-pack](#)).
- `pack-<checksum>.idx`: the object IDs in sorted order, a CRC32 and the pack offset for each, and a 256-entry table saying where the IDs with each first byte begin. Finding an object is a binary search here, then one read at the offset.
- `pack-<checksum>.rev`: the reverse map, from position in the pack to position in the index.

The manual states the principle in one line: "Conceptually there are only four object types: commit, tree, tag and blob. However to save space, an object could be stored as a 'delta' of another 'base' object." Which objects are compared is a heuristic: `git pack-objects` sorts them by type, size and name, compares each with its neighbours inside a window of ten (`--window`), and does not let a chain grow deeper than fifty (`--depth`).

Inside .git. Loose object files disappear, one `.pack`, one `.idx` and one `.rev` appear under `objects/pack/`, and `git gc` also writes `packed-refs` (section 3.9) and `objects/info/commit-graph` (Chapter 26).

See it. An evaluation harness whose test-case file has 200 lines; five commits each change one line of it:

```
$ git log --oneline
14fbba9 Relax case 150 to two sentences
d076308 Relax case 120 to two sentences
5c0b0dc Relax case 90 to two sentences
709d5d8 Relax case 60 to two sentences
213918f Relax case 30 to two sentences
609e81f Add evaluation cases
$ wc -l eval/cases.jsonl
  200 eval/cases.jsonl
$ git count-objects -v
count: 25
size: 100
in-pack: 0
packs: 0
size-pack: 0
prune-packable: 0
garbage: 0
size-garbage: 0
```

Twenty-five loose objects: six commits, twelve trees, six versions of `cases.jsonl` and one small configuration file. `count` and `size` describe loose objects, and `size` is disk space in KiB: every one of the 25 files occupies a 4 KiB block, however small its content.

```
$ git gc
$ git count-objects -v
count: 0
size: 0
in-pack: 25
packs: 1
size-pack: 4
prune-packable: 0
garbage: 0
size-garbage: 0
$ find .git/objects -type f | sort
.git/objects/info/commit-graph
.git/objects/info/packs
.git/objects/pack/pack-8027c3005e9ed6293511e1bef76969aa63a131bc.idx
.git/objects/pack/pack-8027c3005e9ed6293511e1bef76969aa63a131bc.pack
.git/objects/pack/pack-8027c3005e9ed6293511e1bef76969aa63a131bc.rev
```

The same 25 objects, now `in-pack`, in 4 KiB instead of 100. Look inside:

```
$ git verify-pack -v .git/objects/pack/pack-*.idx
14fbb9f81a145d88e699ad6379b7844b71e855a commit 232 163 12
d076308a75023ff48e849cd91e1acac4fce7f4c6 commit 232 166 175
5c0b0dc76a1661cca61de1475a80959793e9280c commit 231 164 341
709d5d8beab89236b8649acadca336e7406c6132 commit 231 163 505
213918f93a4e5886620977a101f7181e57459140 commit 231 165 668
609e81f2752953bc7efa6215b74b1d08bde2ceef commit 173 128 833
a4e58256c921e15fecaaaf744fbdb04b37907a40 blob 16789 1372 961
26489f627fb36b56d87f8b615855f57c7888fd60 blob 17 27 2333
794774c4549fba17c4a64943e946feeac7c0ed4d tree 31 40 2360
89f3a9862e1b2373ab6e9bcac1e76d158b8c478f tree 78 85 2400
8c9b4e17e9e4d4e2aa09bd48ba220549944e06d2 tree 31 40 2485
5a92fde4e91ed06f182b9a45d78fa51d92e87e7f tree 78 85 2525
739088cda5ca15c1fd59dc40238f8852f7f677de blob 22 35 2610 1 a4e58256c921e15fecaaaf744fbdb04b37907a40
59ded879fc1920533dd82f4e9f2a55ecc7480549 tree 31 41 2645
9513db28fdebd67318d477e402e045a33336949c tree 78 84 2686
d77675f2a7b00158d41dc879b35011a742015f48 blob 17 30 2770 2 739088cda5ca15c1fd59dc40238f8852f7f677de
b19c9a4c02f3742eab5f0ccfc40e1fc2b4f66b3c tree 31 40 2800
5b0701b0d0b031208d1e4972513376c5cd0ae392 tree 78 85 2840
d900a91b8bd7c1bafc34c1c8baef1c92f5af3832 blob 22 35 2925 3 d77675f2a7b00158d41dc879b35011a742015f48
be320f0afa9a2877aa5134ee84b85c6ada640031 tree 31 40 2960
b99c6ada4476e39a26ae54f9ab22db3aec8d1f1d tree 78 84 3000
782b8e9b7662fd152695234c4ba08fde34c18f73 blob 18 30 3084 4 d900a91b8bd7c1bafc34c1c8baef1c92f5af3832
b803d0495a5e56f35f19109a922d0ca78f64c184 tree 31 41 3114
a1356effbbb0a03fd3fa3115791bcf98670794b4 tree 78 84 3155
0928d48c558a6457e11c9fa47f347fb2a190259f blob 22 35 3239 5 782b8e9b7662fd152695234c4ba08fde34c18f73
non delta: 20 objects
chain length = 1: 1 object
chain length = 2: 1 object
chain length = 3: 1 object
chain length = 4: 1 object
chain length = 5: 1 object
.git/objects/pack/pack-8027c3005e9ed6293511e1bef76969aa63a131bc.pack: ok
```

The columns are object ID, type, size, size in the pack and offset; a deltified object has two more, the depth of its chain and the ID of its base. One version of the file, `a4e58256`, is stored whole: 16,789 bytes that compress to 1,372. The other five are deltas of about twenty bytes each, and for those the size column is the size of the delta. The summary counts chain lengths.

Picture. The chain that the listing describes. Each arrow reads "is stored as a delta against":

```
newest version                                oldest version
a4e58256 <--- 739088cd <--- d77675f2 <--- d900a91b <--- 782b8e9b <--- 0928d48c
```

whole	depth 1	depth 2	depth 3	depth 4	depth 5	
1372 bytes	35 bytes	30 bytes	35 bytes	30 bytes	35 bytes	in the pack
16789 bytes	16788	16787	16786	16785	16784	as an object

The newest version is the whole one, and the old versions are the deltas: the opposite of what "Git stores diffs" would suggest. To read the oldest version, Git reads the base and applies five deltas. The object you get is complete:

[illegible]

`%(objectsize)` is the size of the object, `%(objectsize:disk)` is what its entry costs in the pack, and `%(delabase)` is all zeros for an object that is stored whole. Three statements keep the two levels apart:

1. A delta is between two objects chosen for similarity, not "the change a commit made"; the base can be another version of the same file or a different file.
2. IDs are computed from full content, so packing changes no ID, no tree and no commit, and `git cat-file`, `git show` and `git checkout` always deliver full content.
3. The choice of base "is arbitrary and is subject to change during a repack" (`git help cat-file`), so a size on disk says little about which commit "caused" it.

The files themselves:

```
$ ls .git/objects/pack
pack-8027c3005e9ed6293511e1bef76969aa63a131bc.idx
pack-8027c3005e9ed6293511e1bef76969aa63a131bc.pack
pack-8027c3005e9ed6293511e1bef76969aa63a131bc.rev
# The first twelve bytes of the pack: signature, version, number of objects (0x19 is 25).
$ head -c 12 .git/objects/pack/pack-*.pack | xxd
00000000: 5041 434b 0000 0002 0000 0019          PACK.....
# The last twenty bytes: a checksum of everything before them. It is also the name of the file.
$ tail -c 20 .git/objects/pack/pack-*.pack | xxd -p
8027c3005e9ed6293511e1bef76969aa63a131bc
```

```
# The index starts with a magic number and its own version:
$ head -c 8 .git/objects/pack/pack-*.idx | xxd
00000000: ff74 4f63 0000 0002                                .t0c....
# Its entries, decoded: offset in the pack, object ID, CRC32. They are sorted by object ID:
$ git show-index < .git/objects/pack/pack-*.idx | head -4
3239 0928d48c558a6457e11c9fa47f347fb2a190259f (713cb177)
12 14fbbba9f81a145d88e699ad6379b7844b71e855a (197f240b)
668 213918f93a4e5886620977a101f7181e57459140 (d1bda72c)
2333 26489f627fb36b56d87f8b615855f57c7888fd60 (ac6cf501)
# Sorted by offset instead, the same entries give the order of the objects inside the pack:
$ git show-index < .git/objects/pack/pack-*.idx | sort -n | head -4
12 14fbbba9f81a145d88e699ad6379b7844b71e855a (197f240b)
175 d076308a75023ff48e849cd91e1acac4fce7f4c6 (06d8d7ea)
341 5c0b0dc76a1661cca61de1475a80959793e9280c (b18d9a25)
505 709d5d8beab89236b8649acadca336e7406c6132 (7d7faf7e)
# The index ends with the checksum of its pack, then a checksum of itself:
$ tail -c 40 .git/objects/pack/pack-*.idx | xxd -p -c 20
```

```
8027c3005e9ed6293511e1bef76969aa63a131bc
d34871f7ea2e09071ce733054562a3a8be5765c3
```

The pack is named after its own checksum. The index starts with `\377t0c` and version 2, lists the IDs in sorted order with the offsets `12`, `175`, `341` that the pack listing showed, and ends with the checksum of the pack it belongs to. The index is derived data: Lab 16.1 deletes it and rebuilds an identical one with `git index-pack`.

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git gc</code>	unchanged	unchanged	unchanged	same value; the loose file moves into <code>packed-refs</code>	loose objects become a pack; <code>packed-refs</code> and <code>commit-graph</code> written; reflog entries past their expiry removed; unreachable objects past the grace period deleted	unchanged	unchanged

After a `git gc`, new commits are loose objects again, next to the pack, until the next consolidation. That is the normal state of a working repository.

Version note. Older behavior: porcelain commands ran `git gc --auto` when about 6,700 loose objects or 50 packs had accumulated. Current behavior: automatic maintenance uses the "geometric" strategy and does not run `git gc`. Since: Git 2.54 ([maintenance configuration](#)). Recommended: learn the storage model here with `git gc`, and read Chapter 26 for what runs on its own.

In production. The answer to the fourth question of section 3.1: four hundred versions of a line-oriented 16 MB file that differ in a few records cost roughly one compressed copy plus four hundred small deltas. Three limits apply. Deltas need similar bytes, so compressed archives and model weights gain nothing. Files above `core.bigFileThreshold` (512 MiB by default) are stored without any attempt at deltas. And a clone transfers a pack, so everything reachable is downloaded however well it compresses (Chapters 22 and 26).

3.8 Reachability, `git fsck`, and where Git checks its hashes

In one sentence. An object is reachable if you can arrive at it from a starting point by following the IDs stored inside objects; `git fsck` verifies that everything reachable is present and well-formed, and lists what nothing reaches.

Analogy. A library with a catalogue. A book is findable if a card names it or a findable book cites it. A book that nothing names is still on its shelf, complete, until the next clear-out. The analogy breaks at direction: in Git, citations only run from newer to older (a commit names its parents, never its children), so losing the card for the newest commit of a line makes the whole line unfindable at once.

Precisely. The glossary: one object is reachable from another "if we can reach the one from the other by a chain that follows tags to whatever they tag, commits to their parents or trees, and trees to the trees or blobs that they contain". An unreachable object is one that no starting point reaches; a dangling object is an unreachable object that no other unreachable object refers to, the tip of a lost line. The starting points of `git fsck` are "the index file, all SHA-1 references in the `refs` namespace, and all reflogs" (`git help fsck`), plus `HEAD`. Garbage collection keeps the same set alive and deletes an unreachable object only after a grace period, two weeks by default (`gc.pruneExpire`).

Inside .git. `git fsck` changes nothing. `git gc` removes unreachable objects from `objects/` once they are past the grace period, and `git reflog expire` removes lines from `logs/`.

See it.

```
# A healthy repository: no output, exit status 0.
$ git fsck
[exit status: 0]
```

```
# Stage a file, then change your mind and unstage it.
$ printf 'api_token = "test-0000-not-a-real-token"\n' > secrets.toml
$ git add secrets.toml
$ git rm --cached --quiet secrets.toml
$ git status --short
?? secrets.toml
# The index entry is gone. The blob that "git add" wrote is not:
$ git fsck
dangling blob e6773e7e30f399953039ee95529a70c172b7bf5e
$ git cat-file -p e6773e7e
api_token = "test-0000-not-a-real-token"
```

Unstaging removed an index entry, not the blob that `git add` had written: anyone with access to this directory can still read the token (Chapter 21B), and the same fact saves you the day you lose a staged file.

```
# A commit on a branch, and then the branch is deleted.
$ git switch --quiet -c experiment/cache
$ printf 'cache_ttl_s = 300\n' >> config.toml
$ git commit --quiet -am "Add cache TTL"
$ git switch --quiet main
$ git branch -D experiment/cache
Deleted branch experiment/cache (was 728ac29).
# Starting from refs and the index only (--no-reflogs), four objects are unreachable:
# the commit, its tree, its new blob, and the blob that was unstaged earlier.
$ git fsck --no-reflogs --unreachable
unreachable blob e6773e7e30f399953039ee95529a70c172b7bf5e
unreachable commit 728ac2949c899feb5120ad1d50640f48e299c311
unreachable tree 73c1adebf83ff39f155fe1236171279e58109bc6
unreachable blob 35376fb465bd033dbe3a382c6a66467bef321292
# Only the objects that nothing at all points to are called dangling:
$ git fsck --no-reflogs
dangling blob e6773e7e30f399953039ee95529a70c172b7bf5e
dangling commit 728ac2949c899feb5120ad1d50640f48e299c311
```

Four objects are unreachable from the refs and the index, and two of them are dangling. The tree `73c1ade` and the blob `35376fb` are not dangling, because the unreachable commit refers to them. Find the dangling commit and you have found everything behind it.

```
# The HEAD reflog still records the commit, which is what keeps it safe for now:
$ git reflog -3
0c2cf43 HEAD@{0}: checkout: moving from experiment/cache to main
728ac29 HEAD@{1}: commit: Add cache TTL
0c2cf43 HEAD@{2}: checkout: moving from main to experiment/cache
$ git log --oneline -1 "HEAD@{1}"
728ac29 Add cache TTL
# With reflog entries as starting points again, only the blob is left over:
$ git fsck
dangling blob e6773e7e30f399953039ee95529a70c172b7bf5e
```

Picture.

starting points	objects	
refs/heads/main ----->	0c2cf43 ----> trees and blobs, parents ...	reachable
refs/tags/..., index, HEAD		
reflog entry HEAD@{1} ----->	728ac29 ----> 73c1ade ----> 35376fb	reachable
	(commit) (tree) (blob)	through the
reflog only		

```
nothing ----- e6773e7 dangling
                  (blob that was staged and unstaged)
```

One reflog entry stands between the deleted branch and the garbage collector. Entries expire (90 days by default, 30 when the commit is no longer reachable from the ref's tip; the lab configuration switches both off), and a deleted branch loses its own reflog at once. Chapter 13 builds its procedures on these facts.

```
Observed behavior : git fsck reports "dangling commit" for a commit that git reflog still lists
Git state         : the only reference to the commit is a reflog entry dated later than the current
time
Mechanism         : git fsck notes the time at which it starts and ignores reflog entries newer than
that
Root cause        : the entry was written with a committer date in the future: a wrong system clock,
or a script that sets GIT_COMMITTER_DATE
Why Git does this : fsck must not be confused by commits that are created while it runs; skipping
newer reflog entries is the "coarse solution" its commit message describes
Correct fix       : none is needed; git gc does not apply the rule, and the commit is not deleted
Prevention        : read "dangling" as information; compare with git reflog before pruning anything
```

```
# Reflog entries in the past count as starting points. Nothing is dangling:
$ git -C dated-2001 reflog --date=short
2cc599e HEAD@{2001-01-01}: reset: moving to HEAD~1
778d03a HEAD@{2001-01-01}: commit: Raise retry limit
2cc599e HEAD@{2001-01-01}: commit (initial): Add configuration
$ git -C dated-2001 fsck
[exit status: 0]
# Reflog entries dated after "now" are skipped, so the dropped commit looks dangling:
$ git -C dated-2099 reflog --date=short
a3c4e27 HEAD@{2099-01-01}: reset: moving to HEAD~1
279be34 HEAD@{2099-01-01}: commit: Raise retry limit
a3c4e27 HEAD@{2099-01-01}: commit (initial): Add configuration
$ git -C dated-2099 fsck
dangling commit 279be34d74a15f6b04f2ec46f2113b8dc8291065
[exit status: 0]
# The commit is in the reflog either way, and the reflog is what you recover from:
$ git -C dated-2099 log --oneline -1 "HEAD@{1}"
279be34 Raise retry limit
```

```
# Garbage collection does not apply the clock rule. Even with no grace period, the commit stays:
$ git -C dated-2099 gc --quiet --prune=now
$ git -C dated-2099 cat-file -t "HEAD@{1}"
commit
$ git -C dated-2099 count-objects -v | grep -e count -e in-pack
count: 0
in-pack: 6
# fsck still calls it dangling. "Dangling" is a statement about fsck starting points, not a verdict:
$ git -C dated-2099 fsck
dangling commit 279be34d74a15f6b04f2ec46f2113b8dc8291065
[exit status: 0]
```

Version note. Older behavior: `git fsck` treated every reflog entry as a starting point. Current behavior: entries dated after the moment `fsck` starts are skipped; `fsck` reads the system clock for this. Since: Git 2.53.0, with the commit "[fsck: snapshot default refs before object walk](#)"; builtin/`fsck.c` has the check at the [v2.53.0 tag](#) and not at v2.52.0. Verified locally: Git 2.55.0 applies it, Apple's Git 2.50.1 does not. Recommended: nothing to configure; know that the two Gits on your Mac can disagree about one repository.

Where the hash is checked, and where it is not

An object ID is a checksum, and Git does not recompute it on every read. A file under `objects/` that is a well-formed object with the wrong content is served as it is:

```

$ git rev-parse HEAD:src/server.py
e2238784c412f0a5151764c07c7a44e7b3a9c073
$ git cat-file -p HEAD:src/server.py
def predict(text):
    return len(text)
# Overwrite that loose object with a valid zlib stream: same header, same length, other content.
$ chmod u+w .git/objects/e2/238784c412f0a5151764c07c7a44e7b3a9c073
$ python3 -c "import sys, zlib; open(sys.argv[1], 'wb').write(zlib.compress(b'blob 40\ndef
predict(text):\n    return 999999999\n'))" .git/objects/e2/238784c412f0a5151764c07c7a44e7b3a9c073
# Ordinary commands read the object by its name and do not recompute the hash:
$ git cat-file -p HEAD:src/server.py
def predict(text):
    return 999999999
[exit status: 0]
$ git status --short
[exit status: 0]

```

This substitution was crafted. Random damage, such as a flipped bit, normally breaks the zlib stream and fails loudly on the first read (Lab 16.2). Two operations do recompute IDs from content:

```

# git fsck hashes what it finds and compares the result with the file name:
$ git fsck
error: bf8a76ba21df71f4f19002c42f44ea784df0f99d: hash-path mismatch, found at:
.git/objects/e2/238784c412f0a5151764c07c7a44e7b3a9c073
missing blob e2238784c412f0a5151764c07c7a44e7b3a9c073
[exit status: 3]
# A fetch-style transfer rebuilds every ID on the receiving side, so the bad object cannot travel:
$ git clone --quiet --no-local . ../clone-over-transport
fatal: did not receive expected object e2238784c412f0a5151764c07c7a44e7b3a9c073
fatal: fetch-pack: invalid index-pack output
[exit status: 128]
# A plain file copy has no such check:
$ git clone --quiet . ../clone-by-file-copy
[exit status: 0]
$ git -C ../clone-by-file-copy cat-file -p HEAD:src/server.py
def predict(text):
    return 999999999

```

`git fsck` hashes what it finds and compares the result with the name. A transfer through Git's transport sends object content without IDs, and the receiving side computes every ID itself, so an object cannot arrive under a name that its content does not have. A clone from a local path skips the transport and copies or hard-links the files, bad object included; `--no-local` forces the transport.

```

# The working tree still holds the true content, and content alone determines the ID.
$ git hash-object src/server.py
e2238784c412f0a5151764c07c7a44e7b3a9c073
# Git will not rewrite an object it believes it has, so remove the bad file first.
$ rm -f .git/objects/e2/238784c412f0a5151764c07c7a44e7b3a9c073
$ git hash-object -w src/server.py
e2238784c412f0a5151764c07c7a44e7b3a9c073
$ git fsck
[exit status: 0]
$ git cat-file -p HEAD:src/server.py
def predict(text):
    return len(text)

```

The repair is the property of section 3.3 used in reverse: content determines the ID, so the same content from anywhere restores the object.

In production. Run `git fsck` on a schedule against the repositories you could not afford to lose (mirrors, backups, the build server's clone); treat `missing` and `error:` lines as an incident and `dangling` lines as routine. Take backups through Git's transport (`git clone --mirror` from a URL, or `--no-local` from a path) when you want every hash recomputed on the way, and as a file copy when you also need reflogs and hooks. The reachability rule

is also the precise form of an uncomfortable fact about leaked secrets: removing a commit from every branch leaves it in the object database of every clone that has it.

3.9 Refs in depth: loose refs, packed-refs and symbolic refs

In one sentence. A ref is a name that holds an object ID or the name of another ref, and in the default `files` format a ref is a small file under `refs/`, a line in `packed-refs`, or both.

Analogy. Signposts. Each carries a name and points at one house (an object); a few point at another signpost (symbolic refs). Moving a signpost is cheap and changes no house. The analogy breaks at demolition: in Git, a house that no signpost leads to is eventually torn down (section 3.8).

Precisely. The glossary defines a ref as "a name that points to an object name or another ref (the latter is called a symbolic ref)". Names either start with `refs/` or sit in the root of the hierarchy (section 3.10). The namespaces under `refs/`:

Namespace	Names	Moved by
<code>refs/heads/<name></code>	a local branch	<code>git commit</code> , <code>git merge</code> , <code>git reset</code> , <code>git rebase</code> , <code>git branch -f</code>
<code>refs/tags/<name></code>	a tag, pointing at a commit or at a tag object	<code>git tag</code> ; meant to stay put
<code>refs/remotes/<remote>/<name></code>	a remote-tracking branch: the last known position of a branch in that remote	<code>git fetch</code> , <code>git push</code>
<code>refs/stash</code>	the newest stash entry; older entries live in its reflog	<code>git stash</code>
<code>refs/bisect/</code> , <code>refs/notes/</code> , <code>refs/replace/</code> , others	refs owned by a tool	that tool

In the `files` format, a loose ref is a file holding the ID and a newline, a symbolic ref is a file holding `ref: <name>`, and `packed-refs` is one sorted text file with many refs. Lookup reads the loose file first and `packed-refs` second, and "subsequent updates to branches always create new files under `$GIT_DIR/refs`" ([git-pack-refs](#)). The plumbing works the same in every format: `git for-each-ref` lists and formats, `git show-ref` lists and verifies, `git update-ref` 🟡 writes or deletes one ref with an optional check of its old value, `git symbolic-ref` 🟡 reads and writes symbolic refs, and `git pack-refs` 🟢 moves loose refs into `packed-refs` without changing a value.

Inside `.git`. Files under `refs/`, the file `packed-refs`, and for every update a line in the corresponding file under `logs/`.

See it. The demo repository with a remote, a stash entry and two tags:

```
$ find .git/refs -type f | sort
.git/refs/heads/feature/batching
.git/refs/heads/main
.git/refs/remotes/origin/HEAD
.git/refs/remotes/origin/main
.git/refs/stash
.git/refs/tags/v1.0.0
.git/refs/tags/v1.0.0-rc1
$ cat .git/refs/heads/main
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
$ cat .git/refs/tags/v1.0.0
0b624edf4a556c702b6ed110ef2702570ced1558
# Two symbolic refs. They hold a ref name, not an object ID:
$ cat .git/HEAD
ref: refs/heads/main
```

```
$ cat .git/refs/remotes/origin/HEAD
ref: refs/remotes/origin/main
```

```
$ git for-each-ref
62001eb879c6506f6d3c8e625dfadfa5029367b7 commit refs/heads/feature/batching
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit refs/heads/main
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit refs/remotes/origin/HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit refs/remotes/origin/main
0c3d1b1c205d3248bab3061f235ef2309f653b49 commit refs/stash
0b624edf4a556c702b6ed110ef2702570ced1558 tag refs/tags/v1.0.0
4f2cc0c5f842120f109977a97bc72acef5aa5ccd commit refs/tags/v1.0.0-rc1
$ git for-each-ref --format='%(refname:short) -> %(objectname:short) %(upstream:short)' refs/heads
feature/batching -> 62001eb
main -> 0c2cf43 origin/main
# show-ref can peel annotated tags (the ^{} lines) and test for existence:
$ git show-ref --tags --dereference
0b624edf4a556c702b6ed110ef2702570ced1558 refs/tags/v1.0.0
0c2cf4371cac2e5d412153dc58293c0cb484c4ba refs/tags/v1.0.0^{
4f2cc0c5f842120f109977a97bc72acef5aa5ccd refs/tags/v1.0.0-rc1
$ git show-ref --verify refs/heads/main
0c2cf4371cac2e5d412153dc58293c0cb484c4ba refs/heads/main
[exit status: 0]
$ git show-ref --exists refs/heads/no-such-branch
error: reference does not exist
[exit status: 2]
```

`for-each-ref` prints ID, type and full name by default, and `--format` chooses any field; `%(upstream:short)` is the information that `git branch -vv` prints after each branch. `show-ref --dereference` adds a `^{}` line with the commit that an annotated tag peels to. `--verify` takes a full name and `--exists` answers by exit status.

```
# A branch is a ref, so plumbing can create one: name, new value, reflog message.
$ git update-ref -m "experiment: start at the release candidate" refs/heads/experiment v1.0.0-rc1
$ git branch --list --verbose experiment
  experiment 4f2cc0c Add readiness handler
# With a third argument the update happens only if the ref still has that old value.
$ git update-ref refs/heads/experiment main b602c1fa61adb577fc9ca3113292b7cf734e856a
fatal: update_ref failed for ref 'refs/heads/experiment': cannot lock ref 'refs/heads/experiment': is
at 4f2cc0c5f842120f109977a97bc72acef5aa5ccd but expected b602c1fa61adb577fc9ca3113292b7cf734e856a
[exit status: 128]
$ git update-ref refs/heads/experiment main v1.0.0-rc1
[exit status: 0]
$ git reflog show experiment
0c2cf43 experiment@{0}:
4f2cc0c experiment@{1}: experiment: start at the release candidate
# Two refusals: an object that does not exist, and a branch that would not name a commit.
$ git update-ref refs/heads/broken 1234567890123456789012345678901234567890
fatal: update_ref failed for ref 'refs/heads/broken': trying to write ref 'refs/heads/broken' with
nonexistent object 1234567890123456789012345678901234567890
[exit status: 128]
$ git update-ref refs/heads/broken HEAD:config.toml
fatal: update_ref failed for ref 'refs/heads/broken': trying to write non-commit object
60a72d9888260645df622ecd424e474d09d40560 to branch 'refs/heads/broken'
[exit status: 128]
$ git update-ref -d refs/heads/experiment
```

`update-ref` is how a branch comes into being: a name, a value and a reflog message, with no checkout involved. With a third argument it is a compare-and-swap: the update happens only if the ref still has the expected old value, which keeps concurrent writers from overwriting each other and is the idea behind `git push --force-with-lease` (Chapter 12). It refuses an ID that names no object and a non-commit under `refs/heads/`. `--stdin` applies several updates as one transaction: if one fails, none is written.

```
$ git symbolic-ref HEAD
refs/heads/main
$ git symbolic-ref --short HEAD
main
$ git symbolic-ref refs/remotes/origin/HEAD
```

```

refs/remotes/origin/main
# Any ref can be symbolic. This one makes "latest" another name for main:
$ git symbolic-ref refs/heads/latest refs/heads/main
$ cat .git/refs/heads/latest
ref: refs/heads/main
$ git rev-parse latest main
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
$ git symbolic-ref --delete refs/heads/latest
# Detached HEAD: the file holds an object ID, so there is no symbolic ref to read.
$ git switch --quiet --detach v1.0.0-rc1
$ cat .git/HEAD
4f2cc0c5f842120f109977a97bc72acef5aa5ccd
$ git symbolic-ref HEAD
fatal: ref HEAD is not a symbolic ref
[exit status: 128]
$ git switch --quiet main

```

A symbolic ref is a name for a name: `HEAD` is the one you use every day, `refs/remotes/origin/HEAD` records the default branch of the remote, and a detached `HEAD` holds an ID instead.

```

$ git pack-refs --all
$ cat .git/packed-refs
# pack-refs with: peeled fully-peeled sorted
0c2cf4371cac2e5d412153dc58293c0cb484c4ba refs/heads/main
0c2cf4371cac2e5d412153dc58293c0cb484c4ba refs/remotes/origin/main
0c3d1b1c205d3248bab3061f235ef2309f653b49 refs/stash
0b624edf4a556c702b6ed110ef2702570ced1558 refs/tags/v1.0.0
^0c2cf4371cac2e5d412153dc58293c0cb484c4ba
4f2cc0c5f842120f109977a97bc72acef5aa5ccd refs/tags/v1.0.0-rc1
# The loose files are gone, except the symbolic ref, which cannot be packed:
$ find .git/refs -type f | sort
.git/refs/remotes/origin/HEAD
# The next update writes a loose file again. It wins over the stale packed line:
$ printf 'max_tokens = 256\n' >> config.toml
$ git commit --quiet -am "Add token limit"
$ find .git/refs -type f | sort
.git/refs/heads/main
.git/refs/remotes/origin/HEAD
$ grep refs/heads/main .git/packed-refs
0c2cf4371cac2e5d412153dc58293c0cb484c4ba refs/heads/main
$ git rev-parse main
a8eea183b656f17da1e2456a2e49995e8388fcb4

```

The header lists the traits of the file: `peeled` and `fully-peeled` promise that every annotated tag is followed by a `^` line with the commit it points at, and `sorted` that the lines are in order. Symbolic refs are never packed. The last commands show the rule that trips scripts: after the next update, the loose file is back and wins, and the line in `packed-refs` is stale.

```

Observed behavior : a release script runs "cat .git/refs/heads/main" and finds no file; a variant
greps
                    packed-refs and stamps the build with last week's commit
Git state          : main was packed by git gc (or arrived packed from git clone), then moved on
Mechanism          : lookup reads the loose file first and packed-refs second; maintenance moves loose
                    refs into packed-refs; a later update writes a new loose file and leaves the old
line
Root cause         : the scripts read one of several storage locations instead of asking Git
Why Git does this  : one file per ref wastes space and is slow with thousands of refs
Correct fix        : git rev-parse --verify refs/heads/main
Prevention         : never read .git/refs or packed-refs from a script; use rev-parse, for-each-ref,
                    update-ref and symbolic-ref, which also work with reftable

```

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git update-ref refs/heads/ <id></code>	unchanged	unchanged	unchanged	moves if is the current branch, and then no longer matches the index and working tree	the ref and a line in its reflog	unchanged	unchanged
<code>git symbolic-ref HEAD refs/heads/</code>	unchanged	unchanged	now names 	unchanged	a line in <code>logs/HEAD</code>	unchanged	unchanged
<code>git pack-refs --all</code>	unchanged	unchanged	unchanged	same value, stored in <code>packed-refs</code>	loose ref files removed except symbolic refs	unchanged	unchanged

In production. Put your automation on `git for-each-ref --format=...`; it needs no parsing, and `git branch --format` and `git tag --format` accept the same language (`git for-each-ref --sort=--committerdate --format='%(committerdate:short) %(refname:short)'` `refs/heads` lists branches by age). Treat `update-ref` on a checked-out branch and `symbolic-ref HEAD` as repairs that leave the index and the working tree behind. And read a stale `packed-refs` line as what it is: harmless, as long as nothing reads it by hand.

3.10 Root refs, pseudorefs, and the state of an operation

In one sentence. A few refs live directly in the repository directory instead of under `refs/`, with names in capital letters that end in `_HEAD`; two of them, `FETCH_HEAD` and `MERGE_HEAD`, are not refs at all in the strict sense and are called pseudorefs.

Precisely. The glossary's naming rule for a root ref: upper-case letters and underscores only, and the name is `HEAD` or ends in `_HEAD`, plus five irregular names such as `AUTO_MERGE` and `MERGE_AUTOSTASH`. A pseudoref is "a ref that has different semantics than normal refs. These refs can be read via normal Git commands, but cannot be written to by commands like `git-update-ref(1)`"; the glossary lists two: `FETCH_HEAD`, which "may refer to multiple object IDs", each annotated with its source, and `MERGE_HEAD`, which "contains all commit IDs which are being merged" ([gitglossary](#)).

Version note. Older behavior: the glossary called every special file under `.git` a pseudoref and gave `MERGE_HEAD` and `CHERRY_PICK_HEAD` as examples. Current behavior: "pseudoref" means `FETCH_HEAD` and `MERGE_HEAD` only; the others are root refs, stored by the ref backend like any other ref. Since: Git 2.46 ([glossary at v2.45.0](#), [at v2.46.0](#)). Recommended: say "root ref" for `ORIG_HEAD` and its relatives, and notice that older tutorials and the description of `--include-root-refs` in `git help for-each-ref` still use the old wording.

The names, who writes them, and when they go away (`git help revisions`, and Lab 17.3, which visits each one):

Name	Written by	Holds	Gone when
<code>HEAD</code>	<code>git init</code> , <code>git switch</code> , <code>git checkout</code>	the current branch, or a commit ID when detached	never
<code>ORIG_HEAD</code>	<code>git merge</code> , <code>git rebase</code> , <code>git reset</code> , <code>git am</code> ; on Git 2.55 also <code>git stash push</code>	where <code>HEAD</code> was before	overwritten by the next such command
<code>FETCH_HEAD</code>	<code>git fetch</code> , <code>git pull</code>	one line per fetched ref: ID, a <code>not-for-merge</code> marker, origin	overwritten by the next fetch

Name	Written by	Holds	Gone when
MERGE_HEAD	a <code>git merge</code> that stops	the commit or commits being merged	the merge is committed or aborted
CHERRY_PICK_HEAD , REVERT_HEAD	a cherry-pick or revert that stops or runs with <code>--no-commit</code>	the commit being applied	committed, skipped or aborted
REBASE_HEAD	a rebase that stops	the commit at which it stopped	continued past it, or aborted
BISECT_HEAD	<code>git bisect --no-checkout</code>	the commit to test	<code>git bisect reset</code>
AUTO_MERGE	the <code>ort</code> merge machinery when it leaves conflicts, and on Git 2.55 also a clean <code>git stash pop</code>	a tree: what was written to the working tree	the next commit

Beside them sit files that are not refs: `MERGE_MSG` and `MERGE_MODE`, the `rebase-merge/` directory with the branch name and todo list of a rebase, `sequencer/` for a multi-commit cherry-pick or revert, and the `BISECT_*` files.

Inside `.git`. In the `files` format all of these are files in the top of the directory; in refable the root refs are records in the tables and only the two pseudorefs remain files (section 3.13).

See it. The two pseudorefs hold more than a single ID:

```
$ git fetch origin
# One line per fetched branch: object ID, a marker for "git pull", and where it came from.
$ cat .git/FETCH_HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba          branch 'main' of ../server
62001eb879c6506f6d3c8e625dfadfa5029367b7        not-for-merge  branch 'feature/batching' of ../server
# Used as a revision, FETCH_HEAD means its first line:
$ git rev-parse FETCH_HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
```

```
# A merge of two branches at once, stopped before the commit is made:
$ git merge --no-commit topic/metrics topic/tracing
Fast-forwarding to: topic/metrics
Trying simple merge with topic/tracing
Automatic merge went well; stopped before committing as requested
# MERGE_HEAD holds one line per branch being merged. ORIG_HEAD holds where main was:
$ cat .git/MERGE_HEAD
228726e45b0adbf88785b4d004845222a7d5e973
67b8eefa35c8ad680fc5cb88ec2d028ba2c09cc6
$ cat .git/ORIG_HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba
$ ls -A .git
COMMIT_EDITMSG
config
description
FETCH_HEAD
HEAD
hooks
index
info
logs
MERGE_HEAD
MERGE_MODE
MERGE_MSG
objects
ORIG_HEAD
refs
```

That is the "different semantics": a list of IDs with notes, where a ref holds exactly one. Git reads the first line when you use the name as a revision, and `update-ref` keeps its hands off both files:

```
# The refs outside refs/, as Git lists them. The two pseudorefs are not in the list:
$ git for-each-ref --include-root-refs | grep -v refs/
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit HEAD
0c2cf4371cac2e5d412153dc58293c0cb484c4ba commit ORIG_HEAD
# A root ref is an ordinary ref, so update-ref writes it. A pseudoref is refused:
$ git update-ref ORIG_HEAD HEAD~1
[exit status: 0]
$ git update-ref MERGE_HEAD HEAD~1
fatal: update_ref failed for ref 'MERGE_HEAD': refusing to update pseudoref 'MERGE_HEAD'
[exit status: 128]
$ git reflog exists ORIG_HEAD; echo "exit status: $"
exit status: 1
$ git merge --abort
```

`for-each-ref --include-root-refs` lists `HEAD` and `ORIG_HEAD` and leaves out the two pseudorefs. `ORIG_HEAD` accepts an update like any ref and has no reflog of its own.

The property that matters in an incident: these names, `HEAD` excepted, are not starting points for reachability. Section 3.8 listed the starting points: `refs/`, `HEAD`, the reflogs and the index.

```
# A branch of a teammate, fetched by path without a destination ref. It lands in FETCH_HEAD only:
$ git fetch ../teammate fix/timeout
From ../teammate
 * branch          fix/timeout -> FETCH_HEAD
$ git log --oneline -1 FETCH_HEAD
25623b2 Lower the timeout to 20 seconds
# No ref under refs/ and no reflog entry names that commit, so fsck reports it.
# (The dangling tree is the result of the two-branch merge that was aborted above.)
$ git fsck
dangling commit 25623b2b4f51bc94cebfa8e80099a56f6b754270
dangling tree 7396eea3ac2000acc286f477277694a681054728
# A garbage collection without a grace period deletes it. The name stays behind and names nothing:
$ git gc --quiet --prune=now
$ cat .git/FETCH_HEAD
25623b2b4f51bc94cebfa8e80099a56f6b754270      branch 'fix/timeout' of ../teammate
$ git log --oneline -1 FETCH_HEAD
fatal: bad object FETCH_HEAD
[exit status: 128]
```

`git gc --prune=now`  removes every unreachable object at once instead of after the two-week grace period, and so destroys the recovery material that `git fsck` had listed. Preview it with `git fsck --unreachable` and `git prune -n`; recover afterwards only from another clone or a backup; use it in a repository whose unreachable objects you have examined and want gone, for example after removing a leaked secret from history (Chapter 21B), and never as routine maintenance.

Root cause. A commit fetched by URL into `FETCH_HEAD`, a position saved in `ORIG_HEAD`, or a tree in `AUTO_MERGE` is protected by nothing except the grace period. Give it a ref before you rely on it: `git fetch <url> <branch>:refs/heads/review/<name>`, or `git switch -c review/<name> FETCH_HEAD`.

In production. Automation that asks "is a merge in progress?" should resolve the name through Git, `git rev-parse --verify --quiet MERGE_HEAD`, rather than test for a file, because root refs are not files in every format (section 3.13).

3.11 Reflog storage under `logs/`

In one sentence. A reflog is an append-only record of the values a ref has had, one file per ref under `logs/` in the `files` format, written whenever the ref moves.

In one sentence. The index is one binary file that records, for every tracked path, a mode, a blob ID, a stage number and a copy of the file's `stat` data, followed by extensions that let Git skip work; it is at once the proposed next commit and the cache that makes `git status` fast.

Analogy. A packing list for the next shipment, with an inspection sticker on every line saying when the item was last checked and how big it was. The list becomes the commit's tree; the stickers let the clerk skip boxes nobody touched. The analogy breaks during a conflict, when the list carries three lines for one item.

Precisely. The format is specified in `git help gitformat-index` ([gitformat-index](#)). A 12-byte header: the signature DIRC, a version (2, 3 or 4), the number of entries. Then the entries, sorted by path as bytes and then by stage. Each entry has 40 bytes of stat data (change and modification time with nanoseconds, device, inode, mode, user, group, size), the object ID, and 16 bits of flags: an assume-valid bit, an "extended" bit, the two-bit stage and the length of the path; version 3 adds 16 bits with the skip-worktree and intent-to-add flags. The path follows, padded with NUL bytes to a multiple of eight; version 4 compresses each path against the previous one and drops the padding. Then the extensions, each a four-letter signature, a length and data: TREE caches the tree IDs that unchanged parts of the index already correspond to, REUC keeps the stages of a resolved conflict so that `git checkout -m` can recreate it, and others serve the split index, the untracked cache and the filesystem monitor. The file ends with a checksum, unless `index.skipHash` replaces it by zeros.

Inside .git. One file, `index`, rewritten as a whole by every command that changes it, including `git status`, which refreshes the stat data and writes the result back ([git-status](#)).

See it. The eight entries of the demo repository, as `git ls-files --stage` shows them and as the bytes begin:

```
$ git ls-files --stage
100644 60a72d9888260645df622ecd424e474d09d40560 0      config.toml
120000 ac850302da978fa7e1ffb2915871a2a50175cfea 0      current-model
100644 ba6f678b335f69ce9ddd1551c01e264366b80a6b 0      models/v2.bin
100755 ea66be4ca05594e98649f4f08ac9fa9ff25578dc 0      run.sh
100644 7dd511683bc85b8cb7d31982e55a6d1a06f814f9 0      src/handlers/health.py
100644 1ea8895d490a68d48ff799a4b567792e5fb4c2fb 0      src/handlers/ready.py
100644 e2238784c412f0a5151764c07c7a44e7b3a9c073 0      src/server.py
160000 9eb542d53a49342a2e19fbca1482a3d10d638739 0      tokenizer
# The first twelve bytes of the file: signature, version, number of entries.
$ head -c 12 .git/index | xxd
00000000: 4449 5243 0000 0002 0000 0008          DIRC.....
```

4449 5243 is DIRC, 0000 0002 is version 2, 0000 0008 is eight entries. A sixty-line Python script in the course, `labs/ch03/read-index.py`, decodes the rest by following the manual page:

```
$ python3 "$COURSE_ROOT/labs/ch03/read-index.py" .git/index
header      signature=DIRC version=2 entries=8
entry       100644 60a72d9888260645df622ecd424e474d09d40560 stage=0 size=46      config.toml
entry       120000 ac850302da978fa7e1ffb2915871a2a50175cfea stage=0 size=13      current-model
entry       100644 ba6f678b335f69ce9ddd1551c01e264366b80a6b stage=0 size=11      models/v2.bin
entry       100755 ea66be4ca05594e98649f4f08ac9fa9ff25578dc stage=0 size=42      run.sh
entry       100644 7dd511683bc85b8cb7d31982e55a6d1a06f814f9 stage=0 size=30      src/handlers/health.py
entry       100644 1ea8895d490a68d48ff799a4b567792e5fb4c2fb stage=0 size=29      src/handlers/ready.py
entry       100644 e2238784c412f0a5151764c07c7a44e7b3a9c073 stage=0 size=40      src/server.py
entry       160000 9eb542d53a49342a2e19fbca1482a3d10d638739 stage=0 size=0      tokenizer
extension   TREE (117 bytes)
checksum    valid
```

The stat data is what the volatile demo prints; its numbers come from your filesystem and differ on every machine, so this transcript is not reproduced by `labs/verify-all.sh`:

```
$ git ls-files --debug config.toml run.sh
config.toml
  ctime: 1790892135:837284071
  mtime: 1790892135:837284071
  dev: 16777231 ino: 67018155
  uid: 501      gid: 0
  size: 46      flags: 0
run.sh
  ctime: 1790892135:668514559
  mtime: 1790892135:665748392
```

```
dev: 16777231 ino: 67018022
uid: 501      gid: 0
size: 42     flags: 0
# The same numbers, asked from the filesystem (BSD stat, as on macOS):
$ stat -f 'ctime=%c mtime=%m dev=%d ino=%i uid=%u gid=%g size=%z' config.toml
ctime=1790892135 mtime=1790892135 dev=16777231 ino=67018155 uid=501 gid=0 size=46
```

Here is the cache at work. Change only the modification time of a file:

```
# Change the modification time of a file, not its content.
$ touch -t 202001010000 config.toml
# Plumbing compares cached stat data only, and reports the path as possibly changed:
$ git diff-files
:100644 100644 60a72d9888260645df622ecd424e474d09d40560 00000000000000000000000000000000 M
config.toml
# Porcelain re-reads the file, finds the same content, and refreshes the cached stat data:
$ git status --short
$ git diff-files
```

`git diff-files`, a plumbing command, compares stat data only and reports the path as possibly changed, with an all-zero ID meaning "look at the working tree". `git status` reads the file, hashes it, finds the blob the index already names, and writes fresh stat data back; the second `diff-files` is silent. Now copy a whole repository:

```
# Copy the whole repository, as a backup restore or a CI cache would.
$ cp -R . ../restored-copy
$ git -C ../restored-copy diff-files --name-status
M      config.toml
M      current-model
M      models/v2.bin
M      run.sh
M      src/handlers/health.py
M      src/handlers/ready.py
M      src/server.py
$ git -C ../restored-copy diff-files --quiet
[exit status: 1]
# New inode numbers and change times: every cached entry is stale. One refresh fixes it.
$ git -C ../restored-copy update-index --refresh
$ git -C ../restored-copy diff-files --quiet
[exit status: 0]
```

Every entry is stale, because the copy has new inode numbers and change times, and no file has changed. One refresh repairs it. This is the whole explanation of "every file shows as modified" after a restore from backup, a CI cache or a `COPY` into a container, and it is never a data problem.

The flags, set by `git update-index --assume-unchanged`, `--skip-worktree` and `git add -N` (Chapter 5 explains what each is for):

```
$ git update-index --assume-unchanged run.sh
$ git update-index --skip-worktree config.toml
$ printf '# Inference service\n' > README.md
$ git add --intent-to-add README.md
# Only the header and the entries that carry a flag:
$ python3 "$COURSE_ROOT/labs/ch03/read-index.py" .git/index | grep -e header -e "\["
header      signature=DIRC version=3 entries=9
entry       100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 stage=0 size=0      README.md [intent-to-
add]
entry       100644 60a72d9888260645df622ecd424e474d09d40560 stage=0 size=46    config.toml [skip-
worktree]
entry       100755 ea66be4ca05594e98649f4f08ac9fa9ff25578dc stage=0 size=42    run.sh [assume-valid]
```

Setting a version-3 flag upgraded the file from version 2 to version 3 on the spot, and the intent-to-add entry names the empty blob. Stages appear when a merge stops:

```
$ git merge feature/timeouts
Auto-merging config.toml
CONFLICT (content): Merge conflict in config.toml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
# One path, three entries: stage 1 is the merge base, 2 is ours, 3 is theirs.
$ git ls-files --unmerged
100644 60a72d9888260645df622ecd424e474d09d40560 1      config.toml
100644 cf333b0c40d2484e3c6d5040d289bdfb06d7e62a 2      config.toml
100644 62235dd3403d45d5edec627dcf862fe1b0ba940e 3      config.toml
$ git cat-file -p :1:config.toml | grep timeout
timeout_s = 30
$ git cat-file -p :2:config.toml | grep timeout
timeout_s = 45
$ git cat-file -p :3:config.toml | grep timeout
timeout_s = 60
```

```
$ git restore --theirs config.toml
$ git add config.toml
$ git ls-files --stage config.toml
100644 62235dd3403d45d5edec627dcf862fe1b0ba940e 0      config.toml
# The three stages moved into the "resolve undo" extension (REUC):
$ python3 "$COURSE_ROOT/labs/ch03/read-index.py" .git/index | grep -e header -e extension -e checksum
header      signature=DIRC version=2 entries=8
extension   TREE (98 bytes)
extension   REUC (93 bytes)
checksum    valid
$ git commit --quiet -m "Merge feature/timeouts"
```

One path, three entries: stage 1 is the merge base, 2 is ours, 3 is theirs (Chapter 8). `git add` replaced the three by one stage-0 entry and moved them into the REUC extension. Finally the version field:

```
$ git update-index --show-index-version
2
$ wc -c < .git/index
898
$ git update-index --index-version 4
$ head -c 12 .git/index | xxd
00000000: 4449 5243 0000 0004 0000 0008          DIRC.....
$ wc -c < .git/index
860
$ git update-index --index-version 2
```

Version 4 saved 38 bytes here by sharing path prefixes; in a repository with a hundred thousand paths the saving is what makes `feature.manyFiles` worth switching on (Chapter 26).

Picture. One entry in version 2, as the reader script walks it:

header	DIRC version entry count	12 bytes
entry	ctime ctime_ns mtime mtime_ns dev ino mode uid gid size	40 bytes of stat
data	object ID	20 bytes
	flags: assume-valid extended stage (2 bits) path length	2 bytes
	path, NUL, padding to a multiple of 8 bytes	variable
...	one entry per path, and per stage during a conflict, sorted by path	
extension	TREE size cached tree IDs REUC size the stages that were resolved	
checksum	hash of everything above	20 bytes

In production. Three incidents and their first command. "Everything is modified after the restore": `git update-index --refresh`. "git status takes a minute on the monorepo": the stat cache already does its job; the next steps are `core.untrackedCache`, `core.fsmonitor` and `feature.manyFiles` (Chapter 26). "fatal: index file corrupt": delete the file and run `git reset`; history and working tree are untouched, and only what was staged and not committed must be staged again (Lab 17.1).

3.13 The reftable backend

In one sentence. reftable stores refs and reflogs in binary, block-structured tables under `.git/reftable/` instead of in loose files, `packed-refs` and `logs/`; it has been available since Git 2.45 and is planned as the default for new repositories in Git 3.0.

Analogy. A stack of ledgers instead of a drawer of index cards. Every transaction adds a thin ledger on top; from time to time thin ledgers are merged into a thick one, and a lookup reads the stack from the newest down. The analogy breaks at correction: a ledger page could be amended, a table never is.

Precisely. The tables live in `$GIT_DIR/reftable/`, and `tables.list` names the ones in use, oldest first; a table name is its first and last update number and a random suffix ([reftable](#)). Each table starts with a 24-byte header: REFT, a version byte, a block size, and the range of update numbers it covers. Two stubs stay for tools that look for them: `.git/HEAD` containing `ref: refs/heads/.invalid`, and `.git/refs/heads` as a regular file. The format is declared by `extensions.refstorage = reftable` with `core.repositoryformatversion = 1`, and a Git that does not know the extension must refuse the repository. Three ways in: `git init --ref-format=reftable` or `git clone --ref-format=reftable`, the setting `init.defaultRefFormat` (Git 2.47), or `git refs migrate --ref-format=reftable` 🟡 for an existing repository (Git 2.46), which cannot migrate a repository with linked worktrees and must not run while anything else writes ([git-refs](#)).

The project's reasons for making it the default: the `files` format cannot store two refs whose names differ only in case on the case-insensitive filesystems of macOS and Windows, deleting a packed ref rewrites the whole `packed-refs` file, and multi-ref updates are not atomic; reftable compacts geometrically after every write and uses prefix compression ([BreakingChanges](#)).

Inside `.git`. `reftable/tables.list` and the tables it names, two extension lines in `config`, the two stubs, and no `logs/`, `packed-refs` or loose ref files.

See it.

```
$ git init --ref-format=reftable inference-service
Initialized empty Git repository in $LAB/ch03/reftable/inference-service/.git/
$ cd inference-service
$ git rev-parse --show-ref-format
reftable
$ cat .git/config
[extensions]
    refstorage = reftable
[core]
    repositoryformatversion = 1
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
```

```
$ ls -A .git
config
description
HEAD
hooks
info
objects
refs
reftable
# Two stub files are kept for tools that look for them. Neither holds a ref:
$ cat .git/HEAD
ref: refs/heads/.invalid
$ cat .git/refs/heads
this repository uses the reftable format
# The real answer comes from Git:
```

```
$ git symbolic-ref HEAD
refs/heads/main
```

```
$ printf 'retry_limit = 3\n' > config.toml
$ git add config.toml
$ git commit --quiet -m "Add service configuration"
$ git branch feature/batching
$ git tag -a v1.0.0 -m "Release 1.0.0"
# Refs and reflogs live in binary tables. tables.list names the tables in use, oldest first.
# A table name is first update number, last update number and a random suffix (masked here).
$ sed -E 's/-[0-9a-f]{8}\./-<random>./' .git/reftable/tables.list
0x000000000001-0x000000000003-<random>.ref
0x000000000004-0x000000000004-<random>.ref
# Compaction merges tables. Git does it on its own; git pack-refs asks for it explicitly:
$ git pack-refs --all
$ sed -E 's/-[0-9a-f]{8}\./-<random>./' .git/reftable/tables.list
0x000000000001-0x000000000004-<random>.ref
# The 24-byte header: REFT, version 1, block size, first and last update number.
$ head -c 24 .git/reftable/*.ref | xxd
00000000: 5245 4654 0100 1000 0000 0000 0000 0001  REFT.....
00000010: 0000 0000 0000 0004                ....
```

The commit wrote update 2, the branch update 3, the tag update 4. Auto-compaction had already merged updates 1 to 3 into one table; `git pack-refs --all` merged all four. The header bytes read `REFT`, version `01`, block size `00 1000` (4,096 bytes), then update numbers 1 and 4 as 64-bit integers.

```
# No logs directory and no packed-refs file. Plumbing answers as in any other repository:
$ ls -A .git
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
objects
refs
reftable
$ git for-each-ref
849357d27dcaa9267a13a0be93d3ea0b5c52ea01 commit refs/heads/feature/batching
849357d27dcaa9267a13a0be93d3ea0b5c52ea01 commit refs/heads/main
90350c8a07136a48c3b158534161e6057047243a tag refs/tags/v1.0.0
$ git reflog show feature/batching
849357d feature/batching@{0}: branch: Created from main
$ git rev-parse main
849357d27dcaa9267a13a0be93d3ea0b5c52ea01
```

No `logs/`, no `packed-refs`, no files under `refs/heads/`, and every plumbing command answers as before. The reflog of the branch is in the same tables as the branch.

```
# A conflicted merge, to see where the refs outside refs/ are kept in this format:
$ git switch --quiet feature/batching && printf 'retry_limit = 4\n' > config.toml && git commit --
quiet -am 'Allow four retries'
$ git switch --quiet main && printf 'retry_limit = 5\n' > config.toml && git commit --quiet -am 'Allow
five retries'
$ git merge feature/batching
Auto-merging config.toml
CONFLICT (content): Merge conflict in config.toml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
# MERGE_HEAD is a file, as in every repository. ORIG_HEAD and AUTO_MERGE are not files here:
$ ls -A .git
COMMIT_EDITMSG
config
description
HEAD
hooks
```

```

index
info
MERGE_HEAD
MERGE_MODE
MERGE_MSG
objects
refs
reftable
$ git for-each-ref --include-root-refs | grep -v refs/
4c07605aead786c4596df1b9212ab37962730b08 tree    AUTO_MERGE
1dd74b5d1a428941e018b4c2f6f98f0559cc5b27 commit HEAD
1dd74b5d1a428941e018b4c2f6f98f0559cc5b27 commit ORIG_HEAD
$ git merge --abort

```

This is the cleanest demonstration of section 3.10: `ORIG_HEAD` and `AUTO_MERGE` are refs and live in the tables; `MERGE_HEAD` and `FETCH_HEAD` are files in every format, which is what makes them pseudorefs.

```

# Names that differ only in case are two refs here, on a filesystem that ignores case:
$ git branch Hotfix
$ git branch hotfix
$ git branch --list "[Hh]otfix"
  Hotfix
  hotfix
# The same two commands in a files-format repository on the same disk:
$ git init --quiet ../files-repo
$ git -C ../files-repo commit --quiet --allow-empty -m "Start"
$ git -C ../files-repo branch Hotfix
$ git -C ../files-repo branch hotfix
fatal: a branch named 'hotfix' already exists
[exit status: 128]
# One rule of the files layout is kept: a ref cannot be both a name and a prefix of names.
$ git branch feature
fatal: 'refs/heads/feature/batching' exists; cannot create 'refs/heads/feature'
[exit status: 128]

```

The last refusal is deliberate: the rule that `feature` and `feature/batching` cannot both exist is a rule of the ref namespace that the `files` layout made necessary, and `reftable` keeps it so that repositories stay interchangeable (Chapter 7).

```

# The two formats that Git 3.0 plans as defaults for new repositories can be chosen today:
$ git init --quiet --object-format=sha256 --ref-format=reftable ../future-repo
$ cat ../future-repo/.git/config
[extensions]
  objectformat = sha256
  refstorage = reftable
[core]
  repositoryformatversion = 1
  filemode = true
  bare = false
  logallrefupdates = true
  ignorecase = true
  precomposeunicode = true

```

Picture. The comparison, as a table:

	files	reftable
Default	yes, in Git 2.55	planned for new repositories in Git 3.0
Names differing only in case, on macOS or Windows	collide	distinct
Deleting a ref	rewrites <code>packed-refs</code> when the ref is packed	appends a record
Several refs in one transaction	not atomic	atomic

	files	reftable
Readable with <code>cat</code>	yes, and that is the trap	no

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git refs migrate --ref-format=reftable</code>	unchanged	unchanged	same target; the file becomes a stub	same value, stored in a table	<code>refs/*</code> files, <code>packed-refs</code> and <code>logs/</code> replaced by <code>reftable/</code> ; <code>extensions.refstorage</code> set	unchanged	unchanged

The ref format is local to each repository: Lab 17.2 fetches with a `files` clone from a reftable server and migrates the clone while the server stays as it is.

In production. Adopt reftable where it solves a problem you have: branch names that collide on a case-insensitive disk (Lab 17.2), tens of thousands of refs, or automation that updates many refs at once. Before migrating a shared repository, check every tool that opens it, stop all writers, and remember that linked worktrees block the migration. The habit that costs nothing and works in both formats: read refs through plumbing.

3.14 SHA-1 with collision detection versus SHA-256

In one sentence. Git names objects with SHA-1 by default, computed by an implementation that detects attempted collision attacks, and can create repositories that use SHA-256 instead; the two kinds cannot exchange objects, and a repository that must live on GitHub stays SHA-1.

Precisely. Since Git 2.13 the default SHA-1 code is the collision-detecting variant ("SHA1_DC"), which recognises the patterns of the known attacks on SHA-1; `git version --build-options` shows it ([RelNotes 2.13](#)). SHA-256 repositories were experimental in 2.29 and are a supported format since 2.42, created with `git init --object-format=sha256`, declared by `extensions.objectformat = sha256`, and fixed for the life of the repository: changing the setting afterwards "will not work and will produce hard-to-diagnose issues" ([extensions configuration](#)). The format applies to object IDs, the index checksum and the pack checksums alike. There is "no interoperability between SHA-256 repositories and SHA-1 repositories" ([object-format-disclaimer](#)); a clone adopts the format of what it clones. The project's reasons for moving are the published attacks on SHA-1; the plan to make SHA-256 the default for new repositories in Git 3.0 is conditional on libraries, applications and forges being ready, with no plan to deprecate SHA-1 ([BreakingChanges](#)).

Inside `.git`. `extensions.objectformat = sha256` and `repositoryformatversion = 1` in `config`; object directories named by the first two of 64 hexadecimal digits; 32-byte IDs inside trees, indexes and packs.

See it.

```
# What this Git was built with:
$ git version --build-options | grep -e SHA -e default
SHA-1: SHA1_DC
SHA-256: SHA256_BLK
default-ref-format: files
default-hash: sha1
```

```
$ git init --quiet sha1-repo
$ git init --quiet --object-format=sha256 sha256-repo
$ cat sha256-repo/.git/config
[extensions]
    objectformat = sha256
```

```
[core]
  repositoryformatversion = 1
  filemode = true
  bare = false
  logallrefupdates = true
  ignorecase = true
  precomposeunicode = true
$ git -C sha1-repo rev-parse --show-object-format
sha1
$ git -C sha256-repo rev-parse --show-object-format
sha256
```

```
# The same sixteen bytes, hashed by each repository:
$ printf 'retry_limit = 3\n' > sha1-repo/config.toml
$ cp sha1-repo/config.toml sha256-repo/config.toml
$ git -C sha1-repo hash-object config.toml
f784b58423ef67be5af8d1cfd9bea5eaa26bae
$ git -C sha256-repo hash-object config.toml
ca399e7aeebdebc3e8ae90cbb2d8cab88b72cd16ebf654d483f48652356f1d68
# The object format is the same; only the hash function differs:
$ printf 'blob 16\0retry_limit = 3\n' | shasum -a 256
ca399e7aeebdebc3e8ae90cbb2d8cab88b72cd16ebf654d483f48652356f1d68 -
$ git -C sha256-repo log --oneline
ecfdab6 Add service configuration
$ git -C sha256-repo cat-file -p HEAD
tree 13e25952da64d7ebd81886b745d0ddef276fe0249a4869054efe6b27a72675b5
author Lab User <you@example.com> 1788756240 +0530
committer Lab User <you@example.com> 1788756240 +0530

Add service configuration
$ cat sha256-repo/.git/refs/heads/main
ecfdab60723aa25daa18b6d1de7fba739cdd5c2013ef47d827cc33d48261bfb5
```

The same `blob 16`, the same NUL, the same sixteen bytes; only the hash function differs, and `shasum -a 256` agrees with Git. Everything in the repository now carries 64-digit IDs.

```
# The two formats cannot exchange objects, in either direction:
$ git -C sha1-repo fetch ../sha256-repo main
fatal: mismatched algorithms: client sha1; server sha256
[exit status: 128]
$ git -C sha1-repo push ../sha256-repo main:refs/heads/from-sha1
fatal: the receiving end does not support this repository's hash algorithm
fatal: the remote end hung up unexpectedly
[exit status: 128]
$ git -C sha256-repo push ../sha1-repo main:refs/heads/from-sha256
fatal: the receiving end does not support this repository's hash algorithm
fatal: the remote end hung up unexpectedly
[exit status: 128]
# A clone adopts the format of what it clones:
$ git clone --quiet sha256-repo sha256-clone
$ git -C sha256-clone rev-parse --show-object-format
sha256
```

```
# A check that assumes forty hexadecimal digits accepts one repository and rejects the other:
$ git -C sha1-repo rev-parse HEAD | grep -E -c '^[0-9a-f]{40}$'
1
$ git -C sha256-repo rev-parse HEAD | grep -E -c '^[0-9a-f]{40}$'
0
```

GitHub, not Git. GitHub had no publicly available support for SHA-256 repositories on 1 October 2026. A private preview is reported by GitLab's blog, by a community comment and by a conference speaker, but no GitHub blog post, changelog entry or documentation page announces it, so the Phase 0 report marks it unverified ([GitLab](#), [community discussion](#)). Forgejo supports SHA-256 repositories since v7.0; GitLab

announced experimental support in August 2024. Every repository in this course that touches GitHub therefore stays SHA-1.

In production. Decide the object format at `git init` and never afterwards. Keep every repository that is pushed to GitHub in SHA-1, and treat a SHA-256 repository as an island. And never write code that assumes forty hexadecimal digits: ask `git rev-parse --show-object-format`, or accept 40 and 64; the length is a property of the repository, not of Git.

3.15 What can go wrong

Symptom	Diagnosis	Fix	Prevention
<code>fatal: index file corrupt</code>	<code>head -c 12 .git/index xxd</code> does not start with <code>DIRC</code>	<code>rm .git/index && git reset</code> ; stage again what was staged (Lab 17.1)	none needed; derived data
Every file shows as modified after a restore, a CI cache or a container copy	<code>git diff-files</code> lists everything, <code>git diff</code> shows no change	<code>git update-index --refresh</code> or any <code>git status</code>	copy repositories with Git rather than <code>cp</code> where you can
A script reads <code>.git/refs/heads/</code> and finds no file or a stale ID	the ref is in <code>packed-refs</code> , or the repository uses <code>reftable</code>	<code>git rev-parse --verify refs/heads/</code>	never read <code>.git</code> files in scripts
<code>git fsck</code> prints <code>dangling commit</code>	<code>git reflog</code> names it, or its <code>reflog</code> entry is dated in the future	nothing, or <code>git branch rescue/<x> <id></code> to keep it	treat <code>dangling</code> as information
<code>hash-path mismatch</code> or missing blob from <code>git fsck</code>	an object's content does not match its name, or the object is gone	restore it from another clone, a backup or the working tree (section 3.8)	scheduled <code>git fsck</code> ; backups through Git's transport
pack has bad object at offset, failed to read delta base object	damage inside a pack; <code>git verify-pack</code> names the offset	restore the pack from a backup or another clone, or recreate the base object and <code>repack</code> (Lab 16.2)	backups and mirrors
<code>warning: no corresponding .idx</code>	the pack index was deleted or never written	<code>git index-pack <pack></code> (Lab 16.1)	none needed; derived data
<code>git fetch</code> complains about a case-insensitive filesystem	<code>git ls-remote</code> shows names that differ only in case; <code>git rev-parse</code> and <code>git for-each-ref</code> disagree	<code>git refs migrate --ref-format=reftable</code> in the clone, or rename a branch on the server (Lab 17.2)	a branch naming rule
mismatched algorithms: client <code>sha1</code> ; server <code>sha256</code> , or the receiving end does not support this repository's hash algorithm	<code>git rev-parse --show-object-format</code> differs between the two repositories	recreate one side in the other format; there is no conversion	decide the object format at <code>git init</code>
There is no merge in progress (<code>MERGE_HEAD</code> missing)	a state file was removed by hand; a commit made now has one parent	rebuild the merge commit with <code>git commit-tree</code> and move the branch (Lab 17.3)	repair <code>.git</code> only with documented procedures

3.16 When not to use it, and dangerous edge cases

Plumbing is for repair, inspection and automation. Use porcelain for daily work, because porcelain keeps the three trees consistent and plumbing does not:

- `git update-ref` on the checked-out branch and `git symbolic-ref HEAD` move the branch and leave the index and the working tree behind; `git status` then reports every difference as your change. Use `git switch` and `git reset` unless you are rebuilding a damaged repository.
- Deleting a file under `.git` by hand is a repair only where this book or the manual says so (`index`, a pack `.idx`); deleting `MERGE_HEAD`, a ref file or anything under `objects/` is damage.
- `git hash-object -w` and `git add` write objects that stay until they are pruned, also after the file is unstaged or the commit is dropped. A secret that was ever staged is in `.git/objects` until a garbage collection removes it, two weeks after it became unreachable at the earliest (Chapter 21B).
- `git gc --prune=now`, `git prune --expire=now`, `git reflog expire --expire=now --all` and `git repack -a -d` destroy what recovery depends on: unreachable objects and reflog entries. Run them in a copy, after `git fsck --unreachable` has shown what will go, and never while another process writes to the repository (`git help gc` carries that warning).
- `git clone <path>` copies or hard-links the object files and verifies nothing. Add `--no-local` when the point of the clone is to check them.
- `extensions.objectFormat` and `extensions.refStorage` are set by `git init` and `git refs migrate`; editing them in `.git/config` corrupts the repository. A tool built on an older Git library refuses such a repository: the repository is fine, the tool is behind.
- `git fsck` racing with a writer reports missing objects; `git gc` racing with one can delete an object the writer is about to reference. Run both on quiet repositories.

3.17 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git cat-file</code> , <code>git ls-tree</code> , <code>git rev-parse</code> , <code>git for-each-ref</code> , <code>git show-ref</code> , <code>git count-objects</code> , <code>git verify-pack</code> , <code>git show-index</code> , <code>git ls-files</code> , <code>git fsck</code>	●	nothing (<code>fsck --lost-found</code> writes copies of dangling objects under <code>.git/lost-found/</code>)	—	—
<code>git hash-object -w</code>	●	adds one object	<code>git hash-object</code> without <code>-w</code>	not needed
<code>git pack-refs --all</code> , <code>git update-index --index-version 4</code>	●	the storage of refs or of the index, not their content	—	not needed
<code>git update-ref <ref> <new> [<old>]</code>	●	one ref and its reflog	<code>git rev-parse <ref></code>	<code>git update-ref <ref> "</code> <code><ref>@{1}"</code> from its reflog
<code>git update-ref -d <ref></code>	●	deletes the ref and its reflog, with no merge check: the safety net for that ref goes with it (Chapter 2, section 2.14 answers the five questions)	<code>git rev-parse <ref></code> , and note the ID	<code>git update-ref <ref> <id></code> ; the HEAD reflog or <code>git fsck</code> find the ID
<code>git symbolic-ref HEAD <ref></code>	●	which branch is current, without touching the index or the working tree	<code>git symbolic-ref HEAD</code>	<code>git symbolic-ref HEAD <previous></code> ; <code>git reflog</code>
<code>git refs migrate --ref-format=<f></code>	●	the ref storage format; values are kept	<code>--dry-run</code>	migrate back, or restore the <code>.git</code> copy you made first
<code>git gc</code>	●	packs objects and refs, expires reflog entries by age,	<code>git fsck --unreachable</code> ; <code>git</code>	within the periods nothing is lost; beyond

Command	Label	What it changes	Preview	Recovery
		deletes unreachable objects older than two weeks	<code>reflog expire --dry-run --all</code>	them, another clone or a backup
<code>git gc --prune=now, git prune, git repack -a -d</code>	●	deletes unreachable objects at once (<code>repack -a -d</code> drops the unreachable objects of the packs it replaces)	<code>git fsck --unreachable; git prune -n</code>	another clone or a backup only
<code>git reflog expire --expire=now --all</code>	●	deletes every reflog entry	<code>git reflog expire --dry-run --expire=now --all</code>	none for the entries; <code>git fsck</code> still finds the objects until they are pruned

The ● commands change the object database or the reflogs and can destroy every unreachable object and every record of where a ref used to point, the safety net of Chapter 13. Preview with the commands in the table, recover only from another copy, and use them after a deliberate history rewrite in a repository you have copied first, never as routine maintenance.

3.18 Version notes

Topic	Older behavior	Current behavior	Since	Recommended
Reflog entries dated in the future	<code>git fsck</code> used every reflog entry as a starting point	entries newer than the moment <code>fsck</code> starts are skipped; <code>git gc</code> is unaffected	Git 2.53.0 (section 3.8)	read <code>dangling</code> as information
"pseudoref"	any special file under <code>.git</code>	only <code>FETCH_HEAD</code> and <code>MERGE_HEAD</code> ; the rest are root refs	Git 2.46	say "root ref"
Ref storage	loose files and <code>packed-refs</code> only	<code>reftable</code> available; <code>git refs migrate</code> converts	2.45; migrate 2.46; <code>init.defaultRefFormat</code> 2.47	plumbing for all ref reads
Object format	SHA-1 only	SHA-256 repositories supported, without interoperability	experimental 2.29; supported 2.42	SHA-1 for anything on GitHub
Automatic maintenance	<code>git gc --auto</code> from porcelain commands	the geometric strategy of <code>git maintenance</code>	2.54	Chapter 26
Git 3.0 plans for new repositories	—	SHA-256 and <code>reftable</code> as defaults, <code>main</code> as the initial branch; conditional, unreleased, no date in the official documentation	announced; 2.98 and 2.99 come first	opt in early with <code>--object-format</code> and <code>--ref-format</code> in throwaway repositories

3.19 Practice

Labs 16.1 and 16.2 in `lab-manual/m16-object-database.md`: loose objects become a pack and a missing pack index is rebuilt; a pack listing, its delta chain, and the recovery from one damaged byte. Labs 17.1 to 17.3 in `lab-manual/m17-index-refs-gitdir.md`: the index as a data structure and the rebuild of a corrupt one; `files` against `reftable` and a case-clash; a tour of every special ref and the repair of a merge whose `MERGE_HEAD` was deleted. Then replay the demos with `labs/run ch03/<name>` and change one thing in each. Answers: `solutions/m16-lab-answers.md`, `solutions/m17-lab-answers.md`.

3.20 Interview questions

1. Compute the object ID of a blob by hand. Which bytes are hashed, and why does the compression level not matter?
2. A tree entry has mode `160000`. What is it, what object type does the ID name, and why can `git cat-file` not show it?
3. What does a commit ID commit you to? Explain why changing a commit message three commits back changes the ID of `HEAD`.
4. We have six versions of a 16 MB file. What does the pack contain, which version is stored whole, and what happens when a byte in that version is damaged?
5. `git fsck` says `dangling commit`. Walk me through what you check before you decide whether anything is at risk.
6. Why can a corrupt object survive `git cat-file` and `git status` but not a clone over the network?
7. A deploy script reads `.git/refs/heads/main`. Name two situations in which that file does not describe the branch, and the command that always does.
8. What is the difference between `ORIG_HEAD`, `FETCH_HEAD` and `refs/heads/main` as far as garbage collection is concerned?
9. Someone copied a repository with `cp -R` and now every file is "modified". Explain the mechanism and the one-command fix.
10. When would you migrate a repository to reftable, what breaks if you do it carelessly, and what stays the same?
11. Why do we keep our repositories in SHA-1 although Git supports SHA-256, and what would have to change for that to change?

3.21 Sources

Primary sources. The local manual pages for the Git you run: `git help gitformat-loose`, `gitformat-pack`, `gitformat-index`, `gitformat-signature`, `gitrepository-layout`, `gitdatamodel`, `gitglossary`, `gitrevisions`, and the pages of every command used above. Online, at the 2.56.0 tag: [gitdatamodel](#), [gitformat-loose](#), [gitformat-pack](#), [gitformat-index](#), [gitformat-signature](#), [gitrepository-layout](#), [glossary](#), [revisions](#), [git-refs at 2.55](#), [the reftable design](#), [repository format versions](#), [hash-function-transition](#), [BreakingChanges](#), the release notes for [2.13](#), [2.29](#), [2.42](#), [2.45](#), [2.46](#), [2.51](#) and [2.53](#), and the commit ["fsck: snapshot default refs before object walk"](#).

Secondary sources. Pro Git, chapter 10: [objects](#), [packfiles](#) and [maintenance and recovery](#); frozen since May 2024, so it predates reftable and the glossary change. The GitHub Blog, [Git's database internals, part I](#). The Phase 0 report of this course for the hosting facts of section 3.14.

Videos. Derrick Stolee, ["Git Internals: a Database Perspective"](#) (Git Merge 2022, 27 min): packfiles and the storage layer; current. John Britton, ["Git Internals"](#) (CS50, 2018, 58 min): the object model, correct and clear; `master`, `checkout`, SHA-1 only, almost nothing on packfiles. Patrick Steinhardt, ["Reftable Backend"](#) (2025, 20 min) and Emily Shaffer, ["SHA-256 at a Hyperscaler"](#) (Git Merge 2026, 32 min): both assume this chapter's level; the second had almost no views when the report checked it.

Further reading. James Coglan, *Building Git*, teaches the formats by reimplementing them; ["Write yourself a Git"](#) does the same in Python. Both predate reftable.

Chapter 4: The Working Tree

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch04/`.

4.1 Why this matters

Three questions a CTO can ask after an ordinary week:

1. "We put `.env` in `.gitignore` months ago. Why did the secret scanner find the key in the repository last night?"
2. "The same commit runs on every laptop and fails in CI with a missing module. What is different?"
3. "One cleanup command deleted two days of labelled evaluation data. Can Git bring it back?"

None of these is about commits, branches or GitHub. All three are about the working tree, the part of Git that most tutorials treat as too plain to explain. The answers are: an ignore rule never applies to a path that is already tracked (section 4.6); the laptops have a case-insensitive filesystem and the CI runner does not (section 4.12); and `git clean -fdx` deletes files that Git never had a copy of, so no (section 4.14).

Hold on to one idea through the chapter. A version of a file can live in three places: in the working tree, in the index, or in a commit. The working tree is the only one of the three with no history and no second copy. Git can rebuild tracked files in it from the index or from a commit. It cannot rebuild content that never left it.

4.2 What the working tree is

In one sentence. The working tree is the directory of ordinary files that you edit, build and run: one checked-out version of the project, plus whatever else is lying in that directory.

Analogy. A workbench in front of a parts cabinet. The cabinet (the object database) holds labelled parts that never change. The bench is where you assemble and modify. Git can lay any stored version out on the bench again, but shavings and half-built parts that never went into the cabinet exist only on the bench. The analogy breaks in two places: Git does not watch the bench, it looks only when a command asks; and the bench may hold things the cabinet has never seen (untracked files), which Git will list but does not protect.

Precisely. The glossary defines the working tree as "the tree of actual checked out files", which "normally contains the contents of the HEAD commit's tree, plus any local changes that you have made but not yet committed" ([gitglossary](#)). It starts at the directory that contains `.git`, called the top level, and every path Git prints is relative to that directory unless a command says otherwise. A repository can have no working tree (a bare repository, the form used on servers and for the local "remotes" in later labs), one (the normal case), or several (linked worktrees, Chapter 25, Worktrees).

Inside `.git`. Nothing. The working tree is not stored in `.git` at all. The only trace of it there is in `.git/index`: for each tracked path the index remembers which blob the file last matched, and the file's size and timestamps, so that Git can find changed files without reading every file (Chapter 5, section 5.14).

See it. `git rev-parse` answers "where am I" questions. From a subdirectory, `--show-prefix` prints the path from the top level to the current directory.

```

$ git init support-bot
Initialized empty Git repository in $LAB/ch04/worktree-basics/support-bot/.git/
$ cd support-bot
$ git rev-parse --show-toplevel
$LAB/ch04/worktree-basics/support-bot
$ git rev-parse --git-dir
.git
$ cd src
$ git rev-parse --show-toplevel --show-prefix --git-dir
$LAB/ch04/worktree-basics/support-bot
src/
$LAB/ch04/worktree-basics/support-bot/.git
$ git rev-parse --is-inside-work-tree
true
$ cd ..

```

A bare repository has the `.git` content and nothing to check out into, so any command that needs a working tree refuses:

```

# A bare repository is a repository without a working tree.
$ git init --bare ../central.git
Initialized empty Git repository in $LAB/ch04/worktree-basics/central.git/
$ git -C ../central.git rev-parse --is-bare-repository
true
$ git -C ../central.git status
fatal: this operation must be run in a work tree
[exit status: 128]

```

Now the asymmetry that the rest of the chapter depends on. Delete two tracked directories and one untracked file, then ask Git to rebuild the working tree:

```

# Tracked content can be rebuilt from the index. Untracked content cannot.
$ echo 'LLM_API_KEY=lab-secret-0001' > .env
$ rm -r src config .env
$ git status --short
D config/settings.yaml
D src/app.py
D src/retriever.py
$ git restore .
$ find . -path ./git -prune -o -type f -print | sort
./config/settings.yaml
./src/app.py
./src/retriever.py
$ git restore .env
error: pathspec '.env' did not match any file(s) known to git
[exit status: 1]

```

The three tracked files came back from the index. `.env` did not: Git has never heard of it, and says so.

Picture.

working tree		index (.git/index)		HEAD commit
+-----+		+-----+		+-----+
plain files you	add	one entry per	commit	a full snapshot:
edit, build and run	---->	tracked path:	---->	tree and blobs,
		mode, blob ID, path		immutable
plus untracked and	<----		<----	
ignored files	restore		restore	
+-----+		+-----+		+-----+
no history,		one proposed snapshot	--staged	every committed snapshot
no second copy				

In production. Any tool that reads the directory sees the working tree, not the commit: a container build that copies the project directory, a packaging step, a test runner, an evaluation script. A clean `git status` does not mean the

directory equals the commit, because ignored files are present and invisible: a local `.env`, compiled caches, a stale `build/`. When a laptop build and a CI build of the same commit behave differently, list what the laptop has that the commit does not with `git status --ignored` (section 4.3) before looking anywhere else. A build that must contain exactly the commit should run in a fresh clone or a fresh worktree.

4.3 Tracked, untracked, ignored

In one sentence. Every path in the working tree is in exactly one of three categories, and the index decides which: tracked if it has an index entry, ignored if it has none and matches an ignore pattern, untracked otherwise.

Precisely.

Category	Defined by	git status	Listed by	Touched by git add .	Touched by git clean
Tracked	The path has an entry in the index	Shown when it differs from the index or from HEAD	<code>git ls-files</code>	Yes, updated	Never
Untracked	No index entry, no ignore pattern matches	Shown under "Untracked files"	<code>git ls-files --others --exclude-standard</code>	Yes, added	Yes
Ignored	No index entry, an ignore pattern matches	Hidden unless <code>--ignored</code>	<code>git ls-files --others --ignored --exclude-standard</code>	No (<code>-f</code> to force)	Only with <code>-x</code> or <code>-X</code>

Two consequences follow from the definitions. "Ignored" is a kind of untracked, so a tracked path can never be ignored. And "tracked" says nothing about commits: a path becomes tracked the moment `git add` writes its index entry.

See it. The repository has four committed files and a three-line `.gitignore`. Four more paths were then created in the working tree.

```
$ cat .gitignore
.env
__pycache__/
data/
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes/

nothing added to commit but untracked files present (use "git add" to track)
```

Status shows one untracked directory and says nothing about `.env`, `data/` or the bytecode cache. `git ls-files` lists each category exactly:

```
# Tracked: every path that has an entry in the index.
$ git ls-files
.gitignore
config/settings.yaml
src/app.py
src/retriever.py
# Untracked and not ignored.
$ git ls-files --others --exclude-standard
notes/ideas.md
# Untracked and ignored.
$ git ls-files --others --ignored --exclude-standard
.env
```

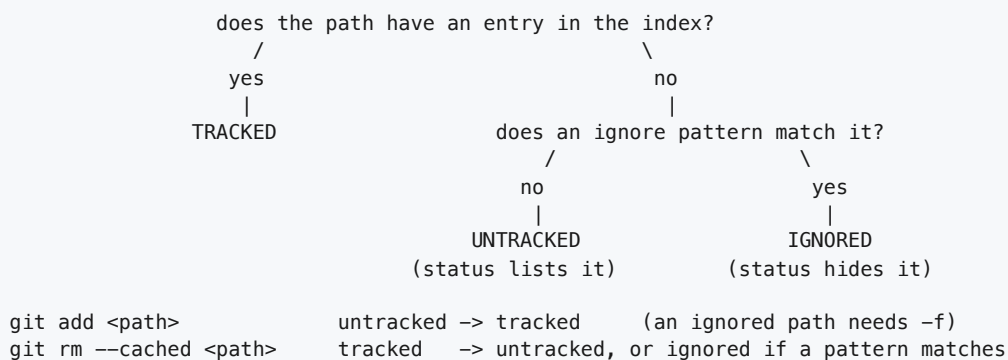
```
data/tickets.jsonl
src/__pycache__/app.cpython-314.pyc
```

Moving between categories is always an index operation:

```
# Untracked becomes tracked the moment the path gets an index entry.
$ git add notes/ideas.md
$ git ls-files notes
notes/ideas.md
# An ignored path is refused unless you force it.
$ git add .env
The following paths are ignored by one of your .gitignore files:
.env
hint: Use -f if you really want to add them.
hint: Disable this message with "git config set advice.addIgnoredFile false"
[exit status: 1]
$ git status --short --ignored
A  notes/ideas.md
!! .env
!! data/
!! src/__pycache__/
```

In the short format `A` is a staged new file and `!!` marks ignored paths. The refusal to add `.env` is a courtesy of `git add`, not a lock: `git add -f .env` would add it.

Picture.



4.4 The anatomy of `git status`

In one sentence. `git status` is two comparisons and one scan: HEAD against the index, the index against the working tree, and a scan of the working tree for paths that have no index entry.

Analogy. A warehouse audit with three clipboards: what the last signed inventory says (HEAD), what the next inventory will say (the index), and what is on the shelves (the working tree). Status reads out the differences between neighbouring clipboards. The analogy breaks because the middle clipboard is not a list of differences: it is a complete inventory, and status computes the differences each time you ask.

Precisely. The manual describes the same three parts: "paths that have differences between the index file and the current HEAD commit, paths that have differences between the working tree and the index file, and paths in the working tree that are not tracked by Git" ([git-status](#)). The first is what `git commit` would record. The second and third are what you could still add.

See it. One repository, one path in each interesting state:

```
$ git status
On branch main
Changes to be committed:
```

```

    (use "git restore --staged <file>..." to unstage)
    modified:    config/settings.yaml
    renamed:     README.md -> docs/README.md
    deleted:     docs/old-notes.md
    new file:    src/prompts.py
    modified:    src/retriever.py

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    deleted:    requirements.txt
    modified:    src/app.py
    modified:    src/retriever.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    scratch/

```

src/retriever.py appears twice. That is not a contradiction: it was staged and then edited again, so both comparisons find a difference. The short format puts the two comparisons in two columns:

```

$ git status --short
M config/settings.yaml
R README.md -> docs/README.md
D docs/old-notes.md
D requirements.txt
M src/app.py
A src/prompts.py
MM src/retriever.py
?? scratch/
$ git status --short --branch
## main
M config/settings.yaml
R README.md -> docs/README.md
D docs/old-notes.md
D requirements.txt
M src/app.py
A src/prompts.py
MM src/retriever.py
?? scratch/

```

The first column (X) is comparison 1, the second (Y) is comparison 2. You can reproduce each column with the diff command that performs the same comparison:

```

# Comparison 1: HEAD against the index. This is the X column.
$ git diff --cached --name-status
M      config/settings.yaml
R100   README.md          docs/README.md
D      docs/old-notes.md
A      src/prompts.py
M      src/retriever.py
# Comparison 2: the index against the working tree. This is the Y column.
$ git diff --name-status
D      requirements.txt
M      src/app.py
M      src/retriever.py
# Third pass: paths in the working tree that have no index entry.
$ git ls-files --others --exclude-standard
scratch/notes.py
scratch/try.py

```

XY	Meaning	In the transcript
M	Modification staged; working tree matches the index	config/settings.yaml
M	Modified in the working tree, not staged	src/app.py
MM	Staged, then modified again	src/retriever.py

XY	Meaning	In the transcript
A	New file staged	src/prompts.py
D	Deletion staged	docs/old-notes.md
D	File deleted in the working tree, deletion not staged	requirements.txt
R	Rename staged (detected, section 4.9)	README.md -> docs/README.md
??	Untracked	scratch/
!!	Ignored, shown only with <code>--ignored</code>	section 4.3
UU, AA, DU and others	Unmerged during a conflict	Chapter 5, section 5.13

For scripts there are two stable formats. `--porcelain` (version 1) looks like the short format but is guaranteed not to change between Git versions or with user configuration, and always prints paths relative to the top level. `--porcelain=v2` adds the file modes and object IDs that status compared:

```
$ git status --porcelain
M config/settings.yaml
R README.md -> docs/README.md
D docs/old-notes.md
D requirements.txt
M src/app.py
A src/prompts.py
MM src/retriever.py
?? scratch/

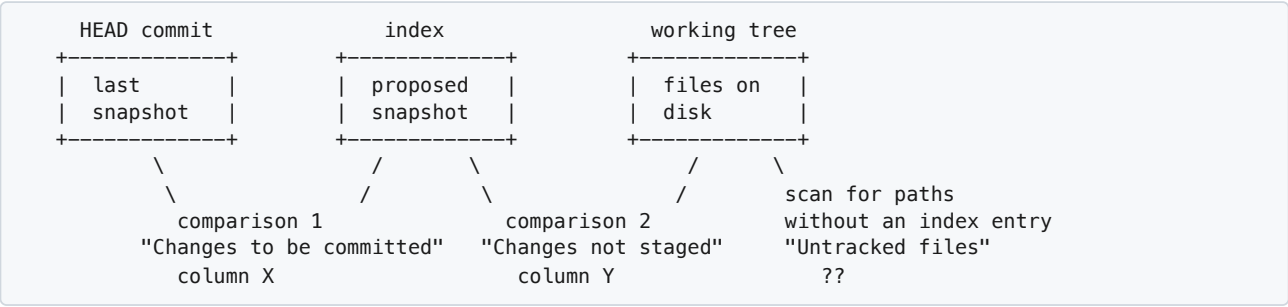
$ git status --porcelain=v2 --branch
# branch.oid 7b557e0425ff4598b69acdedec3ca379116db157
# branch.head main
1 M. N... 100644 100644 100644 4da99e38a05a2a7b436aeea17b946e0288949413
0217758d92bf9cd7d8ebba62175a33a9ef51ba2f config/settings.yaml
2 R. N... 100644 100644 100644 384f3ba8db7735cfcccc8a6644808c37adbe6fba
384f3ba8db7735cfcccc8a6644808c37adbe6fba R100 docs/README.md README.md
1 D. N... 100644 000000 000000 4dddde5373611e10f9ec6bc328a25ae216552487
00000000000000000000000000000000 docs/old-notes.md
1 .D N... 100644 100644 000000 fbce96793add271886cc2acbd6b5bca38d960391
fbce96793add271886cc2acbd6b5bca38d960391 requirements.txt
1 .M N... 100644 100644 100644 9069bfc5af86dd8daf593b43de35ae691a7e682f
9069bfc5af86dd8daf593b43de35ae691a7e682f src/app.py
1 A. N... 000000 100644 100644 0000000000000000000000000000000000000000
5e83731ea002effcf7f6c6c4a2b565cbf889abd8 src/prompts.py
1 MM N... 100644 100644 100644 67b3f03b8c4ab448c68cda278050d05e9c1865f0
ecfe75fa97c452977d0fa9decbbca215f5633779 src/retriever.py
? scratch/
```

Read one version 2 line: `1 .M N... 100644 100644 100644 9069bfc... 9069bfc... src/app.py`. The three modes are HEAD, index and working tree. The two object IDs are HEAD and index. They are equal, so nothing is staged for this path, and there is no third ID because status compares the working tree file with the index entry and does not need to name its content. Add `-z` when file names may contain spaces or newlines: entries are then separated by NUL bytes and never quoted.

Three details matter in practice:

- **Untracked directories are collapsed.** Status printed `scratch/`, not the two files inside it. `--untracked-files=all` (`-uall`) lists every file, and `--untracked-files=no` (`-uno`) skips the scan, which is the expensive part in a large working tree. Run `labs/run ch04/status-anatomy` to see both.
- **Status writes.** By default `git status` refreshes the cached file information in the index and writes the index back, taking a lock while it does. A tool that runs status in the background can therefore collide with your foreground command. The manual's advice for such tools is `git --no-optional-locks status`.
- **Status is local.** Lines such as "Your branch is up to date with 'origin/main'" compare with the last fetched state, not with the server (Chapter 12, Remote Operations).

Picture.



In production. A CI job that must prove "the build left the tree clean" (generated code is committed, the formatter changed nothing) should test `git status --porcelain` for empty output. `git diff --exit-code` is not enough: it performs comparison 2 only, so a generator that created a new, untracked file passes unnoticed.

4.5 Ignore rules

In one sentence. An ignore pattern tells the commands that scan the working tree for untracked paths to leave matching paths out.

Analogy. A "do not list" note taped to the warehouse door for the people doing the audit. It changes what the auditors report. It does not move, lock or hide anything, and it has no effect on items that are already in the inventory. That last clause is where most people's mental model breaks, and section 4.6 is about it.

Precisely. Patterns come from four sources. From highest to lowest precedence ([gitignore](#)):

Source	Scope	Shared with the team	Use it for
Command-line options, for commands that take them (for example <code>git clean -e</code>)	One command	No	A one-off exception
<code>.gitignore</code> in the path's directory or any parent directory; a deeper file overrides a shallower one	That directory and below	Yes, it is a tracked file	Build output, caches, datasets, secrets files: what every developer should ignore
<code>.git/info/exclude</code> (formally <code>\$GIT_COMMON_DIR/info/exclude</code>)	This clone	No	Your own scratch files in this repository
The file named by <code>core.excludesFile</code> ; default <code>\$XDG_CONFIG_HOME/git/ignore</code> , or <code>~/.config/git/ignore</code> when that variable is not set	Every repository on this machine	No	Editor and operating-system noise such as <code>.DS_Store</code> and <code>.idea/</code>

Within one source, the last matching pattern decides.

Pattern form	Meaning	Example in the demo
No slash, or only a trailing slash	Matches at any depth below the <code>.gitignore</code>	<code>*.log</code> matches <code>logs/app.log</code>
Trailing slash	Matches directories only	<code>__pycache__/</code>
Slash at the start or in the middle	Anchored to the directory of the <code>.gitignore</code>	<code>/build/</code> matches <code>build/</code> , not <code>src/build/</code>
<code>*</code> , <code>?</code> , <code>[a-z]</code>	Wildcards that do not match a <code>/</code>	<code>data/*</code>
<code>**/x</code> , <code>x/**</code> , <code>a/**/b</code>	Any number of directories	<code>models/**/*.bin</code>
Leading <code>!</code>	Negation: re-include a path that an earlier pattern excluded	<code>!data/README.md</code>

Pattern form	Meaning	Example in the demo
Leading #	Comment; write \# for a literal hash	

See it. The demo's `.gitignore`:

```
$ cat -n .gitignore
1 # caches and logs, at any depth
2 __pycache__/
3 *.log
4
5 # build output, only at the top level
6 /build/
7
8 # datasets stay out; the README that documents them stays in
9 data/*
10 !data/README.md
11
12 # secrets, with one documented exception
13 .env*
14 !.env.example
15
16 # model weights under models/, at any depth
17 models/**/*.bin
```

`git check-ignore -v` is the debugger for ignore rules. For each path it prints the source file, the line number and the pattern that decided:

```
# Output: <source>:<line>:<pattern> TAB <path>. With -n, a path that matches no pattern prints "::.
$ git check-ignore -v -n src/__pycache__/app.cpython-314.pyc logs/app.log build/out.txt
src/build/helper.py
.gitignore:2:__pycache__/      src/__pycache__/app.cpython-314.pyc
.gitignore:3:*.log           logs/app.log
.gitignore:6:/build/        build/out.txt
::      src/build/helper.py
$ git check-ignore -v -n data/tickets.jsonl data/README.md .env .env.local .env.example
.gitignore:9:data/*          data/tickets.jsonl
.gitignore:10:!data/README.md data/README.md
.gitignore:13:.env*          .env
.gitignore:13:.env*          .env.local
.gitignore:14:!.env.example  .env.example
$ git check-ignore -v -n models/v1/model.bin models/v1/checkpoints/step-100.bin models/v1/card.md
.gitignore:17:models/**/*.bin models/v1/model.bin
.gitignore:17:models/**/*.bin models/v1/checkpoints/step-100.bin
::      models/v1/card.md
```

Read it line by line. `/build/` on line 6 ignores the top-level `build/` and not `src/build/helper.py`, because the leading slash anchors it. `.env.example` matched line 13 and then line 14; the later line wins, and because that line is a negation the path is not ignored. A match on a `!` pattern means "not ignored", so read the pattern, not only the fact that there is output.

Negation has a hard limit. The manual states that "it is not possible to re-include a file if a parent directory of that file is excluded", because Git does not look inside an excluded directory at all. Change `data/*` (ignore the things in data) to `data/` (ignore the directory):

```
# Edit line 10 from "data/*" to "data/": the directory itself is now excluded.
$ grep -n 'data' .gitignore
8:# datasets stay out; the README that documents them stays in
9:data/
10:!data/README.md
$ git check-ignore -v data/README.md
.gitignore:9:data/      data/README.md
$ git status --short --untracked-files=all data
```

Line 10 still says `!data/README.md`, and it no longer has any effect. To keep one file inside an otherwise ignored directory, exclude the directory's contents (`data/*`), not the directory.

Precedence between files. A `.gitignore` deeper in the tree overrides the ones above it:

```
# A .gitignore in a subdirectory overrides the ones above it, for paths below it.
$ printf '!*.log\n' > experiments/.gitignore
$ git check-ignore -v -n experiments/run1/train.log logs/app.log
experiments/.gitignore:1:!.log experiments/run1/train.log
.gitignore:3:*.log      logs/app.log
```

The two personal sources never travel with the repository. In the lab, `XDG_CONFIG_HOME` points into the sandbox; on your machine the default global file is `~/.config/git/ignore`:

```
# Patterns for this clone only, never committed: .git/info/exclude
$ printf 'scratch/\n' >> .git/info/exclude
# Patterns for every repository on this machine: the file named by core.excludesFile.
# Its default is $XDG_CONFIG_HOME/git/ignore, or ~/.config/git/ignore when that variable is unset.
$ mkdir -p "$XDG_CONFIG_HOME/git"
$ printf '.DS_Store\n.idea/\n*.ipynb\n' > "$XDG_CONFIG_HOME/git/ignore"
$ git check-ignore -v scratch/try.py .DS_Store notebooks/scratch.ipynb
.git/info/exclude:7:scratch/      scratch/try.py
$LAB/ch04/gitignore-patterns/home/.config/git/ignore:1:.DS_Store      .DS_Store
$LAB/ch04/gitignore-patterns/home/.config/git/ignore:3:*.ipynb      notebooks/scratch.ipynb
```

And the repository outranks both personal files, so a project can insist on tracking something that one developer ignores globally:

```
# The repository wants one notebook tracked. A per-directory .gitignore outranks the personal files.
$ printf '\n!notebooks/report.ipynb\n' >> .gitignore
$ git check-ignore -v -n notebooks/report.ipynb notebooks/scratch.ipynb
.gitignore:19:!notebooks/report.ipynb      notebooks/report.ipynb
$LAB/ch04/gitignore-patterns/home/.config/git/ignore:3:*.ipynb      notebooks/scratch.ipynb
$ git status --short --untracked-files=all notebooks
?? notebooks/report.ipynb
```

Inside `.git`. Ignore rules leave no trace in the index or the object database. `.gitignore` is an ordinary tracked file; `.git/info/exclude` is a plain file that `git init` creates with a few comment lines; neither is consulted for a path that has an index entry.

In production. For an AI/ML repository the shared `.gitignore` usually starts from a language template and adds what the template lacks. GitHub's Python template covers interpreter, packaging, environment and tool caches and has nothing specific to machine learning ([Python.gitignore](#)). The Phase 0 research for this course lists the usual additions: data directories, checkpoint and export formats (`*.pt`, `*.pth`, `*.ckpt`, `*.safetensors`, `*.onnx`, `*.gguf`), experiment-tracker output (`wandb/`, `mlruns/`) and `.env` files, with editor and operating-system noise kept in each developer's global ignore file. Two rules keep the file honest: put a pattern in the shared file only if every developer and CI should ignore the path, and never treat the file as a security control. It hides paths from `git status`. It does not stop `git add -f`, and it does nothing about history.

GitHub, not Git. The `.gitignore` template that GitHub offers when you create a repository comes from the public [github/gitignore](#) collection. It is a starting file that GitHub commits for you. Git itself ships no templates and attaches no meaning to the choice.

4.6 The already-tracked trap

This is the eighth most-voted Git question on Stack Overflow ("How do I make Git forget about a file that was tracked, but is now in .gitignore?", 8,647 votes on 1 October 2026 according to the Phase 0 report), and it has one cause.

See it. The first commit of a project is made with `git add .` before any `.gitignore` exists, so `.env` is committed. The rule arrives one commit later:

```
$ echo '.env' > .gitignore
$ git add .gitignore
$ git commit -m "Ignore local environment file"
[main ff0b08f] Ignore local environment file
1 file changed, 1 insertion(+)
create mode 100644 .gitignore
$ echo 'LLM_API_KEY=lab-secret-0002' > .env
$ git status --short
M .env
```

`.env` is listed in `.gitignore` and still shows as modified. The debugger explains why:

```
# check-ignore reports nothing for a tracked path: ignore rules are not consulted for it.
$ git check-ignore -v .env
[exit status: 1]
# Ask the same question with the index left out of it.
$ git check-ignore -v --no-index .env
.gitignore:1:.env      .env
[exit status: 0]
# List every path that is tracked and also matches an ignore pattern.
$ git ls-files --cached --ignored --exclude-standard
.env
```

`check-ignore` prints nothing and exits with status 1 for a tracked path, because ignore rules are not consulted for it. `--no-index` asks the same question with the index left out and shows that the pattern itself is fine. The last command is the detector to remember: `git ls-files --cached --ignored --exclude-standard` (short: `-ci --exclude-standard`) lists every path that is tracked and also matches an ignore pattern. In a healthy repository it prints nothing.

Observed behavior	: <code>.env</code> is listed in <code>.gitignore</code> , yet <code>git status</code> reports it as modified and it keeps appearing in commits.
Git state	: <code>.env</code> has an entry in the index. It was added before the rule existed.
Mechanism	: Ignore patterns are consulted only for paths that have no index entry: when <code>status</code> lists untracked files and when <code>add</code> walks a directory. A tracked path is compared with its index entry. No pattern is checked.
Root cause	: The file was tracked first and ignored second. Tracking wins.
Why Git does this	: The manual defines the purpose of ignore files narrowly: "to ensure that certain files not tracked by Git remain untracked". What is tracked changes only when a command says so (<code>git add</code> , <code>git rm</code>).
Correct fix	: <code>git rm --cached <path></code> , commit, keep the pattern in <code>.gitignore</code> . Treat any secret in the file as leaked and rotate it.
Prevention	: Write <code>.gitignore</code> before the first " <code>git add .</code> "; read <code>git status</code> before the first commit; make CI fail when " <code>git ls-files -ci --exclude-standard</code> " prints anything.

The fix. 🟡 `git rm --cached` removes the index entry and leaves the file on disk:

```
$ git rm --cached .env
rm '.env'
$ git status --short --ignored
D .env
!! .env
$ git commit -m "Stop tracking .env"
[main b0ddc57] Stop tracking .env
1 file changed, 1 deletion(-)
delete mode 100644 .env
$ git ls-files
```

```
.gitignore
src/app.py
$ cat .env
LLM_API_KEY=lab-secret-0002
```

Between the `rm` and the commit, status shows the path twice: `D` because the index no longer has what HEAD has, and `!!` because the file on disk is now an untracked path that matches a pattern. After the commit the file is still on disk with your content, and later edits are invisible to status.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git rm --cached <path></code>	unchanged	entry for <path> removed	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git commit</code> afterwards	unchanged	unchanged	new commit whose tree lacks <path>	moves to the new commit	reflogs of HEAD and the branch gain an entry	unchanged until you push	unchanged until you push

What the fix does not do. Two things, and both have caused incidents.

First, it does not remove the file from history. Every earlier commit still contains it:

```
# The fix changed the index and the new commit. It did not change any earlier commit.
$ git log --oneline -- .env
b0ddc57 Stop tracking .env
cd1e384 Add service skeleton
$ git show HEAD~1:.env
LLM_API_KEY=lab-secret-0001
```

If the file held a credential, the credential is in every clone and, once pushed, on the server. Removing the path from the next commit changes nothing about that. The order of operations for a leaked secret is to revoke or rotate it first; Chapter 21 (Security) covers the full response.

Second, the fix is a deletion commit, and Git applies deletions to working trees. A teammate who cloned while `.env` was tracked loses her copy when she pulls:

```
# Asha cloned while .env was still tracked. Her local runs read that file.
$ cd ../asha-clone
$ ls -A
.env
.git
.gitignore
src
$ git pull
From $LAB/ch04/ignore-tracked-trap/support-bot
ff0b08f..b0ddc57  main      -> origin/main
* [new branch]    release-1.0 -> origin/release-1.0
Updating ff0b08f..b0ddc57
Fast-forward
 .env | 1 -
 1 file changed, 1 deletion(-)
 delete mode 100644 .env
$ ls -A
.git
.gitignore
src
# The content is still in the previous commit, so she can get her file back as an ignored file.
$ git restore --source=HEAD~1 .env
$ git status --short --ignored
!! .env
```

From Git's point of view this is correct: the path was tracked in her HEAD, unmodified, and is absent from the new commit, so it is removed like any other deleted file. She can take the content back out of the previous commit with `git restore --source=HEAD~1 .env`; the file is then untracked and ignored, which is the state you wanted for everyone. If she had edited her `.env`, the pull would have stopped with "Your local changes to the following files would be overwritten by merge" instead. Announce a fix of this kind before you push it, with those two commands in the message.

A third consequence, for ignored files in general, is in section 4.16.

4.7 Restoring paths: `git restore`

In one sentence. 📌 `git restore <path>` overwrites files in the working tree with a stored version: the version in the index by default, the version in a commit with `--source`.

Analogy. "Revert to saved" in an editor, where "saved" means "last staged". The analogy breaks twice: there is no undo afterwards, and "saved" is the index, not the last commit, which surprises people who have staged something and forgotten.

Precisely. Two choices define a restore ([git-restore](#)):

- **Where the content comes from.** Without `--source`: the index, or HEAD if `--staged` is given. With `--source=<tree>`: that commit or tree.
- **Where it is written.** The working tree by default (`--worktree`, `-W`); the index with `--staged` (`-S`); both when both options are given.

The command needs at least one pathspec, and the default mode is "no overlay": a tracked path that does not exist in the source is removed, so that the destination matches the source exactly.

Inside `.git`. Restoring the working tree from the index reads the blob that the index entry names and writes it to disk. No object is created and no ref moves, so no reflog records the event. The content that was overwritten is not saved anywhere.

See it. One staged line, one unstaged line:

```
# Stage one change, then make a second change that is not staged.
$ echo 'max_tokens: 512' >> config/settings.yaml
$ git add config/settings.yaml
$ echo 'debug: true' >> config/settings.yaml
$ git status --short
MM config/settings.yaml
# Default source is the index: the unstaged line goes, the staged line stays.
$ git restore config/settings.yaml
$ cat config/settings.yaml
model: small-v1
top_k: 5
max_tokens: 512
$ git status --short
M  config/settings.yaml
```

The file now equals the index, not HEAD: the staged `max_tokens` line survived and `debug: true` is gone. With `--source` the content comes from a commit and, without `--staged`, only the working tree is written:

```
# With --source, the content comes from a commit. Only the working tree is written.
$ git restore --source=HEAD config/settings.yaml
$ cat config/settings.yaml
model: small-v1
top_k: 5
$ git status --short
MM config/settings.yaml
```

```
# Add --staged to write the index as well. Now all three trees agree.
$ git restore --source=HEAD --staged --worktree config/settings.yaml
$ git status --short
```

After the first command the status is `MM`: the index still holds the staged line, and the working tree now lacks it, so both comparisons differ. `--staged --worktree` writes both places and the path is clean. Any commit can be the source:

```
$ git log --oneline
3139437 Raise top_k to 5
aab6b8e Add service skeleton
$ git restore --source=HEAD~1 config/settings.yaml
$ cat config/settings.yaml
model: small-v1
top_k: 3
$ git status --short
M config/settings.yaml
$ git diff
diff --git a/config/settings.yaml b/config/settings.yaml
index 4da99e3..9b521fd 100644
--- a/config/settings.yaml
+++ b/config/settings.yaml
@@ -1,2 +1,2 @@
 model: small-v1
-top_k: 5
+top_k: 3
$ git restore config/settings.yaml
```

Older scripts use `git checkout` for the same job, with one difference that matters:

```
# The older spelling. With a commit named, "git checkout" writes the index too.
$ git checkout HEAD~1 -- config/settings.yaml
$ git status --short
M config/settings.yaml
$ git checkout HEAD -- config/settings.yaml
$ git status --short
```

`git checkout <commit> -- <path>` writes the index as well as the working tree (M above), where `git restore --source=<commit> <path>` wrote only the working tree (M in the previous transcript).

Task	Current command	Older spelling
Discard unstaged changes	<code>git restore <path></code>	<code>git checkout -- <path></code>
Take a path from a commit into index and working tree	<code>git restore --source=<commit> --staged --worktree <path></code>	<code>git checkout <commit> -- <path></code>
Take a path from a commit into the working tree only	<code>git restore --source=<commit> <path></code>	none in one step
Unstage (Chapter 5)	<code>git restore --staged <path></code>	<code>git reset HEAD <path></code>

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git restore <path></code>	<code><path></code> overwritten with the index version; unstaged changes destroyed	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git restore --source=<commit> <path></code>	<code><path></code> overwritten with the version in <code><commit></code> ; tracked paths under <code><path></code>	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
	that <commit> lacks are deleted						
<code>git restore --source=<commit> --staged --worktree <path></code>	as above	entries under <path> replaced by those of <commit>	unchanged	unchanged	unchanged	unchanged	unchanged

What survives. The command is 🚫 because the overwritten content has no other copy, with one exception worth knowing. Anything that was ever staged went into the object database when `git add` ran:

```
# The staged line from the first step was written into the object database by "git add".
$ git fsck
dangling blob e4eb0e6d146a5b1eb62d3a52301f262836a0a224
$ git cat-file -p $(git fsck | cut -d" " -f3)
model: small-v1
top_k: 5
max_tokens: 512
# The line "debug: true" was never added. No object holds it. It is gone.
```

`git fsck` reports a dangling blob: an object that no ref, commit or index entry points to. It is the staged version from the first transcript. The unstaged `debug: true` line was never added, so nothing holds it. Dangling objects are removed by garbage collection after a grace period (`gc.pruneExpire`, two weeks by default), so this is a rescue route, not a storage plan. Chapter 13 (Recovery) builds on it.

For this 🚫 command:

- **What it changes:** files in the working tree (and index entries with `--staged`).
- **What it can destroy:** every unstaged change in the named paths, without confirmation.
- **How to preview:** `git diff -- <path>` shows exactly what a restore from the index will discard; `git diff <commit> -- <path>` shows it for `--source=<commit>`.
- **How to recover:** not through Git, unless the content was staged or committed at some point. Editor history and backups are outside Git.
- **When it is appropriate:** after you have read the diff and decided that the changes are worthless. When in doubt, commit to a throwaway branch or stash (Chapter 11, Reset, Revert, Restore) instead: both keep the content.

In production. `git restore .` at the top level of a repository is the working-tree half of `git reset --hard`. The usual trigger is frustration during a failed experiment, and the usual loss is the one useful change among twenty useless ones. Use `git restore -p` to discard hunk by hunk when the changes are mixed.

4.8 Moving and removing files: `git mv` and `git rm`

In one sentence. `git mv` and `git rm` do in one step what you can do in two: change the working tree with `mv` or `rm`, then tell the index.

Precisely. 🟢 `git mv <source> <destination>` renames the file on disk and renames its index entry. 🟡 `git rm <path>` deletes the file on disk and removes its index entry; it refuses when the file's content is not safely stored in HEAD. Neither creates a commit: "the index is updated after successful completion, but the change must still be committed" (`git-mv`).

See it. The index before and after a `git mv`:

```
$ git ls-files --stage src
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 9d77f9cf9e4a6e1da622b599a95cd3afc3ee77c1 0      src/retriever.py
$ git mv src/retriever.py src/search.py
$ git status --short
R src/retriever.py -> src/search.py
$ git ls-files --stage src
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 9d77f9cf9e4a6e1da622b599a95cd3afc3ee77c1 0      src/search.py
# Same blob ID, new path. The tree that this index would produce:
$ git write-tree
566379be59e0cde72aa450cfea467a891457268c
```

The entry kept its blob ID and changed its path. `git write-tree` prints the ID of the tree that this index describes (Chapter 5, section 5.2). Now undo everything and perform the rename with shell commands:

```
# Undo, then do the same rename with plain shell commands.
$ git restore --staged --worktree --source=HEAD .
$ git status --short
$ mv src/retriever.py src/search.py
$ git status --short
D src/retriever.py
?? src/search.py
$ git add -A src
$ git status --short
R src/retriever.py -> src/search.py
$ git write-tree
566379be59e0cde72aa450cfea467a891457268c
```

The tree ID is identical, 566379b... both times. That is the proof that `git mv` adds nothing that `mv` plus `git add -A` does not: the two routes produce the same index, byte for byte in what a commit would record.

`git rm` has a safety check that plain `rm` lacks:

```
$ git rm docs/old-notes.md
rm 'docs/old-notes.md'
$ git status --short
D docs/old-notes.md
$ ls docs
runbook.md
# git rm refuses to delete content that exists nowhere else.
$ echo 'Escalation contacts.' >> docs/runbook.md
$ git rm docs/runbook.md
error: the following file has local modifications:
    docs/runbook.md
(use --cached to keep the file, or -f to force removal)
[exit status: 1]
$ git rm --cached docs/runbook.md
rm 'docs/runbook.md'
$ git status --short
D docs/old-notes.md
D docs/runbook.md
?? docs/
```

The first removal went through because the file matched HEAD, so the content is recoverable. The second was refused because `docs/runbook.md` had a modification that exists nowhere else. `-f` overrides the check and is  for that reason. `--cached` removes only the index entry.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git mv a b</code>	a renamed to b	entry a removed, entry b added with the same blob ID	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git rm <path></code>	file deleted; its directory too if that leaves it empty	entry removed	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git rm --cached <path></code>	unchanged	entry removed	unchanged	unchanged	unchanged	unchanged	unchanged

Two behaviors found while writing the demos for this chapter are worth knowing before they surprise you. `git rm` deletes a directory when it removes the last file in it (section 4.11). And `git mv` does not create a missing destination directory: `git mv README.md docs/README.md` fails with "renaming 'README.md' failed: No such file or directory" when `docs/` does not exist.

4.9 Why Git does not record renames

In one sentence. A commit stores a snapshot of paths and contents, so there is no place in it where "this file used to be called that" could be written; a rename is a conclusion that Git draws later by comparing two snapshots.

Precisely. The Git User's Manual: "a commit does not itself contain any information about what actually changed; all changes are calculated by comparing the contents of the tree referred to by this commit with the trees associated with its parents. In particular, Git does not attempt to record file renames explicitly, though it can identify cases where the existence of the same file data at changing paths suggests a rename" ([user-manual](#)).

See it. Commit the rename from the previous section and look inside the commit:

```
$ git commit -m "Rename retriever module to search"
[main a7a3aae] Rename retriever module to search
1 file changed, 0 insertions(+), 0 deletions(-)
rename src/{retriever.py => search.py} (100%)
$ git cat-file -p HEAD
tree 566379be59e0cde72aa450cfea467a891457268c
parent f518810d6545244f37e647a360377229b00b306e
author Lab User <you@example.com> 1788756300 +0530
committer Lab User <you@example.com> 1788756300 +0530

Rename retriever module to search
$ git ls-tree -r HEAD
100644 blob 4dddde5373611e10f9ec6bc328a25ae216552487 docs/old-notes.md
100644 blob 3e9f19511d48a09c7080a74f7f78b97f62e2d153 docs/runbook.md
100644 blob 63df51b788f2464137e0f30355056e42a320f705 src/app.py
100644 blob 9d77f9cf9e4a6e1da622b599a95cd3afc3ee77c1 src/search.py
```

The summary line says `rename src/{retriever.py => search.py} (100%)`, but the commit object has a tree, a parent, two identities and a message, and the tree has four paths. No field mentions a rename. The word comes from the comparison:

```
# The rename is computed when two snapshots are compared.
$ git diff --name-status HEAD~1 HEAD
R100 src/retriever.py src/search.py
$ git diff --name-status --no-renames HEAD~1 HEAD
D src/retriever.py
A src/search.py
```

R100 means "a deleted path and an added path whose contents are 100% similar". Turn detection off and the same two commits show a deletion and an addition. When a commit renames and edits, the similarity drops, and whether you see a rename depends on a threshold:

```
# Rename and edit in one commit: similarity drops below 100.
$ git diff --name-status HEAD~1 HEAD
R096    src/search.py    src/vector_search.py
$ git diff --name-status -M98% HEAD~1 HEAD
D       src/search.py
A       src/vector_search.py
```

The default threshold is 50% (`-M50%`). History that follows a file depends on the same detection:

```
$ git log --oneline -- src/vector_search.py
8ae9ce9 Rename search module and raise TOP_K
$ git log --oneline --follow -- src/vector_search.py
8ae9ce9 Rename search module and raise TOP_K
a7a3aae Rename retriever module to search
f518810 Add service skeleton
```

Without `--follow`, the log of `src/vector_search.py` starts at the commit where that path first appears. With it, Git detects the two renames and continues. `--follow` works for a single file only.

In production. Three habits follow from detection being a heuristic. Rename in one commit and edit in the next when the edit is large: a rename combined with a rewrite can fall below the threshold, and then `git log --follow`, `git blame` and the rename handling in merges (Chapter 8, Merge) all treat it as an unrelated new file. Expect very large moves to look like delete plus add: the exhaustive part of detection is skipped when the number of candidate files exceeds `diff.renameLimit`, whose default the 2.55 manual gives as 1000. And do not look for a "rename" flag to audit: there is none in the data.

4.10 File modes and symbolic links

In one sentence. Besides its content, Git records one more thing about a file: its type and whether it is executable.

Precisely. An index entry and a tree entry carry a mode. For files there are three: `100644` (regular file), `100755` (executable file) and `120000` (symbolic link). The User's Manual puts it bluntly: "Git actually only pays attention to the executable bit." Owner, group, the other permission bits and timestamps are not recorded.

See it.

```
$ git ls-files --stage scripts/run_eval.sh
100644 75fd1dcf59cec1b6e5f88119344383b4a77d980b 0      scripts/run_eval.sh
$ chmod +x scripts/run_eval.sh
$ git status --short
M scripts/run_eval.sh
$ git diff
diff --git a/scripts/run_eval.sh b/scripts/run_eval.sh
old mode 100644
new mode 100755
$ git add scripts/run_eval.sh
$ git ls-files --stage scripts/run_eval.sh
100755 75fd1dcf59cec1b6e5f88119344383b4a77d980b 0      scripts/run_eval.sh
# The blob ID did not change. The mode is stored in the index entry, and later in the tree.
```

A mode change is a change: status shows `M`, and the diff has no content lines, only `old mode` and `new mode`. After `git add`, the entry has mode `100755` and the same blob ID as before. The mode is not part of the blob.

When the filesystem cannot express the bit, set it in the index directly:

```
# Set the bit in the index without touching the file on disk.
$ git add --chmod=+x scripts/deploy.sh
$ git ls-files --stage scripts/deploy.sh
100755 d11e0661a7a15ee7be23789363e0fb67052ec70c 0      scripts/deploy.sh
$ test -x scripts/deploy.sh && echo "executable on disk" || echo "not executable on disk"
not executable on disk
$ git status --short
MM scripts/deploy.sh
M  scripts/run_eval.sh
# With core.fileMode=false Git stops comparing the bit on disk with the index.
$ git -c core.fileMode=false status --short
M  scripts/deploy.sh
M  scripts/run_eval.sh
```

After `git add --chmod=+x` the index says executable and the disk does not, so status reports the file as modified in the working tree (MM). With `core.fileMode=false` Git stops comparing the bit, which is what `git init` and `git clone` configure when they detect a filesystem that does not keep it.

A symbolic link is stored as a blob whose content is the link text:

```
$ ln -s settings.yaml config/current.yaml
$ git add config/current.yaml
$ git ls-files --stage config
120000 4cba9211b05dfbfce96b9e2ce5c8a33d4110caf6 0      config/current.yaml
100644 4da99e38a05a2a7b436aaea17b946e0288949413 0      config/settings.yaml
# The blob of a symbolic link holds the link text, not the content of the target.
$ git cat-file -p :config/current.yaml; echo
settings.yaml
$ git cat-file -s :config/current.yaml
13
```

The blob is 13 bytes long: the characters of `settings.yaml`. Git never follows the link and never stores the target's content under the link's path.

In production. A script that was made executable on one machine and committed from another where the bit was not set fails in CI with "Permission denied". Check with `git ls-files --stage <path>`, fix with `git update-index --chmod=+x <path>` or `git add --chmod=+x <path>`, and commit. For links, remember that the stored text is whatever you typed: a link to an absolute path on your laptop is committed faithfully and is broken on every other machine. On a filesystem without symbolic links (`core.symlinks=false`) the link is checked out as a small plain file containing the link text.

4.11 Empty directories

In one sentence. Git tracks files; a directory exists in a commit only because a file path passes through it.

See it.

```
$ mkdir -p data/raw data/processed
$ git status
On branch main
nothing to commit, working tree clean
$ git add data
$ git status --short
$ git ls-files data
```

Two empty directories exist on disk. Status reports a clean working tree, `git add data` adds nothing and says nothing, and the index has no entry under `data/`. There is nothing to add: the index is a list of files (Chapter 5).

The convention is to give the directory a file. A `.gitignore` inside it does two jobs at once:

```
# The index holds file paths only. A directory reaches a commit through a file inside it.
$ printf '%s\n!.gitignore\n' > data/raw/.gitignore
$ git add data/raw/.gitignore
$ git status --short
A data/raw/.gitignore
$ git commit -m "Keep data/raw in the repository, ignore its content"
[main 2105d8a] Keep data/raw in the repository, ignore its content
1 file changed, 2 insertions(+)
create mode 100644 data/raw/.gitignore
$ git ls-tree -r --name-only HEAD
data/raw/.gitignore
docs/old-notes.md
src/app.py
$ echo '{"id": 1}' > data/raw/tickets.jsonl
$ git status --short
```

The pattern `*` ignores everything in `data/raw/`, and `!.gitignore` keeps the placeholder itself tracked. A dataset dropped into the directory stays out of status. You will also meet empty files named `.gitkeep`; the name means nothing to Git and appears nowhere in its documentation.

The rule works in the other direction too. `git rm` on the last tracked file removes the directory, as section 4.8 noted, and a checkout of a commit that no longer has any file in a directory removes that directory unless untracked files are in it.

In production. Code that expects `logs/`, `outputs/` or `data/processed/` to exist after a clone will fail on a fresh CI runner. Either commit a placeholder or, better, have the code create the directory it writes to.

4.12 The case-insensitive filesystem trap

In one sentence. Git treats `Config.py` and `config.py` as two different paths, the default macOS filesystem treats them as one file, and the setting that reconciles the two (`core.ignoreCase`) hides case-only renames.

Precisely. Git stores file names as byte sequences and performs no case folding ([gitfaq](#)). When `git init` or `git clone` creates a repository, it probes the filesystem and sets `core.ignoreCase=true` if names are case-insensitive, as they are on a default APFS volume. The manual describes the effect: if a directory listing finds `makefile` when Git expects `Makefile`, "Git will assume it is really the same file, and continue to remember it as `Makefile`". It also calls the variable internal and warns that changing it by hand "may result in unexpected behavior". The transcripts in this section assume a case-insensitive volume; on a case-sensitive one they come out differently, and that difference is the lesson.

See it.

```
$ git ls-files
src/Config.py
src/app.py
$ mv src/Config.py src/config.py
$ ls src
app.py
config.py
$ git status
On branch main
nothing to commit, working tree clean
$ git add -A
$ git ls-files
src/Config.py
src/app.py
```

The file on disk is now `config.py`. Status is clean, `git add -A` records nothing, and the index still says `src/Config.py`. Every commit you make will keep the old name, and every Linux machine will check out the old name. The rename has to be made in the index:

```
# The file on disk already has the new name. The index does not. "git mv" renames the index entry.
$ git mv src/Config.py src/config.py
$ git status --short
R src/Config.py -> src/config.py
$ git ls-files
src/app.py
src/config.py
```

Observed behavior : A module was renamed from Config.py to config.py. It works on every Mac and fails in CI on Linux, where the file is still called Config.py.

Git state : The index and every commit say src/Config.py. The Mac's disk says config.py.

Mechanism : With core.ignoreCase=true Git matches the directory entry "config.py" to the tracked path "Config.py" and keeps the name it already had.

Root cause : The rename was made in the filesystem only. On a case-insensitive filesystem such a rename never reaches the index, so it is never committed.

Why Git does this : So that tools which change the case of a name on these filesystems do not make a tracked file look deleted and a new one look untracked.

Correct fix : git mv src/Config.py src/config.py, then commit.

Prevention : Rename with git mv. Keep one naming convention. Let CI run on a case-sensitive filesystem and fail on paths that differ only by case.

The mirror image of the problem arrives from the other side. Someone on Linux commits two files whose names differ only by case, which is legal there. On a Mac both names cannot exist:

```
$ cd ..
$ git clone linux-teammate mac-clone
Cloning into 'mac-clone'...
done.
warning: the following paths have collided (e.g. case-sensitive paths
on a case-insensitive filesystem) and only one from the same
colliding group is in the working tree:

'src/Config.py'
'src/config.py'
$ cd mac-clone
$ ls src
config.py
$ git status --short
M src/Config.py
$ git diff
diff --git a/src/Config.py b/src/Config.py
index 1e0b1ad..5fac60c 100644
--- a/src/Config.py
+++ b/src/Config.py
@@ -1,11 @@
- TOP_K = 5
+ TOP_K = 8
```

Git warns during the clone, one file lands on disk, and the other path is reported as modified for ever: when Git reads `src/Config.py` it gets the content of `src/config.py`. No `git restore` can fix it, because writing either file overwrites the other. The documented fix is to stop tracking one of the names:

```
# Detect: fold every tracked path to lower case and look for duplicates.
$ git ls-files | tr '[:upper:]' '[:lower:]' | sort | uniq -d
src/config.py
# Fix: stop tracking one of the two names, commit, and check the other one out again.
$ git rm --cached src/Config.py
rm 'src/Config.py'
$ git commit -q -m "Remove Config.py, which collides with config.py on case-insensitive filesystems"
$ git restore .
$ git status --short
$ git ls-files
src/config.py
```

The one-line detector, `git ls-files | tr '[:upper:]' '[:lower:]' | sort | uniq -d`, is cheap enough to run in CI on every pull request.

In production. This trap is specific to teams that develop on macOS or Windows and deploy on Linux, which describes most backend and ML teams. Python imports, Java class files and Docker COPY paths are all case-sensitive on Linux. macOS has a second, rarer cousin: it may hand back a file name in a different Unicode normalization form than the one you wrote, and `core.precomposeUnicode` (set to true by `git init` on macOS, as the first transcript of the course shows) exists to undo that.

4.13 Line endings, in brief

Git stores the bytes it is given. A file saved with Windows line endings (CRLF) is stored with them unless an attribute or a configuration value tells Git to convert. `git ls-files --eol` shows what is in the index, what is in the working tree, and which attribute applies:

```
# i/ is the content in the index, w/ the content in the working tree, attr/ the attribute in force.
$ git ls-files --eol
i/lf      w/lf      attr/          requirements.txt
i/crlf    w/crlf    attr/          scripts/deploy.bat
```

When an editor on another platform rewrites every line ending, the whole file becomes one change:

```
# An editor on another platform saves requirements.txt with CRLF line endings.
$ git ls-files --eol requirements.txt
i/lf      w/crlf    attr/          requirements.txt
$ git diff --stat
requirements.txt | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
$ git diff --stat --ignore-cr-at-eol
```

Three lines "changed" although no character you can see did, and the second command confirms that the only difference is the carriage return at the end of each line. Such a commit makes `git blame` useless for the file and produces conflicts on every line for anyone with pending work. The durable fix is a committed `.gitattributes` with `* text=auto` and one `git add --renormalize .`, so that the repository holds LF for everyone regardless of personal settings. Chapter 14C (Stash internals, rerere, attributes, hooks) covers attributes, and `core.autocrlf` with its limits, in full.

4.14 `git clean`

In one sentence.  `git clean` deletes untracked files from the working tree, which makes it the one everyday command that removes data Git has no copy of.

Analogy. Sweeping the workbench into the bin. Everything that came out of the cabinet is safe, because the cabinet still has it. Everything else is gone, and Git cannot tell a wood shaving from the prototype you spent a week on. The analogy holds well; its only weakness is that a real bin can be searched afterwards.

Precisely. By default `git clean` removes untracked files that are not ignored, in the current directory and below, and does not enter untracked directories ([git-clean](#)). The options widen that:

Option	Effect
<code>-n</code> , <code>--dry-run</code>	Show what would be removed and remove nothing
<code>-f</code> , <code>--force</code>	Required to delete anything, because <code>clean.requireForce</code> defaults to true. A second <code>-f</code> allows deleting untracked directories that contain their own <code>.git</code>

Option	Effect
<code>-d</code>	Also remove untracked directories
<code>-X</code> (capital)	Remove only ignored files: build products and caches, keeping files you created by hand
<code>-x</code> (small)	Do not use ignore rules at all: remove untracked and ignored files alike
<code>-e</code> <pattern>	Add an exclude pattern for this run
<code>-i</code>	Interactive mode: choose what to delete

`-x` and `-X` differ by one keystroke and by which of your files survive.

Inside `.git`. Nothing changes. Clean reads the index to learn which paths are tracked and the ignore rules to classify the rest. It writes no object, moves no ref, and leaves no record of what it deleted.

See it. The starting point: three untracked paths and four ignored ones.

```
$ git status --short --ignored
?? experiments/
?? scratch.py
?? vendor/
!! .env
!! data/
!! server.log
!! src/__pycache__/
```

Without `-f`, `-n` or `-i`, Git refuses to run. Always start with the dry runs:

```
# Untracked files in the current directory only.
$ git clean -n
Would remove scratch.py
# Add -d: untracked directories too.
$ git clean -n -d
Would remove experiments/
Would remove scratch.py
Would skip repository vendor/tokenizer
# Add -X: only what the ignore rules match.
$ git clean -n -d -X
Would remove .env
Would remove data/
Would remove server.log
Would remove src/__pycache__/
# Add -x instead: ignore rules are not used at all, so everything untracked goes.
$ git clean -n -d -x
Would remove .env
Would remove data/
Would remove experiments/
Would remove scratch.py
Would remove server.log
Would remove src/__pycache__/
Would skip repository vendor/tokenizer
```

Each added option widens the list. The last one includes `.env` and `data/`. The directory `vendor/tokenizer` is a repository of its own and is skipped by every run with fewer than two `-f`.

```
$ git clean -f -d -x
Removing .env
Removing data/
Removing server.log
Removing src/__pycache__/
Skipping repository vendor/tokenizer
$ git status --short --ignored
?? vendor/
$ cat .env
```

```
cat: .env: No such file or directory
[exit status: 1]
$ git fsck
# fsck prints nothing: no object ever held those files. Git cannot bring them back.
```

`git fsck` prints nothing. There is no dangling blob to rescue, because none of these files was ever added. That is the whole difference between this command and the losses in section 4.7.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git clean -n ...</code>	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git clean -f</code>	untracked, non-ignored files in the current directory deleted	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git clean -f -d -x</code>	every untracked path below the current directory deleted, ignored ones included	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged

For this  command:

- **What it changes:** the working tree only.
- **What it can destroy:** untracked files; with `-d` untracked directories; with `-x` also ignored files such as `.env`, downloaded datasets, model checkpoints, virtual environments and local databases.
- **How to preview:** the same command with `-n` in place of `-f`. Read every line of the output.
- **How to recover:** not through Git. Backups, or editor history for single files.
- **When it is appropriate:** to prove a build from a pristine tree, to reset a CI workspace, to clear generated files that the build tool's own clean target misses. Prefer `-X` to `-x`, pass a pathspec (`git clean -n -d -X -- build/`) to narrow the blast radius, and protect what must survive with `-e`.

In production. The typical loss is in an ML repository where `data/`, `checkpoints/` and `.env` are ignored on purpose. Someone copies `git clean -fdx` from a "fix your broken build" answer, and the ignored files, which were the only copies, are deleted in under a second. The command did what its options say. The defence is the habit of running `-n` first and of keeping irreplaceable data outside the working tree or in real storage (Chapter 28, AI/ML Workflows).

4.15 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A file listed in <code>.gitignore</code> keeps showing as modified	<code>git check-ignore -v <path></code> prints nothing; <code>git ls-files -ci -- exclude-standard</code> lists it: the path is tracked	<code>git rm --cached <path></code> , commit; rotate any secret	<code>.gitignore</code> before the first <code>git add .</code> ; the detector in CI
A new file is not offered by <code>git status</code>	<code>git check-ignore -v <path></code> names the pattern and the file it lives in, which may be <code>.git/info/exclude</code> or your global ignore file	Fix the pattern, add a <code>!</code> exception, or <code>git add -f</code> once	Keep personal patterns narrow; keep shared patterns anchored (<code>/build/</code>)
A <code>!</code> exception has no effect	The parent directory is excluded (<code>data/</code> instead of <code>data/*</code>)	Exclude the contents, not the directory	Test new patterns with <code>git check-ignore -v -n</code>

Symptom	Diagnosis	Fix	Prevention
A teammate's local config vanished after <code>git pull</code>	The pull contained a commit that stopped tracking the file (<code>git log --diff-filter=D --name-only</code>)	<code>git restore --source=<commit before> <path></code>	Announce "stop tracking" commits with the restore command
Edits disappeared after <code>git restore <path></code>	Nothing to diagnose: the working tree was overwritten from the index	<code>git fsck</code> finds versions that were staged at some point; otherwise editor history	<code>git diff</code> before restore; stash or commit when unsure
<code>git status</code> is clean but the build differs from CI	Ignored or untracked files in the working tree (<code>git status --ignored</code> , <code>git clean -ndx</code>)	Build in a fresh clone or worktree	Never build release artifacts from a developer working tree
A file is "modified" immediately after a clone on macOS	Two tracked paths differ only by case (the clone printed a warning)	<code>git rm --cached</code> one of them, commit, <code>git restore .</code>	Case-collision check in CI
A renamed file still has its old name on Linux	Case-only rename made with <code>mv</code> on macOS; <code>git ls-files</code> shows the old name	<code>git mv old new</code> , commit	Rename with <code>git mv</code>
A script fails in CI with "Permission denied"	<code>git ls-files --stage <path></code> shows 100644	<code>git update-index --chmod=+x <path></code> , commit	Check modes in review; <code>git diff</code> shows old mode/new mode
Every line of a file shows as changed	<code>git ls-files --eol <path></code> shows i/lf w/crlf; <code>git diff --ignore-cr-at-eol</code> is empty	Restore the endings; adopt <code>.gitattributes</code> (Chapter 14C)	* text=auto committed for everyone
History of a file stops at a rename	The rename commit also rewrote the file; similarity fell below the threshold (<code>git log --follow -M30% -- <path></code> to test)	None for existing history; lower the threshold when reading	Rename and edit in separate commits
Expected directory missing after clone	Git does not track empty directories	Commit a placeholder or create the directory in code	Same

4.16 When not to use it, and dangerous edge cases

Ignored means expendable. Git treats an ignored file as something it may overwrite without asking. Continue the story of section 4.6. Your `.env` is now ignored and holds a value that exists nowhere else. The branch `release-1.0` was cut while `.env` was still tracked:

```
# Back in your clone. Your .env is ignored now and holds a value that exists nowhere else.
$ cd ../support-bot
$ cat .env
LLM_API_KEY=lab-secret-0003
# release-1.0 was cut while .env was still tracked.
$ git switch release-1.0
Switched to branch 'release-1.0'
$ cat .env
LLM_API_KEY=lab-secret-0001
$ git switch main
Switched to branch 'main'
$ cat .env
cat: .env: No such file or directory
[exit status: 1]
```

Switching to the old branch replaced your file with the tracked version, silently. Switching back deleted it, because the path is tracked there and absent here. Your value is gone and no object holds it.

```
Observed behavior : An ignored local file was replaced, then deleted, by two branch switches.
                   No warning was printed.
Git state          : On main the path is untracked and ignored. On release-1.0 it is tracked.
Mechanism          : To check out release-1.0 Git must write the tracked file. An untracked file
```

in the way normally stops the checkout. An ignored file does not: `--overwrite-ignore` is the default for checkout, switch and merge.

Root cause : The same path is ignored on one branch and tracked on another.

Why Git does this : Ignored files are assumed to be regenerable build products, and refusing to switch branches because of them would make ignoring pointless.

Correct fix : None for the lost content. Recreate the file.

Prevention : Give private files a name that no commit on any branch has ever tracked (settings.local.yaml, not the formerly tracked settings.yaml). Keep irreplaceable data outside the working tree. Use `git switch --no-overwrite-ignore` when moving to old branches.

The guard exists, as an option:

```
$ echo 'LLM_API_KEY=lab-secret-0004' > .env
$ git switch --no-overwrite-ignore release-1.0
error: The following untracked working tree files would be overwritten by checkout:
    .env
Please move or remove them before you switch branches.
Aborting
[exit status: 1]
$ cat .env
LLM_API_KEY=lab-secret-0004
```

`--no-overwrite-ignore` is documented for [git checkout](#) and `git merge`; `git switch -h` on 2.55 lists it as well.

`git restore --source` **on a directory deletes files**. Because `restore` runs in no-overlay mode, `git restore --source=HEAD~5 src/` removes every tracked file under `src/` that did not exist five commits ago, together with any unstaged changes in them. Name files, not directories, when you take content from an old commit, or add `--overlay` to forbid deletions.

`git clean` **in the wrong directory**. `Clean` works from the current directory down. Run at the top level with `-d -x` it reaches everything. Run it from the narrowest directory that contains what you want gone.

`git rm --cached` **is a deletion for everyone else**. It is the right tool to stop tracking a file and the wrong tool to unstage a change (Chapter 5, section 5.9).

When not to use `.gitignore`. Not as protection for secrets: it is advice to `git status` and `git add`, nothing more. Not for files the team must share. And not for "ignore my local edits to a tracked file": Git has no such feature, and the two index bits that look like one are covered in Chapter 5, section 5.12.

When not to use `git restore` **or** `git clean`. When you cannot say what will be lost. Both have a preview (`git diff`, `git clean -n`). A stash or a throwaway commit costs seconds and keeps everything.

4.17 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git status</code> , <code>git check-ignore</code> , <code>git ls-files</code> , <code>git diff</code>	● SAFE	Nothing (status may refresh cached file information in the index)	not needed	not needed
<code>git add <path></code> , <code>git add -f <path></code>	● SAFE	Writes a blob and an index entry	<code>git add -n <path></code>	<code>git restore --staged <path></code>
<code>git mv <a> </code>	● SAFE	Renames the file and its index entry	<code>git mv -n <a> </code>	<code>git mv <a></code>
<code>git add --chmod=+x</code> , <code>git update-index --chmod=+x</code>	● SAFE	Mode in the index entry	<code>git ls-files --stage <path></code>	the opposite <code>--chmod=-x</code>

Command	Label	What it changes	Preview	Recovery
<code>git rm <path></code>	⚠ CAUTION	Deletes the file and its index entry; refuses if content is not in HEAD	<code>git rm -n <path></code>	<code>git restore --staged --worktree <path></code>
<code>git rm --cached <path></code>	⚠ CAUTION	Removes the index entry; the next commit deletes the path for everyone who pulls	<code>git rm --cached -n <path></code>	<code>git restore --staged <path></code> before committing
<code>git rm -f <path></code>	🔴 DANGEROUS	As <code>git rm</code> , without the check; destroys uncommitted changes in the file	<code>git diff HEAD -- <path></code>	none for unstaged content
<code>git restore <path></code> , <code>git checkout -- <path></code>	🔴 DANGEROUS	Overwrites working tree files from the index	<code>git diff -- <path></code>	none for content never staged; <code>git fsck</code> for content that was
<code>git restore --source=<commit> [--staged] [--worktree] <path></code>	🔴 DANGEROUS	Overwrites working tree files (and index entries) from a commit; deletes tracked paths the commit lacks	<code>git diff <commit> - <path></code>	as above
<code>git clean -n</code>	🟢 SAFE	Nothing	it is the preview	not needed
<code>git clean -f [-d] [-X or -x]</code>	🔴 DANGEROUS	Deletes untracked (and with <code>-x</code> / <code>-X</code> ignored) files	the same options with <code>-n</code>	none through Git
<code>git switch <branch></code> when an ignored file is tracked on that branch	🔴 DANGEROUS	Overwrites the ignored file	<code>git ls-tree -r --name-only <branch></code> compared with <code>git status --ignored</code>	none; prevent with <code>--no-overwrite-ignore</code>

4.18 Version notes

Version note. Older behavior: `git checkout -- <path>` restored files and `git checkout <branch>` switched branches. Current behavior: `git restore` and `git switch` do the two jobs separately; `git checkout` still works and is not scheduled for removal. Since: introduced in Git 2.23 ([release notes](#)), no longer labelled experimental from Git 2.51 ([release notes](#)). Recommended: write `restore` and `switch`, read `checkout`. Apple's Git 2.50.1 at `/usr/bin/git` still prints the experimental label.

Version note. Older behavior: status hints suggested `git reset HEAD <file>...` to unstage and `git checkout -- <file>...` to discard, and many tutorials and the Pro Git chapter on recording changes still show those hints. Current behavior: Git 2.55 prints the `git restore` forms shown in this chapter. Since: the exact release in which the hints changed was not verified for this chapter. Recommended: treat old hints as valid spellings of the same operations.

Version note. Older behavior: none; the command did not exist. Current behavior: `git check-ignore` explains which pattern ignores a path, and `--no-index` diagnoses paths that were tracked by mistake. Since: Git 1.8.2 for the command and 1.8.5 for `--no-index`, according to the release notes shipped with Git 2.55.0. Recommended: use it before editing any ignore file.

Version note. Older behavior: `git config --get core.ignoreCase`. Current behavior: `git config get core.ignoreCase`, as used in this chapter; the old form still works. Since: Git 2.46. Recommended: the

subcommand form, remembering that it fails on Git older than 2.46.

Unverified. The release in which `git status --porcelain=v2` appeared, and the release in which `git mv` began to handle case-only renames on case-insensitive filesystems, were not found in the sources available for this chapter. Both behaviors were run on Git 2.55.0 and are shown above.

4.19 Practice

- **Lab 2.3, the `.gitignore` trap and its fix**, in the Module 2 lab manual. Do it now: it uses only this chapter.
- Labs 2.1, 2.2 and 2.4 in the same file need Chapter 5.
- Replay any transcript of this chapter with `labs/run ch04/<demo>`, for example `labs/run ch04/gitignore-patterns`. The sandbox stays in place afterwards, so you can continue by hand in it.
- Five short drills, each in a sandbox left by a replay:
 1. In `ch04/gitignore-patterns`, write one pattern that ignores `*.bin` only directly inside `models/v1/` and prove it with `git check-ignore -v -n`.
 2. In `ch04/status-anatomy`, predict the output of `git status --short` after `git restore --staged src/retriever.py`, then run it.
 3. In `ch04/mv-rm-renames`, find the smallest `-M` percentage at which the second rename is still reported as `R`.
 4. In `ch04/clean`, list what `git clean -n -d -X -- src` would remove and explain each line.
 5. In `ch04/case-insensitive`, run `git config get core.ignoreCase` in each of the three repositories and explain the values.

4.20 Interview questions

1. Define tracked, untracked and ignored in terms of the index. Can a path be tracked and ignored at the same time?
2. `git status` shows the same file under "Changes to be committed" and under "Changes not staged for commit". Explain exactly which comparisons produced the two lines.
3. A developer added `config/secrets.yaml` to `.gitignore` and it is still in every new commit. Walk through your diagnosis commands, the fix, and the two things the fix does not solve.
4. What are the four sources of ignore patterns, in precedence order, and which would you use for `.DS_Store`, for `build/`, and for a personal scratch directory?
5. Why can `!data/README.md` fail to re-include a file, and how do you write the rule so that it works?
6. What does `git restore file.py` take its content from? How does that differ from `git restore --source=HEAD file.py`, and from `git checkout HEAD -- file.py`?
7. After `git restore` destroyed some edits, a colleague says "use the reflog". What is wrong with that advice, and under what condition can part of the work still be recovered?
8. Prove to me that Git does not store renames. Then explain what `R087` in a diff means and name two features that depend on it.
9. A Python module imports fine on macOS and fails on the Linux CI runner after someone renamed it. What happened inside Git, and how do you prevent a recurrence?
10. Compare `git clean -fdX` with `git clean -fdx`. Which one would you allow in a CI cleanup step for an ML repository, and what would you run first?
11. Why did switching branches delete a file that was listed in `.gitignore`?
12. What does Git record about file permissions, and how do you make a script executable for everyone when your own filesystem cannot express that?

4.21 Sources

Primary sources

- [git-status](#), [gitignore](#), [git-check-ignore](#), [git-restore](#), [git-rm](#), [git-mv](#), [git-clean](#), [git-ls-files](#), [git-checkout](#), [git-diff](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitfaq](#): "I asked Git to ignore various files, yet they are still tracked" and "Why do I have a file that's always modified?".
- [gitglossary](#) and [gitattributes](#).
- [Git User's Manual](#): renames are not recorded; only the executable bit is.
- Release notes: [2.23.0](#), [2.51.0](#), and 1.8.2 and 1.8.5 as shipped with Git 2.55.0 (`Documentation/RelNotes/` in the Git source tree).

Secondary sources

- Pro Git, [Recording Changes to the Repository](#). Caveat: its status transcripts mix the current `git restore --staged` hint with the older `git reset HEAD <file>` hint.
- GitHub Docs, [Ignoring files](#), and the [github/gitignore](#) template collection.
- GitHub Docs, [Removing sensitive data from a repository](#), for what a committed secret requires beyond `git rm --cached`.
- Perez De Rosso and Jackson, [Purposes, Concepts, Misfits, and a Redesign of Git](#) (OOPSLA 2016): file tracking, untracking a file, file rename and empty directory are four of its seven "operational misfits".
- The Phase 0 report of this course, sections 4, 12 and 15, for the Stack Overflow figures and the ML ignore list.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Git Tutorial for Beginners: Learn Git in One Video](#), CodeWithHarry, Hindi, 12 July 2026: covers staging, ignoring files and tracking empty directories with `git restore --staged`. Caveats from the report: no reset, revert or reflog; local `master`; an early "branch is a copy" phrase that the video corrects later.
- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 24 November 2020: `git restore`, including `-p`. Caveat: `master` naming.

Further reading

- [git-update-index](#), section "Untracked cache", and the "Untracked files and performance" section of `git-status`, for why status is slow in very large working trees. Chapter 26 (Performance) continues from there.

Chapter 5: The Index

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch05/`.

5.1 Why this matters

Three questions a CTO can ask:

1. "You ran the tests, they passed, you committed. CI ran that commit and failed on the line you had fixed. Which of the two is wrong?"
2. "A developer wanted to keep one file out of a commit. The commit deleted that file from the project for everyone. What did they type, and why did Git itself suggest it?"
3. "Half the team hides local configuration edits with `git update-index --assume-unchanged`. Last week a pull was refused on a clean working tree, and then somebody's settings were gone. Why is there no switch for 'ignore my local changes'?"

The three share one root. Git does not build a commit from your files. It builds it from the index, a second complete record of the project that sits between the working tree and HEAD. The answers: the commit contained what was in the index, and the tests ran against something else (sections 5.3 and 5.5); `git rm --cached` deletes an index entry, where unstaging means restoring it (section 5.9); and the bit is a promise that you will not edit the file, not a request to overlook edits (section 5.12).

Understanding usually stops at this layer. "How do I undo 'git add' before commit?" is the fifth most-voted Git question on Stack Overflow (11,626 votes on 1 October 2026, from the Phase 0 report of this course), and none of the ten popular beginner resources that the report surveyed teaches the index as a data structure. Hold on to one idea: the index is not a list of changes. It is a full snapshot.

5.2 The index: the proposed next commit, stored as one file

In one sentence. The index is a single file, `.git/index`, that lists every tracked path together with the ID of a blob, and `git commit` turns exactly that list into the tree of the new commit.

Analogy. The packing list for the next shipment. It names every item that will be in the box, not the items that differ from the last box, and the warehouse ships what the list says even if the shelf has changed since. The analogy breaks in two places: writing a line on this list also puts a copy of the item into storage at that moment (section 5.3), and during a conflict the list can carry three candidate lines for one item (section 5.13).

Precisely. Git's data-model document: "The index, also known as the 'staging area', is a list of files and the contents of each file, stored as a blob", and "Unlike a tree, the index is a flat list of files. When you commit, Git converts the list of files in the index to a directory tree and uses that tree in the new commit" ([gitdatamodel](#)). Each entry has four fields: the file type and mode, the blob ID, a stage number (0 except during a conflict) and the full path; mode, blob ID and path are what the commit's tree receives. Each entry also carries bookkeeping that never leaves your machine: cached filesystem data (section 5.14) and flag bits (sections 5.6 and 5.12). The old name "cache" survives in options such as `--cached`.

The definition corrects the most common wrong model. Nothing in the index is a "change". "Staged changes" are the result of comparing the index with HEAD, and an index with nothing staged is not empty: it describes the same tree as HEAD.

Inside .git. One binary file, created by the first `git add`: a 12-byte header, the entries sorted by path, optional extensions and a checksum ([gitformat-index](#)). Chapter 3 (Git Internals) reads it byte by byte.

See it. `git ls-files --stage` prints the four fields of every entry:

```
# The index: one line per file, with mode, blob ID, stage number and full path.
$ git ls-files --stage
100644 87806e1f4037a90d2243b92a5136fd330bd6570c 0      README.md
100644 4da99e38a05a2a7b436aaea17b946e0288949413 0      config/settings.yaml
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 1e0b1ade696068e087656f8ae5e859fe92aef3b8 0      src/retriever.py
# The tree of HEAD: the same files, arranged as nested tree objects.
$ git ls-tree HEAD
100644 blob 87806e1f4037a90d2243b92a5136fd330bd6570c  README.md
040000 tree d654a67edecb91bd680f03493196872f0b3626eb  config
040000 tree 8e3c2dcba993bebbb0d2d3439234797c8b3cbb0  src
$ git ls-tree -r HEAD
100644 blob 87806e1f4037a90d2243b92a5136fd330bd6570c  README.md
100644 blob 4da99e38a05a2a7b436aaea17b946e0288949413  config/settings.yaml
100644 blob 63df51b788f2464137e0f30355056e42a320f705  src/app.py
100644 blob 1e0b1ade696068e087656f8ae5e859fe92aef3b8  src/retriever.py
```

Four lines with full paths. HEAD's tree has three entries, two of them trees; flattened with `git ls-tree -r`, it gives the same four modes, blob IDs and paths. The file itself starts with the signature `DIRC`, the format version and the entry count:

```
# The index is one binary file. Its first 12 bytes: signature, format version, number of entries.
$ head -c 12 .git/index | hexdump -C
00000000  44 49 52 43 00 00 00 02  00 00 00 04                |DIRC.....|
0000000c
$ git update-index --show-index-version
2
$ git ls-files | wc -l
4
```

 `git write-tree` does on request what `git commit` does every time: it writes the index out as tree objects and prints the ID of the top one. With nothing staged, that is the tree HEAD already has:

```
# Write the index out as a tree object. With nothing staged it is the tree HEAD already has.
$ git write-tree
676700b510b47ab35c5d2aa71977f80d234cdc0c
$ git rev-parse HEAD^{tree}
676700b510b47ab35c5d2aa71977f80d234cdc0c
```

Stage one change. One entry gets a new blob ID (4da99e3 becomes 0217758), and the index describes a different tree:

```
$ echo 'temperature: 0.2' >> config/settings.yaml
$ git add config/settings.yaml
$ git ls-files --stage
100644 87806e1f4037a90d2243b92a5136fd330bd6570c 0      README.md
100644 0217758d92bf9cd7d8ebba62175a33a9ef51ba2f 0      config/settings.yaml
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 1e0b1ade696068e087656f8ae5e859fe92aef3b8 0      src/retriever.py
# One entry has a new blob ID. Every other entry is untouched. The tree this index describes:
$ git write-tree
dd3751c99563c335e9d6db98fd3ebaedfc6ad007
```

```
$ git commit -m "Set sampling temperature"
[main e3aa1bc] Set sampling temperature
1 file changed, 1 insertion(+)
$ git rev-parse HEAD^{tree}
dd3751c99563c335e9d6db98fd3ebaedfc6ad007
$ git status
```

```
On branch main
nothing to commit, working tree clean
```

The commit's tree is `dd3751c`, the ID that `git write-tree` printed before the commit existed. The commit did not read the working tree. Afterwards the index and HEAD describe the same tree again, which is what "nothing to commit" means.

Picture.

.git/index: flat, sorted by path					tree objects, written at commit time	
mode	blob ID	stage	path		676700b	(top-level tree)
100644	87806e1	0	README.md		-- blob 87806e1	README.md
100644	4da99e3	0	config/settings.yaml	==>	-- tree d654a67	config
100644	63df51b	0	src/app.py		-- blob 4da99e3	settings.yaml
100644	1e0b1ad	0	src/retriever.py		-- tree 8e3c2dc	src
					-- blob 63df51b	app.py
					-- blob 1e0b1ad	retriever.py

In production. "What exactly will be in this commit?" has one authoritative answer, the index. Reviewers, CI and every clone receive the tree that was written from it, never your working tree.

GitHub, not Git. GitHub never sees your index. A push transfers commits and the objects they refer to, and everything GitHub shows or checks is derived from those.

5.3 What `git add` writes

In one sentence.  `git add <path>` stores the current content of the file as a blob in the object database and writes that blob's ID into the path's index entry: a copy taken at that moment, not a subscription to the file.

Precisely. The manual: "It only adds the content of the specified file(s) at the time the add command is run; if you want subsequent changes included in the next commit, then you must run `git add` again" ([git-add](#)). Two things are written: one object, unless an identical blob already exists, and one index entry. No ref moves, so no reflog records an add.

See it. `git hash-object` computes the ID that the edited file would get. No such object exists yet:

```
$ git cat-file --batch-all-objects --batch-check
4da99e38a05a2a7b436aaea17b946e0288949413 blob 25
701a11e018456aece1cb6cee6528889539fde3ff tree 33
7bd7d2318a2a4a2b9a16bfbfdd11e7c48a2fda07 commit 165
d654a67edecb91bd680f03493196872f0b3626eb tree 41
$ echo 'max_tokens: 512' >> config/settings.yaml
# hash-object computes the ID this content would get. It writes nothing.
$ git hash-object config/settings.yaml
e4eb0e6d146a5b1eb62d3a52301f262836a0a224
$ git cat-file -t e4eb0e6
fatal: Not a valid object name e4eb0e6
[exit status: 128]
```

```
$ git add config/settings.yaml
$ git cat-file --batch-all-objects --batch-check
4da99e38a05a2a7b436aaea17b946e0288949413 blob 25
701a11e018456aece1cb6cee6528889539fde3ff tree 33
7bd7d2318a2a4a2b9a16bfbfdd11e7c48a2fda07 commit 165
d654a67edecb91bd680f03493196872f0b3626eb tree 41
e4eb0e6d146a5b1eb62d3a52301f262836a0a224 blob 41
$ git ls-files --stage
100644 e4eb0e6d146a5b1eb62d3a52301f262836a0a224 0      config/settings.yaml
```

```
$ git cat-file -p e4eb0e6
model: small-v1
top_k: 5
max_tokens: 512
```

After the add there is a fifth object, a blob with the predicted ID, and the index entry names it. Now edit the file again without adding it:

```
# Edit the file again after the add. The index keeps what it was given.
$ echo 'debug: true' >> config/settings.yaml
$ git status --short
MM config/settings.yaml
$ git show :config/settings.yaml
model: small-v1
top_k: 5
max_tokens: 512
$ git commit -m "Limit response length"
[main 388a0ba] Limit response length
1 file changed, 1 insertion(+)
$ git show HEAD:config/settings.yaml
model: small-v1
top_k: 5
max_tokens: 512
$ git status --short
M config/settings.yaml
```

MM says that both comparisons of `git status` find a difference (Chapter 4, section 4.4). `git show :config/settings.yaml` prints the staged blob, which lacks `debug: true`. The commit records that blob, and the file is still modified afterwards (`M`).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git add <path></code>	unchanged	entry for <code><path></code> created or updated; what was staged for it before is replaced	unchanged	unchanged	one new blob under <code>objects/</code> if the content is new	unchanged	unchanged

Root cause. A fix that was tested but is missing from the commit was edited after the last `git add`. The commit is written from the index, which still held the earlier version; status showed MM. Add the file again and commit, and read `git diff --cached` before every commit. Chapter 1, section 1.12 works through the two-file form of this failure.

What an add leaves behind. Unstaging removes the entry, not the object. Here `git add .` picked up an environment file, and the slip was corrected at once:

```
$ echo 'LLM_API_KEY=lab-secret-0001' > .env
# The slip: everything is added, including the environment file.
$ git add .
$ git status --short
A .env
$ git restore --staged .env
$ git status --short
?? .env
```

```
# The index entry is gone. The object that "git add" wrote is not.
$ git fsck
dangling blob 5620d7c0158994dedf8a5ff7d48b68e5479f513c
$ git cat-file -p 5620d7c
LLM_API_KEY=lab-secret-0001
```

`git fsck` reports a dangling blob, an object that nothing refers to, and `git cat-file -p` prints the secret from it. Has it left the machine?

```
# A push sends the objects that the pushed commits need. Nothing refers to this blob.
$ git push -q ../central.git main
$ git -C ../central.git rev-parse main
a03ed3139b72f324996c67688f6c26074f601f3c
$ git -C ../central.git cat-file -t 5620d7c
fatal: Not a valid object name 5620d7c
[exit status: 128]
$ git cat-file -t 5620d7c
blob
```

The repository that received the push has the commit and not the blob. A push sends what the pushed commits need, and no commit refers to this object.

In production. A secret or a 2 GB checkpoint that was added and then unstaged is in no commit and has not travelled, so there is no history to rewrite. Garbage collection removes the stray object once it is older than `gc.pruneExpire`, two weeks by default (Chapter 13, Recovery). Until then it is also a rescue route: content that was staged once and then lost can be found with `git fsck` (Lab 2.1).

5.4 Three diffs, three comparisons

In one sentence. Three places for a version of a file give three pairwise comparisons, and `git diff` performs a different one depending on its arguments.

Command	Compares	Answers
<code>git diff</code>	index with working tree	What could I still stage?
<code>git diff --cached</code> (synonym: <code>--staged</code>)	HEAD with index	What will the next commit change?
<code>git diff HEAD</code>	HEAD with working tree	What changed since the last commit, staged or not?

See it. `TOP_K` was changed and staged, then `MIN_SCORE` was changed and not staged:

```
# Index against working tree: what you could still stage.
$ git diff
diff --git a/src/retriever.py b/src/retriever.py
index 399da0e..5853020 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -1,5 +1,5 @@
 TOP_K = 8
-MIN_SCORE = 0.2
+MIN_SCORE = 0.35

def retrieve(query, index):
```

```
# HEAD against index: what the next commit will change. --staged is a synonym.
$ git diff --cached
diff --git a/src/retriever.py b/src/retriever.py
index bd144df..399da0e 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -1,4 +1,4 @@
-TOP_K = 5
+TOP_K = 8
 MIN_SCORE = 0.2
```

```
# HEAD against working tree: everything since the last commit, staged or not.
$ git diff HEAD
diff --git a/src/retriever.py b/src/retriever.py
index bd144df..5853020 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -1,5 +1,5 @@
-TOP_K = 5
-MIN_SCORE = 0.2
+TOP_K = 8
+MIN_SCORE = 0.35

def retrieve(query, index):
```

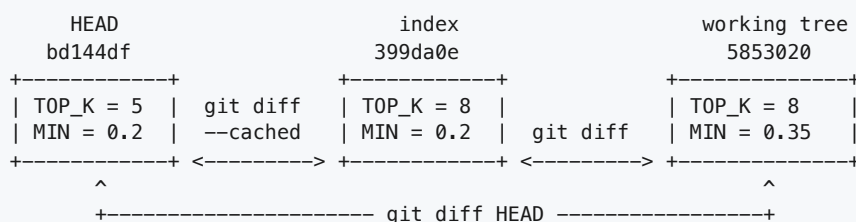
Read the `index` line of each diff: `399da0e..5853020`, `bd144df..399da0e`, `bd144df..5853020`. `bd144df` is the blob in HEAD, `399da0e` the staged blob, and `5853020` the ID that the working tree content would get; no object with that ID exists until you add the file. Each command compares two of the three.

Now put HEAD's content back into the working tree only (Chapter 4, section 4.7):

```
# Put the content of HEAD back into the working tree only. The index still has TOP_K = 8.
$ git restore --source=HEAD src/retriever.py
$ git status --short
MM src/retriever.py
$ git diff HEAD
$ git diff --cached --stat
src/retriever.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git diff --stat
src/retriever.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
# The files on disk match HEAD, yet a commit made now would set TOP_K to 8.
```

`git diff HEAD` is empty, because the files on disk equal HEAD. The other two comparisons each show one changed line, in opposite directions, and a commit made now would set `TOP_K` to 8.

Picture.



In production. Make `git diff --cached` the last command before `git commit`: it is the review of what you are about to record. An empty `git diff HEAD` proves nothing about the next commit, and none of the three shows a file that has no index entry (section 5.6).

5.5 Partial staging: `git add -p`

In one sentence. 🟢 `git add -p` walks through the difference between the index and the working tree one hunk at a time and stages only the hunks you accept, so one edited file can feed several commits and leave a remainder that is never committed.

Analogy. Marking, on a manuscript full of your own edits, the blocks that go to the printer today. The analogy breaks where it matters most: the printer receives a page that never lay on your desk in that form, and nobody has read that page as a whole.

Precisely. A hunk is a block of changed lines with its context, as in `git diff`. Git asks about each hunk and you answer with a letter (`git-add`, "Interactive mode"):

Answer	Effect
y, n	Stage this hunk; leave it
s	Split the hunk. Offered only when unchanged lines separate the changes in it
e	Open the hunk in your editor and stage the edited version
a, d	Stage, or leave, this hunk and all later hunks of the file
q	Stop. Hunks already accepted are staged
j, J, k, K, g, /, p, ?	Move between hunks, search, print the hunk again, print the help

The same selector runs in the other direction in `git restore -p`, `git restore --staged -p` and `git reset -p`.

See it. One file holds a tuning change (`TOP_K`), a bug fix (the guard for a zero vector) and a debug print. The fix must become a commit of its own:

```
# Answers: s (split), n (TOP_K: not in this commit), y (the fix), n (the debug print).
```

```
$ git add -p
```

```
diff --git a/src/retriever.py b/src/retriever.py
```

```
index 177cb37..e548d1b 100644
```

```
--- a/src/retriever.py
```

```
+++ b/src/retriever.py
```

```
@@ -1,12 +1,14 @@
```

```
"""Retrieval for the support bot."""
```

```
import math
```

```
-TOP_K = 5
```

```
+TOP_K = 8
```

```
MIN_SCORE = 0.2
```

```
def normalize(vec):
```

```
    length = math.sqrt(sum(x * x for x in vec))
```

```
+ if length == 0:
```

```
+     return vec
```

```
    return [x / length for x in vec]
```

```
(1/2) Stage this hunk [y,n,q,a,d,k,K,j,J,g/,s,e,p,P,?]? s
```

```
Split into 2 hunks.
```

```
@@ -1,9 +1,9 @@
```

```
"""Retrieval for the support bot."""
```

```
import math
```

```
-TOP_K = 5
```

```
+TOP_K = 8
```

```
MIN_SCORE = 0.2
```

```
def normalize(vec):
```

```
    length = math.sqrt(sum(x * x for x in vec))
```

```
(1/3) Stage this hunk [y,n,q,a,d,k,K,j,J,g/,e,p,P,?]? n
```

```
@@ -5,8 +5,10 @@
```

```
MIN_SCORE = 0.2
```

```
def normalize(vec):
```

```
    length = math.sqrt(sum(x * x for x in vec))
```

```
+ if length == 0:
```

```
+     return vec
```

```
    return [x / length for x in vec]
```

```
(2/3) Stage this hunk [y,n,q,a,d,k,K,j,J,g/,e,p,P,?]? y
```

```

@@ -18,4 +20,5 @@ def retrieve(query_vec, index):
    query_vec = normalize(query_vec)
    scored = [(score(query_vec, vec), doc_id) for doc_id, vec in index]
    scored.sort(reverse=True)
+   print("DEBUG scored:", scored)
    return [(s, doc_id) for s, doc_id in scored[:TOP_K] if s >= MIN_SCORE]
(3/3) Stage this hunk [y,n,q,a,d,K,J,g,/ ,e,p,P,?]? n

```

Git first offered two hunks, because the `TOP_K` line and the guard were close enough to share context. `s` split the first one (`1/2` became `1/3`), and the answers were no, yes, no:

```

$ git status --short
MM src/retriever.py
$ git diff --cached
diff --git a/src/retriever.py b/src/retriever.py
index 177cb37..a7cba42 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -7,6 +7,8 @@ MIN_SCORE = 0.2

def normalize(vec):
    length = math.sqrt(sum(x * x for x in vec))
+   if length == 0:
+       return vec
    return [x / length for x in vec]

```

Inside `.git`. One new blob, `a7cba42`, and one changed index entry; the state table of section 5.3 applies. The blob is HEAD's version plus one hunk, content that has never existed as a file.

Three versions of the file now exist. HEAD (`177cb37`) has none of the three edits. The working tree (`e548d1b`) has all three and is what you ran. The index (`a7cba42`) has the guard alone, and that version has never run anywhere. Before you commit it, copy the staged snapshot out of the index and test that:

```

# The staged snapshot has never existed as files. Export it to test it.
$ git checkout-index --all --prefix=./staged-snapshot/
$ diff ../staged-snapshot/src/retriever.py src/retriever.py
4c4
< TOP_K = 5
---
> TOP_K = 8
22a23
>     print("DEBUG scored:", scored)

```

 `git checkout-index --all --prefix=<dir>/` writes every index entry as a file below `<dir>` (the trailing slash matters). The `diff` confirms that the export lacks the two lines you left out.

When `s` is not offered. Changed lines that touch each other form one hunk:

```

# The changed lines touch each other, so the prompt offers no "s". Typing it anyway:
$ git add -p
diff --git a/config/settings.yaml b/config/settings.yaml
index bfcfe5c..d56fa8e 100644
--- a/config/settings.yaml
+++ b/config/settings.yaml
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
-temperature: 0.7
+top_k: 8
+temperature: 0.2
+debug: true
 max_tokens: 512
(1/1) Stage this hunk [y,n,q,a,d,e,p,P,?]? s

```

```
Sorry, cannot split this hunk
(1/1) Stage this hunk [y,n,q,a,d,e,p,P,?]? q
```

The answer **e** opens the hunk as a patch, with Git's own instructions below it:

```
# Answer "e". In the editor, delete the line "+debug: true", then save and close.
$ git add -p
diff --git a/config/settings.yaml b/config/settings.yaml
index bfcfe5c..d56fa8e 100644
--- a/config/settings.yaml
+++ b/config/settings.yaml
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
-temperature: 0.7
+top_k: 8
+temperature: 0.2
+debug: true
 max_tokens: 512
(1/1) Stage this hunk [y,n,q,a,d,e,p,P,?]? e
--- the hunk file as Git opened it in the editor ---
# Manual hunk edit mode -- see bottom for a quick guide.
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
-temperature: 0.7
+top_k: 8
+temperature: 0.2
+debug: true
 max_tokens: 512
# ---
# To remove '-' lines, make them ' ' lines (context).
# To remove '+' lines, delete them.
# Lines starting with # will be removed.
# If the patch applies cleanly, the edited hunk will immediately be marked for staging.
# If it does not apply cleanly, you will be given an opportunity to
# edit again. If all lines of the hunk are removed, then the edit is
# aborted and the hunk is left unchanged.
--- the hunk as saved (comment lines left out) ---
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
-temperature: 0.7
+top_k: 8
+temperature: 0.2
 max_tokens: 512
```

Deleting a **+** line keeps that addition out of the index. Replacing the **-** of a removed line with a space keeps that removal out. Git then recounts the hunk header and applies the patch to the index. Section 5.17 shows how this goes wrong.

In production. A commit that does one thing can be reviewed, reverted, cherry-picked and bisected on its own (Chapter 6). Debugging sessions do not produce changes in that shape, and partial staging sorts one messy working tree into such commits. The price is a version in the index that never ran. CI is the backstop, and the export is the local check.

5.6 Intent-to-add: `git add -N`

In one sentence. 🟢 `git add -N <path>` gives a new file an index entry without staging its content, so that commands which compare the index with the working tree stop overlooking the file.

Precisely. "Record only the fact that the path will be added later. An entry for the path is placed in the index with no content" ([git-add](#)).

See it. A new file is invisible to `git diff`, because there is no entry to compare it with:

```
$ git status --short
?? src/prompts.py
$ git diff
# A new file does not appear in "git diff": there is no index entry to compare it with.
```

```
$ git add -N src/prompts.py
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    new file:   src/prompts.py

no changes added to commit (use "git add" and/or "git commit -a")
$ git ls-files --stage
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 0      src/prompts.py
```

Status lists the path as a new file under "Changes not staged for commit", and its entry holds `e69de29`, the ID of the empty blob.

```
$ git diff
diff --git a/src/prompts.py b/src/prompts.py
new file mode 100644
index 00000000..bdc5f9
--- /dev/null
+++ b/src/prompts.py
@@ -0,0 +1,2 @@
+SYSTEM = "You are a support assistant."
+MAX_TURNS = 6
$ git diff --cached
# Nothing is staged: the entry is a placeholder, and "git commit" has nothing to record.
$ git commit -m "Add prompts"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    new file:   src/prompts.py

no changes added to commit (use "git add" and/or "git commit -a")
[exit status: 1]
```

`git diff` shows the whole file as an addition, `git diff --cached` shows nothing, and `git commit` refuses: nothing is staged. The hunk selector reads the same comparison as `git diff`, so it has the same blind spot and the same cure:

```
# The hunk selector reads the same comparison as "git diff", so it does not see an untracked file.
$ git add -p src/prompts.py
No changes.
$ git add -N src/prompts.py
$ git add -p src/prompts.py
diff --git a/src/prompts.py b/src/prompts.py
new file mode 100644
index 00000000..bdc5f9
--- /dev/null
+++ b/src/prompts.py
@@ -0,0 +1,2 @@
+SYSTEM = "You are a support assistant."
+MAX_TURNS = 6
(1/1) Stage addition [y,n,q,a,d,e,p,P,?]? y

$ git status --short
M config/settings.yaml
A src/prompts.py
```

Inside `.git`. The entry carries an intent-to-add flag. Index format version 2 has no room for it, so Git writes the index as version 3 while such an entry exists and as version 2 again after the real add (`labs/run ch05/intent-to-add` shows the switch).

In production. The classic "works on my machine" commit lacks a new file. The code runs locally because the file is on disk, `git diff` looked complete, `git commit -a` skipped the file (section 5.10), and CI fails with a missing module. `git add -N` when you create a file puts it into every later `git diff`, `git add -p` and `git commit -a`.

5.7 Staging deletions, renames and binary files

In one sentence. In index terms a deletion is a removed entry, a rename is a removed entry plus an added one, and a binary file is an ordinary entry whose content Git will neither show nor split.

See it.

```
$ git ls-files --stage
100644 4dddde5373611e10f9ec6bc328a25ae216552487 0      docs/old-notes.md
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 67b3f03b8c4ab448c68cda278050d05e9c1865f0 0      src/retriever.py
$ rm docs/old-notes.md
$ git status --short
D docs/old-notes.md
# Staging a deletion removes the index entry. "git add" does it for a path that is gone.
$ git add docs/old-notes.md
$ git status --short
D docs/old-notes.md
$ git ls-files --stage
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 67b3f03b8c4ab448c68cda278050d05e9c1865f0 0      src/retriever.py
```

After `rm`, the entry exists and the file does not (`D`). `git add` on the missing path removes the entry (`D`). `git rm` does both steps at once (Chapter 4, section 4.8).

```
$ mv src/retriever.py src/search.py
$ git status --short
D docs/old-notes.md
D src/retriever.py
?? src/search.py
$ git add src/retriever.py src/search.py
$ git status --short
D docs/old-notes.md
R src/retriever.py -> src/search.py
$ git ls-files --stage
100644 63df51b788f2464137e0f30355056e42a320f705 0      src/app.py
100644 67b3f03b8c4ab448c68cda278050d05e9c1865f0 0      src/search.py
# One entry removed, one entry added with the same blob ID. "Renamed" is a conclusion status draws.
```

Status prints `R`, but the index only lost one entry and gained another with the same blob ID, `67b3f03`. Nothing recorded a rename: status inferred it from the pair (Chapter 4, section 4.9). A binary file, here a small image, is staged like any other file:

```
$ git add assets/logo.png
$ git ls-files --stage assets
100644 5b7d1ca0ae1e65da75b02e4d87cedec1edd00589 0      assets/logo.png
$ git diff --cached --stat -- assets
assets/logo.png | Bin 0 -> 20 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
$ git diff --cached -- assets
diff --git a/assets/logo.png b/assets/logo.png
new file mode 100644
index 0000000..5b7d1ca
Binary files /dev/null and b/assets/logo.png differ
```

```
$ git diff --cached --numstat -- assets
-      -      assets/logo.png
```

The entry looks like any other. The diff says "Binary files ... differ", `--stat` gives byte sizes and `--numstat` prints dashes. When one byte changes, the next add writes a complete new blob, and `git add -p` has nothing to offer:

```
# Change one byte of the image.
$ git diff --stat
assets/logo.png | Bin 20 -> 20 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
# There are no hunks to choose from in a binary file.
$ git add -p assets/logo.png
Only binary files changed.
$ git add assets/logo.png
$ git ls-files --stage assets
100644 9d52479bdc3b8b3068d9f2c4ace19bc88fe1c5d2 0      assets/logo.png
```

Precisely. Unless a `diff` attribute decides, Git inspects the content: "if it looks like text and is smaller than `core.bigFileThreshold`, it is treated as text" ([gitattributes](#)).

In production. Model weights, images and compressed datasets get no partial staging and no readable diff, and every changed version is stored whole when it is added: they belong in Git LFS or outside Git (Chapter 22). A Jupyter notebook is text to Git, so `git add -p` works on it, but its hunks are JSON with embedded outputs (Chapter 28, AI/ML Workflows).

5.8 `git add -u`, `git add -A` and `git add .`

In one sentence. The bulk forms differ in two things: whether untracked files are included, and whether the command is limited to the current directory.

Command	Modified tracked	Deleted tracked	Untracked	Scope
<code>git add -u</code>	staged	staged	not touched	whole working tree
<code>git add -A</code>	staged	staged	staged	whole working tree
<code>git add .</code>	staged	staged	staged	current directory and below
<code>git add -u .</code>	staged	staged	not touched	current directory and below
<code>git add --no-all <path></code>	staged	not touched	staged	the pathspec

No form adds ignored files (Chapter 4, section 4.3).

See it. From inside `src/`, with one modification, one deletion and one new file both inside and outside that directory. `--dry-run` (`-n`) prints what would happen and stages nothing:

```
$ git status --short
M README.md
D docs/guide.md
M src/app.py
D src/retriever.py
?? notes.md
?? src/prompts.py
$ cd src
# From a subdirectory, the short format shows paths relative to where you stand.
$ git status --short
M ../README.md
D ../docs/guide.md
M app.py
D retriever.py
?? ../notes.md
?? prompts.py
```

```
# A pathspec limits the command. "." means this directory and below.
$ git add --dry-run .
add 'src/app.py'
remove 'src/retriever.py'
add 'src/prompts.py'
```

```
# -u: every tracked path in the whole working tree. No new files.
$ git add --dry-run -u
add 'README.md'
remove 'docs/guide.md'
add 'src/app.py'
remove 'src/retriever.py'
# -u with a pathspec: tracked paths in this directory and below.
$ git add --dry-run -u .
add 'src/app.py'
remove 'src/retriever.py'
```

```
# -A: every change in the whole working tree, new files included.
$ git add --dry-run -A
add 'README.md'
remove 'docs/guide.md'
add 'src/app.py'
remove 'src/retriever.py'
add 'notes.md'
add 'src/prompts.py'
```

. stayed inside `src/` and included the new file and the removal. `-u` reached `README.md` and `docs/guide.md` outside the directory and skipped both new files. `-A` took everything.

Outdated advice. Tutorials written before Git 2.0 say that `git add -u` and `git add -A` cover only the current directory and that `git add <path>` ignores removed files. Both changed in 2.0 (section 5.19).

In production. `git add -A` and `git add .` are how files that nobody meant to track get tracked: an `.env` created before its ignore rule, a dataset, the output of a failed experiment. Use `git add -u` for "what I changed in files Git already knows", name new files explicitly, and run a bulk form with `-n` first in a working tree you have not inspected.

5.9 Unstaging: three commands, two meanings

In one sentence. To unstage a path is to make its index entry equal to HEAD's again:  `git restore --staged <path>` and  `git reset <path>` do that, while  `git rm --cached <path>` deletes the entry, which is the same thing only when HEAD has no such path.

See it. A tracked file with a staged modification, and a new file that has only been added:

```
# A tracked file with a staged modification, and a new file that has only been added.
$ echo 'TOP_K = 8' > src/retriever.py
$ echo 'SYSTEM = "You are a support assistant."' > src/prompts.py
$ git add src/retriever.py src/prompts.py
$ git status --short
A  src/prompts.py
M  src/retriever.py
$ git ls-files --stage
100644 5e83731ea002effc7ffce6c4a2b565cbf889abd8 0      src/prompts.py
100644 5fac60c5a32ec062dcb2163acd05c4c1c20dda0c 0      src/retriever.py
```

```
# restore --staged copies the entry from HEAD. A path HEAD lacks loses its entry.
$ git restore --staged src/retriever.py src/prompts.py
$ git status --short
```

```

M src/retriever.py
?? src/prompts.py
$ git ls-files --stage
100644 1e0b1ade696068e087656f8ae5e859fe92aef3b8 0      src/retriever.py

```

The tracked file's entry holds HEAD's blob `1e0b1ad` again, and the path shows `M`. The new file, which HEAD lacks, lost its entry and is untracked. `git reset <path>` leaves the identical index and reports what remains unstaged:

```

# reset <path> does the same to the index, and reports what is left unstaged.
$ git reset src/retriever.py src/prompts.py
Unstaged changes after reset:
M      src/retriever.py
$ git status --short
M src/retriever.py
?? src/prompts.py
$ git ls-files --stage
100644 1e0b1ade696068e087656f8ae5e859fe92aef3b8 0      src/retriever.py

```

```

# rm --cached removes the entry, whatever HEAD has.
$ git rm --cached src/retriever.py src/prompts.py
rm 'src/prompts.py'
rm 'src/retriever.py'
$ git status --short
D src/retriever.py
?? src/
$ git ls-files --stage
# For the tracked file that is a staged deletion: the next commit would remove it from the project.

```

`git rm --cached` removed both entries. For the new file the outcome is the same. For the tracked file it is a staged deletion (`D`): the next commit would remove `src/retriever.py` from the project, although the file stays on your disk.

	<code>git restore --staged</code>	<code>git reset <path></code>	<code>git rm --cached</code>
Path that HEAD has	staged change undone	staged change undone	deletion staged
Path that HEAD lacks	entry removed	entry removed	entry removed
Repository without commits	fails: HEAD cannot be resolved	works	works
Safety check	none	none	refuses when the staged content matches neither HEAD nor the file

The last two rows, as run:

```

# Index, HEAD and working tree now hold three different versions of one file.
$ echo 'TOP_K = 12' > src/retriever.py
$ git rm --cached src/retriever.py
error: the following file has staged content different from both the
file and the HEAD:
    src/retriever.py
(use -f to force removal)
[exit status: 1]

```

```

# A repository with no commit yet has no HEAD to copy from.
$ cd ..
$ git init -q fresh && cd fresh
$ echo 'TOP_K = 5' > retriever.py
$ git add retriever.py
$ git status
On branch main

```

```

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   retriever.py

$ git restore --staged retriever.py
fatal: could not resolve 'HEAD'
[exit status: 128]
$ git reset retriever.py
[exit status: 0]
$ git status --short
?? retriever.py

```

The guard protects a staged version that would otherwise survive only as an unreachable object ([git-rm](#)); the other two commands have no such check. And before the first commit, `git status` itself recommends `git rm --cached`. There the advice is right, because every staged path is new. It is also where the habit starts.

```

Observed behavior : A developer "unstaged" a file to keep it out of a commit. The commit deleted
                    the file from the project, and teammates lost it on their next pull.
Git state          : The path existed in HEAD. After git rm --cached the index had no entry for
                    it, and status showed "D " and "??" for the same path.
Mechanism          : A path that HEAD has and the index lacks is a deletion in the next commit.
Root cause         : git rm --cached was used to unstage. It removes the entry instead of
                    restoring the one from HEAD.
Why Git does this  : rm --cached exists to stop tracking a file (Chapter 4, section 4.6).
Correct fix        : Before the commit: git restore --staged <path>. After it:
                    git restore --source=HEAD~1 --staged <path>, then a new commit (Lab 2.4).
Prevention         : Unstage with git restore --staged. Read "D " as "the next commit deletes this".

```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git restore --staged <path></code> , <code>git reset <path></code>	unchanged	entry replaced by HEAD's, or removed if HEAD has no such path	unchanged	unchanged	unchanged; the blob that was staged stays as an unreachable object	unchanged	unchanged
<code>git rm --cached <path></code>	unchanged	entry removed	unchanged	unchanged	unchanged	unchanged	unchanged

`git reset` without a path unstages everything. And when the working tree has moved on since the add, the unreachable blob is the only copy of the staged version (Lab 2.1 recovers one).

5.10 Two commit forms that bypass what you staged

In one sentence. 🟢 `git commit -a` stages every modified and deleted tracked file and then commits, and 🟢 `git commit <path>` commits the working tree content of the named paths and nothing else: both read the working tree at commit time, whatever you staged before.

Precisely. With paths, the commit takes "the updated working tree contents of the paths specified on the command line, disregarding any contents that have been staged for other paths"; `-i` (`--include`) instead stages the named paths on top of what is already staged, "usually not what you want unless you are concluding a conflicted merge" ([git-commit](#)).

Command	Tree of the new commit	Staged changes to other paths
<code>git commit -a</code>	the index after staging every tracked modification and deletion	committed
<code>git commit <path></code>	HEAD's tree with the named paths taken from the working tree	stay staged, not committed
<code>git commit -i <path></code>	the index after staging the named paths	committed

See it.

```
# One modified file, one deleted file, one new file. Nothing staged.
$ git status --short
  D docs/old-notes.md
  M src/retriever.py
?? src/prompts.py
$ git commit -a -m "Raise TOP_K and drop old notes"
[main 3a8aac6] Raise TOP_K and drop old notes
 2 files changed, 1 insertion(+), 2 deletions(-)
 delete mode 100644 docs/old-notes.md
$ git status --short
?? src/prompts.py
# -a staged the modification and the deletion. The new file is still untracked.
```

```
# settings.yaml is staged. retriever.py is modified and not staged.
$ git status --short
M  config/settings.yaml
M  src/retriever.py
?? src/prompts.py
$ git commit src/retriever.py -m "Raise TOP_K to 10"
[main b271980] Raise TOP_K to 10
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git show --stat --format=%s HEAD
Raise TOP_K to 10

src/retriever.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git status --short
M  config/settings.yaml
?? src/prompts.py
# The commit took retriever.py from the working tree and left the staged settings.yaml for later.
```

The path from committed `retriever.py` alone and left the staged `settings.yaml` for later. It also overrides partial staging of the file it names. Below, a clean version of `retriever.py` is staged and the working tree holds one more line:

```
# Stage a clean version of retriever.py while the working tree also holds a debug line.
$ git diff --cached -- src/retriever.py
diff --git a/src/retriever.py b/src/retriever.py
index 474e4c9..f028408 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -1,1 +1 @@
- TOP_K = 10
+ TOP_K = 12
$ git diff -- src/retriever.py
diff --git a/src/retriever.py b/src/retriever.py
index f028408..66f5344 100644
--- a/src/retriever.py
+++ b/src/retriever.py
@@ -1,1,2 @@
  TOP_K = 12
+ print("DEBUG TOP_K", TOP_K)
$ git commit src/retriever.py -m "Raise TOP_K to 12"
[main aa38475] Raise TOP_K to 12
 1 file changed, 2 insertions(+), 1 deletion(-)
$ git show HEAD:src/retriever.py
TOP_K = 12
print("DEBUG TOP_K", TOP_K)
```

The commit contains the debug line: what you staged for that path was replaced by the file on disk. A path that Git does not know is an error, not an add, and `--dry-run` previews any commit command:

```
$ git commit src/prompts.py -m "Add prompts"
error: pathspec 'src/prompts.py' did not match any file(s) known to git
```

```
[exit status: 1]
```

```
# Preview what a commit command would record without creating the commit.
$ git commit --dry-run --short -a
M src/retriever.py
?? src/prompts.py
$ git status --short
M src/retriever.py
?? src/prompts.py
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git commit -a</code>	unchanged	entries of modified tracked files updated, entries of deleted files removed	unchanged: still names the branch	moves to the new commit	new blobs, trees and a commit; the reflogs of HEAD and the branch gain an entry	unchanged until you push	unchanged until you push
<code>git commit <path></code>	unchanged	entries of the named paths updated; other staged entries kept	as above	as above	as above	as above	as above

In production. `-a` suits a working tree that holds one finished change. After `git add -p` it is the wrong reflex: it stages again every hunk you left out (Lab 2.2 does this on purpose). `git commit <path>` surprises in two ways: it takes the whole file, and it leaves other staged work behind for a later commit.

5.11 Reading the index: `git ls-files`

In one sentence. `git ls-files` is the plumbing view of the index: it lists entries and, with options, compares them with the working tree, without the interpretation that `git status` adds ([git-ls-files](#)).

Option	Lists
none, <code>-c</code>	every path in the index
<code>-s</code> , <code>--stage</code>	mode, blob ID, stage number and path of every entry
<code>-m</code> , <code>-d</code>	tracked paths whose file differs from the entry (a deleted file counts); tracked paths whose file is missing
<code>-o</code> , <code>--others</code>	paths with no entry; ignore rules apply only with <code>--exclude-standard</code>
<code>-i</code> , <code>--ignored</code>	with <code>-o</code> : untracked paths that an ignore rule matches; with <code>-c</code> : tracked paths that one matches

`-u` restricts the list to unmerged entries (section 5.13), `-v` adds a status tag (section 5.12), and `--error-unmatch`, `--format` and `-z` serve scripts.

See it. The repository has a modified file, a deleted file, an untracked file, two ignored ones, and an `.env` that was committed before its ignore rule:

```
# -m: tracked paths whose working tree file differs from the index. A deleted file counts.
$ git ls-files --modified
config/settings.yaml
src/retriever.py
# -d: tracked paths whose working tree file is gone.
$ git ls-files --deleted
config/settings.yaml
```

```
# -o: paths with no index entry. Ignore rules apply only when you ask for them.
$ git ls-files --others
notes.md
server.log
src/__pycache__/app.cpython-314.pyc
$ git ls-files --others --exclude-standard
notes.md
# -o -i: untracked paths that an ignore rule matches.
$ git ls-files --others --ignored --exclude-standard
server.log
src/__pycache__/app.cpython-314.pyc
# -c -i: TRACKED paths that an ignore rule matches. This finds the already-tracked trap.
$ git ls-files --cached --ignored --exclude-standard
.env
```

`--others` alone lists ignored files too, which is why `--exclude-standard` appears in nearly every real use. The last command is the detector for the already-tracked trap of Chapter 4.

```
# Is this path tracked? The exit status answers.
$ git ls-files --error-unmatch src/app.py
src/app.py
[exit status: 0]
$ git ls-files --error-unmatch notes.md
error: pathspec 'notes.md' did not match any file(s) known to git
Did you forget to 'git add'?
[exit status: 1]
# A custom format, one field at a time.
$ git ls-files --abbrev --format='%(%(objectmode) %(objectname) %(objectsize:padded) %(path))'
100644 5620d7c      28 .env
100644 458a0d0      24 .gitignore
100644 4da99e3      25 config/settings.yaml
100755 08029a1      20 scripts/run_eval.sh
100644 63df51b      38 src/app.py
100644 1e0b1ad      10 src/retriever.py
```

In production. `git ls-files -z | xargs -0 <tool>` runs a formatter or linter on tracked files only and survives names with spaces.

5.12 Two bits that are not an ignore mechanism

In one sentence. 🟡 `git update-index --assume-unchanged` and `--skip-worktree` each set a flag on an index entry that tells Git not to examine the working tree file, and neither is a way to keep private edits to a tracked file, because Git has no such feature.

Precisely. Both flags are documented for other jobs ([git-update-index](#)):

	<code>--assume-unchanged</code>	<code>--skip-worktree</code>
Documented purpose	Speed on filesystems with a slow <code>lstat</code> : "the user promises not to change the file"	Sparse checkout: "avoid writing the file to the working directory when reasonably possible"
Tag in <code>git ls-files -v</code>	lower case, <code>h</code>	<code>S</code>
<code>git add <path></code>	does nothing, silently	refused with a sparse-checkout message
A merge that changes the file	refused: local changes would be overwritten	refused in the same way
<code>git restore <path></code>	overwrites the edit without warning	refused: the pathspec matches nothing

The FAQ answers the underlying wish directly: "How do I ignore changes to a tracked file? Git doesn't provide a way to do this", and the two bits "don't work properly for this purpose and shouldn't be used this way" ([gitfaq](#)).

Inside `.git`. One bit in one entry of your `.git/index`. No commit, ref or configuration file records it, so no clone has it, and rebuilding the index (section 5.15) erases it.

See it. A private edit points the service at a local model server:

```
$ git update-index --assume-unchanged config/settings.yaml
# ls-files -v shows the bit as a lower-case tag.
$ git ls-files -v
h config/settings.yaml
H src/app.py
# Point the service at a local model server. A private edit you do not want to commit.
$ cat config/settings.yaml
model: small-v1
api_base: http://localhost:8080
$ git status --short
$ git diff
```

```
# It is hidden from every command that asks "what changed?", including the ones you want.
$ git add config/settings.yaml
$ git status --short
$ git stash
No local changes to save
```

The edit is hidden from status and diff, and also from `git add` and `git stash`, the commands you would use to save it. Then a teammate's change to the same file arrives:

```
# The teammate changed the same file. Git checks the real file before overwriting it.
$ git merge teammate
error: Your local changes to the following files would be overwritten by merge:
    config/settings.yaml
Please commit your changes or stash them before you merge.
Aborting
Updating b52b27b..0fac3ad
[exit status: 1]
$ git status
On branch main
nothing to commit, working tree clean
# A clean status and a refused merge at the same time. And restore silently discards the edit:
$ git restore config/settings.yaml
$ cat config/settings.yaml
model: small-v1
api_base: https://llm.internal.example
$ git update-index --no-assume-unchanged config/settings.yaml
```

A refused merge and a clean status at the same time. Then  `git restore` replaced the edit with the index version, silently. `--skip-worktree` fails differently and no better:

```
$ git update-index --skip-worktree config/settings.yaml
$ git ls-files -v
S config/settings.yaml
H src/app.py
# The same private edit again.
$ git status --short
$ git add config/settings.yaml
The following paths and/or pathspecs matched paths that exist
outside of your sparse-checkout definition, so will not be
updated in the index:
config/settings.yaml
hint: If you intend to update such entries, try one of the following:
hint: * Use the --sparse option.
hint: * Disable or modify the sparsity rules.
hint: Disable this message with "git config set advice.updateSparsePath false"
[exit status: 1]
```

```
$ git merge teammate
error: Your local changes to the following files would be overwritten by merge:
    config/settings.yaml
Please commit your changes or stash them before you merge.
Aborting
Updating b52b27b..0fac3ad
[exit status: 1]
$ git restore config/settings.yaml
error: pathspec 'config/settings.yaml' did not match any file(s) known to git
[exit status: 1]
$ git status --short
```

```
Observed behavior : git status says "nothing to commit, working tree clean". git merge stops
                    with "Your local changes to the following files would be overwritten".
Git state          : The entry of config/settings.yaml has the assume-unchanged bit (git ls-files
                    -v prints "h"). The file on disk differs from the entry.
Mechanism          : Commands that list changes skip flagged entries. A command that must replace
                    the file checks the real file first, finds the edit and stops.
Root cause         : The bit was used to hide a local edit. It is a promise that there is none.
Why Git does this  : It cannot know whether a local change is precious, so it "has to take the
                    safe route and always preserve them" (gitfaq).
Correct fix        : Clear the bit (--no-assume-unchanged, --no-skip-worktree), then commit,
                    stash or discard the edit in the open.
Prevention         : Private settings in a separate, ignored file. Find leftover bits with
                    git ls-files -v | grep -e '[a-z]' -e '^S'
```

Unverified. Do not read "refused" in the table above as a guarantee. Git decides whether a flagged file still matches its entry from the stat data cached in the index (size and timestamps), not by reading the file every time. Two exercise authors each saw, once, a merge or pull overwrite a hidden edit that had the same size as the committed file and was made within the same second; the run could not be reproduced on demand (four further attempts on Git 2.55.0 for this pass were all refused), so no transcript is shown. See the note in the Module 17 solutions, exercise 17.8. The refusal is a best effort that depends on the cached stat data, which is one more reason not to hide edits behind either bit.

In production. The arrangement that works is the one the FAQ recommends: shared defaults in a tracked file, private overrides in an ignored file that no commit on any branch has ever tracked (Chapter 4, section 4.16), and an application that reads both.

```
# Keep the shared defaults tracked. Put private overrides in a file that no commit has ever tracked.
$ echo 'config/settings.local.yaml' >> .gitignore
$ git add .gitignore
$ git commit -m "Ignore the per-developer settings override"
[main ce22e5e] Ignore the per-developer settings override
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore
$ echo 'api_base: http://localhost:8080' > config/settings.local.yaml
$ git status --short --ignored
!! config/settings.local.yaml
# The teammate change to the tracked defaults now merges, and the private file is untouched.
$ git merge teammate
Merge made by the 'ort' strategy.
 config/settings.yaml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ cat config/settings.yaml config/settings.local.yaml
model: small-v2
api_base: https://llm.internal.example
api_base: http://localhost:8080
```

5.13 Index stages during a conflict

In one sentence. When a merge cannot decide the content of a path, the index holds up to three entries for it, at stages 1, 2 and 3, and resolving the conflict means replacing them with one entry at stage 0.

Precisely. "During a merge, stage 1 is the common ancestor, stage 2 is the target branch's version (typically the current branch), and stage 3 is the version from the branch which is being merged" ([gitrevisions](#)). `:<n>:<path>` names the blob at stage n.

See it. Two branches changed the same line:

```
$ git merge tune-retrieval
Auto-merging config/settings.yaml
CONFLICT (content): Merge conflict in config/settings.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
UU config/settings.yaml
$ git ls-files --stage
100644 4da99e38a05a2a7b436aeea17b946e0288949413 1      config/settings.yaml
100644 9b521fd0f699b5fd45c7618b4e7560d00cc24088 2      config/settings.yaml
100644 2e6bc1476312458dcdd5e7eacf04942211ccb27f 3      config/settings.yaml
```

```
# Stage 1: the merge base. Stage 2: the current branch. Stage 3: the branch being merged.
$ git show :1:config/settings.yaml
model: small-v1
top_k: 5
$ git show :2:config/settings.yaml
model: small-v1
top_k: 3
$ git show :3:config/settings.yaml
model: small-v1
top_k: 8
$ cat config/settings.yaml
model: small-v1
<<<<<<< HEAD
top_k: 3
=====
top_k: 8
>>>>>> tune-retrieval
```

```
# Resolve: write the content you want, then "git add" replaces the three entries with one.
$ printf 'model: small-v1\ntop_k: 8\n' > config/settings.yaml
$ git add config/settings.yaml
$ git ls-files --stage
100644 2e6bc1476312458dcdd5e7eacf04942211ccb27f 0      config/settings.yaml
$ git status --short
M  config/settings.yaml
$ git commit -m "Merge tune-retrieval: keep top_k at 8"
[main 531d34d] Merge tune-retrieval: keep top_k at 8
```

One path, three entries, three blob IDs. Each stage can be read as a complete file while the working tree holds the version with markers. `git add` replaced the three entries with one at stage 0.

Take two facts into Chapter 8 (Merge). An index with unmerged entries cannot be written out as a tree, so `git commit` refuses until every path is back at stage 0. And `git add` does not judge a resolution: it stages the file as it is, conflict markers included if you left them in.

5.14 The cached stat data

In one sentence. Each index entry remembers the size and timestamps that the file had when Git last saw it match the entry, so "has this file changed?" normally costs one `lstat` call instead of reading and hashing the file.

Analogy. A librarian who checks whether a book was touched by looking at the dust on it instead of rereading it. Disturbed dust only means that the book must be reread. The analogy breaks at one point: Git knows when dust can lie, an edit that keeps the size and lands in the same timestamp tick as the index write, and rereads then.

Precisely. In Git's own words, "the index entries record the information obtained from the filesystem via `lstat(2)` system call when they were last updated", and the comparison uses the modification and change times, size, inode, owner, group, file type and executable bit ([racy-git](#)). Matching data mean "unchanged". A mismatch means "possibly changed": plumbing commands report the path as changed, while porcelain commands first *refresh* the index, that is, re-read the file and, when the content still equals the blob, store the new stat data.

See it.

```
# The stat fields are your machine and your clock, so this transcript masks their values with N.
$ git ls-files --debug src/app.py | sed -E '/time|dev|uid/s/[0-9]+/N/g'
src/app.py
  ctime: N:N
  mtime: N:N
  dev: N      ino: N
  uid: N      gid: N
  size: 38    flags: 0
# size is the length of the file on disk. Here it equals the size of the blob.
$ git cat-file -s :src/app.py
38
```

```
# Change the modification time of the file. The content stays the same.
$ touch -t 202001010000 src/app.py
# Plumbing compares stat data only, and reports the entry as possibly changed.
$ git diff-files
:100644 100644 63df51b788f2464137e0f30355056e42a320f705 00000000000000000000000000000000 M
src/app.py
```

```
# Porcelain re-reads the file, finds the same content, and stores the new stat data.
$ git status --short
$ git diff-files
```

After `touch`, the plumbing command `git diff-files` reports `M`; the all-zero ID on the right is its notation for a working tree file that is "out of sync with the index" ([git-diff-files](#)). `git status` printed nothing: it re-read the file, found the same content and wrote the new stat data back, so the second `git diff-files` is silent as well. 🟢 `git update-index --refresh` does the same on request and names the files that have a real change:

```
# A real edit: refresh cannot make this entry match, and says so.
$ echo 'TOP_K = 10' > src/retriever.py
$ git update-index --refresh
src/retriever.py: needs update
[exit status: 1]
$ git diff-files
:100644 100644 1e0b1ade696068e087656f8ae5e859fe92aef3b8 00000000000000000000000000000000 M
src/retriever.py
```

In production. Speed: a clean working tree costs about one `lstat` call per tracked file and no file reads, and after anything that gives the files new timestamps or inodes, the next command hashes them all once. Correctness: a script that asks plumbing must refresh first.

```
# A release script asks plumbing: does anything differ from HEAD? Exit status 0 means no.
$ git diff-index --quiet HEAD
[exit status: 0]
# Copy the repository, as a build context or a restored CI cache does. No file content changes.
$ cp -R ../support-bot ../build-copy
$ cd ../build-copy
$ git diff-index --quiet HEAD
[exit status: 1]
$ git diff-files --name-status
```

```
M    config/settings.yaml
M    src/app.py
```

```
# Refresh: re-read the files whose stat data differ, and store the new stat data when the content
matches.
$ git update-index -q --refresh
$ git diff-index --quiet HEAD
[exit status: 0]
$ git diff-files --name-status
```

A copy of a clean repository is "dirty" to `git diff-index --quiet HEAD` until the index is refreshed. This is one way in which a release build of an unchanged commit gets labelled dirty inside a container or after a cache restore. Run `git update-index -q --refresh` before plumbing checks, or use `git status --porcelain`.

5.15 The index file in operation: the lock, and a rebuild

In one sentence. Every writer creates `.git/index.lock`, writes a complete new index into it and renames it over `.git/index`; a leftover lock therefore blocks all writers, and a damaged index can be discarded and rebuilt from HEAD at the price of whatever was only staged.

Precisely. Git's lock-file code states the protocol: the `.lock` file is created "with `O_CREAT|O_EXCL` so that we can notice and fail if somebody else has already locked the file", and exit and signal handlers remove it ([lockfile.h](https://www.kernel.org/doc/Documentation/git-lockfile.txt)). A lock without a process therefore means a crash, a `kill -9` or a power loss. A lock with a live process is normal: an editor, a hook, another terminal.

See it.

```
# Imitate a Git process that died while it was writing the index.
$ touch .git/index.lock
$ echo 'TOP_K = 8' > src/retriever.py
$ git add src/retriever.py
fatal: Unable to create '$LAB/ch05/index-lock-and-rebuild/support-bot/.git/index.lock': File exists.
```

```
Another git process seems to be running in this repository, or the lock file may be stale
[exit status: 128]
# Reading still works. Only writers need the lock.
$ git status --short
M src/retriever.py
# After checking that no Git process is running, remove the stale lock.
$ rm .git/index.lock
$ git add src/retriever.py
$ git status --short
M src/retriever.py
```

Readers work and writers fail. Before you remove a lock 🟡, check that no Git process is running (`pgrep -fl git`).

```
# One staged change (retriever.py) and one unstaged change (settings.yaml) exist.
$ echo 'temperature: 0.2' >> config/settings.yaml
$ git status --short
M config/settings.yaml
M src/retriever.py
# Now the index file is damaged.
$ printf 'garbage' > .git/index
$ git status
fatal: .git/index: index file smaller than expected
[exit status: 128]
```

```
# Delete the damaged file. With no index, every path in HEAD looks deleted and every file looks new.
$ rm .git/index
$ git status --short
D config/settings.yaml
```

```

D src/app.py
D src/retriever.py
?? config/
?? src/
# Rebuild the index from HEAD. A mixed reset writes the index and leaves the working tree alone.
$ git reset
Unstaged changes after reset:
M config/settings.yaml
M src/retriever.py
$ git status --short
M config/settings.yaml
M src/retriever.py

```

```

# The files are intact. What is gone is the knowledge of which change was staged.
$ git diff --stat
config/settings.yaml | 1 +
src/retriever.py      | 2 +-
2 files changed, 2 insertions(+), 1 deletion(-)
$ git fsck
dangling blob 5fac60c5a32ec062dcb2163acd05c4c1c20dda0c

```

Without an index, every path of HEAD looks like a staged deletion and every file looks untracked. 🟡 `git reset` wrote a new index from HEAD and left the files alone. What is lost is the knowledge of what was staged (both changes are unstaged now), together with any intent-to-add entries and the bits of section 5.12. The blob that was staged for `retriever.py` is still in the object database: `git fsck` lists it as dangling. Do not rebuild during a merge, because the conflict stages exist only in the index.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git reset</code> (no path, no commit)	unchanged	rewritten from HEAD's tree; staging state and flag bits gone	unchanged	unchanged	<code>ORIG_HEAD</code> written; the HEAD reflog gains an entry	unchanged	unchanged
<code>git update-index --[no-]assume-unchanged, --[no-]skip-worktree</code>	unchanged	one flag bit of the entry	unchanged	unchanged	unchanged	unchanged	unchanged

In production. The lock message in CI usually means two Git processes in one working tree. Background tools should run `git --no-optional-locks status`, or set `GIT_OPTIONAL_LOCKS=0`, so that their optional refresh of the index never takes the lock ([git-status](#), "Background refresh").

5.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
The commit lacks your latest edit	Still modified after the commit (section 5.3)	<code>git add</code> , then a new commit or <code>--amend</code>	<code>git diff --cached</code> before every commit
The commit contains a line you left out with <code>-p</code>	It was made with <code>-a</code> or with a path	<code>git reset HEAD~1</code> if private, stage again, plain <code>git commit</code> (Lab 2.2)	No <code>-a</code> , no path after partial staging
A new file is missing from the commit	<code>git status</code> shows <code>??</code>	<code>git add</code> , commit	<code>git add -N</code> when you create a file
A commit deleted a file you meant to unstage	<code>git show --stat HEAD</code> says "delete mode" (section 5.9)	<code>git restore --source=HEAD~1 --staged <path></code> , commit (Lab 2.4)	Unstage with <code>git restore --staged</code>

Symptom	Diagnosis	Fix	Prevention
Clean status, yet a merge refuses to overwrite local changes	<code>git ls-files -v</code> shows <code>h</code> or <code>S</code>	Clear the bit; commit, stash or discard the edit	Never hide edits with the bits
"Unable to create ... index.lock"	<code>pgrep -fl git</code>	Wait, or remove a stale lock	<code>--no-optional-locks</code> for background tools
"index file smaller than expected"	<code>git status</code> , <code>git diff</code> and <code>git fsck</code> fail; <code>git log</code> works	<code>rm .git/index</code> , <code>git reset</code> , stage again	None
A script reports a dirty tree in a fresh copy	<code>git diff-files</code> lists every file, <code>git status</code> none	<code>git update-index -q --refresh</code>	Refresh before plumbing checks
A staged version is gone after unstaging	<code>git fsck</code> lists dangling blobs	<code>git cat-file -p <id></code> > <path> (Lab 2.1)	Commit early

5.17 When not to use it, and dangerous edge cases

A hunk edit can stage content that exists nowhere. Take the hunk of section 5.5 again and stage only the `top_k` change: turn `-temperature` into a context line and delete two `+` lines.

```
# Start again. This time stage only the top_k change.
$ git restore --staged config/settings.yaml
# In the editor: turn "-temperature: 0.7" into a context line, delete "+temperature: 0.2" and "+debug: true".
$ git add -p
diff --git a/config/settings.yaml b/config/settings.yaml
index bfcfe5c..d56fa8e 100644
--- a/config/settings.yaml
+++ b/config/settings.yaml
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
-temperature: 0.7
+top_k: 8
+temperature: 0.2
+debug: true
 max_tokens: 512
(1/1) Stage this hunk [y,n,q,a,d,e,p,P,]? e
--- the hunk as saved (comment lines left out) ---
@@ -1,4 +1,5 @@
 model: small-v1
-top_k: 5
 temperature: 0.7
+top_k: 8
 max_tokens: 512
```

```
# The staged file. Compare the order of its lines with HEAD and with the working tree.
$ git show :config/settings.yaml
model: small-v1
temperature: 0.7
top_k: 8
max_tokens: 512
$ git show HEAD:config/settings.yaml
model: small-v1
top_k: 5
temperature: 0.7
max_tokens: 512
$ cat config/settings.yaml
model: small-v1
top_k: 8
temperature: 0.2
```

```
debug: true
max_tokens: 512
```

The patch applied. But in the saved hunk the context line `temperature: 0.7` stands before `+top_k: 8`, so the staged file has the two keys in an order that neither HEAD nor the working tree has. For YAML keys that is harmless. For two statements of a program it is behavior that no file on disk ever had. In the editor, move the `+` line up to its `-` line, and after every `e` read `git diff --cached`.

Do not stage partially what cannot stand alone. If the accepted hunks do not build without the rejected ones, that commit is broken in history although your working tree works, and `git bisect` (Chapter 14A, History investigation) will stop on it one day.

The index is not storage. A blob that only the index refers to becomes unreachable the moment its entry changes, and garbage collection prunes unreachable loose objects after two weeks by default. Work you want to keep belongs in a commit, on a throwaway branch if need be.

The two bits in team documentation. A README that tells every developer to run `--skip-worktree` on a config file creates state that no clone and no CI image reproduces, and it fails at the next upstream change to that file.

When staging adds nothing. One finished change in an otherwise clean working tree: `git commit -a` after reading `git status` is fine. The index earns its keep when the working tree holds more than one change.

5.18 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git diff</code> , <code>git ls-files</code> , <code>git diff-files</code> , <code>git diff-index</code> , <code>git write-tree</code> , <code>git update-index --refresh</code> , <code>git checkout-index --prefix=<dir>/</code>	● SAFE	Nothing, or only tree objects, cached stat data, files below <code><dir></code>	not needed	not needed
<code>git add</code> in all its forms	● SAFE	Blobs and index entries; replaces what was staged for those paths	<code>git add -n</code> ; <code>-p</code> shows each hunk	<code>git restore --staged <path></code>
<code>git commit -a</code> , <code>git commit <path></code>	● SAFE	Index, a new commit, the branch ref	<code>--dry-run</code>	<code>git reset HEAD~1</code> ● while the commit is private (Chapter 11)
<code>git restore --staged <path></code> , <code>git reset [<path>]</code>	● CAUTION	Index entries copied from HEAD	<code>git diff --cached</code>	<code>git add</code> again; <code>git fsck</code> if the file has changed since
<code>git rm --cached <path></code>	● CAUTION	Removes the entry: a deletion if HEAD has the path	<code>git rm --cached -n</code>	<code>git restore --staged <path></code>
<code>git update-index --assume-unchanged</code> , <code>--skip-worktree</code>	● CAUTION	A flag bit that hides later edits	<code>git ls-files -v</code>	the <code>--no-</code> forms
<code>rm .git/index.lock</code>	● CAUTION	Removes a lock; unsafe while a Git process is writing	<code>pgrep -fl git</code>	none needed for a stale lock
<code>rm .git/index</code> , then <code>git reset</code>	● DANGEROUS	Discards staging state, flag bits, conflict stages, outside Git's locking. What it can destroy: the record of what was staged and not committed, which no reflog holds	<code>git diff --cached</code> ; move the file aside instead of deleting it	<code>git fsck</code> for blobs that were staged; move the file back. Appropriate only for a corrupt index (section 5.15)

Command	Label	What it changes	Preview	Recovery
<code>git restore <path></code> on a file with the assume-unchanged bit	 DANGEROUS	Overwrites the hidden edit without warning	clear the bit, then <code>git diff</code>	none (Chapter 4, section 4.7 answers the five questions)

5.19 Version notes

Version note. Older behavior: `git add -u` and `git add -A` without a pathspec covered only the current directory, and `git add <path>` ignored removed files. Current behavior: both cover the whole working tree, and `git add <path>` records removals. Since: Git 2.0 ([release notes](#)). Recommended: `git add -u .` to stay in the current directory.

Version note. Older behavior: a path added with `git add -N` appeared in `git diff` as a change to an empty file, and an index holding only such paths let `git commit` create an empty commit. Current behavior: `git diff` shows a new file, and the commit is refused (section 5.6). Since: Git 2.19 for the diff and 2.11 for the commit ([2.19](#), [2.11](#)). Recommended: nothing to change.

Version note. Older behavior: none; the features did not exist. Current behavior: as shown above. Since: `git add -p` 1.5.4; `git add -N` and `--staged` 1.6.1 ([release notes](#)); `git restore --staged` 2.23 (Chapter 4, section 4.18); `git ls-files --format` 2.38. Recommended: write `restore --staged`, read `reset HEAD`; only `--format` needs a version check.

5.20 Practice

- Labs 2.1 (the three-trees prediction table), 2.2 (partial staging) and 2.4 (unstage three ways) in the Module 2 lab manual. Lab 2.3 there belongs to Chapter 4.
- Replay any transcript with `labs/run ch05/<demo>`, for example `labs/run ch05/add-patch-edit`. The sandbox stays in place for your own experiments.
- Three drills, each in a sandbox left by a replay:
 1. In `ch05/three-diffs`, predict the three diffs after `git add src/retriever.py`, then run them.
 2. In `ch05/add-patch-edit`, repeat the second edit by hand and place `+top_k: 8` so that the staged order is right.
 3. In `ch05/stat-cache`, run `git ls-files --debug` before and after `git status` on a touched file and say which field changed.

5.21 Interview questions

1. What is in the index when nothing is staged? Prove it with two commands.
2. A file shows MM in `git status --short`. Which three versions exist, where is each stored, and which one does `git commit` record?
3. What exactly does `git add` write, and what remains of it after `git restore --staged`? Does a push send it?
4. `git diff HEAD` prints nothing. Can `git commit` still create a non-empty commit? Construct the case.
5. You staged half of a file with `git add -p`, using `e` where `s` was not offered. Why is that commit riskier than a normal one, what can `e` get wrong, and how do you test what you are about to commit?

6. Compare `git restore --staged`, `git reset <path>` and `git rm --cached` for a path that HEAD has, for a path that HEAD lacks, and in a repository without commits.
7. A developer ran `git commit -a` after careful partial staging. What did the commit contain, and what is the lowest-risk fix before and after a push?
8. Why are `--assume-unchanged` and `--skip-worktree` not ways to ignore local changes? What do you recommend to a team that needs per-developer settings?
9. `git status` is clean, and `git diff-index --quiet HEAD` in the release script exits with 1. Explain the mechanism and fix the script.
10. One CI job fails because `.git/index.lock` exists, another because the index file is corrupt. For each: diagnosis, fix, and what is lost.

5.22 Sources

Primary sources

- [git-add](#), [git-diff](#), [git-commit](#), [git-restore](#), [git-reset](#), [git-rm](#), [git-ls-files](#), [git-update-index](#), [git-checkout-index](#), [git-write-tree](#), [git-diff-files](#), [git-status](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text used for the transcripts.
- [gitdatamodel](#), [gitformat-index](#), [gitrevisions](#) and [gitfaq](#) ("How do I ignore changes to a tracked file?").
- [racy-git](#): the cached stat data and its one known weakness. [lockfile.h](#): the lock-file protocol.
- Release notes [2.0.0](#), [1.6.1](#), [2.11.0](#), [2.19.0](#); 1.5.4 and 2.38.0 as shipped with Git 2.55.0.

Secondary sources

- Pro Git, [Interactive Staging](#). Caveat: its prompt and its `git stash save --patch` come from an older Git.
- Pro Git, [Reset Demystified](#): the three trees, with the index as "your proposed next commit". Caveat: it says "working directory" for the working tree.
- The Phase 0 report of this course, sections 11 and 12: the Stack Overflow figure and the survey of ten beginner resources (outline-based; transcripts were not reviewed).

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026: the staging area as part of the data model; the `notes` list `add -p`. Caveats: describes object IDs as SHA-1 only; the demo starts on `master`.
- [Git for Professionals](#), Tobias Günther for freeCodeCamp, September 2021: commit craft with `git add -p`. Caveat: 2021 interface.

Further reading

- [git-sparse-checkout](#), the feature that the skip-worktree bit was built for.

Chapter 6: Commits

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch06/`.

6.1 Why this matters

Three questions a CTO can ask about one week of history.

1. "Security reviewed commit X on Friday. Monday's build was made from commit Y. The diff between X and Y is empty. Did we deploy reviewed code or not?"
2. "`git log` dates this change 24 August. The release notes list it under 'merged since 1 September'. Which system is wrong?"
3. "Half of our release manager's commits appear on GitHub without her avatar, under a name that is not a link. Did somebody else push them?"

None of the three is a bug. (1) A commit ID is the hash of the whole commit object, and that object contains the moment the commit was created. An amend, a rebase or a cherry-pick writes a new object with the same content and a different ID (sections 6.4 and 6.7; Lab 3.2). (2) A commit records two people and two times, and Git prints one date and filters by the other (section 6.5). (3) Git copies a name and an email address from her configuration into each commit and checks neither; GitHub links a commit to an account by that email address (sections 6.11 and 6.13).

Hold on to one idea through the chapter: a commit is a small text object that never changes after it is written. Every command that seems to change one (amend, rebase, cherry-pick, sign) writes another object and moves a ref.

6.2 What a commit is

In one sentence. A commit is an immutable object that names one complete snapshot of the project, the commit or commits it was built on, who wrote the change and when, who created the commit and when, and a message.

Analogy. An entry in a ledger whose page numbers are computed from what is written on the page. Each entry points at a full inventory sheet, cites the page number of the entry before it, and names who did the work and who entered it. Change one character and the page number changes. The analogy breaks at the binding: a ledger is one sequence, while an entry in Git may cite two earlier entries (a merge) or none (a root).

Precisely. The manual lists five required fields ([gitdatamodel](#)):

Field	Content	Count
<code>tree</code>	The tree object of the top-level directory: the full snapshot	one
<code>parent</code>	A commit this one was built on	none (root), one (ordinary), two or more (merge)
<code>author</code>	Name, email, time and time-zone offset of the person who wrote the change	one
<code>committer</code>	The same for the person who created this commit object	one
<code>message</code>	Free text after one empty line	one, possibly empty

Optional headers can follow the committer line: `encoding`, written when `i18n.commitEncoding` declares a legacy encoding ([git-commit](#)), and `gpgsig`, a signature (section 6.11). A commit holds no diff, no branch name and no file names. File names live in trees, and "Git does not store the diff for a commit": `git show` "calculates the diff from its parent on the fly" (`gitdatamodel`).

Inside `.git`. One object per commit in the object database (Chapter 3). Nothing else describes a commit: there is no table of commits and no per-branch list. A branch reaches its commits by following `parent` IDs from its tip.

See it. A new repository, `evalkit`, and its first commit:

```
$ git init evalkit
Initialized empty Git repository in $LAB/ch06/commit-anatomy/evalkit/.git/
$ cd evalkit
$ git add README.md evalkit/metrics.py
$ git commit -m "Add exact-match metric"
[main (root-commit) 51d62b3] Add exact-match metric
2 files changed, 5 insertions(+)
create mode 100644 README.md
create mode 100644 evalkit/metrics.py
```

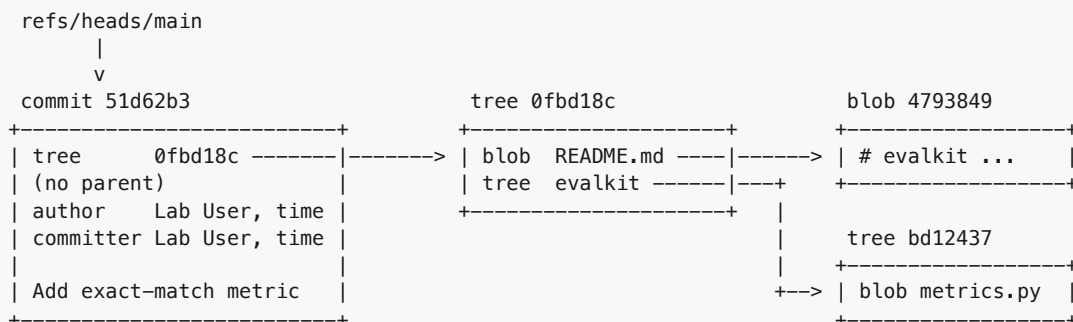
`(root-commit)` in the summary line says that this commit has no parent. Now read the object itself:

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 0fbd18cca19ea00c195639456eecd36debe578ab
author Lab User <you@example.com> 1788755700 +0530
committer Lab User <you@example.com> 1788755700 +0530

Add exact-match metric
$ git cat-file -p 'HEAD^{tree}'
100644 blob 4793849f9215e303b89dd4fb1ab163499fb69815    README.md
040000 tree bd12437744ea02734adf313860ecfb4509b509f    evalkit
```

`git cat-file -p` prints the object as stored. The `tree` line names the snapshot, and `HEAD^{tree}` asks for that tree: a blob for `README.md` and a subtree for `evalkit`. There is no parent line. `author` and `committer` end with a time in seconds since 1 January 1970 UTC and an offset from UTC. One empty line separates headers and message.

Picture.



In production. A commit ID in a deploy record or an evaluation report identifies the code exactly, and nothing outside the commit. A run started from a working tree with uncommitted edits executed code that no commit ID describes, and trackers may not notice: MLflow's classic run context records the commit and does not inspect uncommitted changes ([git context source](#)). Record `git rev-parse HEAD` together with `git status --porcelain`, or refuse a tracked run when that output is not empty (Chapter 28).

6.3 How `git commit` creates a commit

In one sentence. `git commit`  SAFE turns the index into tree objects, wraps the top tree in a new commit object whose parent is the current HEAD commit, and moves the current branch to that object.

Precisely. The steps, in the order in which their results depend on each other:

1. **Content.** The index, also called the staging area, is the proposed snapshot (Chapter 5).
2. **Checks and message.** The `pre-commit` hook runs before the message is obtained, and the `commit-msg` hook can reject the message ([githooks](#); Chapter 14C). `--no-verify` skips both.
3. **Trees.** Git builds one tree per directory from the index entries. A tree that the object database already holds keeps its ID, and nothing new is stored. `git write-tree` performs this step alone.
4. **Parent.** The commit that HEAD resolves to becomes the parent. A first commit has none; a merge in progress adds the commits in `MERGE_HEAD`.
5. **Commit object.** Tree ID, parent IDs, author, committer and message are written as one object. Its hash is the commit ID.
6. **Refs.** The ref that HEAD names is set to the new ID, and the reflogs of HEAD and of the branch each gain a line.

See it. Edit a file, stage it, and compare what the plumbing reports before the commit with the commit itself:

```
# evalkit/metrics.py has been edited: it now also defines f1().
$ git add evalkit/metrics.py
$ git write-tree
27d56e4ea4080b5719bf97b9985f6f543eda349d
$ git rev-parse HEAD
51d62b3de231309f8aba10baba050cf2220fcc61
$ git commit -m "Add F1 metric"
[main 0d77920] Add F1 metric
 1 file changed, 9 insertions(+)
$ git cat-file -p HEAD
tree 27d56e4ea4080b5719bf97b9985f6f543eda349d
parent 51d62b3de231309f8aba10baba050cf2220fcc61
author Lab User <you@example.com> 1788756120 +0530
committer Lab User <you@example.com> 1788756120 +0530

Add F1 metric
```

`git write-tree` printed 27d56e4, and the new commit's `tree` line carries the same ID: the commit recorded the index. `git rev-parse HEAD` printed 51d62b3, which is now the `parent` line.

```
$ cat .git/HEAD
ref: refs/heads/main
$ git rev-parse main
0d77920ea1ff22bc46fc5051fd6e5d86078a54
$ git reflog
0d77920 HEAD@{0}: commit: Add F1 metric
51d62b3 HEAD@{1}: commit (initial): Add exact-match metric
$ git reflog show main
0d77920 main@{0}: commit: Add F1 metric
51d62b3 main@{1}: commit (initial): Add exact-match metric
```

`.git/HEAD` still holds the same text. The commit moved the branch, and HEAD follows only because it names the branch (Chapter 7 builds on this). Both reflogs gained an entry.

Inside `.git`. A second sandbox with a similar edit, this time comparing the object database before and after each command. `git cat-file --batch-all-objects --batch-check` prints ID, type and size of every object:

```
# evalkit/metrics.py has been edited: it now also defines f1().
$ objects() { git cat-file --batch-all-objects --batch-check | sort; }
```

```
$ objects > ../objects-before-add.txt
$ git add evalkit/metrics.py
$ objects | comm -13 ../objects-before-add.txt -
5ca6473ff5c0d818666928cda8551455b0794cd2 blob 154
```

`git add` wrote the blob: the content is in the object database before any commit exists.

```
$ objects > ../objects-before-commit.txt
$ files() { find .git -type f -exec shasum {} + | sort; }
$ files > ../files-before-commit.txt
$ git commit -q -m "Add F1 metric"
$ objects | comm -13 ../objects-before-commit.txt -
1ed5d56b9ea6962262eb6c2e5634e45c7543b391 tree 38
29b018434273c541b9fce4dfc1e29fb92a358447 tree 71
529480e39c5092c92c130d091ce54617310747b7 commit 214
```

The commit wrote three objects: a tree for `evalkit/`, a tree for the top level, and the commit. It wrote no blob, and the new top-level tree points at the `README.md` blob that the first commit already uses.

```
# Every file under .git that is new, or whose content differs from the snapshot taken before the
commit:
$ files | comm -13 ../files-before-commit.txt - | awk '{print $2}' | sort
.git/COMMIT_EDITMSG
.git/index
.git/logs/HEAD
.git/logs/refs/heads/main
.git/objects/1e/d5d56b9ea6962262eb6c2e5634e45c7543b391
.git/objects/29/b018434273c541b9fce4dfc1e29fb92a358447
.git/objects/52/9480e39c5092c92c130d091ce54617310747b7
.git/refs/heads/main
$ cat .git/COMMIT_EDITMSG
Add F1 metric
$ cat .git/HEAD
ref: refs/heads/main
```

Eight files: three objects, the branch ref, two reflogs, the index (same entries, refreshed bookkeeping), and `COMMIT_EDITMSG` with the last message. `.git/HEAD` is not in the list.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git commit</code>	unchanged	entries unchanged; file rewritten	unchanged; resolves to the new commit	set to the new commit	new tree and commit objects; one line in <code>logs/HEAD</code> and in the branch reflog; <code>COMMIT_EDITMSG</code>	unchanged	unchanged
<code>git commit</code> in detached HEAD	unchanged	entries unchanged; file rewritten	set to the new commit ID	no current branch; no branch moves	new objects; one line in <code>logs/HEAD</code> only	unchanged	unchanged

In production. Nothing in a commit says which branch it is on. It goes wherever HEAD pointed when you ran the command: onto `main` if HEAD named `main`, onto no branch if HEAD was detached in a CI checkout. Check `git status` before you commit, not after.

6.4 The commit ID

In one sentence. A commit ID is the hash of the commit object, so one ID names the snapshot, the metadata and, through the parent IDs, the whole history behind the commit.

Precisely. Git hashes the bytes `commit <size>`, one NUL byte, and the object content. With the default object format the hash is SHA-1 and the ID has 40 hexadecimal digits; a SHA-256 repository has 64 (Chapter 3). Two properties follow.

- **Same bytes, same ID**, on any machine. The replays in this book pin the identity and the clock, which is the only reason your IDs can equal the printed ones.
- **Any change, new ID.** The `parent` line contains the parent's ID, so an ID depends on every ancestor. Nobody can alter an old commit without changing the ID of every commit after it.

See it. Compute an ID by hand, then let Git confirm it. `git hash-object -t commit --stdin` hashes the text the way `git commit` does:

```
$ git rev-parse HEAD
1f9c5d8d7bd2b007d3ee4ff4e5c023c01bb40a29
$ git cat-file -s HEAD
214
$ (printf 'commit %s\n' "$(git cat-file -s HEAD)"; git cat-file commit HEAD) | shasum
1f9c5d8d7bd2b007d3ee4ff4e5c023c01bb40a29 -
$ git cat-file commit HEAD | git hash-object -t commit --stdin
1f9c5d8d7bd2b007d3ee4ff4e5c023c01bb40a29
```

Rebuild the same commit from its parts. `git commit-tree` writes a commit object from a tree, a parent and a message; the dates come from the environment:

```
$ git cat-file -p HEAD
tree 29b018434273c541b9fce4dfc1e29fb92a358447
parent d4c9fab9326ab4edbe04ed3a6f5f0b001bf6d86
author Lab User <you@example.com> 1788755640 +0530
committer Lab User <you@example.com> 1788755640 +0530

Add F1 metric
$ mk() { GIT_AUTHOR_DATE="$1" GIT_COMMITTER_DATE="$2" git commit-tree -p "$3" -m "$4" "$5"; }
$ T='@1788755640 +0530'
$ mk "$T" "$T" HEAD~1 'Add F1 metric' 'HEAD^{tree}'
1f9c5d8d7bd2b007d3ee4ff4e5c023c01bb40a29
```

Identical inputs gave `1f9c5d8` again. Now change one input at a time:

```
# Each call changes exactly one input of the call above.
$ mk "$T" "$T" HEAD~1 'Add F1 metric.' 'HEAD^{tree}' # message: one more character
f22660f55b1a794df68ae4053dcaa8248a3d8636
$ mk '@1788755641 +0530' "$T" HEAD~1 'Add F1 metric' 'HEAD^{tree}' # author date: one second later
6ace11eacd33eda2b2c22c2f05b3a30a0f94ac63
$ mk "$T" '@1788755641 +0530' HEAD~1 'Add F1 metric' 'HEAD^{tree}' # committer date: one second later
8d3a6660cc08d014e4d3eb108aa22364504ecb86
$ mk '@1788755640 +0000' "$T" HEAD~1 'Add F1 metric' 'HEAD^{tree}' # author time zone: same instant, other offset
1a5c7c949d901b1455717b67cb1fad6b9e7e0c49
$ mk "$T" "$T" HEAD 'Add F1 metric' 'HEAD^{tree}' # parent: HEAD instead of HEAD~1
5be2270d868fdb4922293b1051405f2c660f5756
$ mk "$T" "$T" HEAD~1 'Add F1 metric' 'HEAD~1^{tree}' # tree: the previous snapshot
097a2da44a07874ffc33633728cb207e29e89358
$ GIT_AUTHOR_EMAIL=You@example.com mk "$T" "$T" HEAD~1 'Add F1 metric' 'HEAD^{tree}' # author email: capital Y
b96c648475d2e36d3c55883a81e1473b2868e4fd
```

Seven changes, seven new IDs. One second on the committer date is enough. The same instant with another offset is a different commit, because the offset is part of the text. So is a capital letter in an email address.

Abbreviated IDs. Any unique prefix of at least four characters names an object. Git prints abbreviations whose length it computes from the size of the repository (`core.abbrev`), seven characters in these sandboxes. A prefix that is unique today can become ambiguous later, so deploy records and model cards should store the full ID.

In production. This is the first CTO question.

Observed behavior : the deployed commit is not the reviewed commit, yet git diff between the two is empty

Git state : two commit objects with the same tree and the same parent; the branch points at the newer one

Mechanism : an amend, a rebase or a cherry-pick wrote a new commit object with a new committer line

Root cause : the commit ID is the hash of the commit object, and the committer time is part of that object

Why Git does this : an ID that covers every byte lets any two repositories agree on history by comparing IDs alone

Correct fix : prove equal content with git rev-parse X^{tree} Y^{tree}; then move the branch back to X or re-approve Y

Prevention : do not rewrite a commit after review or deployment; gate deploys on the approved ID, not on a branch name

6.5 Author and committer, two times, time zones

In one sentence. The author is the person who wrote the change and the committer is the person who created this commit object; each is recorded with a name, an email address, a time and a time-zone offset.

Precisely. For both identities Git reads the environment first (GIT_AUTHOR_NAME , GIT_AUTHOR_EMAIL , GIT_AUTHOR_DATE and the three GIT_COMMITTER_ variables), then user.name and user.email , and as a last resort guesses from the system user name and host name (git-commit, "Commit information"). --author and --date set the author fields of one commit. The committer has no such option: only the environment, the configuration and the clock.

Operation	Author	Committer
git commit	you, now	you, now
git commit --author=... --date=...	as given	you, now
git cherry-pick, git rebase, git commit --amend	copied from the original commit	you, now
git commit --amend --reset-author	you, now	you, now
git am applying a mailed patch	name from the mail's From: line, date from its Date: line (git-am)	you, now

See it. Asha wrote a commit on her branch. You cherry-pick it onto main :

```
$ git log -1 --format=fuller asha/bleu
commit da5c9a3495f90d9d260daeecb19aa556173c4ae0
Author: Asha Rao <asha@example.com>
AuthorDate: Mon Sep 7 10:05:00 2026 +0530
Commit: Asha Rao <asha@example.com>
CommitDate: Mon Sep 7 10:05:00 2026 +0530

    Add BLEU metric skeleton
$ git cherry-pick asha/bleu
[main 0cf479c] Add BLEU metric skeleton
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 evalkit/bleu.py
$ git log -1 --format=fuller
commit 0cf479ca16372d2f084f0f13e3b109a1eafaa883
Author: Asha Rao <asha@example.com>
AuthorDate: Mon Sep 7 10:05:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:08:00 2026 +0530
```

```
Add BLEU metric skeleton
```

The new commit `0cf479c` keeps Asha as author with her time, and records you as committer three minutes later. The author can also be set by hand, here for a file that Ravi mailed from another time zone:

```
# Ravi mailed you evalkit/rouge.py from San Francisco. You commit it under his name and his date.
$ git add evalkit/rouge.py
$ git commit --author='Ravi Menon <ravi@example.com>' --date='2026-09-03T09:30:00-07:00' -m 'Add
ROUGE-L metric skeleton'
[main c757945] Add ROUGE-L metric skeleton
Author: Ravi Menon <ravi@example.com>
Date: Thu Sep 3 09:30:00 2026 -0700
1 file changed, 2 insertions(+)
create mode 100644 evalkit/rouge.py
$ git cat-file -p HEAD
tree e0a2e8aeef531e14e6b6665551ead658c6e70782
parent 0cf479ca16372d2f084f0f13e3b109a1eafaa883
author Ravi Menon <ravi@example.com> 1788453000 -0700
committer Lab User <you@example.com> 1788756060 +0530

Add ROUGE-L metric skeleton
```

```
$ git log -1 --format='author: %ad%ncommitter: %cd'
author: Thu Sep 3 09:30:00 2026 -0700
committer: Mon Sep 7 10:11:00 2026 +0530
$ git log -1 --format='author: %ad%ncommitter: %cd' --date=iso-local
author: 2026-09-03 22:00:00 +0530
committer: 2026-09-07 10:11:00 +0530
$ git log -1 --format='author: %ad%ncommitter: %cd' --date=unix
author: 1788453000
committer: 1788756060
```

The seconds identify the instant; the offset only says what the wall clock showed where the commit was made. By default Git prints each date with its recorded offset. `--date=iso-local` converts to your zone: Ravi's 09:30 at UTC-7 is 22:00 at UTC+5:30. `--date=unix` prints the stored seconds.

An amend keeps the author and renews the committer; `--reset-author` makes you the author as well:

```
# HEAD is the cherry-picked commit again: author Asha, committer you.
$ git commit --amend --no-edit
[main e79045d] Add BLEU metric skeleton
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 evalkit/bleu.py
$ git log -1 --format=fuller
commit e79045d6ec8bd3e6d974c4ce880488e9835e515a
Author: Asha Rao <asha@example.com>
AuthorDate: Mon Sep 7 10:05:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:22:00 2026 +0530

Add BLEU metric skeleton
$ git commit --amend --no-edit --reset-author
[main 42cbfdd] Add BLEU metric skeleton
1 file changed, 2 insertions(+)
create mode 100644 evalkit/bleu.py
$ git log -1 --format=fuller
commit 42cbfdd550e822dbb810fb7d8071b628b0b47db1
Author: Lab User <you@example.com>
AuthorDate: Mon Sep 7 10:24:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:24:00 2026 +0530

Add BLEU metric skeleton
```

Contribution counts depend on which identity is counted:

```
$ git log --format='%h author: %an committer: %cn %s'
c757945 author: Ravi Menon committer: Lab User Add ROUGE-L metric skeleton
0cf479c author: Asha Rao committer: Lab User Add BLEU metric skeleton
d4c9fab author: Lab User committer: Lab User Add exact-match metric
$ git shortlog -sn HEAD
1 Asha Rao
1 Lab User
1 Ravi Menon
$ git shortlog -sn --committer HEAD
3 Lab User
$ git log --oneline --author=Asha
0cf479c Add BLEU metric skeleton
$ git log --oneline --committer=Asha
```

Which date Git uses where. A commit written on 24 August and cherry-picked on 7 September:

```
$ git log -1
commit 0e42f9ba14e36f898a645cbd664537470d01d9d0
Author: Asha Rao <asha@example.com>
Date: Mon Aug 24 15:20:00 2026 +0530

    Add BLEU metric skeleton
$ git log --format='%h authored %as committed %cs %an: %s'
0e42f9b authored 2026-08-24 committed 2026-09-07 Asha Rao: Add BLEU metric skeleton
1c52ba8 authored 2026-08-10 committed 2026-08-10 Lab User: Add exact-match metric
```

```
$ git log --oneline --since=2026-09-01
0e42f9b Add BLEU metric skeleton
$ git log --oneline --until=2026-09-01
1c52ba8 Add exact-match metric
$ git log --since=2026-09-01 --format='%h Date: %ad' --date=short
0e42f9b Date: 2026-08-24
```

Root cause. `git log` prints the author date, and `--since` and `--until` select by committer date. A listing of "everything since 1 September" therefore contains a commit that displays 24 August. Both are correct: the change was written in August and entered this history in September.

Git needs the committer date for its own work: "the history walking machinery assumes that commits have non-decreasing commit timestamps" (`git-am`). The author date is for people: it survives every rebase and cherry-pick.

In production. After a rebase, every commit of the branch has the rebase time as committer date and the original time as author date. A report built with `--since` counts all of them as new, while plain `git log` shows old dates. For release notes, select by a range between two tags, not by date. In an audit, print both dates with `--format=fuller`.

GitHub, not Git. Commits that GitHub creates for you carry a GitHub address as committer email: a ruleset that restricts committer email "must also include `noreply@github.com` for web-based merges and other commits created on GitHub.com" ([available rules for rulesets](#)). An author who differs from the committer is normal and no sign of tampering.

6.6 Parents: root commits, ordinary commits, merge commits

In one sentence. The `parent` lines of a commit are the only links in the history graph: none for a root commit, one for an ordinary commit, two or more for a merge commit.

Precisely. Parents are ordered. The first parent is the commit HEAD pointed at when the commit was made; for a merge, the further parents are the commits you merged in (Chapter 8, section 8.13). Two suffixes walk these links ([gitrevisions](#)): `^<n>` selects the n-th parent of one commit, and `~<n>` follows first parents n times.

See it. `evalkit` with a feature branch written by Asha, before any merge:

```
$ git log --format='%h parents: [%p] %s'
191bbd1 parents: [bb904cd] Document how to run the tests
bb904cd parents: [d4c9fab] Test exact match
d4c9fab parents: [] Add exact-match metric
$ git rev-list --max-parents=0 HEAD
d4c9fabe9326ab4edbe04ed3a6f5f0b001bf6d86
```

`%p` prints the parents. The oldest commit has none, and `git rev-list --max-parents=0` finds it. A repository can have more than one root: `git switch --orphan` starts a history with no parent (Chapter 7), and `git merge --allow-unrelated-histories` joins two of them.

```
$ git merge --no-ff feature/f1
Merge made by the 'ort' strategy.
 evalkit/metrics.py      | 4 ++++
 tests/test_metrics.py  | 4 ++++
 2 files changed, 8 insertions(+)
$ git cat-file -p HEAD
tree bc9f5dde767002a5a4e19d2bd8f0ccae4d29738e
parent 191bbd14a6ce45fcdc2b74169e84f8cbb54826ba
parent 59c914e58743f5a89da4758dfb73b1c93fc62b1e
author Lab User <you@example.com> 1788756120 +0530
committer Lab User <you@example.com> 1788756120 +0530

Merge branch 'feature/f1'
$ git log -1 --format=fuller
commit ae6795c6c675be2cdc3e275aceee9161491b5342
Merge: 191bbd1 59c914e
Author:      Lab User <you@example.com>
AuthorDate: Mon Sep 7 10:12:00 2026 +0530
Commit:     Lab User <you@example.com>
CommitDate: Mon Sep 7 10:12:00 2026 +0530

Merge branch 'feature/f1'
```

A merge commit is an ordinary commit object with a second `parent` line. Its tree is a complete snapshot like any other. Porcelain shows the parents on the `Merge:` line.

```
$ git log --oneline --graph
* ae6795c Merge branch 'feature/f1'
|\
| * 59c914e Test F1 on empty strings
| * 79ff6d7 Add F1 metric
* | 191bbd1 Document how to run the tests
|/
* bb904cd Test exact match
* d4c9fab Add exact-match metric
$ git log -1 --format='%h %s' 'HEAD^1'
191bbd1 Document how to run the tests
$ git log -1 --format='%h %s' 'HEAD^2'
59c914e Test F1 on empty strings
$ git log -1 --format='%h %s' 'HEAD~2'
bb904cd Test exact match
$ git log -1 --format='%h %s' 'HEAD^2~1'
79ff6d7 Add F1 metric
$ git rev-parse --verify --quiet 'HEAD^3'
[exit status: 1]
```

Picture.

```

d4c9fab---bb904cd---191bbd1-----ae6795c  main   (HEAD -> main)
      \                               /
      79ff6d7---59c914e-----+      feature/f1

```

Expression	Reads as	Commit here
HEAD^ or HEAD^1	first parent	191bbd1
HEAD^2	second parent	59c914e
HEAD~2	first parent of the first parent	bb904cd
HEAD^2~1	first parent of the second parent	79ff6d7
HEAD^3	third parent	none: exit status 1

^ chooses among the parents of one commit, ~ goes back in a straight line. HEAD~2 and HEAD^^ are the same commit; HEAD^2 is not.

```

$ git diff --stat 'HEAD^1' HEAD
evalkit/metrics.py | 4 ++++
tests/test_metrics.py | 4 ++++
2 files changed, 8 insertions(+)
$ git diff --stat 'HEAD^2' HEAD
README.md | 2 ++
1 file changed, 2 insertions(+)
$ git log --oneline --first-parent
ae6795c Merge branch 'feature/f1'
191bbd1 Document how to run the tests
bb904cd Test exact match
d4c9fab Add exact-match metric

```

A merge commit has one diff per parent. Against the first parent you see what the merge brought into `main`; against the second, what `main` had that the feature branch lacked. `--first-parent` lists what happened to `main` itself, one line per merge.

In production. A tool that needs "the previous state of this branch" must ask for `HEAD^1` or use `--first-parent`. Plain `git log` interleaves both sides of a merge by date, so the line below a merge can be the tip of the merged branch, which never was a state of `main`.

6.7 `git commit --amend`

In one sentence. `git commit --amend` 🟡 CAUTION writes a new commit that takes the place of the current one: same parent, new tree and message as you choose, new ID.

Precisely. The manual's description is "Replace the tip of the current branch by creating a new commit", and "The new commit has the same parents and author as the current one" ([git-commit](#)). The old commit is neither modified nor deleted. The branch stops pointing at it, and only relog entries still refer to it. Chapter 11, section 11.7, shows the equivalence with a soft reset plus a new commit.

See it. A commit with a typo in its message and a forgotten test file:

```

$ git add evalkit/metrics.py
$ git commit -m "Add F1 metrc"
[main 831f9ff] Add F1 metrc
1 file changed, 4 insertions(+)
$ git status --short
?? tests/
$ git log --oneline
831f9ff Add F1 metrc
d4c9fab Add exact-match metric

```

```
$ git add tests/test_metrics.py
$ git commit --amend -m "Add F1 metric"
[main 8f6fa22] Add F1 metric
Date: Mon Sep 7 10:05:00 2026 +0530
2 files changed, 9 insertions(+)
create mode 100644 tests/test_metrics.py
$ git log --oneline
8f6fa22 Add F1 metric
d4c9fab Add exact-match metric
```

The log shows 8f6fa22 where 831f9ff was. The Date: line in the summary is the author date, carried over by the amend. Both objects exist:

```
$ git reflog
8f6fa22 HEAD@{0}: commit (amend): Add F1 metric
831f9ff HEAD@{1}: commit: Add F1 metric
d4c9fab HEAD@{2}: commit (initial): Add exact-match metric
$ git cat-file -p 'HEAD@{1}'
tree 29b018434273c541b9fce4dfc1e29fb92a358447
parent d4c9fab9326ab4edbe04ed3a6f5f0b001bf6d86
author Lab User <you@example.com> 1788755700 +0530
committer Lab User <you@example.com> 1788755700 +0530

Add F1 metric
$ git cat-file -p HEAD
tree 304b1d11e74d6497529760e3c17e94c881773561
parent d4c9fab9326ab4edbe04ed3a6f5f0b001bf6d86
author Lab User <you@example.com> 1788755700 +0530
committer Lab User <you@example.com> 1788755940 +0530

Add F1 metric
```

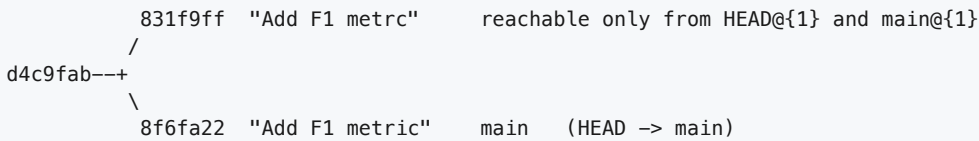
Same parent, same author line; different tree, committer time and message. Two commits that share a parent.

```
$ git branch --contains 'HEAD@{1}'
$ git log --oneline --all
8f6fa22 Add F1 metric
d4c9fab Add exact-match metric
$ git fsck --no-reflogs
dangling commit 831f9ff6c0dc934d7528503ecc13565dfe506791
```

No branch contains the old commit, and git log --all does not list it. git fsck --no-reflogs reports it as dangling: only the reflog still refers to it. Give it a name and it is an ordinary commit again:

```
$ git branch before-amend 'HEAD@{1}'
$ git log --oneline --graph --all
* 8f6fa22 Add F1 metric
| * 831f9ff Add F1 metric
|/
* d4c9fab Add exact-match metric
```

Picture.



Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git commit --	unchanged	entries unchanged;	unchanged; resolves to the	set to the new commit,	new commit object; old	unchanged; if the old commit was	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
amend		they become the tree of the new commit	new commit	whose parent is the old commit's parent	commit kept; reflog lines commit (amend)	pushed, branch and upstream now diverge	

How long the old commit survives. By default a reflog entry that is not reachable from the current tip expires after 30 days, and any other entry after 90. An object that no ref and no reflog entry refers to is pruned by maintenance once it is more than two weeks old (`git-gc: gc.reflogExpireUnreachable, gc.reflogExpire, gc.pruneExpire`). The lab configuration sets the two reflog values to `never`, so replays do not depend on the calendar. Chapter 13 covers recovery in full.

In production. Amend freely while a commit exists only in your repository. `--no-edit` keeps the message, `--only` leaves staged changes out, and `--reset-author` repairs a wrong identity. Everything staged goes into an amend unless you say `--only`; Lab 3.1 breaks a commit that way. Once the commit has been pushed, an amend rewrites shared history (section 6.13).

6.8 Empty commits: `--allow-empty`

In one sentence. An empty commit has the same tree as its parent: a point in history with a message and no change.

See it.

```
$ git commit -m "Re-run the nightly evaluation"
On branch main
nothing to commit, working tree clean
[exit status: 1]
$ git commit --allow-empty -m "Re-run the nightly evaluation"
[main 6642d40] Re-run the nightly evaluation
$ git rev-parse 'HEAD^{tree}' 'HEAD~1^{tree}'
0fbd18cca19ea00c195639456eecd36debe578ab
0fbd18cca19ea00c195639456eecd36debe578ab
$ git show --stat --format=fuller HEAD
commit 6642d40c372fdf1e58fbcc287284a33ad528da84
Author: Lab User <you@example.com>
AuthorDate: Mon Sep 7 10:05:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:05:00 2026 +0530

Re-run the nightly evaluation
```

Without the option Git refuses with exit status 1. With it, the new commit and its parent have the same tree ID, and `--stat` has nothing to list.

In production. The manual says the option "is primarily for use by foreign SCM interface scripts". Teams use it to start a push-triggered pipeline without touching a file. Two cautions: a pipeline filtered by changed paths has no path to match, so an empty commit may not start it; and the commit stays in history, where a manual trigger in the CI system leaves no trace (Chapter 20A).

6.9 Trailers

In one sentence. A trailer is a `Key: value` line in the last paragraph of a commit message: ordinary text, placed where Git and other tools can find and parse it.

See it. `-s` adds a `Signed-off-by` trailer with the committer's identity, and `--trailer` adds any other:

```
$ git add evalkit/judge.py
$ git commit -s -m 'Retry judge calls on HTTP 429' \
  -m 'The judge endpoint rate-limits bursts. Retry with exponential backoff, at most five attempts.' \
  --trailer 'Co-authored-by: Asha Rao <asha@example.com>' --trailer 'Refs: EVAL-212'
[main 7b588dd] Retry judge calls on HTTP 429
1 file changed, 10 insertions(+)
create mode 100644 evalkit/judge.py
$ git cat-file -p HEAD
tree a4e08def686cfbb567d5f427573310e3344a9056
parent 6eab4a90f8944518dce3aef708249b338e8f709a
author Lab User <you@example.com> 1788755700 +0530
committer Lab User <you@example.com> 1788755700 +0530

Retry judge calls on HTTP 429

The judge endpoint rate-limits bursts. Retry with exponential backoff, at most five attempts.

Signed-off-by: Lab User <you@example.com>
Co-authored-by: Asha Rao <asha@example.com>
Refs: EVAL-212
```

The trailers are part of the message, so they are part of the commit object and of its ID. Three ways to read them back, one to count by them:

```
$ git log -1 --format=%B | git interpret-trailers --parse
Signed-off-by: Lab User <you@example.com>
Co-authored-by: Asha Rao <asha@example.com>
Refs: EVAL-212
$ git log -1 --format='% (trailers:key=Refs,valueonly)'
EVAL-212

$ git log --oneline --grep='^Refs: EVAL-212'
7b588dd Retry judge calls on HTTP 429
$ git shortlog -sn --group=author --group=trailer:co-authored-by HEAD
2 Lab User
1 Asha Rao
```

Precisely. Git recognises a trailer block only at the end of the message, after an empty line. Every line of the block must be a trailer, unless at least a quarter of the lines are and one of them has a key that Git generates itself (Signed-off-by, or the (cherry picked from commit ...) line) or a key defined in your configuration ([git-interpret-trailers](#); the two generated prefixes are listed in [trailer.c](#)). The five cases in order: recognised; no empty line before it; a sentence inside the block; the same with `Refs` configured as a key; text after the block.

```
# A trailer block is the last paragraph, and it must look like trailers.
$ printf 'Fix tokenizer\n\nRefs: EVAL-300\n' | git interpret-trailers --parse
Refs: EVAL-300
$ printf 'Fix tokenizer\nRefs: EVAL-300\n' | git interpret-trailers --parse
$ printf 'Fix tokenizer\n\nSee the design note.\nRefs: EVAL-300\n' | git interpret-trailers --parse
$ printf 'Fix tokenizer\n\nSee the design note.\nRefs: EVAL-300\n' | git -c trailer.ticket.key=Refs
interpret-trailers --parse
Refs: EVAL-300
$ printf 'Fix tokenizer\n\nRefs: EVAL-300\n\nThanks to the platform team.\n' | git interpret-trailers
--parse
```

Trailer	Who writes it	What it means
Signed-off-by	<code>git commit -s</code>	A statement defined by the project, commonly the Developer Certificate of Origin . It is text, not a cryptographic signature
Co-authored-by	you, with <code>--trailer</code>	Credit for a further author. Git only stores the line
Reviewed-by, Refs, Fixes and others	you or your tooling	Whatever your team defines; the Git project documents its own set in SubmittingPatches

GitHub, not Git. GitHub reads these trailers: "Add one or more `Co-authored-by` trailers to a commit message to attribute a commit to multiple authors", with an email address associated with each co-author's account ([creating a commit with multiple authors](#)).

In production. A ticket number or an evaluation-run ID in a trailer travels with the commit into every clone; a pull-request label does not. Lab 3.3 builds a report from trailers.

6.10 Messages and atomic commits

How Git reads a message. Two rules matter. First, the title is the text up to the first empty line, not the first line:

```
$ cat msg.txt
Add whitespace tokenizer
F1 and ROUGE need the same token boundaries.
$ git commit -q -F msg.txt
$ git log --oneline -1
0095661 Add whitespace tokenizer F1 and ROUGE need the same token boundaries.
$ git log -1 --format='subject=[%s]%nbody=[%b] '
subject=[Add whitespace tokenizer F1 and ROUGE need the same token boundaries.]
body=[]
```

Without the empty line, both lines became the title and the body is empty. Second, when a message passes through the editor, lines that begin with `#` are comments and are removed. `--edit` sends a message from a file through that path:

```
$ cat msg.txt
Handle empty references in F1

#212 reported a ZeroDivisionError when the gold answer is empty.
Return 0.0 instead.
$ git commit -q -F msg.txt --edit
$ git log -1 --format=%B
Handle empty references in F1

Return 0.0 instead.

$ git commit -q --amend -F msg.txt
$ git log -1 --format=%B
Handle empty references in F1

#212 reported a ZeroDivisionError when the gold answer is empty.
Return 0.0 instead.
```

The sentence about issue 212 disappeared the first time: the default `--cleanup` mode is `strip` when the message is edited and `whitespace` otherwise ([git-commit](#)). Do not start a line with `#`.

Craft, with the reasons.

Rule	Reason
A title of about 50 characters without a full stop	"that title is used throughout Git" (git-commit, "Discussion"): one-line logs, shortlog, branch listings, reflogs, mail subjects and patch file names
An empty line, then the body	Otherwise the body becomes part of the title, as shown above
Imperative mood: "Retry judge calls", not "Retried"	The Git project asks for it, "as if you are giving orders to the codebase" (SubmittingPatches), and Git's own titles (Merge branch ..., Revert ...) read the same way
The body states the problem and why this solution	"The goal of your log message is to convey the why behind your change" (SubmittingPatches). The diff shows what changed; nothing else records why

Rule	Reason
Lines of at most about 72 characters	Git does not re-wrap, and <code>git log</code> indents the message by four spaces
Machine-readable facts as trailers	They can be parsed (section 6.9)

```
$ cat ../msg.txt
Retry judge calls on HTTP 429

Nightly evaluation runs failed about once a week with "judge rate
limit": the judge endpoint rejects bursts, and one rejected call
aborted the whole run after the generation step had already used
its GPU hours.

Retry up to five times with exponential backoff (1, 2, 4, 8, 16 s).
Other HTTP errors still fail at once, because retrying them would
hide real bugs.

Refs: EVAL-212
$ git commit -q -F ../msg.txt
$ git show -s --format=reference HEAD
6083abf (Retry judge calls on HTTP 429, 2026-09-07)
```

`--format=reference` prints the form in which one message should cite another commit. The title appears wherever Git needs one line for a commit. Compare it with the title of the commit above it:

```
$ git log --oneline
24f43cf fix stuff
6083abf Retry judge calls on HTTP 429
6eab4a9 Add README
$ git shortlog HEAD
Asha Rao (1):
    fix stuff

Lab User (2):
    Add README
    Retry judge calls on HTTP 429

$ git branch -v
* main 24f43cf fix stuff
$ git reflog -2
24f43cf HEAD@{0}: commit: fix stuff
6083abf HEAD@{1}: commit: Retry judge calls on HTTP 429
```

```
$ git format-patch -1 --stdout HEAD~1 | head -n 6
From 6083abf1acecb06ad1ccb415b8036c7dfb1229db Mon Sep 17 00:00:00 2001
From: Lab User <you@example.com>
Date: Mon, 7 Sep 2026 10:06:00 +0530
Subject: [PATCH] Retry judge calls on HTTP 429

Nightly evaluation runs failed about once a week with "judge rate
$ git format-patch -1 -o ../outbox HEAD~1
../outbox/0001-Retry-judge-calls-on-HTTP-429.patch
```

Team conventions such as [Conventional Commits](#) add a `type(scope): description` pattern to the title; they are agreements on top of Git, enforced by hooks or CI if at all.

Atomic commits. An atomic commit contains one logical change, complete enough that the project still builds and passes its tests. The Git project's rule is "Make separate commits for logically separate changes" (SubmittingPatches). Two unrelated edits in the working tree, committed separately:

```
$ git status --short
M evalkit/metrics.py
```

```

M requirements.txt
$ git add evalkit/metrics.py
$ git commit -q -m "Return 0.0 from F1 when no tokens overlap"
$ git add requirements.txt
$ git commit -q -m "Bump tokenizers to 0.21.0"
$ git log --oneline --stat -2
edc60de Bump tokenizers to 0.21.0
requirements.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
4ea5f60 Return 0.0 from F1 when no tokens overlap
evalkit/metrics.py | 2 ++
1 file changed, 2 insertions(+)

```

```

# The new tokenizers release breaks the nightly run. Undo that change only.
$ git revert --no-edit HEAD
[main fb38675] Revert "Bump tokenizers to 0.21.0"
Date: Mon Sep 7 10:10:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ cat requirements.txt
tokenizers==0.20.3
pyyaml==6.0.2
$ grep -n "common == 0" evalkit/metrics.py
4:     if common == 0:

```

The dependency bump was undone and the bug fix stayed, because they were two commits. The same separation lets you cherry-pick the fix to a release branch (Chapter 10) and lets `git bisect` stop on a commit small enough to read (Chapter 14A). `git add -p` builds such commits from a mixed working tree (Chapter 5).

6.11 Signatures and attribution

A signed commit carries its signature inside the commit object, in a `gpgsig` header. This demo creates a throwaway SSH key in the sandbox, so its output differs on every run:

```

$ git commit -q -S -m "Describe the project"
$ git cat-file -p HEAD
tree 5dca3a6e589a5d9b648e2563ac9c17bee742e961
parent 150744516440fbc2cc4197e81141ce63d5fa7cb2
author Lab User <you@example.com> 1788755880 +0530
committer Lab User <you@example.com> 1788755880 +0530
gpgsig -----BEGIN SSH SIGNATURE-----
U1NIU0lHAAAAQAAADMAAAALc3NoLWVkmjU1MTkAAAAgeAvXs1QLvASH7oyGmTC81lg0eA
6h9gfo2suPhirxoDgAAADZ2l0AAAAAaZaGE1MTIAAABTAAAC3NzaC1lZDI1NTE5
AAAAQMMcYH0dg+LhdIIe0rd3ubk6j0JVijcMdpjwjHWVTT08y+/dNh80pVbiWWOPltXc0
T8VhfAj9d5XxVa0EFx6gU=
-----END SSH SIGNATURE-----

```

Describe the project

The header sits between the committer line and the message; continuation lines begin with a space ([gitformat-signature](#)). Because it is inside the object, it is covered by the commit ID, and signing a commit that already exists means writing a new commit. Do not confuse the two options: `-S` signs with a key, `-s` adds the Signed-off-by text. Keys, verification and trust are the subject of Chapter 14B.

GitHub, not Git. "GitHub links a commit to a user by matching the email address in the commit header to an email address on a GitHub account" ([troubleshooting commits](#)). Nothing in that match proves that the owner of the address made the commit. A "Verified" label is a separate statement about a signature. Chapter 21B treats identity as a security topic.

6.12 Reading commits

Command	Shows
<code>git cat-file -p <commit></code>	The object as stored
<code>git show --format=raw --no-patch <commit></code>	The same headers with the ID in front and the message indented
<code>git show --format=fuller <commit></code>	Both identities and both dates, then the diff against the parent
<code>git show --stat <commit></code>	A summary of changed paths in place of the diff
<code>git log</code> with <code><n></code> , <code>--oneline</code> , <code>--graph</code> , <code>--stat</code> , <code>-p</code>	A walk backwards from HEAD or from the commits you name
<code>git log --author=<pattern></code> , <code>--committer=<pattern></code> , <code>--grep=<pattern></code> , <code>--no-merges</code> , <code>-- <path></code>	The same walk, filtered

```
$ git show --format=fuller --stat HEAD
commit 0d77920ea1ff22bc46fc5051fd6e5d86078a54
Author:   Lab User <you@example.com>
AuthorDate: Mon Sep 7 10:12:00 2026 +0530
Commit:   Lab User <you@example.com>
CommitDate: Mon Sep 7 10:12:00 2026 +0530
```

Add F1 metric

```
evalkit/metrics.py | 9 ++++++++
1 file changed, 9 insertions(+)
```

```
$ git show --format=raw --no-patch HEAD
commit 0d77920ea1ff22bc46fc5051fd6e5d86078a54
tree 27d56e4ea4080b5719bf97b9985f6f543eda349d
parent 51d62b3de231309f8aba10baba050cf2220fcc61
author Lab User <you@example.com> 1788756120 +0530
committer Lab User <you@example.com> 1788756120 +0530
```

Add F1 metric

```
$ git log --oneline -3
ae6795c Merge branch 'feature/f1'
191bbd1 Document how to run the tests
59c914e Test F1 on empty strings
$ git log -1 --stat feature/f1
commit 59c914e58743f5a89da4758dfb73b1c93fc62b1e
Author: Asha Rao <asha@example.com>
Date:   Mon Sep 7 10:07:00 2026 +0530
```

Test F1 on empty strings

```
tests/test_metrics.py | 4 ++++
1 file changed, 4 insertions(+)
$ git log --format='%h %ad %an: %s' --date=short --no-merges -3
191bbd1 2026-09-07 Lab User: Document how to run the tests
59c914e 2026-09-07 Asha Rao: Test F1 on empty strings
79ff6d7 2026-09-07 Asha Rao: Add F1 metric
$ git log --oneline -- README.md
191bbd1 Document how to run the tests
d4c9fab Add exact-match metric
```

Every diff here is computed when you ask. `git log -- README.md` lists the commits whose snapshot of that path differs from their parent's. Chapter 14A turns these options into investigation techniques.

6.13 What can go wrong

An amended commit that was already pushed. The amend creates a sibling of the published commit:

```
$ git status -sb
## main...origin/main
$ git commit --amend -m "Exact match: strip whitespace"
[main 4f33829] Exact match: strip whitespace
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --all
* 4f33829 Exact match: strip whitespace
| * f8456dc Exact match: strp whitespace
|/
* 4279e65 Add exact-match metric
```

```
$ git push
To $LAB/ch06/amend-pushed/origin.git
! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to '$LAB/ch06/amend-pushed/origin.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

[ahead 1, behind 1] is the diagnosis: your branch has the new commit, the upstream has the old one. Do not follow the hint to pull, which would merge two versions of one commit. If others may have the published commit, return to it with `git reset --soft '@{u}'` and fix the mistake in a new commit. If the branch is yours alone, replace the published commit deliberately (Chapter 12, section 12.8).

A commit under the wrong identity. A repository-local `user.email`, left from another project, overrides the global one:

```
$ git add evalkit/judge.py
$ git commit -q -m "Add judge client"
$ git log --format='%h %an <%ae> %s'
0d3dc20 Lab User <lab.user@personal.example> Add judge client
6eab4a9 Lab User <you@example.com> Add README
```

```
$ git var GIT_AUTHOR_IDENT
Lab User <lab.user@personal.example> 1788755880 +0530
$ git config get --show-scope --show-origin --all user.email
global file:$LAB/ch06/identity/home/.gitconfig you@example.com
local file:.git/config lab.user@personal.example
```

`git var GIT_AUTHOR_IDENT` prints the identity the next commit would record, and `git config get --show-origin --all` names the file each value comes from (`get` needs Git 2.46 or later). Remove the wrong setting, then rewrite the unpublished commit:

```
$ git config unset user.email
$ git var GIT_AUTHOR_IDENT
Lab User <you@example.com> 1788756060 +0530
$ git commit --amend --no-edit --reset-author
[main 2f1be8c] Add judge client
1 file changed, 2 insertions(+)
create mode 100644 evalkit/judge.py
$ git log --format='%h %an <%ae> %s'
2f1be8c Lab User <you@example.com> Add judge client
6eab4a9 Lab User <you@example.com> Add README
```

When no email is configured, Git guesses one from the login name and the host name. `user.useConfigOnly` turns the guess into an error:

```
# In this sandbox user.email is now configured nowhere.
$ git -c user.useConfigOnly=true commit --allow-empty -m "Re-run the nightly evaluation"
Author identity unknown

*** Please tell me who you are.

Run

    git config --global user.email "you@example.com"
    git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: no email was given and auto-detection is disabled
[exit status: 128]
```

Symptom	Diagnosis	Fix	Prevention
<code>git push</code> rejected after an amend	<code>git status -sb</code> shows [ahead 1, behind 1]	<code>git reset --soft '{u}'</code> and a new commit; or a deliberate <code>--force-with-lease</code> on a private branch	Amend only unpublished commits
An amend swallowed unrelated staged changes	<code>git show --stat HEAD</code> lists files that do not belong	<code>git reset --soft 'HEAD@{1}'</code> , unstage, amend again (Lab 3.1)	<code>git status</code> before amending; <code>--only</code> for message fixes
The "old" commit is needed again	<code>git reflog</code> shows it one entry down	<code>git branch <name> 'HEAD@{1}'</code>	None needed
Commits carry the wrong name or email	<code>git log --format='%an <ae>'; git config get -show-origin --all user.email</code>	Last commit: <code>--amend --reset-author</code> . Published commits: leave them	<code>user.useConfigOnly=true</code> ; per-directory identity with <code>includeIf</code> (Chapter 14B)
The deployed ID is no longer on the branch	<code>git merge-base --is-ancestor <id> HEAD</code> exits with 1; <code>git diff --quiet <id> HEAD</code> exits with 0	Move the branch back to the deployed commit (Lab 3.2)	Freeze commits after review
A line of the message is missing	The line began with <code>#</code> and the message went through the editor	Amend with <code>-F <file></code> , or reword the line	Never start a line with <code>#</code>
<code>git log --oneline</code> shows a very long title	No empty line after the first line	Amend the message	Title, empty line, body
A trailer is not found by tooling	<code>git log -1 --format=%B git interpret-trailers --parse</code> prints nothing	Move the trailer to the last paragraph (Lab 3.3)	Add trailers with <code>--trailer</code> , not by hand

6.14 When not to use it, and dangerous edge cases

- **Do not amend what others may have.** On a shared branch an amend produces the divergence of section 6.13 for every colleague.
- **An amend does not delete anything.** A secret committed and then amended away is still in the old commit, in your reflog, and on the server if it was pushed. Rotate the secret (Chapter 21B).

- `--no-verify` **skips the** `pre-commit` **and** `commit-msg` **hooks**, and with them whatever your team relies on them to catch.
- `git commit -a` **commits every tracked change**, including edits you did not mean to publish.
- **Author and committer are assertions.** `--author`, the configuration and the environment accept any text. Only a verified signature is evidence of who created a commit.
- **Git does not validate the clock.** A machine with a wrong date writes that date into the commit, and Git's history walk relies on committer dates (section 6.5).
- **Empty commits are permanent.** Use `--allow-empty` for an event worth recording.

6.15 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git show</code> , <code>git log</code> , <code>git cat-file</code> , <code>git var</code> , <code>git interpret-trailers --parse</code>	● SAFE	Nothing	not needed	not needed
<code>git commit</code> , <code>git commit --allow-empty</code>	● SAFE	Adds objects, moves the current branch forward	<code>git diff --cached</code> , <code>git commit --dry-run</code>	A further commit, or Chapter 11
<code>git commit -a</code>	● SAFE	The same, after staging every tracked change	<code>git diff HEAD</code>	as above
<code>git commit --no-verify</code>	● CAUTION	As <code>git commit</code> , without two hooks	Run the hook's checks by hand	Amend, or a further commit
<code>git commit --amend</code> (with or without <code>--no-edit</code> , <code>--only</code> , <code>--reset-author</code>)	● CAUTION	Replaces the tip commit with a new one	<code>git diff --cached</code> , <code>git log -1</code>	<code>git reset --soft 'HEAD@{1}'</code>
<code>git commit-tree</code>	● SAFE	Adds one commit object and moves no ref	not needed	not needed
<code>git interpret-trailers --in-place <file></code>	● SAFE	Rewrites a message file, never a commit	Run it without <code>--in-place</code>	Edit the file

No command in this chapter is ●. The dangerous moment is the push after a rewrite (Chapter 12).

6.16 Version notes

Version note. Older behavior: trailers were typed by hand or added by a hook that called `git interpret-trailers`. Current behavior: `git commit --trailer <key>=<value>`. Since: Git 2.32 ([release notes](#)). Recommended: `--trailer`, and `-s` for the sign-off.

Version note. Older behavior: `git shortlog` grouped by author or committer only. Current behavior: `--group=trailer:<key>` counts by trailer. Since: Git 2.29 ([release notes](#)). Recommended: use it for review and co-author statistics.

Version note. Older behavior: up to Git 2.55 the trailer parser can take a line that begins with a URL for a trailer. Current behavior: Git 2.56 no longer does (not run here). Since: Git 2.56 ([release notes](#)). Recommended:

keep URLs out of the last paragraph, or put them in a trailer value.

Version note. Older behavior: `git config --get user.email`. Current behavior: `git config get user.email`, as in section 6.13; the old form still works. Since: Git 2.46. Recommended: the subcommand form, which fails on older Git.

6.17 Practice

- **Labs 3.1, 3.2 and 3.3** in the Module 3 lab manual: amend a commit and find the old one; change only the committer date and watch the ID change; write and repair trailers.
- Replay any transcript with `labs/run ch06/<demo>` and continue by hand in the sandbox it leaves behind.
- Two drills. In `ch06/parents`, write two expressions for commit `79ff6d7` that start from `HEAD`. In `ch06/date-filters`, make `git log` print the committer date of each commit.

6.18 Interview questions

1. List every field of a commit object. Which of them can differ between two commits that have identical diffs?
2. A colleague says "I only fixed the commit message, the code is the same commit". What is wrong with that sentence, and how do you prove it?
3. Explain author and committer with three operations that make them differ. Which date does `git log` print, and which one does `--since` use?
4. What exactly does `git commit` write inside `.git`, and what does it leave untouched?
5. After `git commit --amend`, where is the old commit, how long does it stay, and how do you get it back?
6. `HEAD^2`, `HEAD~2`, `HEAD^^`: which commits are these on a merge commit, and which two are the same?
7. Our deploy tool stores abbreviated IDs. What can go wrong, and what should it store?
8. What is the difference between `git commit -s` and `git commit -S`? Where does each leave its mark in the object?
9. Why does GitHub show a teammate's commits without a profile link, and what does that tell you about who pushed them?
10. When does Git fail to recognise a trailer? How would you make ticket IDs queryable across a year of history?
11. A commit dated last month appears in "changes since Monday". Explain it without calling it a bug.

6.19 Sources

Primary sources

- [git-commit](#), [git-show](#), [git-log](#), [git-cat-file](#), [git-commit-tree](#), [git-interpret-trailers](#), [git-shortlog](#), [git-var](#), [git-am](#), [git-gc](#), [githooks](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitdatamodel](#), [gitrevisions](#), [gitformat-signature](#).
- The Git project's [SubmittingPatches](#): separate commits, the message, sign-off and trailers. [trailer.c](#) at 2.55.0 for the trailers Git generates.
- Release notes [2.29](#), [2.32](#) and [2.56](#).
- GitHub Docs: [troubleshooting commits](#), [creating a commit with multiple authors](#), [setting your commit email address](#), [about commit signature verification](#).

Secondary sources

- Pro Git, [Git Internals: Git Objects](#) and the commit guidelines in [Contributing to a Project](#). Caveat: both use `master`.
- Chris Beams, [How to Write a Git Commit Message](#) (2014): the widely quoted seven rules.
- [Conventional Commits 1.0.0](#), a title convention whose footers follow the trailer format.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026: snapshots, history as a graph, content-addressed objects. Caveats: describes object IDs as SHA-1 only; the demo starts on `master`.
- [RubyConf 2018 - Branch in Time](#), Tekin Süleyman: why history quality matters, told as one story. Nothing in it depends on a Git version.
- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 2020: includes amend. Caveat: `master` naming.

Further reading

- Chapter 9, section 9.3, and Chapter 10, section 10.3: the "new object, new ID" rule applied to rebase and cherry-pick.

Chapter 7: Branches

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch07/`.

7.1 Why this matters

Three questions a CTO can ask after an ordinary week.

1. "A contractor says two days of work vanished after she 'checked out a tag to test something'. Is it gone?"
2. "Why does every laptop in the team fail with `cannot lock ref 'refs/heads/feature/login'` since this morning?"
3. "We have 340 branches on the server. Which ones can be deleted without losing anything?"

All three are questions about refs, and none has a Git command as its answer. The commits exist and are recoverable from the reflog (section 7.7, Lab 4.2). Somebody created a branch named `feature`, and a ref cannot be both a name and a directory of names (section 7.12, Lab 4.3). "Merged" has a precise meaning that `git branch --merged` tests, and a squash merge defeats it (section 7.13).

The polls in the Phase 0 report found that 58 percent of practising developers think of a branch as the commits that branched off, and 15 percent as a pointer (jvns.ca). The second group is right in a way that decides outcomes. A branch is one line of text: a name and the ID of one commit. Everything in this chapter follows from that fact.

7.2 A branch is a ref

In one sentence. A branch is a ref, a name under `refs/heads/` that holds the ID of one commit, the tip; the branch's history is whatever that commit reaches through its parents.

Analogy. A bookmark in a book that is still being written. The bookmark marks one page; the chapters before that page are "the story so far", but they are the book's pages, not the bookmark's. Moving the bookmark changes nothing in the book, and two bookmarks can sit on the same page. The analogy breaks at the edge: pages after a bookmark exist too, unless nothing points at them, and then Git may eventually throw them away.

Precisely. The glossary: a branch head is "a named reference to the commit at the tip of a branch", stored "in a file in `$GIT_DIR/refs/heads/` directory, except when using packed refs" ([gitglossary](#)). The ref holds nothing but the ID. There is no list of the branch's commits, no record of where it started, no owner, no creation date. `git log feature` is a walk that starts at the ID in the ref and follows `parent` lines (Chapter 6).

Inside `.git`. With the files backend, which Git 2.55 uses by default, a loose branch is the file `.git/refs/heads/<name>` containing the ID and a newline. Packed refs move the same information into lines of `.git/packed-refs`. The reftable backend keeps refs in a binary store under `.git/reftable/` (Chapter 3). The reflog of a branch lives in `.git/logs/refs/heads/<name>`.

See it. Create a branch with porcelain and read what was written:

```
$ git log --oneline
7aecf06 Add batch runner
69d8252 Add exact-match metric
6eab4a9 Add README
$ git branch feature/retry-backoff
```

```
$ git branch -v
  feature/retry-backoff 7aecf06 Add batch runner
* main                  7aecf06 Add batch runner
```

```
$ cat .git/refs/heads/feature/retry-backoff
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f
$ git rev-parse feature/retry-backoff
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f
$ git for-each-ref refs/heads
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f commit refs/heads/feature/retry-backoff
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f commit refs/heads/main
$ git reflog show feature/retry-backoff
7aecf06 feature/retry-backoff@{0}: branch: Created from main
```

The file holds the full ID of `7aecf06`, and nothing else. `git for-each-ref` is the plumbing listing: for every ref, the ID, the type of object it names, and the full ref name. Both branches name the same commit. The branch also received a reflog whose first line records how it was created.

Plumbing creates branches too. `git update-ref` 🟡 CAUTION writes a ref, with two checks that porcelain also relies on:

```
$ git update-ref refs/heads/hotfix/judge-timeout HEAD~1
$ git branch -v
  feature/retry-backoff 7aecf06 Add batch runner
  hotfix/judge-timeout 69d8252 Add exact-match metric
* main                  7aecf06 Add batch runner
$ git update-ref refs/heads/typo 1234567890123456789012345678901234567890
fatal: update_ref failed for ref 'refs/heads/typo': trying to write ref 'refs/heads/typo' with
nonexistent object 1234567890123456789012345678901234567890
[exit status: 128]
$ git update-ref refs/heads/not-a-commit 'HEAD^{tree}'
fatal: update_ref failed for ref 'refs/heads/not-a-commit': trying to write non-commit object
416a81e352e688b7791f11f2e8159ea3be84dd9c to branch 'refs/heads/not-a-commit'
[exit status: 128]
```

The object must exist, and a ref under `refs/heads/` must name a commit. Writing the file by hand bypasses both checks and the reflog:

```
$ git rev-parse HEAD~2 > .git/refs/heads/by-hand
$ git branch -v
  by-hand                6eab4a9 Add README
  feature/retry-backoff 7aecf06 Add batch runner
  hotfix/judge-timeout 69d8252 Add exact-match metric
* main                  7aecf06 Add batch runner
$ git reflog show by-hand
$ git update-ref -d refs/heads/by-hand
$ git branch --list by-hand
```

`git branch -v` reads the hand-written file like any other, but `git reflog show by-hand` is empty: no Git command wrote this ref, so nothing logged it. `git update-ref -d` deletes the ref. The last snippet is the reason to read refs with plumbing and never by path:

```
$ git pack-refs --all
$ cat .git/refs/heads/main
cat: .git/refs/heads/main: No such file or directory
[exit status: 1]
$ cat .git/packed-refs
# pack-refs with: peeled fully-peeled sorted
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f refs/heads/feature/retry-backoff
69d82526af97115a79ad14d198c1ba24c22d5fb9 refs/heads/hotfix/judge-timeout
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f refs/heads/main
$ git rev-parse main
7aecf069859aa045bb7e47a6b1fb0693f7c3da7f
```

After `git pack-refs --all` the file is gone and the ref lives in `packed-refs`. `git rev-parse main` does not care. A script that reads `.git/refs/heads/main` breaks on the first packed repository, and on every refable repository.

Picture.

```
.git/refs/heads/main      -----> 7aecf06 "Add batch runner"
.git/refs/heads/feature/retry-backoff -----> 7aecf06      |
.git/refs/heads/hotfix/judge-timeout -----> 69d8252 "Add exact-match metric"
                                   |
                                   6eab4a9 "Add README"
```

Three refs, three small files, one history. Creating the second and third branch wrote no commit, no tree and no blob.

In production. Because a branch is a name for a commit, "the branch" has a different value in every repository that holds a copy of it: your clone, each colleague's clone, the server. `main` on the server and `main` in your clone are two refs that happen to share a name (Chapter 12). A statement such as "the fix is on `main`" is only meaningful with a repository attached.

7.3 HEAD is a symbolic ref

In one sentence. HEAD is the ref that says where you are: normally a symbolic ref that names the current branch, and in detached state a direct reference to a commit.

Precisely. A symbolic ref is "a regular file that stores a string that begins with `ref: refs/`" ([git-symbolic-ref](#)). When HEAD contains `ref: refs/heads/main`, every command that needs "the current commit" resolves HEAD to `refs/heads/main` and then to the ID in that ref. HEAD is per worktree: a linked worktree has its own (Chapter 25).

See it. Four ways to read it, from raw to friendly:

```
$ cat .git/HEAD
ref: refs/heads/main
$ git symbolic-ref HEAD
refs/heads/main
$ git symbolic-ref --short HEAD
main
$ git branch --show-current
main
$ git rev-parse --abbrev-ref HEAD
main
$ git rev-parse HEAD
69d82526af97115a79ad14d198c1ba24c22d5fb9
```

A new repository shows that HEAD can name a branch that does not exist yet:

```
$ git init -q ../scratch
$ cat ../scratch/.git/HEAD
ref: refs/heads/main
$ git -C ../scratch rev-parse --verify HEAD
fatal: Needed a single revision
[exit status: 128]
$ git -C ../scratch branch
$ git -C ../scratch status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
$ git -C ../scratch branch feature
fatal: not a valid object name: 'main'
[exit status: 128]
```

HEAD points at `refs/heads/main`, but no such ref exists until the first commit writes it. The glossary calls this an unborn branch. `git rev-parse --verify HEAD` fails, `git branch` lists nothing, and `git branch` feature fails because there is no commit to point the new branch at.

Inside `.git`. Moving HEAD alone shows what `git switch` adds on top of a ref change:

```
$ git symbolic-ref HEAD refs/heads/feature/retry-backoff
$ git status
On branch feature/retry-backoff
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    deleted:    evalkit/retry.py

$ git symbolic-ref HEAD refs/heads/main
$ git status --short
```

`git symbolic-ref HEAD <ref>` 🟡 CAUTION rewrote one file. The index and the working tree still hold the content of `main`, so `git status` reports the difference between the commit HEAD now names and the index as a staged deletion. Switching back makes the report disappear. A branch switch is this ref change plus an update of the index and the working tree (section 7.6).

7.4 What a commit does to the current branch

In one sentence. A commit writes the new commit's ID into the ref that HEAD names, and nothing else moves.

See it. Two branches on the same commit, then one commit on the new branch:

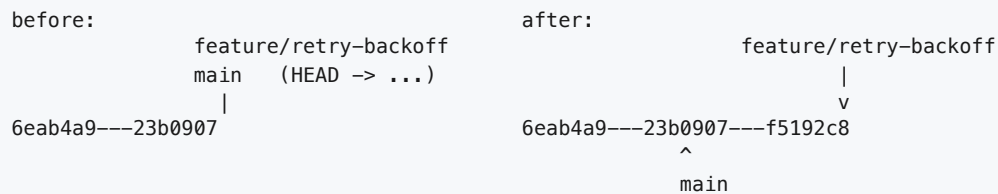
```
$ git switch -c feature/retry-backoff
Switched to a new branch 'feature/retry-backoff'
$ cat .git/HEAD
ref: refs/heads/feature/retry-backoff
$ git for-each-ref --format='%(objectname:short) %(refname)' refs/heads
23b0907 refs/heads/feature/retry-backoff
23b0907 refs/heads/main
```

```
# evalkit/judge.py has been edited: call_judge() now retries on HTTP 429.
$ git commit -am "Retry judge calls on HTTP 429"
[feature/retry-backoff f5192c8] Retry judge calls on HTTP 429
 1 file changed, 10 insertions(+), 2 deletions(-)
```

```
$ cat .git/HEAD
ref: refs/heads/feature/retry-backoff
$ git for-each-ref --format='%(objectname:short) %(refname)' refs/heads
f5192c8 refs/heads/feature/retry-backoff
23b0907 refs/heads/main
$ git reflog show feature/retry-backoff
f5192c8 feature/retry-backoff@{0}: commit: Retry judge calls on HTTP 429
23b0907 feature/retry-backoff@{1}: branch: Created from HEAD
$ git reflog -2
f5192c8 HEAD@{0}: commit: Retry judge calls on HTTP 429
23b0907 HEAD@{1}: checkout: moving from main to feature/retry-backoff
$ git log --oneline --graph --all
* f5192c8 Retry judge calls on HTTP 429
* 23b0907 Add judge client
* 6eab4a9 Add README
```

`.git/HEAD` is unchanged. `feature/retry-backoff` moved to `f5192c8`; `main` stayed on `23b0907`. The branch reflog gained a `commit:` line, and the HEAD reflog shows the switch and the commit. In the graph the two branches now name different commits, and `main` is an ancestor of the feature branch.

Picture.



Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git commit on feature/retry-backoff	unchanged	unchanged	unchanged: ref: refs/heads/feature/retry-backoff	moves to the new commit	main unchanged; new objects; one line in each of two reflogs	unchanged	unchanged

In production. This is why "I committed to the wrong branch" is a ref problem and not a content problem. The commit object is correct; only the name that moved is wrong. Section 7.14 repairs it with two ref writes and no copying.

7.5 git branch: list, create, delete, rename, force

In one sentence. `git branch` lists refs under `refs/heads/`, creates one at a commit, deletes one, renames one, or moves one; it never touches the working tree.

See it. A clone with five local branches, three of which have been pushed. The remote is a bare repository on disk:

```

$ git branch
docs/metrics
feature/f1
fix/typo-readme
* main
spike/judge-cache
$ git branch -v
docs/metrics      a8a1b5f Document the metrics
feature/f1        de7c39b [ahead 1] F1: handle empty reference
fix/typo-readme   6a04691 Fix grammar in README
* main            55144fd [ahead 4] Merge branch 'docs/metrics'
spike/judge-cache 44483e6 Spike: cache judge responses
$ git branch -vv
docs/metrics      a8a1b5f Document the metrics
feature/f1        de7c39b [origin/feature/f1: ahead 1] F1: handle empty reference
fix/typo-readme   6a04691 [origin/fix/typo-readme] Fix grammar in README
* main            55144fd [origin/main: ahead 4] Merge branch 'docs/metrics'
spike/judge-cache 44483e6 Spike: cache judge responses

```

`-v` adds the tip and its title and, for branches with an upstream, how far they are ahead or behind it; `-vv` names the upstream. `spike/judge-cache` has none, so its line shows no relationship. Remote-tracking branches have their own listing:

```

$ git branch -r
origin/feature/f1
origin/fix/typo-readme
origin/main
$ git branch -a
docs/metrics
feature/f1
fix/typo-readme

```

```
* main
  spike/judge-cache
  remotes/origin/feature/f1
  remotes/origin/fix/typo-readme
  remotes/origin/main
```

`-r` lists `refs/remotes/`, `-a` both namespaces. The `remotes/` prefix in `-a` output marks the second kind; section 7.10 explains what they are.

Merged, not merged, contains. `--merged` lists "branches whose tips are reachable from" the named commit, HEAD by default, and `--no-merged` the others ([git-branch](#)). `--contains <commit>` turns the question around: which branches reach this commit?

```
$ git log --oneline --graph --all
* 55144fd Merge branch 'docs/metrics'
|\
| * a8a1b5f Document the metrics
|/
* de7c39b F1: handle empty reference
* 35581b1 Add F1 metric
| * 44483e6 Spike: cache judge responses
|/
| * 6a04691 Fix grammar in README
|/
* d1e8f22 Add exact-match metric
* faf3622 Add README
$ git branch --merged
  docs/metrics
  feature/f1
* main
$ git branch --no-merged
  fix/typo-readme
  spike/judge-cache
$ git branch --contains origin/feature/f1
  docs/metrics
  feature/f1
* main
```

`docs/metrics` and `feature/f1` are merged: `main` reaches their tips, one through a merge commit and one by fast-forward. `fix/typo-readme` and `spike/judge-cache` have commits that `main` does not reach.

Delete. `-d` 🟡 CAUTION refuses a branch that is not fully merged; `-D` 🔴 DANGEROUS deletes regardless, and both delete the branch's reflog with it:

```
$ git branch -d docs/metrics
Deleted branch docs/metrics (was a8a1b5f).
$ git branch -d spike/judge-cache
error: the branch 'spike/judge-cache' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D spike/judge-cache'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
$ git branch -D spike/judge-cache
Deleted branch spike/judge-cache (was 44483e6).
$ git reflog show spike/judge-cache
fatal: ambiguous argument 'spike/judge-cache': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[exit status: 128]
$ git branch spike/judge-cache 44483e6
$ git reflog show spike/judge-cache
44483e6 spike/judge-cache@{0}: branch: Created from 44483e6
```

After `-D` the branch's own reflog is gone, so `git reflog show` has nothing to resolve. The commit itself still exists, and the `was 44483e6` in the deletion message is the ID you need to recreate the branch. If that message has scrolled away, the HEAD reflog still lists the commit if it was ever checked out (Lab 4.2). What `-D` can destroy is the

branch relog and the only name of commits that were never checked out; preview with `git log --oneline main..<branch>` (section 7.8), and recover with `git branch <name> <id>`. It is appropriate for a branch whose commits you have decided to abandon or that you know to be squash-merged (section 7.13).

The rule behind `-d` is about the upstream, not about `main`: "The branch must be fully merged in its upstream branch, or in HEAD if no upstream was set" (`git-branch`).

```
$ git branch -d fix/typo-readme
warning: deleting branch 'fix/typo-readme' that has been merged to
        'refs/remotes/origin/fix/typo-readme', but not yet merged to HEAD
Deleted branch fix/typo-readme (was 6a04691).
$ git branch -d feature/f1
warning: not deleting branch 'feature/f1' that is not yet merged to
        'refs/remotes/origin/feature/f1', even though it is merged to HEAD
error: the branch 'feature/f1' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature/f1'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
```

`fix/typo-readme` is not merged into HEAD, but it is pushed and equal to its upstream, so `-d` deletes it with a warning. `feature/f1` is merged into HEAD, but it has one commit that its upstream lacks, so `-d` refuses. Read both messages before reaching for `-D`.

Rename and force.

```
$ git branch -m feature/f1 feature/f1-metric
$ git branch -vv
feature/f1-metric de7c39b [origin/feature/f1: ahead 1] F1: handle empty reference
* main          55144fd [origin/main: ahead 4] Merge branch 'docs/metrics'
spike/judge-cache 44483e6 Spike: cache judge responses
$ git config get branch.feature/f1-metric.merge
refs/heads/feature/f1
$ git reflog show feature/f1-metric
de7c39b feature/f1-metric@{0}: Branch: renamed refs/heads/feature/f1 to refs/heads/feature/f1-metric
de7c39b feature/f1-metric@{1}: commit: F1: handle empty reference
35581b1 feature/f1-metric@{2}: commit: Add F1 metric
d1e8f22 feature/f1-metric@{3}: branch: Created from HEAD
```

`-m`  CAUTION renames the ref, its relog and its configuration section in one step, and logs the rename. The upstream still points at `origin/feature/f1`, because renaming a local branch changes nothing on the remote (Chapter 12).

```
$ git branch release/0.2 main
$ git branch release/0.2 main~1
fatal: a branch named 'release/0.2' already exists
[exit status: 128]
$ git branch -f release/0.2 main~1
$ git reflog show release/0.2
de7c39b release/0.2@{0}: branch: Reset to main~1
55144fd release/0.2@{1}: branch: Created from main
$ git branch -f main main~1
fatal: cannot force update the branch 'main' used by worktree at '$LAB/ch07/branch-commands/evalkit'
[exit status: 128]
```

Without `-f`, `git branch` refuses to change an existing branch. With it,  CAUTION, the ref moves and the relog records `Reset to main~1`, so the old position is one entry away. A branch that is checked out in a worktree cannot be force-moved at all; `git reset` is the command for that (Chapter 11).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git branch <name>	unchanged	unchanged	unchanged	unchanged	new ref and relog; with a remote-tracking start	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
[<start>]					point, branch. <name>.* configuration		
git branch -d or -D <name>	unchanged	unchanged	unchanged	unchanged	ref and its reflog deleted; branch.<name>.* configuration removed	unchanged	unchanged
git branch -m <old> <new>	unchanged	unchanged	follows the rename if <old> was current	renamed	ref, reflog and configuration renamed; a reflog line records it	unchanged	unchanged
git branch -f <name> <commit>	unchanged	unchanged	unchanged	not allowed	ref set to the commit; reflog line branch: Reset to ...	unchanged	unchanged

7.6 git switch, and git checkout

In one sentence. `git switch`  SAFE changes which branch HEAD names and updates the index and the working tree to that branch's commit, refusing when a local change would be lost.

Precisely. `git switch <branch>` points HEAD at the branch, then makes the index and working tree match its tip. `-c <new>` creates the branch first and is "the transactional equivalent" of `git branch` followed by `git switch` ([git-switch](#)); `-C` resets an existing branch like `git branch -f`. `--detach` points HEAD at a commit instead of a branch (section 7.7). `-` means `@{-1}`, the previously checked-out branch or commit. `--orphan <new>` creates an unborn branch and removes all tracked files from the working tree, so the next commit starts a disconnected history.

See it.

```
$ git switch -c feature/retry
Switched to a new branch 'feature/retry'
$ git switch -c feature/retry
fatal: a branch named 'feature/retry' already exists
[exit status: 128]
$ git switch main
Switched to branch 'main'
$ git switch -C feature/retry main~1
Switched to and reset branch 'feature/retry'
$ git reflog show feature/retry
b01a3f1 feature/retry@{0}: branch: Reset to main~1
2daf400 feature/retry@{1}: branch: Created from HEAD
```

```
$ git switch main
Switched to branch 'main'
$ git switch -
Switched to branch 'feature/retry'
$ git switch -
Switched to branch 'main'
$ git rev-parse --abbrev-ref '@{-1}'
feature/retry
```

```
$ git switch v0.1.0
fatal: a branch is expected, got tag 'v0.1.0'
hint: If you want to detach HEAD at the commit, try again with the --detach option.
[exit status: 128]
$ git switch --detach v0.1.0
HEAD is now at b01a3f1 Add exact-match metric
$ git switch -
```

```
Previous HEAD position was b01a3f1 Add exact-match metric
Switched to branch 'main'
```

`git switch` wants a branch and refuses a tag, with a hint. `--detach` is the explicit way to stand on a commit that is not a branch tip. `git checkout` does the same jobs with older spellings. Learn them to read other people's scripts; write `switch` and `restore` yourself:

```
$ git checkout -b feature/cache
Switched to a new branch 'feature/cache'
$ git checkout main
Switched to branch 'main'
$ git checkout -B feature/cache main~1
Switched to and reset branch 'feature/cache'
$ git checkout -
Switched to branch 'main'
$ git checkout --detach
HEAD is now at 2daf400 Add batch runner
$ git checkout main
Switched to branch 'main'
```

Task	git switch	git checkout
Switch to a branch	<code>git switch <branch></code>	<code>git checkout <branch></code>
Create and switch	<code>git switch -c <new> [<start>]</code>	<code>git checkout -b <new> [<start>]</code>
Reset and switch	<code>git switch -C <new> [<start>]</code>	<code>git checkout -B <new> [<start>]</code>
Previous branch	<code>git switch -</code>	<code>git checkout -</code>
Detach at a commit	<code>git switch --detach <commit></code>	<code>git checkout --detach <commit></code> , or <code>git checkout <commit></code>
Unborn branch	<code>git switch --orphan <new></code>	<code>git checkout --orphan <new></code>
Restore a file	not this command: <code>git restore</code>	<code>git checkout -- <path></code>

`git checkout <name>` guesses whether you mean a branch or a path, which is the ambiguity the two newer commands remove. One difference shows in the next transcript: `switch --orphan` empties the working tree, while `checkout --orphan` keeps the files and stages them.

```
$ git switch --orphan gh-pages
Switched to a new branch 'gh-pages'
$ cat .git/HEAD
ref: refs/heads/gh-pages
$ git status --short
$ ls -A
.git
$ git switch main
Switched to branch 'main'
$ git branch --list gh-pages
$ git checkout --orphan docs-site
Switched to a new branch 'docs-site'
$ git status --short
A README.md
A configs/eval.yaml
A evalkit/metrics.py
A evalkit/runner.py
$ git switch main
Switched to branch 'main'
```

After `git switch --orphan gh-pages`, `HEAD` names a branch that has no commit, and `git branch --list gh-pages` prints nothing after switching away: an unborn branch disappears when you leave it, because no ref was ever written.

Local changes. Switching does not require a clean working tree. A change in a file that is identical on both branches is carried along; a change in a file that differs is refused:

```
$ printf '\nRun the tests with: python -m pytest\n' >> README.md
$ git switch feature/threshold
Switched to branch 'feature/threshold'
M    README.md
$ git switch main
Switched to branch 'main'
M    README.md
$ printf 'judge_model: judge-v2\nthreshold: 0.5\n' > configs/eval.yaml
$ git switch feature/threshold
error: Your local changes to the following files would be overwritten by checkout:
    configs/eval.yaml
Please commit your changes or stash them before you switch branches.
Aborting
[exit status: 1]
$ git status --short
M README.md
M configs/eval.yaml
```

The edited `README.md` travelled to `feature/threshold` and back, marked `M`. The edit to `configs/eval.yaml` could not, because the two branches have different versions of that file, and the switch would have had to overwrite the edit. Commit, stash (Chapter 11) or use `--merge`, which stashes and reapplies for you.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git switch <branch></code>	files that differ between the two tips are replaced; local edits to other files stay	updated to the target tip	<code>ref: refs/heads/<branch></code>	unchanged	reflog line <code>checkout: moving from A to B in logs/HEAD</code>	unchanged	unchanged
<code>git switch -c <new> [<start>]</code>	as above, relative to <code><start></code>	as above	<code>ref: refs/heads/<new></code>	unchanged	new ref and reflog	unchanged	unchanged
<code>git switch --detach <commit></code>	as above	as above	the commit ID	no current branch	reflog line in <code>logs/HEAD</code>	unchanged	unchanged
<code>git switch --orphan <new></code>	tracked files removed	emptied	<code>ref: refs/heads/<new></code>	unborn: no ref yet	nothing until the first commit	unchanged	unchanged

7.7 Detached HEAD

In one sentence. HEAD is detached when it holds a commit ID instead of a branch name; Git works normally, and new commits advance HEAD itself rather than any branch.

Analogy. Reading a book with no bookmark in it. You can read any page and even write notes on new pages, but when you put the book down nothing marks where your notes are. The analogy breaks because Git keeps a diary of where you have been, the reflog, and the notes can be found again through it.

Precisely. The manual: in detached HEAD state "HEAD refers to a specific commit, as opposed to referring to a named branch", and a commit made there "is referenced only by HEAD" ([git-checkout](#), "Detached HEAD"). It is not an error. Git detaches HEAD whenever you ask for a commit that no branch names, and several commands do it for you.

See it. Check out a tag and inspect the state:

```
$ git checkout v0.1.0
Note: switching to 'v0.1.0'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 8c6d240 Add exact-match metric
$ cat .git/HEAD
8c6d240f33727a5e974eed76dadd337c298ed748
$ git status
HEAD detached at v0.1.0
nothing to commit, working tree clean
$ git branch
* (HEAD detached at v0.1.0)
  main
```

The advice block is Git's own explanation, printed once per detach unless `advice.detachedHead` is false.

`.git/HEAD` holds a raw ID; `git status` says `HEAD detached at v0.1.0`; `git branch` lists a pseudo-entry instead of a current branch.

```
$ git symbolic-ref HEAD
fatal: ref HEAD is not a symbolic ref
[exit status: 128]
$ git branch --show-current
$ git rev-parse --abbrev-ref HEAD
HEAD
```

Each of the three commands that answer "which branch am I on" answers differently: `git symbolic-ref HEAD` fails with status 128, `git branch --show-current` prints nothing, and `git rev-parse --abbrev-ref HEAD` prints the literal `HEAD`. Scripts must handle all three.

```
$ printf 'judge_model: judge-v1\nthreshold: 0.8\n' > configs/eval.yaml
$ git commit -am "Experiment: stricter judge threshold"
[detached HEAD f678c98] Experiment: stricter judge threshold
1 file changed, 1 insertion(+), 1 deletion(-)
$ printf 'judge_model: judge-v1\nthreshold: 0.9\n' > configs/eval.yaml
$ git commit -am "Experiment: threshold 0.9"
[detached HEAD b23bce3] Experiment: threshold 0.9
1 file changed, 1 insertion(+), 1 deletion(-)
$ git status
HEAD detached from v0.1.0
nothing to commit, working tree clean
$ git log --oneline --graph --all
* b23bce3 Experiment: threshold 0.9
* f678c98 Experiment: stricter judge threshold
| * d27ae01 Add batch runner
|/
* 8c6d240 Add exact-match metric
* 0381ff6 Add README and eval config
```

Two commits later, the status line says `HEAD detached from v0.1.0`: "at" became "from" because HEAD has moved since it was detached. `git log --all` shows the new commits because `--all` includes HEAD. Now leave:

```
$ git switch main
Warning: you are leaving 2 commits behind, not connected to
any of your branches:

    b23bce3 Experiment: threshold 0.9
    f678c98 Experiment: stricter judge threshold

If you want to keep them by creating a new branch, this may be a good time
to do so with:

git branch <new-branch-name> b23bce3

Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git log --oneline --graph --all
* d27ae01 Add batch runner
* 8c6d240 Add exact-match metric
* 0381ff6 Add README and eval config
```

Git warns, names the commits, and prints the command that keeps them. `git log --all` no longer shows them: no ref reaches them. Section 6.7 of Chapter 6 told the rest of the story: the commits still exist, and the reflog of HEAD still refers to them.

```
$ git branch experiment/judge-threshold b23bce3
$ git log --oneline --graph --all
* b23bce3 Experiment: threshold 0.9
* f678c98 Experiment: stricter judge threshold
| * d27ae01 Add batch runner
|/
* 8c6d240 Add exact-match metric
* 0381ff6 Add README and eval config
```

Inside `.git`. Where `git status` gets its words. It reads the HEAD reflog backwards for the latest `checkout`: moving from `...` to `<X>` entry; X is printed after "at" if HEAD still equals that entry's commit, after "from" otherwise. With no such entry, as in a fresh clone, it prints `Not currently on any branch.` ([wt-status.c at 2.55.0](#)).

Every way it happens.

Cause	What you typed or ran	Seen in
Checking out a tag	<code>git checkout v0.1.0</code> , <code>git switch --detach v0.1.0</code>	above
Checking out a commit by ID or relative name	<code>git checkout <id></code> , <code>git switch --detach HEAD~1</code>	below
Checking out a remote-tracking branch	<code>git checkout origin/main</code> , <code>git switch --detach origin/main</code>	below
<code>git bisect</code>	every step checks out a commit to test	below
<code>git rebase</code> while it runs or is stopped	the rebase replays commits on a detached HEAD (Chapter 9)	below
<code>git clone --branch <tag></code>	a clone of one release, the shape of many deploy scripts	below
<code>git worktree add <path> <tag-or-commit></code>	a linked worktree not on a branch (Chapter 25)	below
<code>git submodule update</code>	the default <code>checkout</code> mode puts each submodule on a detached HEAD (git-submodule ; Chapter 23)	not run here
A CI checkout	on <code>pull_request</code> events the default checkout is a merge commit in detached HEAD (events that trigger workflows ; Chapter 20A)	GitHub Actions, not run here

```
$ git switch --detach origin/main
HEAD is now at d27ae01 Add batch runner
$ git status
```

```

HEAD detached at origin/main
nothing to commit, working tree clean
$ git switch -q main
$ git bisect start HEAD HEAD~2
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[8c6d240f33727a5e974eed76dadd337c298ed748] Add exact-match metric
$ git status
HEAD detached at 8c6d240
You are currently bisecting, started from branch 'main'.
  (use "git bisect reset" to get back to the original branch)

nothing to commit, working tree clean
$ git bisect reset
Previous HEAD position was 8c6d240 Add exact-match metric
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

```

```

$ git rebase -i HEAD~1
--- todo list as Git opened it (comment lines removed) ---
pick d27ae01 # Add batch runner
--- todo list as saved ---
edit d27ae01 # Add batch runner
Rebasing (1/1)
Stopped at d27ae01... # Add batch runner
You can amend the commit now, with

    git commit --amend

Once you are satisfied with your changes, run

    git rebase --continue
$ git branch
* (no branch, rebasing main)
  experiment/judge-threshold
  main
$ cat .git/HEAD
d27ae01097b4cc8528d18538aae3da2c83a38984
$ git rebase --abort
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```

During a stopped rebase, `git branch` prints (no branch, rebasing main) and `.git/HEAD` holds an ID; the rebase moves `main` only at the end. This is why an interrupted rebase looks like a detached HEAD: it is one.

```

$ git switch --detach HEAD~1
HEAD is now at 8c6d240 Add exact-match metric
$ git status
HEAD detached at 8c6d240
nothing to commit, working tree clean
$ git -c advice.detachedHead=false checkout 0381ff6
Previous HEAD position was 8c6d240 Add exact-match metric
HEAD is now at 0381ff6 Add README and eval config
$ git status
HEAD detached at 0381ff6
nothing to commit, working tree clean
$ git switch main
Previous HEAD position was 0381ff6 Add README and eval config
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

```

```

# A deploy script that clones one release. The detached-HEAD advice is switched off to keep the
transcript short.
$ git -c advice.detachedHead=false clone -q --branch v0.1.0 ../origin.git ../deploy
$ cat ../deploy/.git/HEAD
8c6d240f33727a5e974eed76dadd337c298ed748

```

```
$ git -C ../deploy status
Not currently on any branch.
nothing to commit, working tree clean
$ git -C ../deploy branch -a
* (no branch)
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

A clone made with `--branch v0.1.0` has no local branch at all: `git branch -a` shows `(no branch)` and the remote-tracking refs. A deploy script that commits a generated file in such a clone commits onto nothing.

```
$ git worktree add ../evalkit-v0.1.0 v0.1.0
Preparing worktree (detached HEAD 8c6d240)
HEAD is now at 8c6d240 Add exact-match metric
$ git -C ../evalkit-v0.1.0 status
Not currently on any branch.
nothing to commit, working tree clean
$ git branch
* main
$ git worktree remove ../evalkit-v0.1.0
```

Picture.

attached	detached, two commits later
HEAD -> refs/heads/main -> d27ae01	HEAD -> b23bce3 "Experiment: threshold 0.9"
0381ff6---8c6d240---d27ae01 main	0381ff6---8c6d240---d27ae01 main
tag v0.1.0	\
	f678c98---b23bce3

Keeping the work. Three commands create a ref for the commit HEAD is on, as the manual lists them: `git switch -c <name>` attaches HEAD to a new branch there, `git branch <name>` creates the branch and leaves HEAD detached, and `git tag <name>` creates a tag. After you have moved away, find the ID in `git reflog` first (Lab 4.2).

In production. Detached HEAD is the normal state of a CI job, a deploy checkout and a submodule, so a script that assumes a current branch (`git branch --show-current`, `git push` with no `refspec`) fails in exactly those places. Use `git rev-parse HEAD` for the commit and name the branch explicitly when pushing. For people, the rule is one sentence: before you leave a detached HEAD with commits on it, give them a branch.

7.8 Divergence and ancestry

In one sentence. Two branches have diverged when neither tip is an ancestor of the other; their merge base is the best common ancestor, and "ahead" and "behind" are counted from it.

Precisely. A common ancestor is a commit reachable from both tips. "One common ancestor is better than another common ancestor if the latter is an ancestor of the former", and a best one is a merge base ([git-merge-base](#)); there can be more than one (Chapter 8, section 8.5). Ranges are sets of commits ([gitrevisions](#)): `A..B` is everything reachable from B but not from A, and `A...B` is the symmetric difference, reachable from one side but not both. `git merge-base --is-ancestor A B` exits with 0 when A is an ancestor of B and with 1 when it is not.

See it. `main` and `feature/rouge` after both received commits:

```
$ git log --oneline --graph --all
* e12f113 Add CI workflow
* 9500b9e Exact match: strip whitespace
| * eaab34d ROUGE-L: add tests
| * 5351fa7 ROUGE-L: tokenize on whitespace
| * 22c856c Add ROUGE-L metric
```

```
|/  
* 03f74b9 Add exact-match metric  
* 6eab4a9 Add README
```

```
$ git merge-base main feature/rouge  
03f74b9ac8547c31330105153beb4b90c5ddeea8  
$ git log --oneline main..feature/rouge  
eaab34d ROUGE-L: add tests  
5351fa7 ROUGE-L: tokenize on whitespace  
22c856c Add ROUGE-L metric  
$ git log --oneline feature/rouge..main  
e12f113 Add CI workflow  
9500b9e Exact match: strip whitespace
```

The merge base is `03f74b9`. `main..feature/rouge` is what the feature has that `main` lacks (three commits); `feature/rouge..main` is the reverse (two).

```
$ git rev-list --left-right --count main...feature/rouge  
2      3  
$ git log --oneline --left-right main...feature/rouge  
< e12f113 Add CI workflow  
< 9500b9e Exact match: strip whitespace  
> eaab34d ROUGE-L: add tests  
> 5351fa7 ROUGE-L: tokenize on whitespace  
> 22c856c Add ROUGE-L metric  
$ git for-each-ref --format='% (refname:short) %(ahead-behind:main)' refs/heads  
feature/rouge 3 2  
main 0 0
```

`--left-right --count` prints the two numbers in the order of the operands: two commits only on the left side, three only on the right. The `%(ahead-behind:main)` field of `git for-each-ref` computes the same pair for every branch at once, from the branch's point of view: `feature/rouge` is 3 ahead of `main` and 2 behind.

```
$ git merge-base --is-ancestor main feature/rouge  
[exit status: 1]  
$ git merge-base --is-ancestor main~2 feature/rouge  
[exit status: 0]  
$ git merge-base --is-ancestor main~2 main  
[exit status: 0]
```

`main` is not an ancestor of the feature branch, so merging the feature into `main` cannot be a fast-forward (Chapter 8, section 8.3). `main~2`, the merge base, is an ancestor of both.

```
$ git diff --stat main...feature/rouge  
evalkit/rouge.py | 6 ++++++  
test_rouge.py    | 2 ++  
2 files changed, 8 insertions(+)  
$ git diff --stat main..feature/rouge  
ci.yaml          | 1 -  
evalkit/metrics.py | 2 +-  
evalkit/rouge.py | 6 ++++++  
test_rouge.py    | 2 ++  
4 files changed, 9 insertions(+), 2 deletions(-)
```

Notation	In git log and git rev-list	In git diff
<code>A..B</code>	commits reachable from B and not from A	the two tips compared directly; the same as <code>git diff A B</code>
<code>A...B</code>	commits reachable from exactly one side	from the merge base of A and B to B: what B changed

The two meanings are a known trap (Phase 0 report, section 12). In the transcript, `git diff --stat main...feature/rouge` lists the feature's own work, while `main..feature/rouge` also shows `main`'s two commits reversed.

Picture.

```
          9500b9e---e12f113    main          < left: 2 commits
         /
6eab4a9---03f74b9---+         merge base: 03f74b9
         \
          22c856c---5351fa7---eaab34d    feature/rouge    > right: 3 commits
```

In production. [ahead 2, behind 3] in `git status`, the counts on a pull request, "can this be fast-forwarded", and "which commits does this release contain" are all this arithmetic. Compute them with ranges, not with dates (Chapter 6, section 6.5), and state the base: "3 commits ahead" is meaningless until you say ahead of what.

GitHub, not Git. GitHub computes a pull request's changes from a merge base too: "Compare pages and pull request pages can calculate changed files from different merge bases", because "Pull request pages focus on what the pull request introduced, while compare pages reflect the current comparison between two refs" ([pull requests reference](#); Chapter 17).

7.9 Git has no parent-branch concept

In one sentence. Nothing in a repository records which branch a branch was created from; the only trace is a line in a local reflog, and every comparison needs a base that you name.

See it. Create a branch from `feature/rouge` and look for the relationship:

```
$ git switch -c feature/rouge-stemming feature/rouge
Switched to a new branch 'feature/rouge-stemming'
$ cat .git/refs/heads/feature/rouge-stemming
936bfbd7a649344c04602bfd7b614a32596b06a7
$ git cat-file -p HEAD
tree 8dd91332643c665aad37f53a444b610cabdfd9c0
parent 7a1ccc7e383d6b20935992a53b77015c81bbd978
author Lab User <you@example.com> 1788755820 +0530
committer Lab User <you@example.com> 1788755820 +0530

ROUGE-L: tokenize on whitespace
$ git config list --local
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
core.ignorecase=true
core.precomposeunicode=true
$ git reflog show feature/rouge-stemming
936bfbd feature/rouge-stemming@{0}: branch: Created from feature/rouge
```

The new ref holds an ID. The commit it names has no field for a branch. The local configuration has no entry for the branch. The only mention of `feature/rouge` is the reflog line `branch: Created from feature/rouge`, which exists in this clone and nowhere else, and which `git branch -D` or `reflog expiry` removes.

```
$ git log --oneline --graph --all
* 37431c0 ROUGE-L: stem tokens
* 936bfbd ROUGE-L: tokenize on whitespace
* 7a1ccc7 Add ROUGE-L metric
* 69d8252 Add exact-match metric
* 6eab4a9 Add README
$ git branch --contains feature/rouge
feature/rouge
* feature/rouge-stemming
$ git branch --contains main
feature/rouge
```

```
* feature/rouge-stemming
main
```

`git branch --contains` answers "which branches reach this commit", which is reachability, not origin: the commits of `feature/rouge` are on `feature/rouge-stemming` as much as on `feature/rouge`.

```
$ git log --oneline main..feature/rouge-stemming
37431c0 ROUGE-L: stem tokens
936bfbd ROUGE-L: tokenize on whitespace
7a1ccc7 Add ROUGE-L metric
$ git log --oneline feature/rouge..feature/rouge-stemming
37431c0 ROUGE-L: stem tokens
$ git branch -D feature/rouge
Deleted branch feature/rouge (was 936bfbd).
$ git log --oneline main..feature/rouge-stemming
37431c0 ROUGE-L: stem tokens
936bfbd ROUGE-L: tokenize on whitespace
7a1ccc7 Add ROUGE-L metric
```

`main..feature/rouge-stemming` lists three commits; against `feature/rouge` it lists one. Delete `feature/rouge` and the first answer does not change, because the commits never belonged to a branch. The nearest thing to an answer is a guess: `%(is-base:<commit>)` marks "the ref that is most likely the ref used as a starting point for the branch that produced ", chosen by a first-parent heuristic ([git-for-each-ref](#)), and the guess changes with the refs that exist:

```
$ git log --oneline --decorate
ad8abff (HEAD -> feature/rouge-stemming) ROUGE-L: stem tokens
936bfbd (feature/rouge) ROUGE-L: tokenize on whitespace
7a1ccc7 Add ROUGE-L metric
69d8252 (main) Add exact-match metric
6eab4a9 Add README
$ git for-each-ref --format='%(refname:short) %(is-base:feature/rouge-stemming)' refs/heads/main
refs/heads/feature/rouge
feature/rouge (feature/rouge-stemming)
main
$ git branch -D feature/rouge
Deleted branch feature/rouge (was 936bfbd).
$ git for-each-ref --format='%(refname:short) %(is-base:feature/rouge-stemming)' refs/heads/main
main (feature/rouge-stemming)
```

In production. Stacked branches are where this bites. Branch B was created from branch A, A is merged with a squash or deleted, and B's "changes against `main`" suddenly include everything A did, because the base you implied no longer exists. `git rebase --onto` moves B to its real base (Chapter 9, sections 9.5 and 9.9). Commands such as `git rebase` and `git merge` take the target as an argument for the same reason: Git cannot infer it.

GitHub, not Git. A pull request has a base branch, chosen when it is opened and stored by GitHub, not by Git. It is the one place where "created from" is recorded (Chapter 17).

7.10 Remote-tracking branches and upstream: a preview

In one sentence. A remote-tracking branch such as `origin/main` is a ref in your own repository that records the last position of a branch in another repository, and an upstream is two configuration lines that pair a local branch with one.

Precisely. Remote-tracking branches live under `refs/remotes/<remote>/` and are "how Git stores the last-known state of a branch in a remote repository"; `git fetch` updates them ([gitdatamodel](#)). The upstream of branch B is configured by `branch.B.remote` and `branch.B.merge`, and `B@{upstream}` or `@{u}` names the corresponding

remote-tracking branch ([gitrevisions](#)). `git status -sb` and `git branch -v` compare a branch with its upstream; nothing in that comparison talks to the server.

See it.

```
$ git for-each-ref --format='% (objectname:short) %(refname) '
d1e8f22 refs/heads/main
d1e8f22 refs/remotes/origin/main
$ git branch -vv
* main d1e8f22 [origin/main] Add exact-match metric
```

```
$ git rev-parse --abbrev-ref '{@upstream}'
origin/main
$ git rev-parse --symbolic-full-name '{@u}'
refs/remotes/origin/main
$ git config get branch.main.remote
origin
$ git config get branch.main.merge
refs/heads/main
$ git status -sb
## main...origin/main
```

Two refs, same ID, two namespaces. `@{upstream}` resolves through the two configuration entries to `refs/remotes/origin/main`. Now Asha pushes a commit from her own clone:

```
# Asha has pushed one commit to the server. Your repository has not been told.
$ git status -sb
## main...origin/main
$ git rev-parse --short origin/main
d1e8f22
$ git fetch
From $LAB/ch07/upstream-preview/origin
   d1e8f22..0015820  main      -> origin/main
$ git rev-parse --short origin/main
0015820
$ git status -sb
## main...origin/main [behind 1]
```

Before the fetch, `git status -sb` reports no difference, because `origin/main` is what your repository last heard. The fetch moves `origin/main` and the status changes to `behind 1`. "Up to date with origin/main" is a statement about your copy of the remote, never about the remote itself.

```
$ git switch -c feature/rouge
Switched to a new branch 'feature/rouge'
$ git rev-parse --abbrev-ref '{@upstream}'
fatal: no upstream configured for branch 'feature/rouge'
[exit status: 128]
$ git switch -c hotfix/ci origin/main
Switched to a new branch 'hotfix/ci'
branch 'hotfix/ci' set up to track 'origin/main'.
$ git branch -vv
   feature/rouge d1e8f22 Add exact-match metric
* hotfix/ci      0015820 [origin/main] Add CI workflow
   main         d1e8f22 [origin/main: behind 1] Add exact-match metric
```

A branch created from another local branch has no upstream. A branch created from a remote-tracking branch gets one automatically (`branch.autoSetupMerge`, default `true`), which is what the line `set up to track` reports. Chapter 12 covers `fetch`, `push`, `refspecs` and `@{push}`.

7.11 Lightweight tags as refs

In one sentence. A lightweight tag is a ref under `refs/tags/` that names an object directly; it is a label that is not expected to move.

See it.

```
$ git tag v0.1.0
$ cat .git/refs/tags/v0.1.0
69d82526af97115a79ad14d198c1ba24c22d5fb9
$ git cat-file -t v0.1.0
commit
$ git for-each-ref
69d82526af97115a79ad14d198c1ba24c22d5fb9 commit refs/heads/main
69d82526af97115a79ad14d198c1ba24c22d5fb9 commit refs/tags/v0.1.0
```

The tag file holds the commit ID, and `git cat-file -t` on the tag name reaches the commit: no object was created. One commit later, the branch has moved and the tag has not:

```
$ git log --oneline --decorate
e09c144 (HEAD -> main) Add batch runner
69d8252 (tag: v0.1.0) Add exact-match metric
6eab4a9 Add README
```

```
$ git tag -a v0.2.0 -m "Release 0.2.0"
$ git cat-file -t v0.2.0
tag
$ git rev-parse v0.2.0 'v0.2.0^{commit}' HEAD
f850d5e1360ebcb51575a79519e389642a005ad3
e09c14440a4310ac2cb3110c0a778b2f2ec34e0f
e09c14440a4310ac2cb3110c0a778b2f2ec34e0f
```

An annotated tag, `-a`, is different: the ref names a tag object with its own ID, tagger, date and message, and `v0.2.0^{commit}` peels it to the commit. The manual reserves annotated tags for releases and lightweight tags for "private or temporary object labels" ([git-tag](#)); Chapter 14B covers annotated and signed tags.

```
$ ls .git/logs/refs
heads
$ git tag v0.1.0 HEAD
fatal: tag 'v0.1.0' already exists
[exit status: 128]
$ git tag -f v0.1.0 HEAD
Updated tag 'v0.1.0' (was 69d8252)
$ git tag -d v0.1.0
Deleted tag 'v0.1.0' (was e09c144)
```

`.git/logs/refs` has a `heads` directory and no `tags` directory: `core.logAllRefUpdates` creates reflogs for branches, remote-tracking branches, notes and HEAD, not for tags ([git-config](#)). `git tag -f` ⚠ CAUTION and `git tag -d` ⚠ CAUTION therefore leave no trace except the `was ... ID` in their output. Write it down.

```
$ git branch v0.2.0 HEAD~1
$ git log -1 --format='%h %s' v0.2.0
warning: refname 'v0.2.0' is ambiguous.
e09c144 Add batch runner
$ git log -1 --format='%h %s' heads/v0.2.0
69d8252 Add exact-match metric
$ git branch -D v0.2.0
Deleted branch v0.2.0 (was 69d8252).
```

A branch and a tag with the same short name are legal and confusing. Git resolves a short name by trying `refs/tags/<name>` before `refs/heads/<name>` ([gitrevisions](#)), so `v0.2.0` meant the tag, with a warning. `heads/v0.2.0` and `tags/v0.2.0` are unambiguous.

7.12 Naming branches, and the `feature` versus `feature/x` conflict

Conventions. A branch name is a path under `refs/heads/`, so slashes group branches: `feature/retry-backoff`, `fix/judge-timeout`, `release/0.2`, `asha/bleu`. The grouping is what `git branch --list 'feature/*'` and `git for-each-ref refs/heads/feature/` match, and what GitHub's branch rules target by pattern, `qa/*` for example (Chapter 18). Lowercase names with hyphens avoid the case trap of section 7.15 and need no quoting. Ticket numbers in names (`fix/EVAL-212-empty-gold`) tie a branch to its reason without a lookup.

Rules. Git rejects names with a space, `..`, `~`, `^`, `:`, `?`, `*`, `[`, `\`, a control character, a component that starts with `.` or ends with `.lock`, a trailing `.`, the sequence `@{`, a leading or trailing slash, or a double slash; a branch name may not start with `-` (`git-check-ref-format`). `git check-ref-format --branch <name>` tests a candidate without creating anything.

The conflict. One more rule follows from the path structure: a name cannot be both a ref and the prefix of other refs.

```
$ git branch feature
$ git branch feature/login
fatal: cannot lock ref 'refs/heads/feature/login': 'refs/heads/feature' exists; cannot create 'refs/heads/feature/login'
[exit status: 128]
$ ls .git/refs/heads
feature
main
```

With the files backend the reason is visible: `refs/heads/feature` is a file, and `refs/heads/feature/login` would need a directory of the same name. The rule is not a file-system accident, though. It holds for packed refs and in a refutable repository, where there are no such files:

```
$ git pack-refs --all
$ ls .git/refs/heads
$ git branch feature/login
fatal: 'refs/heads/feature' exists; cannot create 'refs/heads/feature/login'
[exit status: 128]
$ git init -q --ref-format=reftable ../reftable-repo
$ git -C ../reftable-repo commit -q --allow-empty -m "Start"
$ git -C ../reftable-repo branch feature
$ git -C ../reftable-repo branch feature/login
fatal: 'refs/heads/feature' exists; cannot create 'refs/heads/feature/login'
[exit status: 128]
```

It also holds in the other direction, and the fix is a rename:

```
$ git branch -m feature feature/base
$ git branch feature/login
$ git branch feature
fatal: cannot lock ref 'refs/heads/feature': 'refs/heads/feature/base' exists; cannot create 'refs/heads/feature'
[exit status: 128]
$ git branch
  feature/base
  feature/login
* main
```

```
$ git branch 'fix bug'
fatal: 'fix bug' is not a valid branch name
hint: See 'git help check-ref-format'
hint: Disable this message with "git config set advice.refSyntax false"
[exit status: 128]
$ git check-ref-format --branch fix..bug
fatal: 'fix..bug' is not a valid branch name
[exit status: 128]
```

```
$ git check-ref-format --branch fix/judge.lock
fatal: 'fix/judge.lock' is not a valid branch name
[exit status: 128]
$ git check-ref-format --branch fix/judge-timeout
fix/judge-timeout
[exit status: 0]
```

```
$ git branch refs/heads/fix/judge-timeout
$ git for-each-ref --format='%(refname)' refs/heads
refs/heads/feature/base
refs/heads/feature/login
refs/heads/main
refs/heads/refs/heads/fix/judge-timeout
$ git branch -D refs/heads/fix/judge-timeout
Deleted branch refs/heads/fix/judge-timeout (was 6eab4a9).
```

The last transcript shows a name that is legal and wrong: `git branch refs/heads/fix/judge-timeout` created `refs/heads/refs/heads/fix/judge-timeout`, because `git branch` takes a short name and prepends `refs/heads/` itself.

```
Observed behavior : since this morning, git fetch fails on every laptop with "cannot lock ref
'refs/heads/feature/login'"
                  or reports "unable to update local ref" for origin/feature/login
Git state          : each clone still has refs/remotes/origin/feature from an earlier fetch; the server
now has feature/login
Mechanism          : the fetch tries to create origin/feature/login while origin/feature exists, and a
ref cannot be
                  a name and a prefix at the same time
Root cause         : a branch named "feature" was deleted on the server after branches under "feature/"
were agreed on,
                  and nobody's clone was told about the deletion
Why Git does this  : ref names form one hierarchy; the rule keeps every ref name unambiguous
Correct fix        : git fetch --prune (or git remote prune origin) deletes the stale origin/feature
and the fetch succeeds
Prevention         : reserve top-level names for namespaces (feature/, fix/) and never create a plain
branch with one of them
```

Lab 4.3 runs this incident from both sides.

7.13 Stale branches

In one sentence. A stale branch is a ref that nobody needs any more, and Git offers three tests for it, each with a blind spot: age, reachability from `main`, and a deleted upstream.

See it. A repository with branches from a whole year. `git for-each-ref --sort=committerdate` lists refs by the date of their git commit:

```
$ git for-each-ref --sort=committerdate --format='%(committerdate:short) %(authorname) | %
(refname:short)' refs/heads
2026-06-30 Lab User | feature/rouge
2026-08-20 Lab User | wip/prompt-tuning
2026-08-31 Lab User | fix/empty-gold
2026-09-07 Lab User | main
$ git for-each-ref --sort=committerdate --exclude=refs/remotes/origin/HEAD --format='%(
committerdate:short) %(authorname) | %(refname:short)' refs/remotes/origin
2026-03-12 Asha Rao | origin/spike/judge-cache
2026-06-30 Lab User | origin/feature/rouge
2026-08-31 Lab User | origin/fix/empty-gold
2026-09-07 Lab User | origin/main
$ git for-each-ref --sort=committerdate --format='%(committerdate:short) %(refname:short)'
refs/remotes/origin | awk '$1 < "2026-07-01"'
2026-03-12 origin/spike/judge-cache
2026-06-30 origin/feature/rouge
```

Age says when the branch last changed, not whether its work landed. The second test is reachability:

```
$ git branch --merged main
  fix/empty-gold
* main
$ git branch --no-merged main
  feature/rouge
  wip/prompt-tuning
$ git branch -r --no-merged origin/main
  origin/feature/rouge
  origin/spike/judge-cache
```

`--merged main` lists `fix/empty-gold`, merged with a merge commit. `feature/rouge` is missing although its content is in `main`: it was squash-merged, so `main` contains a new commit with the same tree and none of the branch's commits. Reachability cannot see that. The third test asks the server, through the remote-tracking refs:

```
# The two merged branches have been deleted on the server.
$ git fetch --prune
From $LAB/ch07/stale-branches/origin
- [deleted]      (none)    -> origin/feature/rouge
- [deleted]      (none)    -> origin/fix/empty-gold
$ git branch -vv
  feature/rouge      842be74 [origin/feature/rouge: gone] ROUGE-L: return a float
  fix/empty-gold     cad8f2c [origin/fix/empty-gold: gone] Exact match: handle empty gold answer
* main              00b2e49 [origin/main] Add CI workflow
  wip/prompt-tuning  9287f33 WIP: stricter judge prompt
$ git for-each-ref --format='% (refname:short) %(upstream:track)' refs/heads
feature/rouge [gone]
fix/empty-gold [gone]
main
wip/prompt-tuning
```

`git fetch --prune` 🟡 CAUTION removes remote-tracking refs for branches the server no longer has, and `[gone]` marks every local branch whose upstream disappeared. On a team that deletes branches on the server after merging, `gone` is the most reliable signal. It still has to be read: `wip/prompt-tuning` was never pushed, so it has no upstream and no `gone`, and nothing but your memory says whether it matters.

```
$ git branch -d fix/empty-gold
Deleted branch fix/empty-gold (was cad8f2c).
$ git branch -d feature/rouge
error: the branch 'feature/rouge' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature/rouge'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
$ git merge-tree --write-tree main feature/rouge
1442f02427f21cdc85e98b91d69535929f65e7ab
$ git rev-parse 'main^{tree}'
1442f02427f21cdc85e98b91d69535929f65e7ab
$ git branch -D feature/rouge
Deleted branch feature/rouge (was 842be74).
```

`git branch -d` deletes the merge-commit branch and refuses the squash-merged one, for the reason above. The refusal is conditional: `-d` tests the branch against its upstream, or against HEAD when it has no upstream. Here the upstream is `gone` (pruned), so Git falls back to HEAD and finds commits that `main` cannot reach; a branch that was never pushed behaves the same. While the remote-tracking ref still exists and contains the branch, `-d` deletes it with a warning instead (section 7.5). The proof that the squash-merged branch is finished is a merge without a working tree: `git merge-tree --write-tree main feature/rouge` prints the tree a merge would produce, and it equals the tree of `main`. Merging the branch would change nothing, so `-D` is safe.

```
$ git branch --delete-merged 'origin/*' 2>&1 | head -n 1
error: unknown option `delete-merged'
```

Version note. Older behavior: cleaning up merged branches needs `git branch --merged` plus judgement, or a script. Current behavior: Git 2.56 adds `git branch --delete-merged <pattern>`, which deletes "local branches whose configured upstream matches, but only when their tip is reachable from that upstream", with `--dry-run` to list them first (not run here: Git 2.55 reports the option as unknown). It skips a branch when "its configured upstream ref no longer exists", so it does not handle [gone] branches, and a branch whose tip is not reachable from its upstream, which a squash-merged one is not, "is silently skipped" ([git-branch at 2.56.0](#)). Since: Git 2.56 ([release notes](#)). Recommended: after upgrading, `git branch --delete-merged 'origin/*' --dry-run` first; keep `-d` with [gone] and `merge-tree` for the rest.

In production. Deleting a local branch deletes a name in your clone. Deleting the branch on the server is a push (`git push origin --delete <branch>`, Chapter 12) and affects everyone's next prune. Before either, answer the CTO's question with the three tests and `git log --oneline main..<branch>`: if that range is empty or every commit in it is squash-merged, nothing is lost.

7.14 What can go wrong

Commits on the wrong branch. Two commits were made on `main` that belong on a feature branch, and the working tree holds a third, uncommitted edit:

```
$ git status -sb
## main...origin/main [ahead 2]
M README.md
$ git log --oneline --decorate
cf0610c (HEAD -> main) Add cached() helper
478ecd8 Cache judge responses in memory
6b3b610 (origin/main) Add judge client
faf3622 Add README
```

[ahead 2] says the commits are local only, so the repair is private. Because branches are refs, it takes two ref writes and copies nothing:

```
$ git switch -c feature/judge-cache
Switched to a new branch 'feature/judge-cache'
$ git branch -f main origin/main
branch 'main' set up to track 'origin/main'.
$ git log --oneline --decorate
cf0610c (HEAD -> feature/judge-cache) Add cached() helper
478ecd8 Cache judge responses in memory
6b3b610 (origin/main, main) Add judge client
faf3622 Add README
$ git status -sb
## feature/judge-cache
M README.md
```

`git switch -c feature/judge-cache` creates a branch at the current commit and attaches HEAD to it; the index and working tree are untouched, so the uncommitted edit comes along. `git branch -f main origin/main` is allowed now that `main` is no longer checked out, and it moves `main` back to the published commit. `set up to track` appears because the start point is a remote-tracking branch.

```
$ git reflog show main -2
6b3b610 main@{0}: branch: Reset to origin/main
cf0610c main@{1}: commit: Add cached() helper
$ git reflog -1
cf0610c HEAD@{0}: checkout: moving from main to feature/judge-cache
$ git branch -vv
* feature/judge-cache cf0610c Add cached() helper
  main                6b3b610 [origin/main] Add judge client
```

Had the commits been pushed, the repair would be the same two writes plus a decision about the server copy (Chapter 12, section 12.8). Chapter 11 shows the same scenario with `git reset`.

Symptom	Diagnosis	Fix	Prevention
Commits "disappeared" after switching branches	<code>git reflog</code> shows commit: lines above the last checkout: moving from <code><id></code> to <code><branch></code> entry	<code>git branch <name> <id></code> (Lab 4.2)	Give a detached HEAD a branch before leaving it
<code>git branch -d</code> refuses although the branch "was merged"	<code>git log --oneline main..<branch></code> lists commits: squash or rebase merge, and the branch has no upstream or its upstream is gone (with an upstream that still contains it, <code>-d</code> deletes with a warning)	Verify with <code>git merge-tree --write-tree main <branch></code> , then <code>-D</code>	Delete the branch in the same step as the squash merge
cannot lock ref 'refs/heads/feature/x'	<code>git branch --list feature</code> shows a branch named feature	<code>git branch -m feature feature/<something></code>	Reserve namespace names
<code>git fetch</code> fails with unable to update local ref	<code>git branch -r</code> shows origin/feature next to a new feature/... on the server	<code>git fetch --prune</code> (Lab 4.3)	<code>fetch.prune=true</code> on a team that uses namespaces
<code>git switch</code> refuses: "Your local changes ... would be overwritten"	<code>git status</code> lists edits in files that differ between the branches	Commit or stash first, or <code>git switch --merge</code>	Commit before switching
<code>git branch -f main <x></code> fails with "used by worktree"	main is checked out here or in a linked worktree	<code>git reset --soft <x></code> while on main, or switch away first	Expected: force-moving the current branch needs <code>reset</code>
<code>git branch feature</code> fails: not a valid object name: 'main'	A new repository with no commit; <code>git log</code> shows nothing	Make the first commit, then branch	Expected on an unborn branch
warning: refname 'v0.2.0' is ambiguous	<code>git for-each-ref --format='%(refname)'</code> '*v0.2.0' lists a tag and a branch	Use <code>heads/v0.2.0</code> or <code>tags/v0.2.0</code> ; rename one	Keep tag and branch names disjoint
<code>git branch -vv</code> shows [origin/x: gone]	The upstream branch was deleted on the server	<code>git branch -d x</code> once its work is confirmed merged	Prune regularly
<code>git status</code> says HEAD detached at <code><id></code> in a script's checkout	<code>cat .git/HEAD</code> shows a raw ID: a clone with <code>--branch <tag></code> , a CI job, a worktree from a tag	Use <code>git rev-parse HEAD</code> ; push with an explicit refspec	Write scripts for detached HEAD
A rename left <code>-vv</code> pointing at the old upstream name	<code>git config get branch.<new>.merge</code> still names <code>refs/heads/<old></code>	<code>git push -u origin <new></code> and delete the old remote branch (Chapter 12)	Rename before the first push

7.15 When not to use it, and dangerous edge cases

Case-insensitive file systems. The default macOS volume format does not distinguish `main` from `MAIN`. With the files backend a loose ref is a file, so a wrong-case name finds the right file and HEAD records the wrong name:

```
$ git config get core.ignoreCase
true
$ git switch MAIN
Switched to branch 'MAIN'
$ cat .git/HEAD
ref: refs/heads/MAIN
$ git branch
```

```
main
$ git status
On branch MAIN
nothing to commit, working tree clean
```

`git switch MAIN` succeeded, HEAD says `refs/heads/MAIN`, and `git branch` lists `main` without an asterisk: the current branch is not in the list. A commit in this state moves the file that exists, `refs/heads/main`:

```
$ printf '\nRun the tests with: python -m pytest\n' >> README.md
$ git commit -am "Document how to run the tests"
[MAIN 6e419d0] Document how to run the tests
1 file changed, 2 insertions(+)
$ git log --oneline --decorate
6e419d0 (HEAD, main) Document how to run the tests
6eab4a9 Add README
$ git for-each-ref --format='%(objectname:short) %(refname)' refs/heads
6e419d0 refs/heads/main
```

`git log --decorate` prints `(HEAD, main)` rather than `(HEAD -> main)`: HEAD and `main` name the same commit, but HEAD does not point at `main`. Switching to the correctly spelled name repairs it, and nothing was lost:

```
$ git switch main
Switched to branch 'main'
$ git branch
* main
$ git log --oneline --decorate -1
6e419d0 (HEAD -> main) Document how to run the tests
```

```
$ git branch Main
fatal: a branch named 'Main' already exists
[exit status: 128]
$ git pack-refs --all
$ git branch Main
$ git for-each-ref --format='%(objectname:short) %(refname)' refs/heads
6e419d0 refs/heads/Main
6e419d0 refs/heads/main
```

Once the refs are packed, the file system is out of the picture and `Main` is created as a second ref. The repository now has two branches that differ only in case. A case-sensitive file system keeps them apart; on this volume they collide again as soon as one of them is written back as a loose file. The report's reasons for making `reftable` the default in Git 3.0 include exactly this class of clash (Phase 0 report, section 1). Use lowercase names, and check `cat .git/HEAD` when `git branch` shows no asterisk.

Other edge cases.

- `git branch -D` **deletes the branch's reflog**. For a branch whose commits were never checked out, nothing but the printed `was <id>` and the object grace period remains. Preview with `git log --oneline main..<branch>`.
- **A plain** `git branch -d` **can delete** `main`. There is nothing special about the name; if `main` is not checked out and its tip is reachable from its upstream, `-d` deletes it. Recreate it from `origin/main`.
- **Deleting a branch does not delete its commits**, and it does not remove them from the server, from colleagues' clones or from a pull request. A branch deletion is never a way to withdraw content.
- `git switch` **can overwrite an ignored file** that is tracked on the other branch (Chapter 4, section 4.16).
- `git switch --discard-changes` (also `-f`) is  **DANGEROUS**. It changes the index and working tree to the target and destroys uncommitted edits in the files it touches, with no preview beyond `git status` and no recovery for content that was never staged. It is appropriate only when you have decided to throw the edits away.
- **Hand-written ref files are**  **DANGEROUS**. `echo <id> > .git/refs/heads/x` skips validation, locking and the reflog; a typo produces a broken ref that `git log --all` refuses to read (Lab 4.1 repairs one). Use

`git update-ref`.

- **Tags have no reflog by default.** `git tag -f` and `git tag -d` leave only the ID in their output.

7.16 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git branch</code> , <code>git for-each-ref</code> , <code>git symbolic-ref HEAD</code> , <code>git rev-parse</code> , <code>git merge-base</code> , <code>git rev-list</code> , <code>git log</code>	● SAFE	Nothing	not needed	not needed
<code>git branch <name></code> [<code><start></code>], <code>git switch -c</code> , <code>git tag <name></code>	● SAFE	Adds a ref (and for branches a reflog)	<code>git rev-parse <start></code>	Delete the ref
<code>git switch <branch></code> , <code>git switch --detach</code> , <code>git switch -</code> , <code>git switch --orphan</code> , the <code>git checkout</code> equivalents	● SAFE	HEAD, index and working tree (<code>--orphan</code> removes tracked files); all refuse to overwrite local edits	<code>git status</code>	<code>git switch -</code> , or the <code>checkout:</code> line in <code>git reflog</code>
<code>git branch -m</code> , <code>git branch -f</code> , <code>git switch -C</code> , <code>git update-ref <ref> <id></code> , <code>git symbolic-ref HEAD <ref></code>	● CAUTION	Renames or moves a ref; the old value stays in the reflog	<code>git branch -vv</code> , <code>git rev-parse <ref></code>	<code>git branch -f <ref> <ref>@{1}</code> ; rename back
<code>git branch -d</code> , <code>git tag -d</code> , <code>git tag -f</code> , <code>git fetch --prune</code>	● CAUTION	Deletes or moves a ref; <code>-d</code> checks merge status first; tags and remote-tracking refs have no reflog by default	<code>git branch --merged</code> , <code>git remote prune origin --dry-run</code>	Recreate from the <code>was <id></code> output or from <code>git reflog</code>
<code>git branch -D</code> , <code>git update-ref -d</code>	● DANGEROUS	Deletes a ref and its reflog with no merge check	<code>git log --oneline main..<branch></code>	<code>git branch <name> <id></code> while the objects exist; the HEAD reflog if the commits were checked out
<code>git switch --discard-changes</code> , <code>git checkout -f</code>	● DANGEROUS	Overwrites uncommitted changes in the index and working tree	<code>git status</code> , <code>git diff</code>	None for content that was never staged
Writing a file under <code>.git/refs/</code> by hand	● DANGEROUS	A ref without validation, locking or reflog	<code>git update-ref</code> instead	<code>git update-ref</code> after deleting the broken file (Lab 4.1)

7.17 Version notes

Version note. Older behavior: `git checkout` switched branches, created them with `-b`, detached HEAD and restored files. Current behavior: `git switch` and `git restore` split those jobs; `git checkout` still does all of them and is not scheduled for removal. Since: Git 2.23 ([release notes](#)); no longer labelled experimental from Git 2.51 ([release notes](#)). Recommended: write `switch`, read `checkout`.

Version note. Older behavior: the current branch had to be parsed out of `git branch` or `git symbolic-ref`. Current behavior: `git branch --show-current`, which prints nothing in detached HEAD. Since: Git 2.22

([release notes](#)). Recommended: use it in scripts, and treat empty output as detached.

Version note. Older behavior: `git checkout -m <branch>`, and `git switch -m`, gave one chance to resolve conflicts between local edits and the target branch. Current behavior: the local changes are saved in a stash and reapplied. Since: Git 2.55 ([release notes](#)). Recommended: commit or stash yourself; use `--merge` when you understand the stash it may leave behind.

Version note. Older behavior: no built-in way to ask which branch a commit was probably based on. Current behavior: `%(is-base:<commit>)` in `git for-each-ref`. Since: Git 2.47 ([release notes](#)). Recommended: a hint for humans, never an input to automation.

Version note. Current behavior: Git 2.56 adds `git branch --delete-merged` and `--forked <branch>`, which lists branches whose configured upstream matches ([git-branch at 2.56.0](#)). Not run here; section 7.13 has the details and the limits.

Version note. Older behavior: `git config --get branch.x.merge`. Current behavior: `git config get branch.x.merge`, as used in this chapter; the old form still works. Since: Git 2.46. Recommended: the subcommand form.

Unverified. The releases that introduced `%(ahead-behind:<commit>)` for `git for-each-ref` and `git refs verify` were not found in the release notes available for this chapter. Both ran on Git 2.55.0 as shown.

7.18 Practice

- **Labs 4.1 to 4.4** in the Module 4 lab manual: refs by hand, the detached-HEAD rescue, the `feature` versus `feature/x` conflict, and counting divergence.
- Replay any transcript with `labs/run ch07/<demo>` and continue by hand in the sandbox it leaves behind.
- Three drills. In `ch07/divergence`, predict the output of `git rev-list --left-right --count feature/rouge...main` before running it. In `ch07/detached-head`, find the two commits of the experiment with `git reflog` alone, without scrolling back. In `ch07/stale-branches`, write one `git for-each-ref` command that lists every local branch with its upstream state and the date of its tip.

7.19 Interview questions

1. Define a branch in one sentence that mentions neither "copy" nor "line of development", and prove the definition with two commands.
2. What is in `.git/HEAD` in the three states: on a branch, detached, and on an unborn branch? How does each state change what `git commit` does?
3. A colleague's two days of commits are "gone" after she checked out a tag and later switched back to `main`. Walk through the recovery and explain why it works.
4. Why does `git branch -d` refuse a branch that was squash-merged once its upstream is gone (or when it never had one), why does it delete the same branch with a warning while the remote-tracking ref still contains it, and how do you prove the branch is safe to delete?

5. Explain the `-d` rule in terms of the upstream. Give one case where `-d` deletes a branch that `main` does not contain, and one where it refuses a branch that `main` does contain.
6. What does `git switch -c fix origin/main` write to `.git`, including configuration?
7. `git log main..feature` and `git diff main..feature`: how do the two dots differ in meaning? Which one do you want for a review?
8. Does Git know which branch a branch was created from? What is the closest it can offer, and why is a pull request's base branch not the same thing?
9. Why does `git fetch` fail on every clone after someone pushes `feature/login`, and what is the one-command fix?
10. Why does `git tag -d` have no undo through the reflog, while `git branch -D` sometimes does?
11. On macOS, `git branch` shows no current branch and `git log --decorate` says `(HEAD, main)`. What happened?
12. Which commands in this chapter are dangerous enough that you would want a preview, and what is the preview for each?

7.20 Sources

Primary sources

- [git-branch](#), [git-switch](#), [git-checkout](#) ("Detached HEAD"), [git-symbolic-ref](#), [git-update-ref](#), [git-for-each-ref](#), [git-rev-parse](#), [git-merge-base](#), [git-rev-list](#), [git-tag](#), [git-check-ref-format](#), [git-pack-refs](#), [git-merge-tree](#), [git-fetch](#), [git-clone](#), [git-worktree](#), [git-submodule](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitrevisions](#), [gitglossary](#), [gitdatamodel](#).
- Git source at 2.55.0, [wt-status.c](#), for how `git status` chooses "detached at", "detached from" and "Not currently on any branch".
- [git-branch at 2.56.0](#) and release notes [2.22](#), [2.23](#), [2.47](#), [2.51](#), [2.55](#), [2.56](#).
- GitHub Docs: [pull requests reference](#), [events that trigger workflows](#).

Secondary sources

- Pro Git, [Branches in a Nutshell](#) and [Git References](#). Caveats: `master` throughout, and branch switching with `git checkout`.
- Julia Evans, [git branches: intuition & reality](#) and [how HEAD works in git](#); the [2024 poll results](#) quoted in section 7.1.
- The Phase 0 report of this course, sections 1 and 12, for the reftable rationale and the misconception table.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Git Internals by John Britton of GitHub - CS50 Tech Talk](#), 2018: refs as files holding an ID, branches as pointers. Caveats: `master` and `checkout`; SHA-1 only.
- [Git For Beginners](#), Telusko, 2023: asks whether a branch copies the project and explains why it does not. Caveat: no undo or rebase.
- [Intern DELETED a Git Branch!](#), Chai aur Code, Hindi, 2026: recovering a deleted branch through the reflog. Caveat: a single scenario.

Outdated advice. Several popular tutorials introduce a branch as a copy of the project and HEAD as the latest commit; the report lists [Apna College's 2023 tutorial](#) among them. Both statements fail the transcripts of sections 7.2 and 7.3.

Further reading

- Chapter 8, section 8.2, for merge bases with more than one candidate; Chapter 12 for everything about `origin/`; Chapter 13 for the reflog as a recovery tool.